从 C++到 AI 计算：第三册实践与代码卷

本地量化推理引擎、完整源码、KV Cache、后端优化与验收

CPU Performance Study

本卷是《从 C++ 到 AI 计算：第三册》的配套实践与代码卷，围绕 `linux_cpu_inference` 贯穿项目展开：模型资产、tokenizer、张量布局、int8 kernel、reference decoder、KV cache、trace、7B 账本、服务调度、CPU/GPU 后端边界和工程验收都要落到可构建代码、完整源码、测试和报告。

前言

第三册原理卷把 AI 推理系统拆成模型资产、typed graph、张量布局、量化、KV cache、后端、服务运行时和验收体系。本实践与代码卷只做一件事：把这些概念压到同一个可运行项目里，并把完整源码阅读路径并入同一目录，让读者知道每个概念对应哪段代码、哪条命令、哪种输入、哪份报告和哪条失败路径。

贯穿项目是 `books/cpu-volume-3/labs/linux_cpu_inference`。它不是玩具目录，而是第三册已经持续维护的本地 CPU 量化推理切片：包含教学模型格式、tokenizer、safetensors manifest gate、int8 linear、packed/AVX2 路径、tiny reference decoder、GPT-2 reference path、KV cache trace、7B compute ledger、serving SLO probe、自研 GPT-2 smoke、真实 small 开源模型 smoke、CLI、benchmark 和 CTest。本卷不维护第二份实现；源码章节通过 `lstinputlisting` 直接读取同一份工程文件，所有练习、讲解和完整代码都回到同一份源码。

本卷的学习顺序是从小证据到大系统：先让一个 tiny MLP 算对，再解释张量地址流和 int8 累加；再让 reference decoder、sampler 和 KV cache trace 固定语义；然后接模型导入、shape verifier、memory planner 和 IR lowering；最后用 CPU/GPU 后端边界、工业专项和回归门禁收束。代码走读不再被放到最后当附录，而是插入到相同主题部分中：讲完项目入口就读构建文件和调用链，讲完量化就读公共合同、tiny MLP 和 int8 kernel，讲到真实模型时读 tokenizer、safetensors 和 GPT-2，收束验收时读工具、benchmark、smoke 和测试。每章都要求读者交付代码运行结果或报告字段，而不是只抄概念定义。

核心思想

> 第三册实践的目标不是“写一个看起来像大模型的 demo”，而是建立一条能被复查的推理证据链：资产能校验，shape 能验证，kernel 能对照，trace 能定位，性能能降级，服务指标能解释，回归门禁能长期维护。

常见缩写和术语先导

缩写	全称	本卷中的含义
AI	Artificial Intelligence	这里特指本地推理工程，不讨论泛泛应用口号。
LLM	Large Language Model	decoder-only 文本生成模型是本卷主线。
KV Cache	Key/Value Cache	attention 历史 K/V 状态，属于请求生命周期。
GEMM	General Matrix Multiply	矩阵乘，是预填充和训练常见核心算子。
GEMV	General Matrix Vector Multiply	矩阵向量乘，是单 token decode 的核心形态。
SIMD	Single Instruction Multiple Data	一条指令处理多个 lane 的 CPU 执行方式。
AVX2	Advanced Vector Extensions 2	x86-64 上的 256-bit SIMD 指令集之一。
FMA	Fused Multiply-Add	乘加融合指令，常出现在浮点 kernel 中。
GGUF	GGML Universal File	llama.cpp 生态常见模型文件，保存权重、量化格式、tokenizer 和模型元数据。
GGML	Georgi Gerganov Machine Learning	llama.cpp 生态底层张量、量化 block 和 CPU kernel 体系；本卷按它的 block 布局读取 `Q4_K/Q5_0/Q6_K/Q8_0`。
Q4_K/Q5_0 Q6_K/Q8_0	Quantized GGML formats	GGML 量化权重格式名。数字表示主要位宽，后缀表示 block 布局族；它们不是 C++ 类型名，进入代码前必须先经过格式 verifier。
RMSNorm	Root Mean Square Normalization	Transformer 中按 hidden 维做均方根归一化的层。
RoPE	Rotary Position Embedding	旋转位置编码，把位置信息注入 Q/K 向量。
GQA	Grouped-Query Attention	多个 query head 共享一组 KV head 的 attention 形状。
SwiGLU	Swish-Gated Linear Unit	LLaMA-like MLP 常用的门控激活结构。
SLO	Service Level Objective	服务目标，例如 TTFT、TPOT、p95/p99。
TTFT	Time To First Token	从请求进入到首 token 输出的时间。
TPOT	Time Per Output Token	decode 阶段每个输出 token 的平均或尾部时间。
IR	Intermediate Representation	编译器内部表示，用于 pass、lowering 和计划生成。
RAG	Retrieval-Augmented Generation	检索增强生成，需要把外部文档和 prompt 编排起来。
MoE	Mixture of Experts	多专家模型，会改变路由、负载和内存账本。
LoRA	Low-Rank Adaptation	低秩适配权重，常用于微调和多租户加载。

这些缩写不要死记。每个缩写在正文里都要落到一个代码对象或报告字段：KV cache 对应 trace 里的 slot 和 position，SLO 对应 serving probe 的 CSV，IR 对应 executable plan，SIMD/AVX2 对应 kernel 反汇编和 benchmark，GGUF/GGML 对应 manifest、tensor bytes、direct/fallback tensor count 和 caw A/B 报告。

本卷结构

本卷按第三册原书的四个大块组织，但目录不再采用“前面做实践、后面贴源码”的割裂方式。每个大块都把实践任务、代码走读和验收证据放在同一条主线里：`linux_cpu_inference`。

第一部分从模型资产到量化 kernel。你会运行 tiny MLP，先读构建入口、项目边界和调用链总图，再检查 tokenizer 和 safetensors manifest，手算张量地址，比较 scalar、packed 和 AVX2 路径，并在公共合同、tiny MLP 与 int8 kernel 源码里解释 scale、accumulator 和 requantize 的边界。

第二部分进入 Transformer、KV cache、sampler 和服务运行时。你会跑 reference decoder trace，解释每个 checkpoint 的 shape、stride、dtype 和 layout，构造 KV cache 状态表，分析 admission、queue wait、TTFT、TPOT、取消和拒绝。

第三部分处理模型导入、真实模型加载闭环、memory planner 和 IR lowering。你会把 manifest、shape verifier、tensor role、state edge、workspace、liveness 和 executable plan 写成可检查报告，并在同一部分中读 reference decoder、tokenizer、safetensors 和 GPT-2 reference path 的完整实现，显式运行 LCQI 自研 GPT-2 safetensors smoke 和一个 small 级别开源 GGUF 模型 smoke，而不是只停留在 tiny fixture。

第四部分收束到后端优化和验收。你会比较 CPU kernel 证据、GPU 后端边界、工业专项能力和回归门禁，并直接读工具、smoke、benchmark、ledger、serving probe 和测试源码。最终交付不是“跑通一次”，而是可复查的工程档案：命令、输入、输出、trace、benchmark、不可验证边界和回归规则。

目录

第零部分

推理链路、张量账本与量化

第 1 章实践：端到端推理链路与项目总入口

本章对应原书中的第三册“端到端链路：从模型文件到下一个 token”。不要把它当作概念复述，本章要把模型目录、manifest、tokenizer、typed graph、backend plan、request session、KV cache、oracle 和 profiler 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `tiny_mlp_i8.txt`，核心命令或测试入口是 `lcqi_cli`。

Listing 1.1 第 1 章实践：端到端推理链路与项目总入口的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_cli / tiny_mlp_i8.txt
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：只跑一个 tiny MLP 推理请求。读者要先手写预期结果，再运行项目观察输出。动作是：把 `readfiles->tokenize->run_inference->logits->predicted_class` 串起来。如果输出里没有 `predicted_class` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：模型文件存在不等于推理链路可信；只有 CLI 输出、测试断言、输入合同和失败路径同时成立，才算完成入口闭环这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：最终大作业的总目录：每章报告都放入 `reports/volume3-practice/`，第一章提交项目地图、命令记录和一次成功/失败输入对照。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

代码走读：构建入口、项目边界与调用链总图

源码阅读覆盖范围

LCQI 的源码边界由三个层次组成。第一层是仓库构建入口，它决定 CMake 怎样把第三册的实验代码纳入构建和测试。第二层是 `labs/linux_cpu_inference`，它包含真正的库、CLI、benchmark、trace、ledger、serving probe 和测试目标。第三层是工具脚本，它们下载或复用外部模型资产，生成 smoke 和 benchmark 报告。

源码阅读任务

> 先读本章的 CMake 文件，再读 README。读者要能回答三个问题：一是 `lcqi` 静态库包含哪些实现文件；二是哪些可执行文件只依赖自研代码，哪些依赖可选外部库；三是真实 GPT-2 和 SmolLM2 smoke 为什么不进入默认 CTest。

根构建入口

第三册根目录的 CMake 文件只做项目级选择：是否构建第三册 labs，以及如何把 `labs/linux_cpu_inference` 接入。它应该保持薄层，避免把具体目标定义分散到多个地方。

Listing 1.2 cpu-volume-3/README.md

````
# 从 C++ 到 AI 计算：第三册

第三册用于承接 AI、AI 编译器、算子开发和本地大模型推理引擎。第二册只讨论硬件原理、多核、高性能计算、并发、锁、无锁数据结构、并行算法和分布式计算，不再直接进入 AI。

本册贯穿项目是一台能推理开源 7B 左右 decoder-only 大模型的本地引擎，并把 AI 编译器作为引擎内部主线：模型图导入、typed tensor IR、算子融合、内存规划、CPU/GPU lowering、kernel selection 和 runtime dispatch 都要能落回编译原理与底层机器账本。工程路线从 CPU 上可解释的 reference 和量化切片开始，逐步扩展到 tokenizer、模型加载、Transformer 层、KV cache、CPU SIMD/多线程后端、GPU 后端、正确性报告和性能对标。

正式书稿工程位于：

```text
source/latex/
```

构建命令：

```bash
make -C source/latex check
make -C source/latex pdf
make -C source/latex epub
```

生成文件：

```text
````
````

~~~~markdown
source/latex/main.pdf
source/latex/main.epub
```

当前保留的第一阶段切片：

```text
labs/linux_cpu_inference/
```

该目录目前包含最小 MLP/int8 权重教学切片、tiny reference decoder、GPT-2 reference path、int8 scalar baseline、packed layout、AVX2 SIMD 路径和 shape sweep benchmark，用来验证张量、线性层、量化权重、tokenizer、safetensors、GPT-2 forward、KV cache、oracle 和 benchmark 的基本边界。当前 x86-64 AVX2 结果见：

```text
results/lcqi-x86-avx2-benchmark-2026-06-25.csv
results/lcqi-x86-avx2-decode4096-2026-06-25.csv
```

自研 GPT-2 smoke 入口如下：

```bash
make cpu3-gpt2-smoke
make cpu3-gpt2-benchmark-compare
```

第一条命令下载 `openai-community/gpt2` 的标准 safetensors 资产到用户缓存目录，用 LCQI C++ reference path 生成 1 个 token，并把报告写入：

```text
results/lcqi-gpt2-smoke.txt
```

第二条命令用同一个 GPT-2 F32 模型对比 LCQI full-prefix 和 cached-KV 生成路径，并在本机存在 llama.cpp/SmolLM2 缓存时附带成熟引擎参考，报告写入：

```text
results/lcqi-gpt2-benchmark-compare.txt
```

它仍然不是生产级 AI Infra。生产级门禁见：

```text
reports/ai-infra-maturity-audit.md
```

第三册后续必须把 lab 推进到“小型 CPU 推理 runtime + 手写高性能算子 + 严格 benchmark 报告 + 真实系统对标”，否则只能证明系统学习，不能证明生产级能力。
~~~~

Listing 1.3 cpu-volume-3/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.31)
project(CPUVolume3 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

option(CPU_VOLUME_3_BUILD_LABS "Build CPU Volume 3 labs" ON)

if(CPU_VOLUME_3_BUILD_LABS)
    enable_testing()
    add_subdirectory(labs/linux_cpu_inference)
endif()
```

实践与代码卷也提供一个薄 CMake 入口，它把同一份 LCQI 源码作为可测试对象接入。这样本卷不会复制实现，只复用第三册主工程。

Listing 1.4 cpu-volume-3-practice/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.31)
project(CPUVolume3Practice LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

enable_testing()
add_subdirectory(../cpu-volume-3/labs/linux_cpu_inference linux_cpu_inference)
```

本卷自身的 Makefile 负责构建 PDF/EPUB，并通过 `test` 目标回到第三册真实 LCQI 工程做 CMake/CTest 验证。实践与代码卷如果只排版、不测试，就不能宣称“完整源码可运行”。

Listing 1.5 cpu-volume-3-practice/Makefile

```
LATEX_DIR := source/latex

.PHONY: all check pdf epub test clean

all: pdf epub

check:
    $(MAKE) -C $(LATEX_DIR) check

pdf:
    $(MAKE) -C $(LATEX_DIR) pdf

epub:
    $(MAKE) -C $(LATEX_DIR) epub

```

```
test:
    cmake -S . -B build/lcqi-debug -DCMAKE_BUILD_TYPE=Debug
    cmake --build build/lcqi-debug
    ctest --test-dir build/lcqi-debug --output-on-failure

clean:
    $(MAKE) -C $(LATEX_DIR) clean
```

Listing 1.6 cpu-volume-3-practice/source/latex/Makefile

```
LATEXMK ?= latexmk
ENGINE ?= -xelatex
MAIN ?= main.tex
EPUB_BUILDER ?= ../../../cpu-volume-1/source/latex/scripts/build_epub.py

.PHONY: all pdf epub clean check

all: pdf epub

pdf:
    $(LATEXMK) $(ENGINE) -interaction=nonstopmode -halt-on-error $(MAIN)

epub:
    @BOOK_TITLE="从 C++ 到 AI 计算：第三册实践与代码卷" \
    BOOK_IMPORT_TITLE="CPU Volume 3 Practice Code" \
    BOOK_ID_SEED="cpu-volume-3-practice-code-weread-20260629" \
    EPUB_ASCII_TITLES=1 \
    BOOK_SUBTITLE="本地量化推理引擎、完整源码、KV Cache、后端优化与验收" \
    BOOK_AUTHOR="CPU Performance Study" \
    python3 $(EPUB_BUILDER) --book-dir . --output main.epub --no-cover-page --forbid-images --linearize-tables --wechat-compatible --no-embed-fonts

clean:
    $(LATEXMK) -C

check:
    @python3 scripts/check_inputs.py
    @command -v xelatex >/dev/null 2>&1 || { echo "missing xelatex"; exit 1; }
    @command -v latexmk >/dev/null 2>&1 || { echo "missing latexmk"; exit 1; }
```

LCQI 目标定义

`linux_cpu_inference/CMakeLists.txt` 是代码走读中的第一份核心文件。这里能看到 `lcqi` 库、`lcqi_cli`、`lcqi_bench`、`lcqi_trace`、`lcqi_gpt2`、`lcqi_ledger`、`lcqi_serving_probe`、可选 `lcqi_onednn_bench` 和 `lcqi_tests` 的完整关系。

Listing 1.7 linux_cpu_inference/CMakeLists.txt

```
find_package(Threads REQUIRED)

add_library(lcqi STATIC
    src/f32_kernels.cpp
```

```cmake
    src/ggml_matvec.cpp
    src/ggml_tensors.cpp
    src/gguf.cpp
    src/gpt2_reference.cpp
    src/inference.cpp
    src/int8_kernels.cpp
    src/llama_gguf.cpp
    src/model.cpp
    src/q4_k.cpp
    src/reference_decoder.cpp
    src/safetensors.cpp
    src/tokenizer.cpp
)

if(CMAKE_SYSTEM_PROCESSOR MATCHES "x86_64|AMD64|i[3-6]86")
    if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
        target_sources(lcqi PRIVATE src/f32_kernels_avx2.cpp)
        target_sources(lcqi PRIVATE src/ggml_matvec_avx2.cpp)
        target_sources(lcqi PRIVATE src/int8_kernels_avx2.cpp)
        target_sources(lcqi PRIVATE src/q4_k_avx2.cpp)
        set_source_files_properties(src/f32_kernels_avx2.cpp PROPERTIES COMPILE_OPTIONS "-mavx2;-mfma")
        set_source_files_properties(src/ggml_matvec_avx2.cpp PROPERTIES COMPILE_OPTIONS "-mavx2")
        set_source_files_properties(src/int8_kernels_avx2.cpp PROPERTIES COMPILE_OPTIONS "-mavx2;-mfma")
        set_source_files_properties(src/q4_k_avx2.cpp PROPERTIES COMPILE_OPTIONS "-mavx2")
        target_compile_definitions(lcqi PRIVATE LCQI_ENABLE_AVX2=1)
    endif()
endif()

if(NOT CMAKE_SYSTEM_NAME STREQUAL "Linux")
    message(WARNING "LCQI is written for the book's local Linux CPU target; current system is ${CMAKE_SYSTEM_NAME}.")
endif()

target_include_directories(lcqi
    PUBLIC
        ${CMAKE_CURRENT_SOURCE_DIR}/include
)

target_compile_features(lcqi PUBLIC cxx_std_20)
target_link_libraries(lcqi PUBLIC Threads::Threads)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_cli
    src/main.cpp
)
```

```
target_link_libraries(lcqi_cli PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_cli PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_bench
    src/bench.cpp
)

target_link_libraries(lcqi_bench PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_bench PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_gguf_q4_bench
    src/gguf_q4_bench.cpp
)

target_link_libraries(lcqi_gguf_q4_bench PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_gguf_q4_bench PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_trace
    src/trace.cpp
)

target_link_libraries(lcqi_trace PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_trace PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_gpt2
    src/gpt2_cli.cpp
)

target_link_libraries(lcqi_gpt2 PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_gpt2 PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_llama_gguf
    src/llama_cli.cpp
)

target_link_libraries(lcqi_llama_gguf PRIVATE lcqi)
```

```cmake

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_llama_gguf PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_ledger
    src/ledger.cpp
)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_ledger PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_serving_probe
    src/serving_probe.cpp
)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_serving_probe PRIVATE -Wall -Wextra -Wpedantic)
endif()

find_package(dnnl QUIET)
if(dnnl_FOUND)
    add_executable(lcqi_onednn_bench
        src/onednn_bench.cpp
    )
    target_link_libraries(lcqi_onednn_bench PRIVATE DNNL::dnnl)
    target_compile_features(lcqi_onednn_bench PRIVATE cxx_std_20)
    if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
        target_compile_options(lcqi_onednn_bench PRIVATE -Wall -Wextra -Wpedantic)
    endif()
endif()

add_executable(lcqi_tests
    tests/lcqi_tests.cpp
)

target_link_libraries(lcqi_tests PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_tests PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_test(NAME lcqi_tests COMMAND $<TARGET_FILE:lcqi_tests>)
add_test(NAME lcqi_gpt2_tiny COMMAND $<TARGET_FILE:lcqi_gpt2>)
set_tests_properties(lcqi_gpt2_tiny PROPERTIES
    PASS_REGULAR_EXPRESSION "generated_ids 1 2 3 3 3"
)
add_test(NAME lcqi_trace COMMAND $<TARGET_FILE:lcqi_trace>)
set_tests_properties(lcqi_trace PROPERTIES
    PASS_REGULAR_EXPRESSION "tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4"
```

```
)
add_test(NAME lcqi_ledger COMMAND $<TARGET_FILE:lcqi_ledger>)
set_tests_properties(lcqi_ledger PROPERTIES
    PASS_REGULAR_EXPRESSION "bandwidth_limit,int4_at_100_gbps,23.602,tokens/s"
)
add_test(NAME lcqi_serving_probe COMMAND $<TARGET_FILE:lcqi_serving_probe>)
set_tests_properties(lcqi_serving_probe PROPERTIES
    PASS_REGULAR_EXPRESSION "request,req_too_long,0,memory_budget_exceeded"
)
```

项目说明与模型资产

README 是复现实验的入口。它写清楚当前能力、后续边界、构建命令、tiny 推理命令、benchmark 字段、reference trace 字段、GPT-2 smoke、7B ledger、serving SLO probe 和 SmolLM2 small smoke。读源码前先读 README，可以避免把 tiny fixture、GPT-2 reference path 和外部 llama.cpp smoke 混成同一个能力等级。

Listing 1.8 linux_cpu_inference/README.md

```
# Linux CPU Quantized Inference

这是第三册贯穿项目的第一阶段切片：Linux 本地 CPU 量化线性层和最小推理流程。第三册的贯穿目标不是这个 MLP 本身，而是一台能推理开源 7B 左右 decoder-only 大模型的本地引擎，并逐步覆盖 tokenizer、模型加载、Transformer 层、KV cache、CPU/GPU 后端、正确性报告和性能对标。

目标平台是 x86-64 Linux 实验环境。完整性能报告应记录 CPU 型号、内核版本、编译器版本、是否能使用 PMU、NUMA 和线程亲和能力。若运行环境缺少 perf、NUMA、线程亲和性或硬件性能计数器权限，报告必须把相关结论降级为“未验证”，不能把源码路径存在当作实测性能证据。后续 GPU 后端会作为独立 backend 接入，不改变当前切片的语义合同。

当前版本支持：

- 教学文本模型格式 `LCQI_MODEL_V1`
- 最小 tokenizer 合同格式 `LCQI_TOKENIZER_V1`
- safetensors header manifest 解析：metadata、tensor name、dtype、shape、data offsets、offset 边界和 dtype/shape 字节数校验
- 两层 MLP 教学切片
- int8 权重加 per-layer scale
- float 输入和激活
- 量化线性层、ReLU、分类 argmax
- int8 linear scalar baseline、packed layout、AVX2 路径和 shape sweep benchmark
- GPT-2 F32 dot kernel：cached generation 热路径使用可替换 dot 边界，x86-64/GNU/Clang 构建接入 AVX2/FMA 实现，其他平台保留标量 fallback
- tiny reference decoder：RMSNorm、float linear、RoPE、GQA KV cache、decode attention、SwiGLU、lm head
- GPT-2 reference path：byte-level BPE tokenizer、`config.json`、F32/F16/BF16 safetensors 张量读取、HuggingFace GPT-2 name mapping、LayerNorm、绝对位置嵌入、packed QKV、causal attention、GELU、tied lm head、full-prefix greedy generation、KV cache greedy generation 和分阶段 benchmark
- reference trace CSV：从 prompt/tokenizer 合同到 logits/sampler/token，逐
```

````
checkpoint 输出 shape、stride、dtype、layout、checksum、max_abs、values 摘要
- 7B ledger CSV：输出典型 7B shape 的 FLOPs、KV 容量、权重/KV 字节和 bandwidth-only tokens/s 上限
- serving SLO probe CSV：输出 admission、batch state、KV 预算、queue wait、TTFT、TPOT、取消回收和拒绝率
- 自研 GPT-2 冒烟：加载 `openai-community/gpt2` 的 `config.json`、`vocab.json`、`merges.txt`、`model.safetensors`，在 LCQI C++ reference path 上生成 1 个 token，并把文件 hash、prompt ids、generated ids 和文本输出写入报告
- 自研 GPT-2 benchmark 对比：同一个 GPT-2 F32 safetensors 模型分别跑 full-prefix 和 cached-KV，并在可用时附带 llama.cpp/SmolLM2 Q4_K_M 作为成熟引擎参考
- 自研 GPT-2 优化 A/B：从指定 baseline commit 建临时 worktree，当前实现和 baseline 都通过 `books/cpu-volume-3-practice` 的 Release CMake 入口构建，再用同一个 GPT-2 F32 模型多轮测量 full-prefix 与 cached-KV
- 真实 small 开源模型冒烟：通过 llama.cpp 加载 SmolLM2-135M-Instruct 的 GGUF 量化文件，在本地 CPU 上生成一段 assistant 文本，并把模型 hash、命令、输出和性能行写入报告
- 自研 SmolLM2/GGUF reference decode：读取同一个 GGUF 内的 tokenizer tokens/merges 和 `F32/Q8_0/Q5_0/Q6_K/Q4_K` 混合权重，加载 30 层 SmolLM2，执行 RMSNorm、normal RoPE、GQA KV cache、SwiGLU 和 tied lm head；默认把 shape 兼容的 `Q4_K` linear tensor 留在 GGUF block bytes 中直接执行，其余格式走 f32 fallback；`LCQI_LLAMA_GGML_DIRECT=1` 可打开 `Q5_0/Q6_K/Q8_0` 实验直算路径用于分析
- CLI 推理和简单 benchmark
- CTest 正确性测试

后续扩展方向：

- 真实 7B 级模型 metadata、tokenizer 和量化权重导入
- SentencePiece tokenizer、safetensors 分片加载和更多模型方言 name mapping
- GPT-2 reference path 的 Transformers/PyTorch golden logits 对齐报告
- KV cache 分页、内存规划和可选量化
- CPU packed weight、SIMD、NUMA 和线程亲和优化
- GPU backend adapter 和 device kernel
- 与现有框架的 prefill/decode/内存/误差对标报告

构建：

```bash
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
```

运行：

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_cli \
  books/cpu-volume-3/labs/linux_cpu_inference/models/tiny_mlp_i8.txt \
  1.0 2.0 -1.0 0.5 1000
````
````

````
```

kernel benchmark:

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_bench 200
```

输出 CSV 字段：

```text
target_arch,input_size,output_size,output_block,repeat,backend,available,average_us,max_abs_diff,checksum
```

当前 benchmark 按 backend 逐行输出：`scalar`、`packed_scalar`、`avx2`。x86-64 构建会编译 AVX2 路径；不支持 AVX2/FMA 的机器会把该 backend 标为 `available=0`，避免把源码存在误报成当前环境实测性能。

这还不是生产级高性能 kernel。当前 benchmark 建立 scalar baseline、packed layout、AVX2 基线正确性和 shape sweep 口径。要达到完整 AI Infra 证据，还必须继续补反汇编分析、AVX-512、perf counter、oneDNN/OpenBLAS/llama.cpp/ggml 对照、sanitizer、fuzz 和 CI。

safetensors manifest gate:

`lcqi_tests` 会运行时生成一个最小 safetensors fixture，验证 header 长度、`__metadata__`、tensor name、dtype、shape、data offsets、payload 边界和 dtype/shape 推导出的字节数。它还会生成 offset 超出 payload、dtype/shape 字节数不匹配的坏文件，确认加载器会拒绝。这个 gate 只覆盖模型资产目录表，不等于已经支持完整 LLaMA/Qwen 权重映射、分片合并或 mmap 执行；后续真实模型加载必须在这个 manifest 上继续接 name mapping、shape verifier、quant descriptor 和 first token trace。

reference decoder trace:

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_trace
```

输出 CSV 字段：

```text
name,token_position,layer_id,shape,stride,dtype,layout,checksum,max_abs,values
```

这条 trace 固定 prompt fixture `tok1 tok2 tok3`，tokenizer 输出完整 `tokens=[0,1,2,3,4]`，其中 `0/4` 是 BOS/EOS；decoder trace 会剥离 BOS/EOS 后进入 tiny decoder。这样报告能同时固定 tokenizer 合同和 decoder 输入合同，避免把二者混成一个不可解释的 token 数组。随后 trace 把 embedding、RMSNorm、Q/K/V、RoPE、KV cache slot、attention、SwiGLU、layer output、final norm、logits、argmax sampler 和 generated token 串成可回归检查的硬链路。`
````
````

````
lcqi_tests` 会检查 tokenizer contract hash、关键 checkpoint 的 shape、stride、dtype、layout 和 logits golden，CTest 也会直接运行 `lcqi_trace`。

当前 tokenizer fixture 的关键输出：

```text
tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4
tokenizer_contract,0,-1,scalar,scalar,u64,fnv1a,13452902845388333734,0,tiny-chat-template-v1
```

GPT-2 自研 reference smoke：

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_gpt2
make cpu3-gpt2-smoke
make cpu3-gpt2-benchmark-compare
make cpu3-gpt2-hotspot-profile
python3 books/cpu-volume-3/tools/run_gpt2_optimization_ab.py --rounds 3
python3 books/cpu-volume-3/tools/run_gpt2_hotspot_profile.py --token-counts 4,16 --rounds 3
```

第一条命令不需要下载模型，直接运行仓库内 tiny GPT-2 fixture，输出固定 prompt ids、logits、predicted token 和 greedy generated ids。`lcqi_tests` 会覆盖 tiny GPT-2 forward/generation、byte-level BPE 编码解码、F32/F16/BF16 safetensors 读取、HuggingFace 风格 tiny GPT-2 checkpoint 加载和坏 shape 拒绝。

第二条命令显式依赖网络，只下载 `openai-community/gpt2` 的 `config.json`、`vocab.json`、`merges.txt`、`model.safetensors` 到 `~/.cache/lcqi-gpt2-smoke/openai-community--gpt2`，模型文件不进入 Git。脚本会校验四个文件的 bytes 和 SHA256，构建 `lcqi_gpt2`，运行：

```text
Hello, my name is
```

当前验证报告写到：

```text
books/cpu-volume-3/results/lcqi-gpt2-smoke.txt
```

最近一次本机报告的关键行是：

```text
prompt_ids 15496 11 616 1438 318
generated_ids 15496 11 616 1438 318 1757
generated_text Hello, my name is John
validation=PASS
```
````

````markdown

这说明 LCQI 自研 C++ reference path 已经能跑标准 GPT-2 safetensors 的 tokenizer、权重导入、forward 和 greedy generation。默认真实模型路径现在使用 `cached_kv`，也可以用 `--engine full` 复现旧的 full-prefix 重算路径：

```bash
books/cpu-volume-3/build/lcqi-release/labs/linux_cpu_inference/lcqi_gpt2 \
  --benchmark --engine cached --threads 0 \
  ~/.cache/lcqi-gpt2-smoke/openai-community--gpt2 \
  "Hello, my name is" 4
```

`--threads 0` 表示自动选择 cached decoder worker 数；`--threads 1` 强制串行；`--threads N` 固定 worker 数。当前 auto 上限是 16，避免在短 decode 上把线程调度开销放大到超过收益。

`make cpu3-gpt2-benchmark-compare` 会生成：

```text
books/cpu-volume-3/results/lcqi-gpt2-benchmark-compare.txt
```

`run_gpt2_optimization_ab.py` 会生成：

```text
books/cpu-volume-3/results/lcqi-gpt2-optimization-ab.txt
```

这个 A/B 报告专门回答“当前 LCQI 代码相对指定 baseline 改快了吗”。脚本默认 baseline 是 `4febf3f`，会临时创建 baseline worktree，分别构建 baseline 和当前工作区的 Release `lcqi_gpt2`，再多轮运行同一条 prompt。上一轮工作区复用优化的 2 轮 smoke 摘要保存在 `books/cpu-volume-3/reports/lcqi-gpt2-optimization-ab-summary.md`；本轮线程优化的正式 raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-threaded-manual-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-threaded-optimization-summary.md`。以 `73f40c7` 作为 baseline，在 `caw` 上 5 轮中位数显示：4 token cached-KV 生成阶段从 `395.397 ms` 降到 `76.4141 ms`，16 token 从 `989.060 ms` 降到 `185.479 ms`，生成吞吐分别提升到 `52.3464 token/s` 和 `86.2633 token/s`。随后的 F32 dot kernel 优化以 `cb4d89f` 为 baseline，raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-dot-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-dot-optimization-summary.md`。在同一台 `caw`、同一 GPT-2 F32 模型、同样 `--threads 0` 自动 16 workers 下，4 token 生成阶段中位数从 `74.4247 ms` 到 `62.3628 ms`，16 token 从 `181.257 ms` 到 `157.230 ms`；decode token/s 分别提升到 `130.073` 和 `127.405`。再往后，row-range kernel 和去 `std::function` 调度开销的 raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-rowrange-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-rowrange-optimization-summary.md`。这一轮相对 `ab72ccb` 的端到端提升很小，16 token 生成中位数约 `153.354 ms` 到 `152.574 ms`；但直接比较 row-range 与 4-row AVX2 row-range 时，4 token 从 `62.0243 ms` 到 `60.3965 ms`，16 token 从 `154.087 ms` 到 `151.222 ms`。本轮 prefill-skip 优化的 raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-prefill-skip-ab-caw.txt`，汇总保存在
````

````
`books/cpu-volume-3/reports/lcqi-gpt2-prefill-skip-optimization-summary.md`。它跳过 prompt 前缀 token 上不会被使用的 `lm_head` 预测：4 token 生成中位数从 `60.5154 ms` 到 `52.1949 ms`，16 token 从 `153.574 ms` 到 `147.424 ms`，生成文本和 token id 全部一致。端到端 GPT-2 数字受调度、温度和共享机器状态影响很大，因此正式结论必须看多轮中位数、min/max 和生成文本一致性，不能只挑一次最快结果。

`run_gpt2_hotspot_profile.py` 专门回答“现在热点到底在哪”。它通过 `--profile-hotspots` 打开 cached decode 内部计时，字段包括 `hotspot_lm_head_pct`、`hotspot_mlp_fc_pct`、`hotspot_mlp_projection_pct`、`hotspot_qkv_projection_pct` 和 `hotspot_attention_pct`。`caw` 上的 3 轮 Release profile 摘要保存在 `books/cpu-volume-3/reports/lcqi-gpt2-hotspot-profile-summary.md`，原始报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-hotspot-profile-caw.txt`。关键结论是：4 token 与 16 token 两个口径下，`lm_head` 中位数约 `30.2%`，MLP 两个线性投影合计约 `46%`，QKV 投影约 `17%`，cached attention 本体只有 `0.06%` 到 `0.11%`。所以当前短上下文 GPT-2 的慢点不是 KV attention，而是标量 F32 matvec 和 `50257 x 768` vocab 扫描。

当前优化点集中在 cached-KV generation：`Gpt2CachedGreedyDecoder` 把模型级 shape validation、KV cache、临时 workspace 和 worker pool 的生命周期提升到整段生成；`Gpt2ForwardWorkspace` 复用 hidden、normed、Q/K/V、packed QKV、attention、MLP、scores 和 logits 缓冲区；生成路径只需要 greedy token 时走 logits-free argmax，避免把完整 logits vector 作为 API 结果反复分配；prompt prefill 前缀通过 `advance_without_prediction()` 只更新 KV cache，不跑 final norm 和 `lm_head`，因为这些预测结果不会被用作输出；QKV、attention output projection、MLP 两个 projection 和 tied `lm_head` 都在 cached session 内按输出行并行；每个线程 chunk 再通过 `linear_f32_rows_unchecked` 或 `max_dot_f32_rows_unchecked` 进入 F32 row-range kernel，x86-64 可用时使用 AVX2/FMA，其他平台走标量 fallback。KV cache 的 checked public API 仍保留，内部 cached attention 在一次边界校验后使用私有 unchecked span 扫描热路径。它不改变 GPT-2 数学语义，只改变会话生命周期、任务分片、无用工作剔除和热循环调度。

`make cpu3-gpt2-benchmark-compare` 仍用于和外部成熟引擎放在同一报告里看工程差距。同机 llama.cpp/SmolLM2 Q4_K_M 是成熟引擎参照，不是同模型质量排名：LCQI 跑的是 GPT-2 F32 reference path，llama.cpp 跑的是 GGUF 量化 small instruct 模型。它揭示的是工程差距：LCQI 已经消除了“每步重算前缀”的主要算法错误，补上 cached F32 row parallelism，并把热路径接到 AVX2/FMA row-range kernel；但还缺 packed/quantized weight、GGUF 量化 block、mmap/lazy load、采样器、分页 KV cache、NUMA/亲和和成熟调度。

它仍然不是生产级推理引擎：当前 GPT-2 路径还没有把 logits 和 Transformers/PyTorch 逐数值对齐成外部 golden 报告，也没有 GGUF 自研导入、分页 KV cache、量化权重执行和高性能 SIMD/packed kernel。

7B ledger:

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_ledger \
  > books/cpu-volume-3/results/lcqi-sevenb-ledger-2026-06-24.csv
```

````

````markdown
输出 CSV 字段：

```text
section,item,value,unit,notes
```

这份账本固定 `L=32,H=4096,n_q=32,n_kv=8,head_dim=128,context=4096`，报告 decode FLOPs、KV 容量、fp16/int8/int4 每 token 字节和不同有效带宽下的 tokens/s 上限。CTest 会检查 `int4_at_100_gbps=23.602 tokens/s` 这一行，防止账本公式漂移。

serving SLO probe：

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_serving_probe \
  > books/cpu-volume-3/results/lcqi-serving-slo-probe-2026-06-24.csv
```

输出 CSV 字段：

```text
row_type,request_id,accepted,rejection_reason,prompt_tokens,reserved_tokens,kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,decode_step_p95_ms,tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,metric_name,metric_value
```

probe 固定四个请求：短请求、RAG 请求、取消请求和超长请求。它演示 admission 先按 `prompt_tokens + max_new_tokens` 预留 KV，取消请求释放预算，超长请求返回 `memory_budget_exceeded`，并输出 `ttft_p95_ms`、`queue_wait_p95_ms`、`rejection_rate` 等服务化指标。

真实 small 开源模型 smoke：

```bash
make cpu3-smollm2-smoke
```

这条命令显式依赖网络和外部构建，因此不放进默认 CTest。脚本会把 llama.cpp 和模型文件缓存到 `~/.cache/lcqi-smollm2-small-smoke`，模型不进入 Git。固定模型是 `HuggingFaceTB/SmolLM2-135M-Instruct` 的 GGUF 量化版本，来源仓库为 `bartowski/SmolLM2-135M-Instruct-GGUF`，文件为 `SmolLM2-135M-Instruct-Q4_K_M.gguf`。脚本会校验文件大小 `105454432` 字节和 SHA256：

```text
2e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d
```

脚本构建固定 commit 的 `llama-simple-chat`，用 `-ngl 0` 强制 CPU 路径，输入短 prompt，检查 assistant response 非空，并生成：

````

````markdown
```text
books/cpu-volume-3/results/lcqi-smollm2-small-model-smoke.txt
```

这份报告只能证明 llama.cpp 成熟外部引擎上的真实 GGUF small 模型已经完成加载、tokenizer/chat template、prefill、decode 和文本输出。它不能证明 LCQI 自研 C++ 引擎已经达到 llama.cpp 的性能，也不能替代 tiny fixture 的逐层 golden trace。

LCQI 自研 SmolLM2/GGUF reference decode:

```bash
books/cpu-volume-3/build/lcqi-release/labs/linux_cpu_inference/lcqi_llama_gguf \
  ~/.cache/lcqi-smollm2-small-smoke/models/SmolLM2-135M-Instruct-Q4_K_M.gguf \
  --prompt "Hello" --max-new 2 --benchmark --decode-text
```

caw 上的最新报告写到:

```text
books/cpu-volume-3/results/lcqi-smollm2-gguf-reference-decode-caw.txt
```

关键行是:

```text
weight_execution gguf_mixed_q4_k_direct
generated_text ... <|im_start|>assistant
Hello!
benchmark_prompt_tokens 31
benchmark_generated_tokens 2
benchmark_decode_tokens_per_second 64.8586
benchmark_quantized_weight_bytes 103668480
benchmark_f32_weight_bytes 481436928
benchmark_direct_quantized_weight_bytes 7962624
benchmark_fallback_dequantized_weight_bytes 481436928
benchmark_q4_k_direct_tensors 16
benchmark_ggml_direct_tensors 0
benchmark_f32_fallback_tensors 256
benchmark_hotspot_w_down_ms 63.5371
benchmark_hotspot_q4_k_direct_calls 512
benchmark_hotspot_ggml_direct_calls 0
benchmark_hotspot_f32_fallback_calls 6208
```

这条路径证明 LCQI 自研代码已经能读取同一个 SmolLM2 GGUF、解析 tokenizer arrays、处理混合量化权重并端到端生成文本。默认执行路径不再只是“加载时全部解成 f32”: shape 兼容的 `Q4_K` linear tensor 会保留 GGUF block bytes，并在真实 decoder 里通过 `matvec_q4_k_q8` 直接执行; `Q5_0/Q6_K/Q8_0/F32` tensor 仍然 materialize 成 f32 fallback。当前只有约 `7.68%` 的加载权重字节进入默认 direct Q4_K 路径，prefill 仍逐 token 执行; 与 llama.cpp 的差距主要来自更
````

````markdown
完整的低比特权重覆盖、prompt batching、packed kernel、线程调度、prefetch、KV cache 和整图执行。

实验路径 `LCQI_LLAMA_GGML_DIRECT=1` 会尝试让 shape 兼容的 `Q5_0/Q6_K/Q8_0` linear tensor 也保留 GGUF block bytes，并通过 `matvec_ggml_quantized_q8_0` 执行。caw 同输入报告显示它把 direct 量化字节覆盖率从 `0.076809` 提高到 `0.708479`，但 prefill 中位数从默认 `370.847 ms` 退化到 `1211.710 ms`，decode step 从 `15.4182 ms` 退化到 `42.4420 ms`。原因是这些 GGML direct kernel 仍按 block/row 标量组织，热点 `benchmark_hotspot_ggml_direct_ms` 达到 `1207.13 ms`，慢于当前 f32 AVX2 fallback。因此它保留为实验路径，不默认启用。

LCQI 与 llama.cpp 同输入对比：

```bash
python3 books/cpu-volume-3/tools/run_smollm2_same_input_compare.py \
  --cache-dir ~/.cache/lcqi-smollm2-same-input-compare \
  --model-path ~/.cache/lcqi-smollm2-small-smoke/models/SmolLM2-135M-Instruct-Q4_K_M.gguf \
  --build-dir books/cpu-volume-3/build/lcqi-release \
  --rounds 5 --max-new 2
```

这个脚本会构建固定 commit 的 llama.cpp，运行 `llama-tokenize`、`llama-simple` 和 `llama-bench`，并确认 LCQI 和 llama.cpp 看到的 prompt token ids 完全一致。caw 上的同输入报告保存在：

```text
books/cpu-volume-3/reports/lcqi-smollm2-ggml-direct-compare-caw.txt
```

关键结论：同一个模型、同一个展开 chat prompt、同样 31 个 token id、同样 `max-new=2` 下，当前默认 LCQI Q4_K direct prefill 中位数 `370.847 ms`，llama.cpp `22.840 ms`，LCQI 慢 `16.24x`；LCQI decode step 中位数 `15.4182 ms`，llama.cpp `2.880 ms`，LCQI 慢 `5.35x`。同一二进制里关闭 `LCQI_LLAMA_Q4_DIRECT=0` 后，LCQI prefill 为 `393.992 ms`，decode step 为 `15.8782 ms`，`w_down` 热点为 `91.6812 ms`；默认 Q4_K direct 后分别为 `370.847 ms`、`15.4182 ms`、`63.5371 ms`。所以这轮默认优化的保守 A/B 证据是：prefill `1.062x`、decode step `1.030x`、`w_down` `1.443x`，并减少 `56,623,104` 字节 f32 materialized 权重。实验 GGML direct 证明“覆盖更多量化格式”本身不是优化结论：没有 packed multi-row/SIMD/threaded kernel 时，端到端会倒退。
````

tiny MLP 模型文件是最短闭环的资产。它决定输入维度、hidden 维度、输出维度、权重量化值、scale 和 bias；测试中的 logits golden 依赖这份文件。

Listing 1.9 models/tiny_mlp_i8.txt

```
LCQI_MODEL_V1
input_size 4
hidden_size 3
output_size 2
hidden_scale 0.1
```

```
hidden_weights
5 -3 2 1
-4 6 1 -2
3 2 -5 4
hidden_bias
0.1 -0.2 0.0
output_scale 0.05
output_weights
4 -6 2
-3 5 8
output_bias
-0.5 0.4
```

tiny tokenizer fixture 固定 BOS、EOS、UNK、词表和模板 hash。tokenizer 合同如果漂移，后面的 decoder trace 和测试都会被污染。

Listing 1.10 models/tiny_tokenizer.txt

```
LCQI_TOKENIZER_V1
bos_id 0
eos_id 4
unk_id -1
chat_template_hash tiny-chat-template-v1
vocab_size 5
vocab
0 <bos>
1 tok1
2 tok2
3 tok3
4 <eos>
```

为什么要先讲调用链

实践与代码卷如果只把 `*.cpp` 和 `*.hpp` 全部贴出来，读者仍然不知道从哪里下手。LCQI 的代码同时包含 tiny MLP、int8 kernel、tokenizer、safetensors、reference decoder、GPT-2、benchmark、trace、ledger、serving probe 和外部模型脚本；这些文件不是平铺关系，而是几条调用链。先讲调用链，再给完整源码，读者才能把每个文件放到正确位置。

核心思想

> 读 LCQI 的顺序不是“哪个文件长先读哪个”，而是“入口、合同、资产、执行、证据”。入口说明目标怎样构建，合同说明类型怎样连接，资产说明模型从哪里来，执行说明 token 或张量怎样流动，证据说明测试和报告守住了哪些边界。

一、全工程入口：哪些文件决定能不能复现

文件	负责什么	读源码时要确认什么
第三册 README	第三册 AI 计算工程路线、当前 LCQI 能力、GPT-2 smoke 和结果报告位置。	能力声明必须能在 `labs/linux_cpu_inference`、`tools` 和 `results` 中找到证据。

第三册CMake	第三册根 CMake，负责把 `linux_cpu_inference` 接入。	根 CMake 应保持薄层；真正 target 定义必须在 lab 内部，避免入口文件承担实现细节。
本卷CMake	本卷复用同一份 LCQI lab。	本卷不复制源码，只把同一工程接入自己的测试构建，防止两份代码漂移。
本卷Makefile	本卷的书籍构建入口。	它构建 PDF/EPUB，也提供 `test` 目标去构建第三册 LCQI；实践与代码卷不能只排版不验证。
LCQICMake	`lcqi` 库、CLI、benchmark、trace、GPT-2、ledger、serving probe、CTest 目标。	读这里能看到哪些文件进入库，哪些是可执行程序，哪些依赖可选外部库。
LCQIREADME	本地构建、tiny 推理、benchmark、trace、GPT-2、SmolLM2 smoke 的命令。	先按 README 跑命令，再读源码；否则不知道某个函数是否真的能从命令行触达。

上表里的短标签对应下面这些真实路径：

- `books/cpu-volume-3/README.md`
- `books/cpu-volume-3/CMakeLists.txt`
- `books/cpu-volume-3-practice/CMakeLists.txt`
- `books/cpu-volume-3-practice/Makefile`
- `books/cpu-volume-3/labs/linux_cpu_inference/CMakeLists.txt`
- `books/cpu-volume-3/labs/linux_cpu_inference/README.md`

这些入口文件回答“代码在哪里”和“怎样运行”。读者进入本卷代码走读章时，先不要直接跳到 `gpt2_reference.cpp`。先确认 `lcqi` 静态库包含 `model.cpp`、`inference.cpp`、`int8_kernels.cpp`、`f32_kernels.cpp`、`gguf.cpp`、`ggml_tensors.cpp`、`q4_k.cpp`、`llama_gguf.cpp`、`reference_decoder.cpp`、`tokenizer.cpp`、`safetensors.cpp` 和 `gpt2_reference.cpp`；再确认 `lcqi_cli`、`lcqi_bench`、`lcqi_gguf_q4_bench`、`lcqi_llama_gguf`、`lcqi_trace`、`lcqi_gpt2`、`lcqi_ledger`、`lcqi_serving_probe` 和 `lcqi_tests` 分别从哪条路径进入。

二、Tiny MLP 调用链：最短闭环怎样跑通

Tiny MLP 是最小推理闭环，用来证明“模型资产、shape 检查、int8 linear、激活函数、logits、argmax、CLI 输出”可以完整连起来。它不是大模型能力本身，但它是后面所有复杂路径的可手算底座。

文件	关键类型或函数	这段代码的意思
`models/tiny_mlp_i8.txt`	输入维度、hidden 维度、输出维度、int8 权重、scale、bias。	这是模型资产。测试中的 golden logits 来自这里，资产改动必须带着测试改动。
`include/lcqi/model.hpp`	`QuantizedLinearLayer`、`TinyMlpModel`、`load_model`。	定义 tiny MLP 如何在内存中表示；只负责模型结构和加载入口，不做推理。
`src/model.cpp`	`read_i8_vector`、`validate_layer`、`load_model`。	解析文本模型，拒绝坏 key、坏维度、权重数量不匹配和非法 scale，把错误挡在执行前。
`include/lcqi/inference.hpp`	`InferenceResult`、`linear_i8`、`relu_inplace`、`run_inference`。	定义推理输出要包含 hidden、logits 和 predicted class，方便测试定位错误层。

`src/inference.cpp`	输入 shape 检查、两层 int8 linear、ReLU、argmax。	最短执行路径：模型加输入变成 logits。这里的代码应保持清楚，不能混入 CLI 或文件解析。
`src/main.cpp`	解析命令行输入并打印结果。	CLI 是薄层；它只接模型路径和输入，不复制 `run_inference` 逻辑。

这条链路按下面顺序读：`main()` 取得模型路径和输入向量，调用 `load_model()` 把 `tiny_mlp_i8.txt` 变成 `TinyMlpModel`，再调用 `run_inference()`。在 `run_inference()` 内部，第一层 `linear_i8()` 把 float 输入和 int8 权重相乘，乘以 scale 后加 bias；`relu_inplace()` 把负 hidden 截断；第二层 `linear_i8()` 得到 logits；最后 `argmax()` 给出 predicted class。

读这个路径时要抓住边界条件。输入长度不等于 `input_size` 要立即拒绝；权重数量不是 `rows*columns` 要在加载时拒绝；logits 为空不能取 argmax。这些错误如果拖到 SIMD kernel 或 benchmark 里才暴露，定位成本会高很多。

三、Int8 Kernel 调用链：reference、packed、AVX2 和 benchmark 怎样对照

文件	关键代码	这段代码的意思
`include/lcqi/int8_kernels.hpp`	`PackedLinearI8`、`KernelBenchmarkCase`、`benchmark_linear_i8_case`。	定义 kernel 对标口径：shape、repeat、backend、平均耗时、checksum、max diff。
`src/int8_kernels.cpp`	`pack_linear_i8`、`linear_i8_packed`、`linear_i8_packed_with_workspace`、deterministic fixture。	scalar 和 packed 路径是正确性基线，workspace 路径减少重复分配，fixture 让 benchmark 和测试可复现。
`src/int8_kernels_avx2.cpp`	`linear_i8_packed_avx2_available`、`linear_i8_packed_avx2`。	平台优化路径。必须先判断机器能力，再执行 AVX2/FMA；不可用时报告 unavailable，而不是假装测过。
`include/lcqi/f32_kernels.hpp` 与 `src/f32_kernels.cpp`	`dot_f32`、`linear_f32_rows_unchecked`、`max_dot_f32_rows_unchecked`。	GPT-2 F32 hot path 的 kernel 边界。单行 dot 是基础，row-range linear 一次写一段输出行，row-range max 给 logits-free greedy `lm_head` 用。
`src/f32_kernels_avx2.cpp`	`dot_f32_avx2_available`、`dot_f32_avx2_unchecked`、4-row row-range kernel。	x86 平台优化路径。它在一段输出行内部复用 input load，同时不改变 GPT-2 层结构、线程分片或 greedy tie-breaking。
`include/lcqi/gguf.hpp` 与 `src/gguf.cpp`	`GgufManifest`、`GgufTensorInfo`、`load_gguf_manifest`、`read_gguf_tensor_bytes`。	读取 GGUF header、metadata、tensor 表、alignment 和真实 tensor payload；它只负责格式边界，不做推理数学。
`include/lcqi/ggml_tensors.hpp` 与 `src/ggml_tensors.cpp`	`dequantize_ggml_tensor`、`ggml_f16_to_f32`。	把 GGUF 中的 `F32`、`Q8_0`、`Q5_0`、`Q6_K` 和 `Q4_K` 张量按 GGML block 布局解释出来。它是格式 oracle，不决定某个 tensor 最终走 f32 fallback 还是量化直算。

include/lcqi/q4_k.hpp 与 src/q4_k.cpp	dequantize_q4_k、quantize_q8_k_input、matvec_q4_k_q8。	实现 llama.cpp 同格式的 Q4_K block 解码和 Q4_KxQ8_K 单算子路径；单测用手写 block 固定 nibble 顺序、scale/min 解包和误差边界。
include/lcqi/ggml_matvec.hpp、src/ggml_matvec.cpp 与 src/ggml_matvec_avx2.cpp	matvec_ggml_quantized_f32、matvec_ggml_quantized_q8_0、quantize_q8_0_input。	Q5_0/Q6_K/Q8_0 direct matvec 的实验路径。它能显著提高量化字节覆盖率，但 caw 证明当前 block/row 组织端到端更慢，所以只用 LCQI_LLAMA_GGML_DIRECT=1 显式打开。
include/lcqi/llama_gguf.hpp 与 src/llama_gguf.cpp	load_llama_gguf_reference_model、load_gpt2_tokenizer_from_gguf、llama_gguf_generate_greedy。	自研 SmolLM2/GGUF decoder path。它从同一个 GGUF 读取 tokenizer tokens/merges、混合量化权重、RMSNorm、normal RoPE、GQA KV cache、SwiGLU 和 tied lm head；默认执行 Q4_K direct 加 f32 fallback，实验开关可接入 Q5_0/Q6_K/Q8_0 direct。
src/llama_cli.cpp	lcqi_llama_gguf。	命令行加载真实 SmolLM2 GGUF，支持 token id prompt 和文本 prompt，输出生成 token、文本、load/prefill/decode 时间、KV 字节、量化权重字节和解码后 f32 字节。
src/q4_k_avx2.cpp	q4_k_q8_avx2_available、matvec_q4_k_q8_avx2_unchecked。	x86 AVX2 路径。它用 maddubs 把 32 个 unsigned nibble 与 32 个 signed int8 激活先归约成整数和，再套用 Q4_K 的 scale/min；报告必须同时给 scalar 与 auto 后端，防止把机器波动写成优化收益。
src/gguf_q4_bench.cpp	lcqi_gguf_q4_bench。	从真实 SmolLM2 GGUF 选择 Q4_K 矩阵，比较 f32 解码基线、Q4_Kxf32 和 Q4_KxQ8_K，输出速度、误差、checksum 和字节账本。
src/bench.cpp	shape 列表、repeat 参数、backend 输出。	benchmark 程序只负责测 kernel 入口。性能结论必须和 max_abs_diff、CPU、编译器、构建类型一起报告。

kernel 文件要按“可信基线到优化路径”的顺序读。先看 deterministic layer 和 input 怎么生成，保证每次运行输入相同；再看 `linear_i8_packed()` 怎样把 int8 权重、scale、bias 和 float 输入算成输出；再看 workspace 路径怎样复用临时缓冲；最后看 AVX2 路径怎样在支持平台上替换内层循环。Q4_K 路径也遵守同一顺序：先读 scalar 解码，再读 AVX2 `maddubs` 子块归约，最后看 benchmark 怎样强制 scalar 和 auto 后端并列输出。测试用 packed 与 scalar 的 `max_abs_diff` 对照，benchmark 再比较耗时。

四、Tokenizer、Safetensors 与 Reference Decoder：真实模型前的三道门

文件	关键类型或函数	这段代码的意思
models/tiny_tokenizer.txt	BOS、EOS、UNK、词表、模板 hash。	固定 tokenizer 合同，避免 prompt 到 token id 的规则漂移。

`include/lcqi/tokenizer.hpp` 与 `src/tokenizer.cpp`	`TokenizerModel`、`load_tokenizer`、`encode_prompt`、`tokenizer_contract_hash`。	把 prompt 切成稳定 token id；未知 token 是否允许，必须由 UNK 合同决定。
`include/lcqi/safetensors.hpp` 与 `src/safetensors.cpp`	`SafeTensorManifest`、`SafeTensorFile`、manifest parser、dtype 转换。	解析模型字节之前先验证 header、dtype、shape、offset 和 payload 边界，防止坏资产进入数学层。
`include/lcqi/reference_decoder.hpp` 与 `src/reference_decoder.cpp`	embedding、RMSNorm、RoPE、KV cache、attention、SwiGLU、trace checkpoint。	Tiny decoder-only reference path。它用 checkpoint 解释每一步张量状态，不只返回最终 token。
`src/trace.cpp`	trace CSV 输出。	把 reference decoder 的 checkpoint 打印成可复查表格，定位错误发生在哪个阶段。

这三道门分别解决三个问题。Tokenizer 解决“人写的 prompt 如何变成 token id”；Safetensors 解决“磁盘里的模型字节如何变成带 shape 的张量”；Reference Decoder 解决“token 和权重如何经过 decoder-only 层生成 logits”。任何一层含糊，后面的 GPT-2 路径都会变成看似能跑、实则无法定位的黑盒。

五、GPT-2 调用链：从 HuggingFace 资产到生成 token

GPT-2 路径是本卷代码走读里最长的一条链，因为它要处理真实模型格式。读 `gpt2_reference.cpp` 时按模块读，不要从第一行一路滚到最后。

文件	关键代码	这段代码的意思
`include/lcqi/gpt2_reference.hpp`	配置、模型、KV cache、cached decoder、tokenizer 与 forward/generate API。	公开 GPT-2 reference path 的合同：配置、权重、KV cache、tokenizer、full-prefix、cached generation，以及 prompt prefill 中只推进 cache、不做无用预测的入口。
`src/gpt2_reference.cpp`	JSON parser、byte-level BPE、safetensors loader、LayerNorm、QKV、causal attention、GELU、lm head、KV cache。	正确性优先实现。它把 HuggingFace GPT-2 权重名、Conv1D 转置、tokenizer 和 forward 规则全部显式写出来。
`src/gpt2_cli.cpp`	tiny mode、真实模型目录模式、generation、benchmark 输出。	命令行入口。无参数跑 tiny fixture；传模型目录时加载真实 GPT-2 资产；benchmark 对比 full-prefix 和 cached-KV。
`tools/run_gpt2_smoke.py`	下载标准 GPT-2 资产，调用 `lcqi_gpt2`。	证明自研 safetensors、BPE、forward、KV cache 和 greedy generation 能跑真实开源模型。
`tools/run_gpt2_benchmark_compare.py`	运行 LCQI full-prefix、cached-KV，并可附带成熟引擎参考。	用同一模型口径比较算法差距和 kernel 差距，不把不同模型的速度混成一个结论。

GPT-2 代码按五段读。第一段是配置和权重加载：`load_gpt2_config()` 读 `config.json`，`load_gpt2_reference_model()` 从 safetensors 中取 embedding、每层 LayerNorm、QKV、MLP 和 final norm。HuggingFace GPT-2 的 Conv1D 权重形状和普通线性层方向不同，所以代码显式调用转置函数，不能跳过。

第二段是 tokenizer：`load_gpt2_tokenizer()` 读 `vocab.json` 和 `merges.txt`，`gpt2_encode()` 执行 byte-level BPE，`gpt2_decode()` 把 id 还原为文本。这里的重点是字节到 Unicode 映射、BPE merge rank 和 unknown token 边界。

第三段是 full-prefix forward：`run_gpt2_forward()` 每生成一个新 token 都用完整 token 前缀重新跑所有层。它慢，但容易验证。流程是 token embedding 加 position embedding，逐层执行 LayerNorm、packed QKV、causal attention、残差、MLP、残差，最后 final LayerNorm 和 tied lm head 得到 logits。

第四段是 cached-KV forward：`Gpt2KvCache` 保存每层每个 position 的 key/value，`run_gpt2_forward_cached()` 每次只处理最新 token，并让 attention 读取过去 cache。这里要特别看 `validate_written()`：读取未写入 cache slot 必须报错，否则生成错误会很难定位。

第五段是 greedy generation：`gpt2_generate_greedy()` 用 full-prefix 路径，`gpt2_generate_greedy_cached()` 用 KV cache 路径。两个函数都要检查 prompt 非空、`max_new_tokens` 非负、不会超过 `max_positions`，并在生成 EOS 时停止。

六、报告、服务探针、外部对标和测试

文件	关键代码	这段代码的意思
`src/ledger.cpp`	7B 参数量、KV cache、bandwidth-only 上限。	输出的是工程账本，不是实测速度；它帮助读者估算权重和 KV 内存压力。
`src/serving_probe.cpp`	短请求、RAG 请求、取消请求、超长请求，TTFT/TPOT 字段。	把服务层边界变成字段：哪些请求接受，哪些拒绝，取消后资源是否回收。
`src/onednn_bench.cpp`	oneDNN matmul benchmark。	可选外部库对标入口。没有 oneDNN 环境时不能声称已验证。
`tools/run_smollm2_small_smoke.py`	下载 SmolLM2-135M-Instruct GGUF，构建固定 commit 的 llama.cpp smoke。	这是成熟外部引擎上的真实 small 模型 smoke，用来建立外部参照，不代表 LCQI 已具备同等能力。
`tools/run_lcqi_benchmark.sh`	收集 LCQI 本地 benchmark。	在固定机器上重复运行同一组 shape，形成可复查 CSV 或文本报告。
`tests/lcqi_tests.cpp`	tiny MLP、tokenizer、safetensors、GPT-2 tiny、HF-style checkpoint、bad shape、decoder、trace、kernel。	这是本卷最重要的代码证据。新增功能如果没有进入这里，就不能写成已经守住的能力。

读测试文件时，不要只看最后是否返回 0。它里面的 fixture 本身就是源码讲解的一部分：测试会手写 safetensors header，构造 F16/BF16/F32 payload，生成 tiny GPT-2 权重，验证 bad shape 会被拒绝，比较 packed int8 kernel 和 scalar 路径。这些测试告诉读者“代码真正承诺了什么”。如果一个能力只出现在 README 或 CLI 输出里，却没有进入测试或 smoke 报告，就只能算实验入口，不能算稳定能力。

源码阅读任务

> 读完本章后，再去读相邻主题中的完整源码。每看到一个文件，都把它归到五类之一：入口、合同、资产解析、执行路径、证据。归不进去的文件，就说明工程边界需要重新解释或重构。

第 2 章实践：张量布局、地址流与第一个 MatVec

本章对应原书中的第三册“张量布局、地址流与第一个矩阵向量内核”。不要把它当作概念复述，本章要把 TensorView、dtype、shape、stride、row-major、地址公式、bias、ReLU、checksum 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `linear_i8`，核心命令或测试入口是 `lcqi_tests`。

Listing 2.1 第 2 章实践：张量布局、地址流与第一个 MatVec 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / linear_i8
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：手算一个 2 行 4 列权重矩阵的地址偏移。读者要先手写预期结果，再运行项目观察输出。动作是：把 `weight[out,k]` 展开成 `base+(out*K+k)*sizeof(i8)`，再对应到 `linear_i8` 的循环。如果输出里没有 `logit` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：张量不是裸数组；每个 kernel 入口都要能解释 dtype、shape、stride、scale 和输出缓冲这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一页地址流账本：输入向量、hidden 权重、输出 logits 各自的 shape、连续性、读写次数和错误边界。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 3 章实践：GEMM、packing 与 SIMD 证据

本章对应原书中的第三册“从矩阵向量乘走向 GEMM、packing 与 SIMD”。不要把它当作概念复述，本章要把 packed weight、scalar baseline、AVX2 backend、output block、shape sweep、反汇编和降级放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `output_block`，核心命令或测试入口是 `lcqi_bench`。

Listing 3.1 第 3 章实践：GEMM、packing 与 SIMD 证据的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_bench / output_block
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：同一组输入同时跑 `scalar`、`packed_scalar` 和 `avx2`。读者要先手写预期结果，再运行项目观察输出。动作是：比较 CSV 中 `backend`、`available`、`average_us`、`max_abs_diff` 和 `checksum`。如果输出里没有 `max_abs_diff` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：SIMD 路径只有在结果和 baseline 对齐后才有资格谈速度；`available=0` 也是证据，不是失败这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一个 shape sweep 表，至少解释一个 SIMD 有收益、一个收益不明显、一个当前环境不可验证的场景。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 4 章实践：int8 量化、累加器与重新量化

本章对应原书中的第三册“量化系统总览”和“int8 量化、累加器与重新量化”。不要把它当作概念复述，本章要把 int8 weight、float activation、scale、accumulator、requantize、oracle tolerance 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `TinyMlpModel`，核心命令或测试入口是 `lcqi_tests`。

Listing 4.1 第 4 章实践：int8 量化、累加器与重新量化的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / TinyMlpModel
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：构造一行 int8 权重 `[2,-1,3,0]` 和输入 `[1,2,-1,0.5]`。读者要先手写预期结果，再运行项目观察输出。动作是：先手算 int32/float accumulator，再和 `linear_i8` 输出比较。如果输出里没有 `scale` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：量化不是把 float 强转成 int8；必须记录 scale 作用位置、累加宽度、误差阈值和 golden 输出这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交量化合同卡：每个 tensor 的 dtype、scale 粒度、accumulator 类型、误差门限和拒绝条件。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

代码走读：公共合同、Tiny MLP 与 Int8 Kernel

为什么先读头文件

头文件是系统对外承诺的最小合同。实现可以重写，kernel 可以替换，benchmark 可以补充平台路径，但头文件里暴露的数据结构、函数入口和返回字段决定了测试、CLI、工具脚本和章节讲解怎样调用这套工程。

核心思想

> 读 LCQI 时不要从最长的 `gpt2_reference.cpp` 开始。先读公共头文件，明确“模型如何表示”“推理返回什么”“trace 记录什么”“KV cache 怎样寻址”“tokenizer 的坏输入怎样处理”，再进入实现。这样读到长循环和 shape 检查时，知道它们在维护哪份合同。

Tiny MLP 与推理入口

`model.hpp` 定义 tiny MLP 的模型对象、权重、bias、scale 和加载入口。它的责任是把文本模型资产解释成 C++ 对象，不负责执行推理。

Listing 4.2 include/lcqi/model.hpp

```cpp
#pragma once

#include <cstdint>
#include <filesystem>
#include <string>
#include <vector>

namespace lcqi {

struct QuantizedLinearLayer {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    float scale = 1.0F;
    std::vector<std::int8_t> weights;
    std::vector<float> bias;
};

struct TinyMlpModel {
    std::int32_t input_size = 0;
    std::int32_t hidden_size = 0;
    std::int32_t output_size = 0;
    QuantizedLinearLayer hidden;
    QuantizedLinearLayer output;
};

TinyMlpModel load_model(const std::filesystem::path& path);

}  // namespace lcqi
```

`inference.hpp` 定义推理结果和 `run_inference` 入口。返回值包含 hidden、logits 和 predicted class，测试可以用这些字段定位是加载错、kernel 错，还是 argmax 错。

Listing 4.3 include/lcqi/inference.hpp

```cpp
#pragma once

#include <cstdint>
#include <span>
#include <vector>

#include <lcqi/model.hpp>

namespace lcqi {

struct InferenceResult {
    std::vector<float> logits;
    std::int32_t predicted_class = 0;
};

void linear_i8(const QuantizedLinearLayer& layer,
               std::span<const float> input,
               std::span<float> output);

void relu_inplace(std::span<float> values);

InferenceResult run_inference(const TinyMlpModel& model,
                              std::span<const float> input);

}  // namespace lcqi
```

Int8 Kernel、Tokenizer 与 Safetensors

int8 kernel 头文件定义 packed linear、deterministic fixture、scalar/workspace/AVX2 路径和误差比较。注意 AVX2 是运行时能力，不是源码存在就等于当前机器已验证。

Listing 4.4 include/lcqi/int8_kernels.hpp

```cpp
#pragma once

#include <cstdint>
#include <span>
#include <vector>

#include <lcqi/model.hpp>

namespace lcqi {

struct PackedLinearI8 {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    std::int32_t output_block_size = 0;
    float scale = 1.0F;
    std::vector<std::int8_t> packed_weights;
```

```
    std::vector<float> bias;
};

struct KernelBenchmarkCase {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    std::int32_t output_block_size = 0;
    std::int32_t repeat = 0;
};

struct KernelBackendBenchmark {
    const char* name = "";
    bool available = false;
    double average_us = 0.0;
    float max_abs_diff = 0.0F;
    float checksum = 0.0F;
};

struct KernelBenchmarkResult {
    KernelBenchmarkCase benchmark_case;
    std::vector<KernelBackendBenchmark> backends;
};

PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
                              std::int32_t output_block_size);

void linear_i8_packed(const PackedLinearI8& layer,
                      std::span<const float> input,
                      std::span<float> output);

void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
                                     std::span<const float> input,
                                     std::span<float> output,
                                     std::span<float> accumulator_workspace);

bool linear_i8_packed_avx2_available() noexcept;

void linear_i8_packed_avx2(const PackedLinearI8& layer,
                           std::span<const float> input,
                           std::span<float> output);

QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
                                                 std::int32_t output_size,
                                                 float scale);

std::vector<float> make_deterministic_input(std::int32_t input_size);

float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs);

KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase& ↩
    ↪ benchmark_case);

```

```
}  // namespace lcqi
```

F32 kernel 头文件定义 GPT-2 热路径使用的点积和 row-range 边界。读这里要区分四层：`dot_f32()` 是 checked API，先确认两个 span 长度一致；`dot_f32_unchecked()` 是单行点积热路径入口；`linear_f32_rows_unchecked()` 一次计算一段输出行，用在 QKV、attention projection 和 MLP；`max_dot_f32_rows_unchecked()` 一次扫描一段 vocab 行并返回局部最大值，用在 logits-free greedy `lm_head`。AVX2/FMA 是否可用是运行时能力，不是 CMake 把文件编进来就等于一定执行。

Listing 4.5 include/lcqi/f32_kernels.hpp

```cpp
#pragma once

#include <cstddef>
#include <span>

namespace lcqi {

struct F32RowMax {
    std::size_t row = 0;
    float value = 0.0F;
};

[[nodiscard]] float dot_f32(std::span<const float> lhs, std::span<const float> rhs);

[[nodiscard]] float dot_f32_scalar_unchecked(const float* lhs,
                                             const float* rhs,
                                             std::size_t size) noexcept;

[[nodiscard]] float dot_f32_unchecked(const float* lhs,
                                      const float* rhs,
                                      std::size_t size) noexcept;

[[nodiscard]] bool dot_f32_avx2_available() noexcept;

[[nodiscard]] float dot_f32_avx2_unchecked(const float* lhs,
                                           const float* rhs,
                                           std::size_t size) noexcept;

void linear_f32_rows_scalar_unchecked(const float* weights,
                                      const float* input,
                                      const float* bias,
                                      std::size_t input_size,
                                      std::size_t row_begin,
                                      std::size_t row_end,
                                      float* output) noexcept;

void linear_f32_rows_unchecked(const float* weights,
                               const float* input,
                               const float* bias,
                               std::size_t input_size,
```

```cpp
                              std::size_t row_begin,
                              std::size_t row_end,
                              float* output) noexcept;

void linear_f32_rows_avx2_unchecked(const float* weights,
                                    const float* input,
                                    const float* bias,
                                    std::size_t input_size,
                                    std::size_t row_begin,
                                    std::size_t row_end,
                                    float* output) noexcept;

[[nodiscard]] F32RowMax max_dot_f32_rows_scalar_unchecked(const float* weights,
                                                          const float* input,
                                                          std::size_t input_size,
                                                          std::size_t row_begin,
                                                          std::size_t row_end) noexcept;

[[nodiscard]] F32RowMax max_dot_f32_rows_unchecked(const float* weights,
                                                   const float* input,
                                                   std::size_t input_size,
                                                   std::size_t row_begin,
                                                   std::size_t row_end) noexcept;

[[nodiscard]] F32RowMax max_dot_f32_rows_avx2_unchecked(const float* weights,
                                                        const float* input,
                                                        std::size_t input_size,
                                                        std::size_t row_begin,
                                                        std::size_t row_end) noexcept;

}  // namespace lcqi
```

tokenizer 头文件固定 BOS/EOS/UNK、词表和 template hash。prompt 进入模型前，必须先变成稳定 token id；未知 token 行为也必须被写进合同。

Listing 4.6 include/lcqi/tokenizer.hpp

```cpp
#pragma once

#include <cstdint>
#include <filesystem>
#include <string>
#include <string_view>
#include <vector>

namespace lcqi {

struct TokenEntry {
```

```cpp
    std::string text;
    std::int32_t id = 0;
};

struct TokenizerModel {
    std::int32_t bos_id = -1;
    std::int32_t eos_id = -1;
    std::int32_t unk_id = -1;
    std::string chat_template_hash;
    std::vector<TokenEntry> vocab;
};

[[nodiscard]] TokenizerModel load_tokenizer(const std::filesystem::path& path);

[[nodiscard]] std::vector<std::int32_t> encode_prompt(
    const TokenizerModel& tokenizer,
    std::string_view prompt);

[[nodiscard]] std::uint64_t tokenizer_contract_hash(const TokenizerModel& tokenizer);

}  // namespace lcqi
```

safetensors 头文件固定 dtype、shape、offset 和 payload 边界。它负责模型字节怎样被解释，不应该和数学推理逻辑混在一起。

Listing 4.7 include/lcqi/safetensors.hpp

```cpp
#pragma once

#include <cstdint>
#include <filesystem>
#include <span>
#include <string>
#include <string_view>
#include <utility>
#include <vector>

namespace lcqi {

struct SafeTensorEntry {
    std::string name;
    std::string dtype;
    std::vector<std::int64_t> shape;
    std::uint64_t data_begin = 0;
    std::uint64_t data_end = 0;

    [[nodiscard]] std::uint64_t byte_size() const;
};

struct SafeTensorManifest {
    std::uint64_t header_size = 0;
    std::uint64_t data_start_offset = 0;
```

```
    std::vector<std::pair<std::string, std::string>> metadata;
    std::vector<SafeTensorEntry> tensors;

    [[nodiscard]] const SafeTensorEntry* find_tensor(std::string_view name) const;
};

struct SafeTensorFile {
    std::filesystem::path path;
    SafeTensorManifest manifest;
    std::vector<std::uint8_t> bytes;
};

[[nodiscard]] SafeTensorManifest load_safetensors_manifest(
    const std::filesystem::path& path);

[[nodiscard]] SafeTensorFile load_safetensors_file(const std::filesystem::path& path);

[[nodiscard]] std::vector<float> read_safetensor_f32_tensor(
    const SafeTensorFile& file,
    std::string_view name);

[[nodiscard]] std::span<const std::uint8_t> read_safetensor_tensor_bytes(
    const SafeTensorFile& file,
    std::string_view name);

}  // namespace lcqi
```

GGUF 与 Q4_K 合同

GGUF 头文件固定 llama.cpp 生态模型文件的读取边界：版本、metadata、alignment、tensor 名、shape、ggml type 和 tensor payload offset。它也保留 tokenizer 需要的字符串数组 metadata，例如 `tokenizer.ggml.tokens` 和 `tokenizer.ggml.merges`。它不负责模型图执行，只把"真实 GGUF 文件里有哪些 tensor、每个 tensor 在哪里、哪些 metadata 能支撑 tokenizer"变成可检查对象。

Listing 4.8 include/lcqi/gguf.hpp

```
#pragma once

#include <cstdint>
#include <filesystem>
#include <span>
#include <string>
#include <string_view>
#include <vector>

namespace lcqi {

enum class GgmlType : std::int32_t {
    f32 = 0,
```

```
    f16 = 1,
    q4_0 = 2,
    q4_1 = 3,
    q5_0 = 6,
    q5_1 = 7,
    q8_0 = 8,
    q8_1 = 9,
    q2_k = 10,
    q3_k = 11,
    q4_k = 12,
    q5_k = 13,
    q6_k = 14,
    q8_k = 15,
    i8 = 24,
    i16 = 25,
    i32 = 26,
    i64 = 27,
    f64 = 28,
    bf16 = 30,
    q1_0 = 41,
};

struct GgmlTypeLayout {
    std::int64_t block_size = 1;
    std::uint64_t type_size = 0;
    const char* name = "";
    bool quantized = false;
};

struct GgufMetadataEntry {
    std::string key;
    std::string type;
    std::string value_preview;
    std::vector<std::string> string_values;
};

struct GgufTensorInfo {
    std::string name;
    std::vector<std::int64_t> shape;
    GgmlType type = GgmlType::f32;
    std::uint64_t relative_offset = 0;
    std::uint64_t absolute_offset = 0;
    std::uint64_t byte_size = 0;

    [[nodiscard]] std::uint64_t element_count() const;
};

struct GgufManifest {
    std::uint32_t version = 0;
    std::uint64_t tensor_count = 0;
    std::uint64_t metadata_count = 0;
    std::uint64_t alignment = 32;
```

```
    std::uint64_t data_offset = 0;
    std::vector<GgufMetadataEntry> metadata;
    std::vector<GgufTensorInfo> tensors;

    [[nodiscard]] const GgufTensorInfo* find_tensor(std::string_view name) const;
    [[nodiscard]] const GgufMetadataEntry* find_metadata(std::string_view key) const;
};

[[nodiscard]] GgmlTypeLayout ggml_type_layout(GgmlType type);

[[nodiscard]] const char* ggml_type_name(GgmlType type);

[[nodiscard]] std::uint64_t ggml_tensor_byte_size(GgmlType type,
                                                  std::span<const std::int64_t> shape);

[[nodiscard]] GgufManifest load_gguf_manifest(const std::filesystem::path& path);

[[nodiscard]] std::vector<std::uint8_t> read_gguf_tensor_bytes(
    const std::filesystem::path& path,
    const GgufTensorInfo& tensor);

}  // namespace lcqi
```

GGML tensor 头文件是 GGUF 与模型数学之间的格式层。SmolLM2 的 `Q4_K_M` 文件内部并不是所有矩阵都是 `Q4_K`：远端 caw 扫描显示它包含 `F32`、`Q8_0`、`Q5_0`、`Q6_K` 和 `Q4_K`。因此完整模型加载不能只写一个 Q4_K 解码器，必须把这些格式统一变成后续执行层能消费的张量。

Listing 4.9 include/lcqi/ggml_tensors.hpp

```
#pragma once

#include <lcqi/gguf.hpp>

#include <cstdint>
#include <span>
#include <vector>

namespace lcqi {

constexpr std::int64_t LCQI_Q8_0_BLOCK_VALUES = 32;
constexpr std::int64_t LCQI_Q8_0_BLOCK_BYTES = 34;
constexpr std::int64_t LCQI_Q5_0_BLOCK_VALUES = 32;
constexpr std::int64_t LCQI_Q5_0_BLOCK_BYTES = 22;
constexpr std::int64_t LCQI_Q6_K_BLOCK_VALUES = 256;
constexpr std::int64_t LCQI_Q6_K_BLOCK_BYTES = 210;

[[nodiscard]] float ggml_f16_to_f32(std::uint16_t bits);

[[nodiscard]] std::vector<float> dequantize_ggml_tensor(
```

```cpp
    GgmlType type,
    std::span<const std::uint8_t> bytes,
    std::int64_t element_count);

void dequantize_ggml_tensor_to(GgmlType type,
                               std::span<const std::uint8_t> bytes,
                               std::span<float> output);

}  // namespace lcqi
```

Q4_K 头文件固定本轮量化切片的数学合同。一个 `Q4_K` block 解码出 256 个权重，`Q8_K` input 是每 256 个激活临时量化一次。这里暴露的 `matvec_q4_k_q8` 是单算子入口，不是完整 LLaMA-like decoder；完整 decoder 还要接 RMSNorm、RoPE、attention、SwiGLU、KV cache 和 sampler。

Listing 4.10 include/lcqi/q4_k.hpp

```cpp
#pragma once

#include <array>
#include <cstdint>
#include <span>
#include <vector>

namespace lcqi {

constexpr std::int64_t LCQI_QK_K_BLOCK_VALUES = 256;
constexpr std::int64_t LCQI_Q4_K_BLOCK_BYTES = 144;
constexpr std::int64_t LCQI_Q8_K_BLOCK_BYTES = 292;
constexpr std::int64_t LCQI_Q4_K_SUBBLOCK_VALUES = 32;
constexpr std::int64_t LCQI_Q4_K_SUBBLOCKS = 8;
constexpr std::int64_t LCQI_Q8_K_BSUM_GROUPS = 16;

struct Q8KBlock {
    float d = 0.0F;
    std::array<std::int8_t, LCQI_QK_K_BLOCK_VALUES> qs{};
    std::array<std::int16_t, LCQI_Q8_K_BSUM_GROUPS> bsums{};
};

[[nodiscard]] std::vector<float> dequantize_q4_k(std::span<const std::uint8_t> bytes,
                                                 std::int64_t element_count);

void dequantize_q4_k_to(std::span<const std::uint8_t> bytes,
                        std::span<float> output);

[[nodiscard]] std::vector<Q8KBlock> quantize_q8_k_input(std::span<const float> input);

void quantize_q8_k_input_to(std::span<const float> input,
                            std::span<Q8KBlock> output);

[[nodiscard]] bool q4_k_q8_avx2_available() noexcept;

```

```
[[nodiscard]] const char* q4_k_q8_active_backend() noexcept;

float dot_q4_k_f32(std::span<const std::uint8_t> q4_blocks,
                   std::span<const float> input);

float dot_q4_k_q8(std::span<const std::uint8_t> q4_blocks,
                  std::span<const Q8KBlock> input);

void matvec_q4_k_f32(std::span<const std::uint8_t> q4_rows,
                     std::int64_t row_count,
                     std::int64_t column_count,
                     std::span<const float> input,
                     std::span<float> output);

void matvec_q4_k_q8(std::span<const std::uint8_t> q4_rows,
                    std::int64_t row_count,
                    std::int64_t column_count,
                    std::span<const Q8KBlock> input,
                    std::span<float> output);

}  // namespace lcqi
```

GGML matvec 头文件固定 `Q5_0/Q6_K/Q8_0` direct 实验路径的合同。`matvec_ggml_quantized_f32` 是 exact oracle：它不先 materialize 整个矩阵，而是直接从 GGUF block bytes 和 f32 input 做点积，用来对照解码后的 f32 权重。`quantize_q8_0_input` 把 activation 每 32 个元素临时量化一次，`matvec_ggml_quantized_q8_0` 再执行低比特权重乘 `Q8_0` activation。这个头文件只承诺单算子数学和边界检查，不承诺端到端更快；是否能进入默认路径必须看 caw 同输入 A/B。

Listing 4.11 include/lcqi/ggml_matvec.hpp

```
#pragma once

#include <lcqi/gguf.hpp>

#include <array>
#include <cstdint>
#include <span>
#include <vector>

namespace lcqi {

constexpr std::int64_t LCQI_Q8_0_INPUT_BLOCK_VALUES = 32;

struct Q8_0InputBlock {
    float d = 0.0F;
    std::array<std::int8_t, LCQI_Q8_0_INPUT_BLOCK_VALUES> qs{};
    std::int16_t sum = 0;
};

[[nodiscard]] bool ggml_type_has_f32_direct_matvec(GgmlType type) noexcept;

```

```
[[nodiscard]] bool ggml_type_has_q8_0_direct_matvec(GgmlType type) noexcept;

[[nodiscard]] std::vector<Q8_0InputBlock> quantize_q8_0_input(std::span<const float> input);

void quantize_q8_0_input_to(std::span<const float> input,
                            std::span<Q8_0InputBlock> output);

void matvec_ggml_quantized_f32(GgmlType type,
                               std::span<const std::uint8_t> rows,
                               std::int64_t row_count,
                               std::int64_t column_count,
                               std::span<const float> input,
                               std::span<float> output);

void matvec_ggml_quantized_q8_0(GgmlType type,
                                std::span<const std::uint8_t> rows,
                                std::int64_t row_count,
                                std::int64_t column_count,
                                std::span<const Q8_0InputBlock> input,
                                std::span<float> output);

}  // namespace lcqi
```

LLaMA GGUF 头文件把真实 GGUF 文件接到 decoder 数学层。当前实现有三种权重执行状态：关闭 direct 时是 `f32_dequantized_reference`；默认是 `gguf_mixed_q4_k_direct`，只让 shape 兼容的 `Q4_K` linear tensor 保留原始 block bytes；显式设置 `LCQI_LLAMA_GGML_DIRECT=1` 时才进入 `gguf_mixed_quantized_direct_experimental`，尝试把 `Q5_0/Q6_K/Q8_0` 也接入 direct。这个边界很重要，因为它防止把“能跑真实模型”误写成“已经拥有 llama.cpp 同等级的低比特执行引擎”。

Listing 4.12 include/lcqi/llama_gguf.hpp

```
#pragma once

#include <lcqi/gpt2_reference.hpp>
#include <lcqi/gguf.hpp>
#include <lcqi/reference_decoder.hpp>

#include <cstdint>
#include <filesystem>
#include <span>
#include <string>
#include <vector>

namespace lcqi {

enum class LlamaGgufLinearStorage : std::uint8_t {
    f32,
    q4_k,
    ggml_quantized,
};

```

```cpp
struct LlamaGgufLinearWeights {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    GgmlType source_type = GgmlType::f32;
    LlamaGgufLinearStorage storage = LlamaGgufLinearStorage::f32;
    std::vector<float> f32_weights;
    std::vector<std::uint8_t> quantized_bytes;
};

struct LlamaGgufDecoderLayerWeights {
    std::vector<float> rms_attention_weight;
    LlamaGgufLinearWeights wq;
    LlamaGgufLinearWeights wk;
    LlamaGgufLinearWeights wv;
    LlamaGgufLinearWeights wo;
    std::vector<float> rms_mlp_weight;
    LlamaGgufLinearWeights w_gate;
    LlamaGgufLinearWeights w_up;
    LlamaGgufLinearWeights w_down;
};

struct LlamaGgufDecoderModel {
    DecoderConfig config;
    std::vector<float> token_embedding;
    std::vector<LlamaGgufDecoderLayerWeights> layers;
    std::vector<float> final_norm_weight;
    LlamaGgufLinearWeights lm_head;
};

struct LlamaGgufModel {
    LlamaGgufDecoderModel decoder;
    std::string architecture;
    std::string name;
    std::int32_t bos_token_id = -1;
    std::int32_t eos_token_id = -1;
    std::int32_t unknown_token_id = -1;
    bool lm_head_tied_to_embedding = true;
};

struct LlamaGgufLoadReport {
    double manifest_ms = 0.0;
    double weights_ms = 0.0;
    std::uint64_t quantized_weight_bytes = 0;
    std::uint64_t f32_weight_bytes = 0;
    std::uint64_t direct_quantized_weight_bytes = 0;
    std::uint64_t fallback_dequantized_weight_bytes = 0;
    std::int64_t tensors_loaded = 0;
    std::int64_t q4_k_direct_tensors = 0;
    std::int64_t ggml_direct_tensors = 0;
    std::int64_t f32_fallback_tensors = 0;
};

```

```cpp
struct LlamaGgufLoadedModel {
    LlamaGgufModel model;
    LlamaGgufLoadReport report;
};

struct LlamaGgufHotspotReport {
    double rms_norm_ms = 0.0;
    double attention_ms = 0.0;
    double rope_ms = 0.0;
    double wq_ms = 0.0;
    double wk_ms = 0.0;
    double wv_ms = 0.0;
    double wo_ms = 0.0;
    double w_gate_ms = 0.0;
    double w_up_ms = 0.0;
    double w_down_ms = 0.0;
    double lm_head_ms = 0.0;
    double q4_k_direct_ms = 0.0;
    double ggml_direct_ms = 0.0;
    double f32_fallback_ms = 0.0;
    std::uint64_t q4_k_direct_calls = 0;
    std::uint64_t ggml_direct_calls = 0;
    std::uint64_t f32_fallback_calls = 0;
};

struct LlamaGgufGenerationResult {
    std::vector<std::int32_t> prompt_ids;
    std::vector<std::int32_t> generated_ids;
    LlamaGgufHotspotReport hotspots;
    double load_ms = 0.0;
    double prefill_ms = 0.0;
    double decode_ms = 0.0;
    double total_ms = 0.0;
    std::size_t kv_cache_bytes = 0;
    std::size_t prefill_steps = 0;
    std::size_t decode_steps = 0;
    std::int32_t predicted_first_token = -1;
};

[[nodiscard]] LlamaGgufLoadedModel load_llama_gguf_reference_model(
    const std::filesystem::path& gguf_path);

[[nodiscard]] Gpt2Tokenizer load_gpt2_tokenizer_from_gguf(
    const std::filesystem::path& gguf_path);

[[nodiscard]] std::vector<std::int32_t> llama_gguf_chat_prompt_ids(
    const Gpt2Tokenizer& tokenizer,
    std::string_view user_prompt,
    std::int32_t bos_token_id);

[[nodiscard]] LlamaGgufGenerationResult llama_gguf_generate_greedy(
    const LlamaGgufModel& model,
```

```
    std::span<const std::int32_t> prompt_ids,
    std::int32_t max_new_tokens);

} // namespace lcqi
```

Reference Decoder 与 GPT-2 合同

reference decoder 头文件定义张量、层配置、trace checkpoint、采样结果和 decode 函数。它是定位推理错误的主要合同，不只返回最终 token。

Listing 4.13 include/lcqi/reference_decoder.hpp

```
#pragma once

#include <cstdint>
#include <span>
#include <string>
#include <vector>

namespace lcqi {

struct LinearWeightsF32 {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    std::vector<float> weights;
    std::vector<float> bias;
};

struct DecoderConfig {
    std::int32_t hidden_size = 0;
    std::int32_t query_heads = 0;
    std::int32_t kv_heads = 0;
    std::int32_t head_dim = 0;
    std::int32_t intermediate_size = 0;
    std::int32_t vocab_size = 0;
    std::int32_t max_context = 0;
    float rms_epsilon = 1.0e-5F;
    float rope_base = 10000.0F;
};

struct DecoderLayerWeightsF32 {
    std::vector<float> rms_attention_weight;
    LinearWeightsF32 wq;
    LinearWeightsF32 wk;
    LinearWeightsF32 wv;
    LinearWeightsF32 wo;
    std::vector<float> rms_mlp_weight;
    LinearWeightsF32 w_gate;
    LinearWeightsF32 w_up;
    LinearWeightsF32 w_down;
};

```

```cpp
struct ReferenceDecoderModel {
    DecoderConfig config;
    std::vector<float> token_embedding;
    std::vector<DecoderLayerWeightsF32> layers;
    std::vector<float> final_norm_weight;
    LinearWeightsF32 lm_head;
};

struct ReferenceDecodeResult {
    std::vector<float> logits;
    std::int32_t predicted_token = 0;
};

struct ReferenceTensorCheckpoint {
    std::string name;
    std::int32_t token_position = 0;
    std::int32_t layer_id = 0;
    std::vector<std::int32_t> shape;
    std::vector<std::int32_t> stride;
    std::string dtype;
    std::string layout;
    float checksum = 0.0F;
    float max_abs = 0.0F;
    std::vector<float> values;
};

struct ReferenceDecodeTraceResult {
    ReferenceDecodeResult result;
    std::vector<ReferenceTensorCheckpoint> checkpoints;
};

class ReferenceKVCache {
public:
    ReferenceKVCache(const DecoderConfig& config, std::int32_t layer_count);

    void append(std::int32_t layer_id,
                std::int32_t model_position,
                std::span<const float> key,
                std::span<const float> value);

    [[nodiscard]] std::span<const float> key(std::int32_t layer_id,
                                             std::int32_t model_position,
                                             std::int32_t kv_head) const;

    [[nodiscard]] std::span<const float> value(std::int32_t layer_id,
                                               std::int32_t model_position,
                                               std::int32_t kv_head) const;

    [[nodiscard]] std::int32_t layer_count() const noexcept;
    [[nodiscard]] std::int32_t filled_tokens() const noexcept;

private:
```

```cpp
    [[nodiscard]] std::size_t base_offset(std::int32_t layer_id,
                                          std::int32_t model_position,
                                          std::int32_t kv_head) const;
    void validate_address(std::int32_t layer_id,
                          std::int32_t model_position,
                          std::int32_t kv_head) const;

    DecoderConfig config_;
    std::int32_t layer_count_ = 0;
    std::int32_t filled_tokens_ = 0;
    std::vector<float> keys_;
    std::vector<float> values_;
    std::vector<std::uint8_t> written_;
};

void rms_norm(std::span<const float> input,
              std::span<const float> weight,
              float epsilon,
              std::span<float> output);

void linear_f32(const LinearWeightsF32& layer,
                std::span<const float> input,
                std::span<float> output);

void apply_rope(std::span<float> heads,
                std::int32_t head_count,
                std::int32_t head_dim,
                std::int32_t model_position,
                float rope_base);

void attention_decode(const DecoderConfig& config,
                      const ReferenceKVCache& cache,
                      std::int32_t layer_id,
                      std::int32_t model_position,
                      std::span<const float> query,
                      std::span<float> output);

ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
                                           std::span<const std::int32_t> token_ids);

ReferenceDecodeTraceResult run_reference_decode_with_trace(
    const ReferenceDecoderModel& model,
    std::span<const std::int32_t> token_ids);

ReferenceDecoderModel make_tiny_reference_decoder_model();

}  // namespace ℓcqi
```

GPT-2 reference 头文件定义 GPT-2 配置、byte-level BPE tokenizer、HuggingFace safetensors 加载、forward、cached forward 和 greedy generation。它单独存在，是因为 GPT-2 使用 LayerNorm、绝对位置嵌入、packed QKV、GELU 和 tied lm head，不能直接塞进 LLaMA-like reference decoder。

读 `Gpt2CachedGreedyDecoder` 时要区分三个入口：`step()` 推进一个 token 并返回 greedy 预测；`step_with_logits()` 保留完整 logits 给测试和对照；`advance_without_prediction()` 只推进 embedding、Transformer layers 和 KV cache，用在 prompt prefill 的前缀 token 上。第三个入口不是“少算一点但可能改变结果”，而是删掉不会被任何 caller 使用的预测结果。

Listing 4.14 include/lcqi/gpt2_reference.hpp

```cpp
#pragma once

#include <cstdint>
#include <filesystem>
#include <memory>
#include <span>
#include <string>
#include <string_view>
#include <unordered_map>
#include <vector>

namespace lcqi {

struct Gpt2Config {
    std::int32_t hidden_size = 0;
    std::int32_t head_count = 0;
    std::int32_t layer_count = 0;
    std::int32_t max_positions = 0;
    std::int32_t vocab_size = 0;
    std::int32_t intermediate_size = 0;
    std::int32_t bos_token_id = -1;
    std::int32_t eos_token_id = -1;
    float layer_norm_epsilon = 1.0e-5F;
    std::string activation_function = "gelu_new";
};

struct Gpt2LinearF32 {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    std::vector<float> weights;
    std::vector<float> bias;
};

struct Gpt2LayerWeightsF32 {
    std::vector<float> ln_1_weight;
    std::vector<float> ln_1_bias;
    Gpt2LinearF32 c_attn;
    Gpt2LinearF32 c_proj;
    std::vector<float> ln_2_weight;
    std::vector<float> ln_2_bias;
    Gpt2LinearF32 c_fc;
    Gpt2LinearF32 mlp_c_proj;
};

struct Gpt2ReferenceModel {
    Gpt2Config config;
```

```
    std::vector<float> token_embedding;
    std::vector<float> position_embedding;
    std::vector<Gpt2LayerWeightsF32> layers;
    std::vector<float> final_ln_weight;
    std::vector<float> final_ln_bias;
    std::vector<float> lm_head_weight;
    bool tie_lm_head_to_embedding = true;
};

struct Gpt2ForwardResult {
    std::vector<float> logits;
    std::int32_t predicted_token = 0;
};

struct Gpt2HotspotProfile {
    std::int64_t decoder_steps = 0;
    std::int64_t layer_steps = 0;
    double total_step_ms = 0.0;
    double embedding_ms = 0.0;
    double layer_norm_ms = 0.0;
    double final_norm_ms = 0.0;
    double qkv_projection_ms = 0.0;
    double kv_append_ms = 0.0;
    double attention_ms = 0.0;
    double attention_projection_ms = 0.0;
    double residual_add_ms = 0.0;
    double mlp_fc_ms = 0.0;
    double gelu_ms = 0.0;
    double mlp_projection_ms = 0.0;
    double lm_head_ms = 0.0;
    double logits_result_ms = 0.0;
};

struct Gpt2ExecutionOptions {
    std::int32_t worker_count = 0;
    std::int32_t parallel_min_rows = 512;
};

class Gpt2KvCache;

namespace detail {
class Gpt2ParallelWorkerPool;

void attend_cached_position(const Gpt2KvCache& cache,
                            std::int32_t layer_id,
                            std::int32_t model_position,
                            std::span<const float> query,
                            std::span<float> scores,
                            std::span<float> output);
}  // namespace detail

class Gpt2ForwardWorkspace {
```

```cpp
public:
    explicit Gpt2ForwardWorkspace(const Gpt2Config& config);

    void reset_for_config(const Gpt2Config& config);

    [[nodiscard]] std::span<float> hidden() noexcept;
    [[nodiscard]] std::span<float> normed() noexcept;
    [[nodiscard]] std::span<float> query() noexcept;
    [[nodiscard]] std::span<float> key() noexcept;
    [[nodiscard]] std::span<float> value() noexcept;
    [[nodiscard]] std::span<float> attention() noexcept;
    [[nodiscard]] std::span<float> projected() noexcept;
    [[nodiscard]] std::span<float> qkv_packed() noexcept;
    [[nodiscard]] std::span<float> mlp_fc() noexcept;
    [[nodiscard]] std::span<float> mlp_out() noexcept;
    [[nodiscard]] std::span<float> logits() noexcept;
    [[nodiscard]] std::span<float> scores_prefix(std::int32_t count);

private:
    Gpt2Config config_;
    std::vector<float> hidden_;
    std::vector<float> normed_;
    std::vector<float> query_;
    std::vector<float> key_;
    std::vector<float> value_;
    std::vector<float> attention_;
    std::vector<float> projected_;
    std::vector<float> qkv_packed_;
    std::vector<float> mlp_fc_;
    std::vector<float> mlp_out_;
    std::vector<float> logits_;
    std::vector<float> scores_;
};

class Gpt2KvCache {
public:
    explicit Gpt2KvCache(const Gpt2Config& config);

    void append(std::int32_t layer_id,
                std::int32_t model_position,
                std::span<const float> key,
                std::span<const float> value);

    [[nodiscard]] std::span<const float> key(std::int32_t layer_id,
                                             std::int32_t model_position,
                                             std::int32_t head) const;

    [[nodiscard]] std::span<const float> value(std::int32_t layer_id,
                                               std::int32_t model_position,
                                               std::int32_t head) const;

    [[nodiscard]] std::int32_t filled_tokens() const noexcept;
```

```
    [[nodiscard]] std::size_t byte_size() const noexcept;

private:
    friend void detail::attend_cached_position(const Gpt2KvCache& cache,
                                               std::int32_t layer_id,
                                               std::int32_t model_position,
                                               std::span<const float> query,
                                               std::span<float> scores,
                                               std::span<float> output);

    [[nodiscard]] std::size_t base_offset(std::int32_t layer_id,
                                          std::int32_t model_position,
                                          std::int32_t head) const;
    [[nodiscard]] std::size_t base_offset_unchecked(std::int32_t layer_id,
                                                    std::int32_t model_position,
                                                    std::int32_t head) const noexcept;
    [[nodiscard]] std::span<const float> key_unchecked(std::int32_t layer_id,
                                                       std::int32_t model_position,
                                                       std::int32_t head) const noexcept;
    [[nodiscard]] std::span<const float> value_unchecked(std::int32_t layer_id,
                                                         std::int32_t model_position,
                                                         std::int32_t head) const noexcept;
    void validate_address(std::int32_t layer_id,
                          std::int32_t model_position,
                          std::int32_t head) const;
    void validate_written(std::int32_t layer_id,
                          std::int32_t model_position) const;

    Gpt2Config config_;
    std::int32_t filled_tokens_ = 0;
    std::vector<float> keys_;
    std::vector<float> values_;
    std::vector<std::uint8_t> written_;
};

class Gpt2CachedGreedyDecoder {
public:
    explicit Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& model);
    Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& model,
                            Gpt2HotspotProfile* hotspot_profile);
    Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& model,
                            Gpt2HotspotProfile* hotspot_profile,
                            const Gpt2ExecutionOptions& execution_options);
    ~Gpt2CachedGreedyDecoder();

    Gpt2CachedGreedyDecoder(const Gpt2CachedGreedyDecoder&) = delete;
```

```
    Gpt2CachedGreedyDecoder& operator=(const Gpt2CachedGreedyDecoder&) = delete;
    Gpt2CachedGreedyDecoder(Gpt2CachedGreedyDecoder&&) noexcept;
    Gpt2CachedGreedyDecoder& operator=(Gpt2CachedGreedyDecoder&&) noexcept;

    void advance_without_prediction(std::int32_t token_id);
    [[nodiscard]] std::int32_t step(std::int32_t token_id);
    [[nodiscard]] Gpt2ForwardResult step_with_logits(std::int32_t token_id);
    [[nodiscard]] const Gpt2KvCache& cache() const noexcept;
    [[nodiscard]] std::int32_t filled_tokens() const noexcept;
    [[nodiscard]] std::size_t kv_cache_bytes() const noexcept;
    [[nodiscard]] std::int32_t worker_count() const noexcept;

private:
    const Gpt2ReferenceModel* model_;
    Gpt2HotspotProfile* hotspot_profile_;
    Gpt2ExecutionOptions execution_options_;
    Gpt2KvCache cache_;
    Gpt2ForwardWorkspace workspace_;
    std::unique_ptr<detail::Gpt2ParallelWorkerPool> worker_pool_;
};

struct Gpt2Tokenizer {
    std::int32_t bos_token_id = -1;
    std::int32_t eos_token_id = -1;
    std::unordered_map<std::string, std::int32_t> vocab;
    std::vector<std::string> id_to_token;
    std::unordered_map<std::string, std::int32_t> bpe_ranks;
};

[[nodiscard]] Gpt2Tokenizer load_gpt2_tokenizer(
    const std::filesystem::path& vocab_json,
    const std::filesystem::path& merges_txt,
    std::int32_t bos_token_id,
    std::int32_t eos_token_id);

[[nodiscard]] std::vector<std::int32_t> gpt2_encode(
    const Gpt2Tokenizer& tokenizer,
    std::string_view text);

[[nodiscard]] std::string gpt2_decode(
    const Gpt2Tokenizer& tokenizer,
    std::span<const std::int32_t> token_ids);

[[nodiscard]] Gpt2Config load_gpt2_config(const std::filesystem::path& config_json);

[[nodiscard]] Gpt2ReferenceModel load_gpt2_reference_model(
    const std::filesystem::path& config_json,
    const std::filesystem::path& safetensors_path);

[[nodiscard]] Gpt2ReferenceModel load_gpt2_from_directory(
```

```
    const std::filesystem::path& model_directory);

[[nodiscard]] Gpt2ForwardResult run_gpt2_forward(
    const Gpt2ReferenceModel& model,
    std::span<const std::int32_t> token_ids);

[[nodiscard]] Gpt2ForwardResult run_gpt2_forward_cached(
    const Gpt2ReferenceModel& model,
    Gpt2KvCache& cache,
    std::int32_t token_id);

[[nodiscard]] std::vector<std::int32_t> gpt2_generate_greedy(
    const Gpt2ReferenceModel& model,
    std::span<const std::int32_t> prompt_token_ids,
    std::int32_t max_new_tokens);

[[nodiscard]] std::vector<std::int32_t> gpt2_generate_greedy_cached(
    const Gpt2ReferenceModel& model,
    std::span<const std::int32_t> prompt_token_ids,
    std::int32_t max_new_tokens);

[[nodiscard]] Gpt2ReferenceModel make_tiny_gpt2_reference_model();

}  // namespace lcqi
```

最短闭环：从模型文件到 logits

tiny MLP 路径不是为了证明 LCQI 已经是完整大模型引擎，而是为了建立最短的、可手算的、可测试的推理闭环。读者要把模型加载、输入检查、int8 linear、ReLU、第二层 logits、argmax 和 CLI 输出连起来看。

模型加载与基础推理

`model.cpp` 负责解析 tiny 文本模型、权重、scale、bias 和 shape。坏格式、坏维度和坏数值都应该在这里被拒绝，不能拖到 kernel 内部才崩溃。

Listing 4.15 src/model.cpp

```
#include <lcqi/model.hpp>

#include <fstream>
#include <limits>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr std::int32_t LCQI_INT8_MIN_VALUE = -128;
constexpr std::int32_t LCQI_INT8_MAX_VALUE = 127;

std::string read_key(std::istream& input) {
    std::string key;
```

```cpp
    if (!(input >> key)) {
        throw std::runtime_error("unexpected end of model file");
    }
    return key;
}

void expect_key(std::istream& input, const std::string& expected) {
    const std::string key = read_key(input);
    if (key != expected) {
        throw std::runtime_error("expected key '" + expected + "', got '" + key + "'");
    }
}

std::int32_t read_i32(std::istream& input, const std::string& key) {
    expect_key(input, key);
    std::int32_t value = 0;
    if (!(input >> value)) {
        throw std::runtime_error("invalid int32 value for key '" + key + "'");
    }
    return value;
}

float read_float(std::istream& input, const std::string& key) {
    expect_key(input, key);
    float value = 0.0F;
    if (!(input >> value)) {
        throw std::runtime_error("invalid float value for key '" + key + "'");
    }
    return value;
}

std::vector<std::int8_t> read_i8_vector(std::istream& input,
                                        const std::string& key,
                                        std::int32_t count) {
    expect_key(input, key);
    if (count < 0) {
        throw std::runtime_error("negative vector size for key '" + key + "'");
    }
    std::vector<std::int8_t> values(static_cast<std::size_t>(count));
    for (std::int32_t i = 0; i < count; ++i) {
        std::int32_t raw = 0;
        if (!(input >> raw)) {
            throw std::runtime_error("invalid int8 payload for key '" + key + "'");
        }
        if (raw < LCQI_INT8_MIN_VALUE || raw > LCQI_INT8_MAX_VALUE) {
            throw std::runtime_error("int8 payload out of range for key '" + key + "'");
        }
        values[static_cast<std::size_t>(i)] = static_cast<std::int8_t>(raw);
    }
```

```cpp
    return values;
}

std::vector<float> read_float_vector(std::istream& input,
                                     const std::string& key,
                                     std::int32_t count) {
    expect_key(input, key);
    if (count < 0) {
        throw std::runtime_error("negative vector size for key '" + key + "'");
    }
    std::vector<float> values(static_cast<std::size_t>(count));
    for (std::int32_t i = 0; i < count; ++i) {
        if (!(input >> values[static_cast<std::size_t>(i)])) {
            throw std::runtime_error("invalid float payload for key '" + key + "'");
        }
    }
    return values;
}

void validate_layer(const QuantizedLinearLayer& layer, const std::string& name) {
    if (layer.input_size <= 0 || layer.output_size <= 0) {
        throw std::runtime_error(name + " layer has invalid shape");
    }
    const std::size_t expected_weights =
        static_cast<std::size_t>(layer.input_size) *
        static_cast<std::size_t>(layer.output_size);
    if (layer.weights.size() != expected_weights) {
        throw std::runtime_error(name + " layer weight size mismatch");
    }
    if (layer.bias.size() != static_cast<std::size_t>(layer.output_size)) {
        throw std::runtime_error(name + " layer bias size mismatch");
    }
}

std::int32_t checked_element_count(std::int32_t rows,
                                   std::int32_t columns,
                                   const std::string& name) {
    if (rows <= 0 || columns <= 0) {
        throw std::runtime_error(name + " layer has invalid shape");
    }
    const std::int64_t count =
        static_cast<std::int64_t>(rows) * static_cast<std::int64_t>(columns);
    if (count > static_cast<std::int64_t>(std::numeric_limits<std::int32_t>::max())) {
        throw std::runtime_error(name + " layer element count is too large");
    }
    return static_cast<std::int32_t>(count);
}

}  // namespace
```

```cpp

TinyMlpModel load_model(const std::filesystem::path& path) {
    std::ifstream input(path);
    if (!input) {
        throw std::runtime_error("cannot open model file: " + path.string());
    }

    expect_key(input, "LCQI_MODEL_V1");

    TinyMlpModel model;
    model.input_size = read_i32(input, "input_size");
    model.hidden_size = read_i32(input, "hidden_size");
    model.output_size = read_i32(input, "output_size");

    const std::int32_t hidden_weight_count =
        checked_element_count(model.input_size, model.hidden_size, "hidden");
    const std::int32_t output_weight_count =
        checked_element_count(model.hidden_size, model.output_size, "output");

    model.hidden.input_size = model.input_size;
    model.hidden.output_size = model.hidden_size;
    model.hidden.scale = read_float(input, "hidden_scale");
    model.hidden.weights = read_i8_vector(
        input,
        "hidden_weights",
        hidden_weight_count);
    model.hidden.bias = read_float_vector(input, "hidden_bias", model.hidden_size);

    model.output.input_size = model.hidden_size;
    model.output.output_size = model.output_size;
    model.output.scale = read_float(input, "output_scale");
    model.output.weights = read_i8_vector(
        input,
        "output_weights",
        output_weight_count);
    model.output.bias = read_float_vector(input, "output_bias", model.output_size);

    validate_layer(model.hidden, "hidden");
    validate_layer(model.output, "output");
    return model;
}

}  // namespace lcqi
```

`inference.cpp` 执行 int8 linear、ReLU、第二层 logits 和 predicted class。它是系统中最短的“模型资产到输出”路径。

Listing 4.16 src/inference.cpp

```cpp
#include <lcqi/inference.hpp>

```

```cpp
#include <algorithm>
#include <cmath>
#include <stdexcept>

namespace lcqi {
namespace {

void validate_input_size(std::span<const float> input, std::int32_t expected) {
    if (input.size() != static_cast<std::size_t>(expected)) {
        throw std::runtime_error("input size does not match model shape");
    }
}

std::int32_t argmax(std::span<const float> values) {
    if (values.empty()) {
        throw std::runtime_error("cannot take argmax of empty logits");
    }
    std::int32_t best_index = 0;
    float best_value = values[0];
    for (std::int32_t i = 1; i < static_cast<std::int32_t>(values.size()); ++i) {
        const float candidate = values[static_cast<std::size_t>(i)];
        if (candidate > best_value) {
            best_value = candidate;
            best_index = i;
        }
    }
    return best_index;
}

}  // namespace

void linear_i8(const QuantizedLinearLayer& layer,
               std::span<const float> input,
               std::span<float> output) {
    if (input.size() != static_cast<std::size_t>(layer.input_size)) {
        throw std::runtime_error("linear_i8 input size mismatch");
    }
    if (output.size() != static_cast<std::size_t>(layer.output_size)) {
        throw std::runtime_error("linear_i8 output size mismatch");
    }

    for (std::int32_t out = 0; out < layer.output_size; ++out) {
        const std::size_t row_base =
            static_cast<std::size_t>(out) * static_cast<std::size_t>(layer.input_size);
        float acc = layer.bias[static_cast<std::size_t>(out)];
        for (std::int32_t in = 0; in < layer.input_size; ++in) {
            const std::int8_t weight =
                layer.weights[row_base + static_cast<std::size_t>(in)];
            acc += static_cast<float>(weight) * layer.scale *
                   input[static_cast<std::size_t>(in)];
```

```cpp
        }
        output[static_cast<std::size_t>(out)] = acc;
    }
}

void relu_inplace(std::span<float> values) {
    for (float& value : values) {
        value = std::max(0.0F, value);
    }
}

InferenceResult run_inference(const TinyMlpModel& model,
                              std::span<const float> input) {
    validate_input_size(input, model.input_size);

    std::vector<float> hidden(static_cast<std::size_t>(model.hidden_size), 0.0F);
    std::vector<float> logits(static_cast<std::size_t>(model.output_size), 0.0F);

    linear_i8(model.hidden, input, hidden);
    relu_inplace(hidden);
    linear_i8(model.output, hidden, logits);

    InferenceResult result;
    result.predicted_class = argmax(logits);
    result.logits = std::move(logits);
    return result;
}

}  // namespace lcqi
```

main.cpp 把模型路径和输入接到 run_inference，并打印结果。CLI 应保持薄层，不复制模型解析和推理逻辑。

Listing 4.17 src/main.cpp

```cpp
#include <lcqi/inference.hpp>
#include <lcqi/model.hpp>

#include <chrono>
#include <cstdint>
#include <cstdlib>
#include <filesystem>
#include <iostream>
#include <span>
#include <stdexcept>
#include <string>
#include <vector>

namespace {

constexpr std::int32_t LCQI_DEFAULT_REPEAT = 1000;
```

```cpp
constexpr int LCQI_EXPECTED_ARGC_MIN = 2;
constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;

std::vector<float> parse_input(int argc, char** argv, std::int32_t expected_size) {
    if (argc < LCQI_EXPECTED_ARGC_MIN + expected_size) {
        throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats...> [repeat]");
    }
    std::vector<float> input(static_cast<std::size_t>(expected_size));
    for (std::int32_t i = 0; i < expected_size; ++i) {
        input[static_cast<std::size_t>(i)] =
            std::stof(argv[LCQI_EXPECTED_ARGC_MIN + i]);
    }
    return input;
}

std::int32_t parse_repeat(int argc, char** argv, std::int32_t input_size) {
    const int repeat_index = LCQI_EXPECTED_ARGC_MIN + input_size;
    if (argc <= repeat_index) {
        return LCQI_DEFAULT_REPEAT;
    }
    const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[repeat_index]));
    if (repeat <= 0) {
        throw std::runtime_error("repeat must be positive");
    }
    return repeat;
}

}  // namespace

int main(int argc, char** argv) {
    try {
        if (argc < LCQI_EXPECTED_ARGC_MIN) {
            throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats...> [repeat]");
        }

        const std::filesystem::path model_path = argv[1];
        const lcqi::TinyMlpModel model = lcqi::load_model(model_path);
        const std::vector<float> input = parse_input(argc, argv, model.input_size);
        const std::int32_t repeat = parse_repeat(argc, argv, model.input_size);

        const lcqi::InferenceResult result = lcqi::run_inference(model, input);
        std::cout << "predicted_class " << result.predicted_class << "\n";
        std::cout << "logits";
        for (const float value : result.logits) {
            std::cout << ' ' << value;
        }
        std::cout << "\n";
```

```

        const auto begin = std::chrono::steady_clock::now();
        std::int32_t checksum = 0;
        for (std::int32_t i = 0; i < repeat; ++i) {
            checksum += lcqi::run_inference(model, input).predicted_class;
        }
        const auto end = std::chrono::steady_clock::now();
        const std::chrono::duration<double> elapsed = end - begin;
        const double average_us =
            elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>(repeat);
        std::cout << "repeat " << repeat << "\n";
        std::cout << "average_us " << average_us << "\n";
        std::cout << "checksum " << checksum << "\n";
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << "\n";
        return EXIT_FAILURE;
    }
}
```

Int8 Kernel 基础实现

`int8_kernels.cpp` 包含 scalar、packed、workspace、deterministic fixture 和误差比较。它是 AVX2 路径的 reference 对照，也是 benchmark 的正确性底座。

Listing 4.18 src/int8_kernels.cpp

```
#include <lcqi/int8_kernels.hpp>

#include <lcqi/inference.hpp>

#include <algorithm>
#include <chrono>
#include <cmath>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr std::int32_t LCQI_I8_PATTERN_MODULUS = 23;
constexpr std::int32_t LCQI_I8_PATTERN_CENTER = 11;
constexpr std::int32_t LCQI_INPUT_PATTERN_MODULUS = 17;
constexpr std::int32_t LCQI_INPUT_PATTERN_CENTER = 8;
constexpr float LCQI_INPUT_PATTERN_SCALE = 0.125F;
constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
constexpr std::int32_t LCQI_SIMD_OUTPUT_BLOCK = 8;
constexpr std::size_t LCQI_BENCHMARK_BACKEND_COUNT = 3;
constexpr const char* LCQI_BACKEND_SCALAR = "scalar";
constexpr const char* LCQI_BACKEND_PACKED_SCALAR = "packed_scalar";
constexpr const char* LCQI_BACKEND_AVX2 = "avx2";
```

```cpp

void require_positive(std::int32_t value, const char* name) {
    if (value <= 0) {
        throw std::runtime_error(std::string(name) + " must be positive");
    }
}

std::size_t checked_size(std::int32_t value, const char* name) {
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
}

void validate_quantized_layer(const QuantizedLinearLayer& layer) {
    require_positive(layer.input_size, "input_size");
    require_positive(layer.output_size, "output_size");
    const std::size_t expected_weights =
        checked_size(layer.input_size, "input_size") *
        checked_size(layer.output_size, "output_size");
    if (layer.weights.size() != expected_weights) {
        throw std::runtime_error("quantized layer weight size mismatch");
    }
    if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("quantized layer bias size mismatch");
    }
}

std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
    require_positive(value, "value");
    require_positive(block_size, "block_size");
    return ((value + block_size - 1) / block_size) * block_size;
}

float checksum(std::span<const float> values) {
    float sum = 0.0F;
    for (const float value : values) {
        sum += value;
    }
    return sum;
}

void add_unavailable_backend(KernelBenchmarkResult& result, const char* name) {
    result.backends.push_back(KernelBackendBenchmark{
        .name = name,
        .available = false,
        .average_us = 0.0,
        .max_abs_diff = 0.0F,
        .checksum = 0.0F,
    });
}

```

```cpp
template <typename Callable>
double measure_average_us(std::int32_t repeat, Callable&& callable) {
    require_positive(repeat, "repeat");
    const auto begin = std::chrono::steady_clock::now();
    for (std::int32_t index = 0; index < repeat; ++index) {
        callable();
    }
    const auto end = std::chrono::steady_clock::now();
    const std::chrono::duration<double> elapsed = end - begin;
    return elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>(repeat);
}

}  // namespace

PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
                              std::int32_t output_block_size) {
    validate_quantized_layer(layer);
    require_positive(output_block_size, "output_block_size");

    PackedLinearI8 packed;
    packed.input_size = layer.input_size;
    packed.output_size = layer.output_size;
    packed.output_block_size = output_block_size;
    packed.scale = layer.scale;
    packed.bias = layer.bias;

    const std::int32_t padded_outputs =
        round_up_to_block(layer.output_size, output_block_size);
    packed.packed_weights.assign(
        checked_size(padded_outputs, "padded_outputs") *
            checked_size(layer.input_size, "input_size"),
        0);

    std::size_t packed_index = 0;
    for (std::int32_t output_block = 0; output_block < padded_outputs;
         output_block += output_block_size) {
        for (std::int32_t input_index = 0; input_index < layer.input_size; ++input_index) {
            for (std::int32_t offset = 0; offset < output_block_size; ++offset) {
                const std::int32_t output_index = output_block + offset;
                if (output_index < layer.output_size) {
                    const std::size_t source =
                        checked_size(output_index, "output_index") *
                            checked_size(layer.input_size, "input_size") +
                        checked_size(input_index, "input_index");
                    packed.packed_weights[packed_index] = layer.weights[source];
                }
                ++packed_index;
            }
```

```cpp
        }
    }
    return packed;
}

void linear_i8_packed(const PackedLinearI8& layer,
                      std::span<const float> input,
                      std::span<float> output) {
    std::vector<float> accumulators(
        checked_size(layer.output_block_size, "output_block_size"),
        0.0F);
    linear_i8_packed_with_workspace(layer, input, output, accumulators);
}

void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
                                     std::span<const float> input,
                                     std::span<float> output,
                                     std::span<float> accumulator_workspace) {
    require_positive(layer.input_size, "packed input_size");
    require_positive(layer.output_size, "packed output_size");
    require_positive(layer.output_block_size, "packed output_block_size");
    if (input.size() != checked_size(layer.input_size, "input_size")) {
        throw std::runtime_error("linear_i8_packed input size mismatch");
    }
    if (output.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("linear_i8_packed output size mismatch");
    }
    if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("linear_i8_packed bias size mismatch");
    }

    const std::int32_t padded_outputs =
        round_up_to_block(layer.output_size, layer.output_block_size);
    const std::size_t expected_packed_size =
        checked_size(padded_outputs, "padded_outputs") *
        checked_size(layer.input_size, "input_size");
    if (layer.packed_weights.size() != expected_packed_size) {
        throw std::runtime_error("linear_i8_packed packed weight size mismatch");
    }
    if (accumulator_workspace.size() != checked_size(layer.output_block_size, "output_block_size")) {
        throw std::runtime_error("linear_i8_packed accumulator workspace size mismatch");
    }

    std::size_t packed_index = 0;
    for (std::int32_t output_block = 0; output_block < padded_outputs;
         output_block += layer.output_block_size) {
        std::fill(accumulator_workspace.begin(), accumulator_workspace.end(), 0.0F);
        for (std::int32_t offset = 0; offset < layer.output_block_size; ++
```

```
offset) {
            const std::int32_t output_index = output_block + offset;
            if (output_index < layer.output_size) {
                accumulator_workspace[checked_size(offset, "offset")] =
                    layer.bias[checked_size(output_index, "output_index")];
            }
        }

        for (std::int32_t input_index = 0; input_index < layer.input_size; ++input_index) {
            const float input_value = input[checked_size(input_index, "input_index")];
            for (std::int32_t offset = 0; offset < layer.output_block_size; ++offset) {
                const std::int32_t output_index = output_block + offset;
                const std::int8_t weight = layer.packed_weights[packed_index++];
                if (output_index < layer.output_size) {
                    accumulator_workspace[checked_size(offset, "offset")] +=
                        static_cast<float>(weight) * layer.scale * input_value;
                }
            }
        }

        for (std::int32_t offset = 0; offset < layer.output_block_size; ++offset) {
            const std::int32_t output_index = output_block + offset;
            if (output_index < layer.output_size) {
                output[checked_size(output_index, "output_index")] =
                    accumulator_workspace[checked_size(offset, "offset")];
            }
        }
    }
}

QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
                                                  std::int32_t output_size,
                                                  float scale) {
    require_positive(input_size, "input_size");
    require_positive(output_size, "output_size");
    QuantizedLinearLayer layer;
    layer.input_size = input_size;
    layer.output_size = output_size;
    layer.scale = scale;
    const std::size_t weight_count =
        checked_size(input_size, "input_size") * checked_size(output_size, "output_size");
    layer.weights.resize(weight_count);
    layer.bias.resize(checked_size(output_size, "output_size"));
    for (std::int32_t output_index = 0; output_index < output_size; ++output_index) {
        layer.bias[checked_size(output_index, "output_index")] =
```

```cpp
            static_cast<float>((output_index % LCQI_INPUT_PATTERN_MODULUS) -
                               LCQI_INPUT_PATTERN_CENTER) *
            LCQI_INPUT_PATTERN_SCALE;
        for (std::int32_t input_index = 0; input_index < input_size; ++input_index) {
            const std::int32_t raw =
                ((output_index * input_size + input_index) % LCQI_I8_PATTERN_MODULUS) -
                LCQI_I8_PATTERN_CENTER;
            layer.weights[checked_size(output_index, "output_index") *
                              checked_size(input_size, "input_size") +
                          checked_size(input_index, "input_index")] =
                static_cast<std::int8_t>(raw);
        }
    }
    return layer;
}

std::vector<float> make_deterministic_input(std::int32_t input_size) {
    require_positive(input_size, "input_size");
    std::vector<float> input(checked_size(input_size, "input_size"));
    for (std::int32_t input_index = 0; input_index < input_size; ++input_index) {
        input[checked_size(input_index, "input_index")] =
            static_cast<float>((input_index % LCQI_INPUT_PATTERN_MODULUS) -
                               LCQI_INPUT_PATTERN_CENTER) *
            LCQI_INPUT_PATTERN_SCALE;
    }
    return input;
}

float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs) {
    if (lhs.size() != rhs.size()) {
        throw std::runtime_error("max_abs_diff size mismatch");
    }
    float diff = 0.0F;
    for (std::size_t index = 0; index < lhs.size(); ++index) {
        diff = std::max(diff, std::fabs(lhs[index] - rhs[index]));
    }
    return diff;
}

KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase& benchmark_case) {
    require_positive(benchmark_case.input_size, "benchmark input_size");
    require_positive(benchmark_case.output_size, "benchmark output_size");
    require_positive(benchmark_case.output_block_size, "benchmark output_block_size");
    require_positive(benchmark_case.repeat, "benchmark repeat");

    const QuantizedLinearLayer layer =
        make_deterministic_i8_layer(benchmark_case.input_size,
```

```cpp
                                    benchmark_case.output_size,
                                    0.015625F);
    const PackedLinearI8 packed = pack_linear_i8(layer, benchmark_case.output_block_size);
    const PackedLinearI8 packed_simd = pack_linear_i8(layer, LCQI_SIMD_OUTPUT_BLOCK);
    const std::vector<float> input = make_deterministic_input(benchmark_case.input_size);
    std::vector<float> scalar_output(checked_size(benchmark_case.output_size, "output_size"),
                                     0.0F);
    std::vector<float> packed_output(scalar_output.size(), 0.0F);
    std::vector<float> simd_output(scalar_output.size(), 0.0F);
    std::vector<float> packed_workspace(
        checked_size(benchmark_case.output_block_size, "output_block_size"),
        0.0F);

    linear_i8(layer, input, scalar_output);
    linear_i8_packed_with_workspace(packed, input, packed_output, packed_workspace);

    KernelBenchmarkResult result;
    result.benchmark_case = benchmark_case;
    result.backends.reserve(LCQI_BENCHMARK_BACKEND_COUNT);
    result.backends.push_back(KernelBackendBenchmark{
        .name = LCQI_BACKEND_SCALAR,
        .available = true,
        .average_us = measure_average_us(benchmark_case.repeat,
                                         [&]() { linear_i8(layer, input, scalar_output); }),
        .max_abs_diff = 0.0F,
        .checksum = checksum(scalar_output),
    });
    result.backends.push_back(KernelBackendBenchmark{
        .name = LCQI_BACKEND_PACKED_SCALAR,
        .available = true,
        .average_us = measure_average_us(
            benchmark_case.repeat,
            [&]() { linear_i8_packed_with_workspace(packed,
                                                    input,
                                                    packed_output,
                                                    packed_workspace); }),
        .max_abs_diff = max_abs_diff(scalar_output, packed_output),
        .checksum = checksum(packed_output),
    });
    if (linear_i8_packed_avx2_available()) {
        linear_i8_packed_avx2(packed_simd, input, simd_output);
        result.backends.push_back(KernelBackendBenchmark{
            .name = LCQI_BACKEND_AVX2,
            .available = true,
            .average_us = measure_average_us(
                benchmark_case.repeat,
```

```cpp
                [&]() { linear_i8_packed_avx2(packed_simd, input, simd_output); }),
            .max_abs_diff = max_abs_diff(scalar_output, simd_output),
            .checksum = checksum(simd_output),
        });
    } else {
        add_unavailable_backend(result, LCQI_BACKEND_AVX2);
    }
    return result;
}

}  // namespace lcqi

#if !defined(LCQI_ENABLE_AVX2)
namespace lcqi {

bool linear_i8_packed_avx2_available() noexcept {
    return false;
}

void linear_i8_packed_avx2(const PackedLinearI8&, std::span<const float>, std::span<float>) {
    throw std::runtime_error("AVX2 packed kernel is not enabled in this build");
}

}  // namespace lcqi
#endif
```

`int8_kernels_avx2.cpp` 是平台优化路径。读者要先看可用性判断，再看向量化实现；如果机器不支持 AVX2/FMA，报告必须写 unavailable。

Listing 4.19 src/int8_kernels_avx2.cpp

```cpp
#include <lcqi/int8_kernels.hpp>

#if defined(LCQI_ENABLE_AVX2)

#include <immintrin.h>

#include <algorithm>
#include <cstdint>
#include <cstring>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr std::int32_t LCQI_AVX2_OUTPUT_BLOCK = 8;

void require_positive(std::int32_t value, const char* name) {
    if (value <= 0) {
```

```
        throw std::runtime_error(std::string(name) + " must be positive");
    }
}

std::size_t checked_size(std::int32_t value, const char* name) {
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
}

std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
    require_positive(value, "value");
    require_positive(block_size, "block_size");
    return ((value + block_size - 1) / block_size) * block_size;
}

void validate_avx2_contract(const PackedLinearI8& layer,
                            std::span<const float> input,
                            std::span<float> output) {
    require_positive(layer.input_size, "packed input_size");
    require_positive(layer.output_size, "packed output_size");
    if (layer.output_block_size != LCQI_AVX2_OUTPUT_BLOCK) {
        throw std::runtime_error("AVX2 packed kernel requires output_block_size == 8");
    }
    if (input.size() != checked_size(layer.input_size, "input_size")) {
        throw std::runtime_error("AVX2 packed kernel input size mismatch");
    }
    if (output.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("AVX2 packed kernel output size mismatch");
    }
    if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("AVX2 packed kernel bias size mismatch");
    }
    const std::int32_t padded_outputs =
        round_up_to_block(layer.output_size, layer.output_block_size);
    const std::size_t expected_packed_size =
        checked_size(padded_outputs, "padded_outputs") *
        checked_size(layer.input_size, "input_size");
    if (layer.packed_weights.size() != expected_packed_size) {
        throw std::runtime_error("AVX2 packed kernel packed weight size mismatch");
    }
}

}  // namespace

bool linear_i8_packed_avx2_available() noexcept {
#if defined(__GNUC__) || defined(__clang__)
    __builtin_cpu_init();
    return __builtin_cpu_supports("avx2") && __builtin_cpu_supports("fma");
```

```
#else
    return true;
#endif
}

void linear_i8_packed_avx2(const PackedLinearI8& layer,
                           std::span<const float> input,
                           std::span<float> output) {
    if (!linear_i8_packed_avx2_available()) {
        throw std::runtime_error("AVX2/FMA is not available on this CPU");
    }
    validate_avx2_contract(layer, input, output);

    const std::int32_t padded_outputs =
        round_up_to_block(layer.output_size, layer.output_block_size);
    std::size_t packed_index = 0;
    alignas(32) float block_output[LCQI_AVX2_OUTPUT_BLOCK] = {};

    for (std::int32_t output_block = 0; output_block < padded_outputs;
         output_block += LCQI_AVX2_OUTPUT_BLOCK) {
        __m256 accumulator = _mm256_setzero_ps();

        for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset) {
            const std::int32_t output_index = output_block + offset;
            block_output[checked_size(offset, "offset")] =
                output_index < layer.output_size
                    ? layer.bias[checked_size(output_index, "output_index")]
                    : 0.0F;
        }
        accumulator = _mm256_load_ps(block_output);

        for (std::int32_t input_index = 0; input_index < layer.input_size; ++input_index) {
            std::uint64_t packed_bytes = 0;
            std::memcpy(&packed_bytes,
                        layer.packed_weights.data() + packed_index,
                        sizeof(packed_bytes));
            const __m128i packed_i8 =
                _mm_cvtsi64_si128(static_cast<long long>(packed_bytes));
            packed_index += LCQI_AVX2_OUTPUT_BLOCK;
            const __m256i weights_i32 = _mm256_cvtepi8_epi32(packed_i8);
            const __m256 weights_f32 = _mm256_cvtepi32_ps(weights_i32);
            const __m256 input_scaled =
                _mm256_set1_ps(input[checked_size(input_index, "input_index")] * layer.scale);
            accumulator = _mm256_fmadd_ps(weights_f32, input_scaled, accumulator);
        }

        _mm256_store_ps(block_output, accumulator);
        for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset
```

```
      ) {
            const std::int32_t output_index = output_block + offset;
            if (output_index < layer.output_size) {
                output[checked_size(output_index, "output_index")] =
                    block_output[checked_size(offset, "offset")];
            }
        }
    }
}

}  // namespace lcqi

#endif
```

第零部分

Transformer、KV Cache 与服务运行时

第 5 章实践：Transformer 切片与 KV Cache 状态

本章对应原书中的第三册“AI 推理原理、KV cache 与计算账本”和“KV cache 实现细节”。不要把它当作概念复述，本章要把 RMSNorm、Q/K/V、RoPE、GQA、KV slot、position、state edge 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `kv_cache`，核心命令或测试入口是 `lcqi_trace`。

Listing 5.1 第 5 章实践：Transformer 切片与 KV Cache 状态的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / kv_cache
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：固定 prompt `tok1tok2tok3`，观察第一个 decode step。读者要先手写预期结果，再运行项目观察输出。动作是：把 trace 中 tokenizer、q/k/v、`kv_cache_slot`、attention 和 logits 串成状态表。如果输出里没有 `token_position` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：KV cache 是请求状态，不是临时 activation；position、layer、head、layout 和生命周期必须进入 trace 这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 KV cache 状态表：每层每个 token 写入哪里、何时读取、何时禁止复用。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 6 章实践：Reference Decoder、logits 与采样

本章对应原书中的第三册“Reference decoder 实现”和“推理算法：logits、softmax、采样”。不要把它当作概念复述，本章要把 reference decoder、logits golden、argmax sampler、tokenizer contract hash、BOS/EOS 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `argmax`，核心命令或测试入口是 `lcqi_trace`。

Listing 6.1 第 6 章实践：Reference Decoder、logits 与采样的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / argmax
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：让 trace 固定输出 tokenizer contract 和 logits golden。读者要先手写预期结果，再运行项目观察输出。动作是：比较 generated token、logits 数组和 tokenizer hash。如果输出里没有 `tokenizer_contract` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：采样前必须先证明 logits 可复查；sampler 的随机性、温度和 top-k 只能建立在 reference 通过之后这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交采样报告：argmax、top-k、temperature 三种策略各自需要哪些额外字段才能复现。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 7 章实践：服务调度、SLO 与资源预算

本章对应原书中的第三册“业务服务实战：请求、调度、队列、取消与资源预算”。不要把它当作概念复述，本章要把 admission、KV budget、queue wait、TTFT、TPOT、cancel、reject、p95 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `ttft_ms`，核心命令或测试入口是 `lcqi_serving_probe`。

Listing 7.1 第 7 章实践：服务调度、SLO 与资源预算的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_serving_probe / ttft_ms
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：运行 serving probe 中短请求、RAG 请求、取消请求和超长请求。读者要先手写预期结果，再运行项目观察输出。动作是：解释 `memory_budget_exceeded`、cancel reclaim、`queue_wait_p95_ms` 和 `rejection_rate`。如果输出里没有 `rejection_rate` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：推理服务不是只看 tokens/s；是否接收请求、如何预留 KV、取消后是否回收，决定系统是否可运营这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 SLO 表：每类请求的 prompt/new token、预算、是否接受、TTFT、TPOT、拒绝原因和未验证边界。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 8 章实践：RAG、Agent 与状态图边界

本章对应原书中的第三册“上层 AI 应用编排：RAG、Agent、工具和状态图”。不要把它当作概念复述，本章要把 retrieval、tool call、prompt assembly、state graph、budget、trace id 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `req_rag`，核心命令或测试入口是 `lcqi_serving_probe`。

Listing 8.1 第 8 章实践：RAG、Agent 与状态图边界的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_serving_probe / req_rag
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把 RAG 请求拆成 retrieve、rerank、prompt build、prefill、decode 五段。读者要先手写预期结果，再运行项目观察输出。动作是：为每段写输入输出和失败后能否重试。如果输出里没有 `prompt_tokens` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：上层编排不能把外部工具结果和模型状态混成字符串；每个边界都要有身份、预算和可重放记录这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一个 RAG 状态图：节点、边、超时、重试、缓存键、prompt 版本和审计字段。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第零部分

模型导入、AI 编译器与内存计划

第 9 章实践：模型导入、manifest 与 Shape Verifier

本章对应原书中的第三册“模型导入、manifest 与 shape verifier”。不要把它当作概念复述，本章要把 safetensors header、metadata、tensor name、dtype、shape、`data_offsets`、payload boundary 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `load_safetensors_manifest`，核心命令或测试入口是 `lcqi_tests`。

Listing 9.1 第 9 章实践：模型导入、manifest 与 Shape Verifier 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / load_safetensors_manifest
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：加载测试临时 safetensors fixture。读者要先手写预期结果，再运行项目观察输出。动作是：故意构造 offset 越界和 dtype/shape 字节数不匹配。如果输出里没有 `data_offsets` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：模型导入的第一职责是早拒绝：名字、shape、dtype、offset 和 payload 都必须在 kernel 运行前被验证这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 manifest gate 报告：合法 fixture 和两个坏 fixture 的字段、错误原因和拒绝位置。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 10 章实践：真实模型加载闭环与首 Token Trace

本章对应原书中的第三册“真实模型加载闭环：Tokenizer、权重格式与首 token trace”。不要把它当作概念复述，本章要把 tokenizer、weight mapping、chat template、first token trace、checksum 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

本章新增一个明确边界：tiny fixture 负责手算和逐层 oracle；LCQI GPT-2 reference path 负责证明自研代码已经能加载标准 GPT-2 safetensors、执行 byte-level BPE、跑 GPT-2 forward 并 greedy 生成；SmolLM2 small smoke 负责证明外部 llama.cpp 路径能加载 GGUF small 模型。三者不能互相冒充。tiny MLP 算对，不等于 GPT-2 完成；GPT-2 reference 能出一个 token，不等于高性能 KV cache runtime 完成；SmolLM2 能输出文本，也不等于 LCQI 自研 GGUF 后端完成。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里有三组核心对象。第一组是 `tiny_tokenizer.txt`、reference decoder 和 `lcqi_trace`，用于手算 token 合同和逐 checkpoint 追踪。第二组是 `gpt2_reference.hpp/.cpp`、`lcqi_gpt2` 和 `tools/run_gpt2_smoke.py`，用于自研 GPT-2 safetensors 路径。第三组是 `tools/run_smollm2_small_smoke.py`，用于下载并校验 SmolLM2-135M-Instruct 的 GGUF 量化文件，再通过 llama.cpp 在 CPU 上实际生成 assistant 文本。

Listing 10.1 第 10 章实践：真实模型加载闭环与首 Token Trace 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / tiny_tokenizer.txt
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_gpt2
make cpu3-gpt2-smoke
make cpu3-smollm2-smoke
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：从 tokenizer fixture 开始，不跳到真实 7B。读者要先手写预期结果，再运行项目观察输出。动作是：解释 BOS/EOS、UNK、template hash 和 decoder 输入 tokens 的边界。如果输出里没有 `tokens=[0,1,2,3,4]` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：真实模型加载不是下载权重；必须先让 tokenizer 合同、权重 role、shape verifier 和首 token trace 形成闭环。这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、自研 GPT-2 reference smoke：先把标准 safetensors 跑通

GPT-2 不是 LLaMA。它使用 LayerNorm，不是 RMSNorm；使用 learned absolute position embedding，不是 RoPE；attention 里的 Q/K/V 在 HuggingFace 权重中是 packed Conv1D 权重；MLP 使用 GELU，不是 SwiGLU；`lm_head` 通常和 token embedding 共享权重。把 GPT-2 硬塞进原来的 LLaMA-like tiny decoder，会让概念混乱，所以 LCQI 新增了独立的 `gpt2_reference` 模块。

仓库内最小验收分两层。第一层不下载模型，运行 tiny GPT-2 fixture：

Listing 10.2 仓库内 tiny GPT-2 数学闭环

```
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_gpt2
```

它固定 prompt ids 为 `123`，检查 logits、predicted token 和 greedy generated ids。对应测试还会临时写出一个 HuggingFace 风格 `model.safetensors`，再用 loader 读回来，覆盖 tensor name mapping、shape verifier、Conv1D 转置和 tied lm head。坏 shape 必须在 loader 阶段被拒绝。

第二层下载真实 GPT-2 资产：

Listing 10.3 LCQI 自研 GPT-2 smoke

```
make cpu3-gpt2-smoke
```

脚本下载 `openai-community/gpt2` 的 `config.json`、`vocab.json`、`merges.txt` 和 `model.safetensors` 到用户缓存目录，不提交模型文件。最近一次报告写到：

Listing 10.4 GPT-2 smoke 报告关键字段

```
books/cpu-volume-3/results/lcqi-gpt2-smoke.txt
prompt_ids 15496 11 616 1438 318
generated_ids 15496 11 616 1438 318 1757
generated_text Hello, my name is John
validation=PASS
```

这说明自研 LCQI C++ reference path 已经能跑标准 GPT-2 safetensors 的 tokenizer、权重导入、forward 和 greedy generation。边界也要写清：当前实现为了可读和可验证，会重跑完整前缀，没有 KV cache、没有 packed weight、没有多线程，也没有把 logits 与 PyTorch/Transformers 的逐数值 golden 报告纳入仓库。因此它是正确性闭环，不是性能结论。

四、真实 small 模型 smoke：不用 tiny，先让开源模型实际跑起来

本章的真实模型 smoke 固定为 `SmolLM2-135M-Instruct`，规模是 135M 参数，不是 tiny 随机夹具。脚本使用 GGUF 文件 `SmolLM2-135M-Instruct-Q4_K_M.gguf`，来源仓库是 `bartowski/SmolLM2-135M-Instruct-GGUF`，原模型组织是 `HuggingFaceTB`，license 字段为 `apache-2.0`。脚本不会把模型文件提交到仓库，而是缓存到用户目录；验收时检查文件大小 `105454432` 字节和 SHA256：

Listing 10.5 SmolLM2 small GGUF 文件身份

```
model_source_id=HuggingFaceTB/SmolLM2-135M-Instruct
model_gguf_repo_id=bartowski/SmolLM2-135M-Instruct-GGUF
model_file=SmolLM2-135M-Instruct-Q4_K_M.gguf
model_expected_bytes=105454432
model_expected_sha256=2↩
    ↪ e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d
```

运行命令是：

Listing 10.6 真实 small 模型 CPU 冒烟

```
make cpu3-smollm2-smoke
```

这条命令会做四步。第一，下载或复用 GGUF 文件，并校验 SHA256，防止“文件名对了但内容不对”。第二，拉取固定 commit 的 `llama.cpp` 并构建 `llama-simple-chat`，避免依赖本机临时安装。第三，用 `-ngl0` 强制 CPU 路径，输入一个短 prompt，确认程序能走过 tokenizer、chat template、prefill、decode 和 sampler。第四，把报告写到：

Listing 10.7 真实模型 smoke 报告路径

```
books/cpu-volume-3/results/lcqi-smollm2-small-model-smoke.txt
```

报告里必须至少出现这些字段：`model_expected_sha256`、`llama_cpp_commit`、`command`、`exit_code=0`、`validation=PASS` 和 `assistant_response`。如果只有下载日志，没有 assistant response，不能算通过；如果 assistant response 内容答错了，也不能改写成“质量通过”。本章只验证真实模型资产可加载、可 tokenize、可 decode、可生成非空文本。

五、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：运行 GPT-2 smoke。报告不许只贴“生成了 John”，必须解释 `config.json`、`vocab.json`、`merges.txt`、`model.safetensors` 各自对应哪段代码，为什么 GPT-2 需要 byte-level BPE，为什么 HuggingFace Conv1D 权重要转置，为什么 `lm_head` 可以 tied 到 embedding。

任务四：运行真实 small 模型 smoke。报告不许只贴“命令执行成功”，必须解释脚本为什么校验模型 hash、为什么固定 `llama.cpp` commit、为什么使用 `-ngl0`，以及为什么 assistant response 非空只是 smoke，不是质量评测。

任务五：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。GPT-2 反例可以是：tensor name 缺失、shape 和 config 不一致、vocab 中缺少 BPE merge 后 token、`max_new_tokens` 超过 context。真实模型 smoke 的反例可以是：模型 SHA256 不匹配、网络不可用、`cmake` 缺失、assistant response 为空、命令超时。每个反例都要写清期望处理方式。

六、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。新增 GPT-2 smoke 后，最低验收还包括：真实 GPT-2 文件身份有 hash，prompt ids 可解释，generated ids 可解释，文本反解码可解释，并明确声明这不是高性能推理结论。新增 SmolLM2 smoke 后，最低验收还包括：模型身份有 hash，模型规模不是 tiny，CPU 路径实际运行，报告里有非空 assistant response，并明确声明这不是自研 LCQI 完整 GGUF 支持。

递进大作业：提交 GPT-2 reference path 的下一阶段迁移清单：KV cache、prefill/decode 分离、packed weight、线程并行、Transformers golden logits、更多模型方言 name mapping、分片 safetensors 和性能报告。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

代码走读：Reference Decoder、Tokenizer、Safetensors 与 GPT-2

从 tiny MLP 走向 decoder-only 模型

Reference decoder 和 GPT-2 reference path 是 LCQI 从教学闭环走向真实模型导入的关键层。这里的代码更长，是因为真实模型不只有矩阵乘法：它还需要 tokenizer、位置、归一化、QKV 拆分、attention mask、KV cache、激活函数、lm head、权重命名映射、dtype 转换和坏资产拒绝。

Tiny Reference Decoder

`reference_decoder.cpp` 实现 embedding、RMSNorm、Q/K/V、RoPE、KV cache、attention、SwiGLU、final norm、logits 和 sampler，并在关键位置生成 trace checkpoint。读者应按 checkpoint 顺序读，而不是从最终 logits 倒推。

Listing 10.8 src/reference_decoder.cpp

```cpp
#include <lcqi/reference_decoder.hpp>

#include <algorithm>
#include <array>
#include <cmath>
#include <limits>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr float LCQI_SOFTMAX_NEGATIVE_INFINITY = -std::numeric_limits<float>::↩
    ↪ infinity();
constexpr std::int32_t LCQI_ROPE_PAIR_WIDTH = 2;
constexpr const char* LCQI_TRACE_DTYPE_F32 = "f32";
constexpr const char* LCQI_TRACE_LAYOUT_CONTIGUOUS = "contiguous";
constexpr const char* LCQI_TRACE_LAYOUT_ROW_MAJOR = "row_major";
constexpr const char* LCQI_TRACE_LAYOUT_KV_CONTIGUOUS = "kv_contiguous";

void require_positive(std::int32_t value, const char* name) {
    if (value <= 0) {
        throw std::runtime_error(std::string(name) + " must be positive");
    }
}

void validate_config(const DecoderConfig& config) {
    require_positive(config.hidden_size, "hidden_size");
    require_positive(config.query_heads, "query_heads");
    require_positive(config.kv_heads, "kv_heads");
    require_positive(config.head_dim, "head_dim");
    require_positive(config.intermediate_size, "intermediate_size");
    require_positive(config.vocab_size, "vocab_size");
```

```cpp
    require_positive(config.max_context, "max_context");
    if (config.query_heads % config.kv_heads != 0) {
        throw std::runtime_error("query_heads must be divisible by kv_heads");
    }
    if (config.hidden_size != config.query_heads * config.head_dim) {
        throw std::runtime_error("hidden_size must equal query_heads * head_dim");
    }
    if (config.head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
        throw std::runtime_error("head_dim must be even for RoPE");
    }
    if (config.rope_base <= 1.0F) {
        throw std::runtime_error("rope_base must be greater than 1");
    }
}

std::size_t checked_size(std::int32_t value, const char* name) {
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
}

std::int32_t kv_width(const DecoderConfig& config) {
    return config.kv_heads * config.head_dim;
}

void validate_linear(const LinearWeightsF32& layer, const char* name) {
    require_positive(layer.input_size, "linear input_size");
    require_positive(layer.output_size, "linear output_size");
    const std::size_t expected_weights =
        checked_size(layer.input_size, name) * checked_size(layer.output_size, name);
    if (layer.weights.size() != expected_weights) {
        throw std::runtime_error(std::string(name) + " weight size mismatch");
    }
    if (!layer.bias.empty() &&
        layer.bias.size() != checked_size(layer.output_size, name)) {
        throw std::runtime_error(std::string(name) + " bias size mismatch");
    }
}

void validate_span_size(std::size_t actual, std::int32_t expected, const char* name) {
    if (actual != checked_size(expected, name)) {
        throw std::runtime_error(std::string(name) + " size mismatch");
    }
}

std::int32_t argmax(std::span<const float> values) {
    if (values.empty()) {
        throw std::runtime_error("cannot take argmax of empty values");
```

```cpp
    }
    std::int32_t best_index = 0;
    float best_value = values[0];
    for (std::int32_t index = 1; index < static_cast<std::int32_t>(values.size()); ++index) {
        const float candidate = values[static_cast<std::size_t>(index)];
        if (candidate > best_value) {
            best_value = candidate;
            best_index = index;
        }
    }
    return best_index;
}

float silu(float value) {
    return value / (1.0F + std::exp(-value));
}

std::span<const float> row_span(std::span<const float> values,
                                std::int32_t row,
                                std::int32_t row_width) {
    const std::size_t begin = checked_size(row, "row") * checked_size(row_width, "row_width");
    return values.subspan(begin, checked_size(row_width, "row_width"));
}

void add_inplace(std::span<float> target, std::span<const float> source) {
    if (target.size() != source.size()) {
        throw std::runtime_error("residual add size mismatch");
    }
    for (std::size_t index = 0; index < target.size(); ++index) {
        target[index] += source[index];
    }
}

void swiglu_mlp(const DecoderLayerWeightsF32& layer,
                std::span<const float> input,
                std::span<float> output) {
    std::vector<float> gate(checked_size(layer.w_gate.output_size, "gate"), 0.0F);
    std::vector<float> up(checked_size(layer.w_up.output_size, "up"), 0.0F);
    std::vector<float> hidden(gate.size(), 0.0F);
    linear_f32(layer.w_gate, input, gate);
    linear_f32(layer.w_up, input, up);
    if (gate.size() != up.size()) {
        throw std::runtime_error("SwiGLU gate/up size mismatch");
    }
    for (std::size_t index = 0; index < hidden.size(); ++index) {
        hidden[index] = silu(gate[index]) * up[index];
    }
    linear_f32(layer.w_down, hidden, output);
}
```

```cpp

float checksum(std::span<const float> values) {
    float sum = 0.0F;
    for (const float value : values) {
        sum += value;
    }
    return sum;
}

float max_abs(std::span<const float> values) {
    float result = 0.0F;
    for (const float value : values) {
        result = std::max(result, std::fabs(value));
    }
    return result;
}

std::vector<std::int32_t> contiguous_stride(std::span<const std::int32_t> shape) {
    std::vector<std::int32_t> stride(shape.size(), 1);
    std::int32_t running = 1;
    for (std::int32_t index = static_cast<std::int32_t>(shape.size()) - 1; index >= 0; --index) {
        stride[checked_size(index, "stride index")] = running;
        running *= shape[checked_size(index, "shape index")];
    }
    return stride;
}

void add_checkpoint(std::vector<ReferenceTensorCheckpoint>* checkpoints,
                    const char* name,
                    std::int32_t token_position,
                    std::int32_t layer_id,
                    std::span<const std::int32_t> shape,
                    const char* layout,
                    std::span<const float> values) {
    if (checkpoints == nullptr) {
        return;
    }
    ReferenceTensorCheckpoint checkpoint;
    checkpoint.name = name;
    checkpoint.token_position = token_position;
    checkpoint.layer_id = layer_id;
    checkpoint.shape.assign(shape.begin(), shape.end());
    checkpoint.stride = contiguous_stride(shape);
    checkpoint.dtype = LCQI_TRACE_DTYPE_F32;
    checkpoint.layout = layout;
    checkpoint.checksum = checksum(values);
    checkpoint.max_abs = max_abs(values);
    checkpoint.values.assign(values.begin(), values.end());
    checkpoints->push_back(std::move(checkpoint));
}
```

```cpp

void swiglu_mlp_with_trace(const DecoderLayerWeightsF32& layer,
                           std::span<const float> input,
                           std::span<float> output,
                           std::int32_t token_position,
                           std::int32_t layer_id,
                           std::vector<ReferenceTensorCheckpoint>* checkpoints) {
    std::vector<float> gate(checked_size(layer.w_gate.output_size, "gate"), 0.0F);
    std::vector<float> up(checked_size(layer.w_up.output_size, "up"), 0.0F);
    std::vector<float> hidden(gate.size(), 0.0F);
    linear_f32(layer.w_gate, input, gate);
    linear_f32(layer.w_up, input, up);
    if (gate.size() != up.size()) {
        throw std::runtime_error("SwiGLU gate/up size mismatch");
    }
    const std::array<std::int32_t, 1> intermediate_shape{layer.w_gate.output_size};
    add_checkpoint(checkpoints,
                   "mlp_gate",
                   token_position,
                   layer_id,
                   intermediate_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   gate);
    add_checkpoint(checkpoints,
                   "mlp_up",
                   token_position,
                   layer_id,
                   intermediate_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   up);
    for (std::size_t index = 0; index < hidden.size(); ++index) {
        hidden[index] = silu(gate[index]) * up[index];
    }
    add_checkpoint(checkpoints,
                   "mlp_hidden",
                   token_position,
                   layer_id,
                   intermediate_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   hidden);
    linear_f32(layer.w_down, hidden, output);
}

void forward_layer(const DecoderConfig& config,
                   const DecoderLayerWeightsF32& layer,
                   std::int32_t layer_id,
                   std::int32_t model_position,
                   ReferenceKVCache& cache,
                   std::span<float> hidden,
```

```
                   std::vector<ReferenceTensorCheckpoint>* checkpoints = nullptr) {
    validate_span_size(hidden.size(), config.hidden_size, "hidden");

    std::vector<float> normed(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> query(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> key(checked_size(kv_width(config), "kv_width"), 0.0F);
    std::vector<float> value(checked_size(kv_width(config), "kv_width"), 0.0F);
    std::vector<float> attention(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> attention_projected(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> mlp(checked_size(config.hidden_size, "hidden_size"), 0.0F);

    const std::array<std::int32_t, 1> hidden_shape{config.hidden_size};
    const std::array<std::int32_t, 2> query_shape{config.query_heads, config.head_dim};
    const std::array<std::int32_t, 2> kv_shape{config.kv_heads, config.head_dim};
    const std::array<std::int32_t, 4> kv_slot_shape{
        1, 1, config.kv_heads, config.head_dim};

    rms_norm(hidden, layer.rms_attention_weight, config.rms_epsilon, normed);
    add_checkpoint(checkpoints,
                   "attn_norm",
                   model_position,
                   layer_id,
                   hidden_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   normed);
    linear_f32(layer.wq, normed, query);
    linear_f32(layer.wk, normed, key);
    linear_f32(layer.wv, normed, value);
    add_checkpoint(checkpoints,
                   "q_before_rope",
                   model_position,
                   layer_id,
                   query_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   query);
    add_checkpoint(checkpoints,
                   "k_before_rope",
                   model_position,
                   layer_id,
                   kv_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   key);
    add_checkpoint(checkpoints,
                   "v",
```

```
                   model_position,
                   layer_id,
                   kv_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   value);
    apply_rope(query, config.query_heads, config.head_dim, model_position, config.rope_base);
    apply_rope(key, config.kv_heads, config.head_dim, model_position, config.rope_base);
    add_checkpoint(checkpoints,
                   "q_after_rope",
                   model_position,
                   layer_id,
                   query_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   query);
    add_checkpoint(checkpoints,
                   "k_after_rope",
                   model_position,
                   layer_id,
                   kv_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   key);

    cache.append(layer_id, model_position, key, value);
    add_checkpoint(checkpoints,
                   "kv_cache_key_slot",
                   model_position,
                   layer_id,
                   kv_slot_shape,
                   LCQI_TRACE_LAYOUT_KV_CONTIGUOUS,
                   key);
    attention_decode(config, cache, layer_id, model_position, query, attention);
    add_checkpoint(checkpoints,
                   "attention_out",
                   model_position,
                   layer_id,
                   hidden_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   attention);
    linear_f32(layer.wo, attention, attention_projected);
    add_inplace(hidden, attention_projected);
    add_checkpoint(checkpoints,
                   "post_attention_residual",
                   model_position,
                   layer_id,
                   hidden_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   hidden);

    rms_norm(hidden, layer.rms_mlp_weight, config.rms_epsilon, normed);
```

```cpp
    add_checkpoint(checkpoints,
                   "mlp_norm",
                   model_position,
                   layer_id,
                   hidden_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   normed);
    if (checkpoints == nullptr) {
        swiglu_mlp(layer, normed, mlp);
    } else {
        swiglu_mlp_with_trace(layer, normed, mlp, model_position, layer_id, checkpoints);
    }
    add_inplace(hidden, mlp);
    add_checkpoint(checkpoints,
                   "layer_out",
                   model_position,
                   layer_id,
                   hidden_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   hidden);
}

LinearWeightsF32 make_linear(std::int32_t input_size,
                             std::int32_t output_size,
                             std::vector<float> weights,
                             std::vector<float> bias = {}) {
    LinearWeightsF32 layer;
    layer.input_size = input_size;
    layer.output_size = output_size;
    layer.weights = std::move(weights);
    layer.bias = std::move(bias);
    validate_linear(layer, "tiny linear");
    return layer;
}

std::vector<float> zeros(std::int32_t count) {
    return std::vector<float>(checked_size(count, "zero count"), 0.0F);
}

}  // namespace

ReferenceDecodeTraceResult run_reference_decode_with_trace(
    const ReferenceDecoderModel& model,
    std::span<const std::int32_t> token_ids) {
    validate_config(model.config);
    if (token_ids.empty()) {
        throw std::runtime_error("reference decode needs at least one token");
    }
    if (token_ids.size() > checked_size(model.config.max_context, "max_context"))) {
        throw std::runtime_error("token_ids exceed max_context");
```

```cpp
    }
    if (model.token_embedding.size() !=
        checked_size(model.config.vocab_size, "vocab_size") *
            checked_size(model.config.hidden_size, "hidden_size")) {
        throw std::runtime_error("token embedding size mismatch");
    }
    if (model.final_norm_weight.size() != checked_size(model.config.hidden_size, "hidden_size")) {
        throw std::runtime_error("final norm weight size mismatch");
    }
    validate_linear(model.lm_head, "lm_head");

    ReferenceDecodeTraceResult trace;
    ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers.size()));
    std::vector<float> hidden(checked_size(model.config.hidden_size, "hidden_size"), 0.0F);
    const std::array<std::int32_t, 1> hidden_shape{model.config.hidden_size};
    for (std::int32_t position = 0; position < static_cast<std::int32_t>(token_ids.size());
         ++position) {
        const std::int32_t token_id = token_ids[checked_size(position, "position")];
        if (token_id < 0 || token_id >= model.config.vocab_size) {
            throw std::runtime_error("token id out of vocabulary");
        }
        const std::span<const float> embedding =
            row_span(model.token_embedding, token_id, model.config.hidden_size);
        std::copy(embedding.begin(), embedding.end(), hidden.begin());
        add_checkpoint(&trace.checkpoints,
                       "embedding",
                       position,
                       -1,
                       hidden_shape,
                       LCQI_TRACE_LAYOUT_ROW_MAJOR,
                       hidden);
        for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(model.layers.size());
             ++layer_id) {
            forward_layer(model.config,
                          model.layers[checked_size(layer_id, "layer_id")],
                          layer_id,
                          position,
                          cache,
                          hidden,
                          &trace.checkpoints);
        }
    }

    std::vector<float> normed(checked_size(model.config.hidden_size, "hidden_size"), 0.0F);
```

```cpp
    std::vector<float> logits(checked_size(model.config.vocab_size, "vocab_size"), 0.0F);
    const std::array<std::int32_t, 1> logits_shape{model.config.vocab_size};
    const std::int32_t last_position = static_cast<std::int32_t>(token_ids.size()) - 1;
    rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed);
    add_checkpoint(&trace.checkpoints,
                   "final_norm",
                   last_position,
                   -1,
                   hidden_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   normed);
    linear_f32(model.lm_head, normed, logits);
    add_checkpoint(&trace.checkpoints,
                   "logits",
                   last_position,
                   -1,
                   logits_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   logits);

    trace.result.predicted_token = argmax(logits);
    trace.result.logits = std::move(logits);
    return trace;
}

ReferenceKVCache::ReferenceKVCache(const DecoderConfig& config, std::int32_t layer_count)
    : config_(config),
      layer_count_(layer_count) {
    validate_config(this->config_);
    require_positive(this->layer_count_, "layer_count");
    const std::size_t element_count =
        checked_size(this->layer_count_, "layer_count") *
        checked_size(this->config_.max_context, "max_context") *
        checked_size(this->config_.kv_heads, "kv_heads") *
        checked_size(this->config_.head_dim, "head_dim");
    this->keys_.assign(element_count, 0.0F);
    this->values_.assign(element_count, 0.0F);
    this->written_.assign(
        checked_size(this->layer_count_, "layer_count") *
            checked_size(this->config_.max_context, "max_context"),
        0);
}

void ReferenceKVCache::append(std::int32_t layer_id,
                              std::int32_t model_position,
                              std::span<const float> key,
                              std::span<const float> value) {
    validate_span_size(key.size(), kv_width(this->config_), "KV key");
```

```cpp
    validate_span_size(value.size(), kv_width(this->config_), "KV value");
    this->validate_address(layer_id, model_position, 0);
    for (std::int32_t kv_head = 0; kv_head < this->config_.kv_heads; ++kv_head) {
        const std::size_t target = this->base_offset(layer_id, model_position, kv_head);
        const std::size_t source =
            checked_size(kv_head, "kv_head") * checked_size(this->config_.head_dim, "head_dim");
        std::copy_n(key.begin() + static_cast<std::ptrdiff_t>(source),
                    this->config_.head_dim,
                    this->keys_.begin() + static_cast<std::ptrdiff_t>(target));
        std::copy_n(value.begin() + static_cast<std::ptrdiff_t>(source),
                    this->config_.head_dim,
                    this->values_.begin() + static_cast<std::ptrdiff_t>(target));
    }
    const std::size_t written_index =
        checked_size(layer_id, "layer_id") * checked_size(this->config_.max_context, "max_context") +
        checked_size(model_position, "model_position");
    this->written_[written_index] = 1;
    this->filled_tokens_ = std::max(this->filled_tokens_, model_position + 1);
}

std::span<const float> ReferenceKVCache::key(std::int32_t layer_id,
                                             std::int32_t model_position,
                                             std::int32_t kv_head) const {
    this->validate_address(layer_id, model_position, kv_head);
    const std::size_t offset = this->base_offset(layer_id, model_position, kv_head);
    return std::span<const float>(this->keys_.data() + offset,
                                  checked_size(this->config_.head_dim, "head_dim"));
}

std::span<const float> ReferenceKVCache::value(std::int32_t layer_id,
                                               std::int32_t model_position,
                                               std::int32_t kv_head) const {
    this->validate_address(layer_id, model_position, kv_head);
    const std::size_t offset = this->base_offset(layer_id, model_position, kv_head);
    return std::span<const float>(this->values_.data() + offset,
                                  checked_size(this->config_.head_dim, "head_dim"));
}

std::int32_t ReferenceKVCache::layer_count() const noexcept {
    return this->layer_count_;
}

std::int32_t ReferenceKVCache::filled_tokens() const noexcept {
```

```cpp
    return this->filled_tokens_;
}

std::size_t ReferenceKVCache::base_offset(std::int32_t layer_id,
                                          std::int32_t model_position,
                                          std::int32_t kv_head) const {
    return (((checked_size(layer_id, "layer_id") *
              checked_size(this->config_.max_context, "max_context")) +
             checked_size(model_position, "model_position")) *
                checked_size(this->config_.kv_heads, "kv_heads") +
            checked_size(kv_head, "kv_head")) *
           checked_size(this->config_.head_dim, "head_dim");
}

void ReferenceKVCache::validate_address(std::int32_t layer_id,
                                        std::int32_t model_position,
                                        std::int32_t kv_head) const {
    if (layer_id < 0 || layer_id >= this->layer_count_) {
        throw std::runtime_error("KV layer_id out of range");
    }
    if (model_position < 0 || model_position >= this->config_.max_context) {
        throw std::runtime_error("KV model_position out of range");
    }
    if (kv_head < 0 || kv_head >= this->config_.kv_heads) {
        throw std::runtime_error("KV head out of range");
    }
}

void rms_norm(std::span<const float> input,
              std::span<const float> weight,
              float epsilon,
              std::span<float> output) {
    if (input.size() != weight.size() || input.size() != output.size()) {
        throw std::runtime_error("RMSNorm size mismatch");
    }
    if (input.empty()) {
        throw std::runtime_error("RMSNorm input is empty");
    }
    float sum_square = 0.0F;
    for (const float value : input) {
        sum_square += value * value;
    }
    const float mean_square = sum_square / static_cast<float>(input.size());
    const float scale = 1.0F / std::sqrt(mean_square + epsilon);
    for (std::size_t index = 0; index < input.size(); ++index) {
        output[index] = input[index] * scale * weight[index];
    }
}

void linear_f32(const LinearWeightsF32& layer,
                std::span<const float> input,
                std::span<float> output) {
```

```
    validate_linear(layer, "linear_f32");
    validate_span_size(input.size(), layer.input_size, "linear input");
    validate_span_size(output.size(), layer.output_size, "linear output");
    for (std::int32_t out = 0; out < layer.output_size; ++out) {
        const std::size_t row_base =
            checked_size(out, "out") * checked_size(layer.input_size, "input_size");
        float accumulator = layer.bias.empty() ? 0.0F : layer.bias[checked_size(out, "out")];
        for (std::int32_t in = 0; in < layer.input_size; ++in) {
            accumulator += layer.weights[row_base + checked_size(in, "in")] *
                           input[checked_size(in, "in")];
        }
        output[checked_size(out, "out")] = accumulator;
    }
}

void apply_rope(std::span<float> heads,
                std::int32_t head_count,
                std::int32_t head_dim,
                std::int32_t model_position,
                float rope_base) {
    require_positive(head_count, "head_count");
    require_positive(head_dim, "head_dim");
    validate_span_size(heads.size(), head_count * head_dim, "RoPE heads");
    if (head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
        throw std::runtime_error("RoPE head_dim must be even");
    }
    if (model_position < 0) {
        throw std::runtime_error("RoPE model_position cannot be negative");
    }
    if (rope_base <= 1.0F) {
        throw std::runtime_error("RoPE base must be greater than 1");
    }
    for (std::int32_t head = 0; head < head_count; ++head) {
        for (std::int32_t offset = 0; offset < head_dim; offset += LCQI_ROPE_PAIR_WIDTH) {
            const float exponent = static_cast<float>(offset) / static_cast<float>(head_dim);
            const float theta = 1.0F / std::pow(rope_base, exponent);
            const float angle = static_cast<float>(model_position) * theta;
            const float cosine = std::cos(angle);
            const float sine = std::sin(angle);
            const std::size_t first =
                checked_size(head, "head") * checked_size(head_dim, "head_dim") +
                checked_size(offset, "offset");
            const float x0 = heads[first];
            const float x1 = heads[first + 1];
            heads[first] = x0 * cosine - x1 * sine;
            heads[first + 1] = x0 * sine + x1 * cosine;
        }
```

```
    }
}

void attention_decode(const DecoderConfig& config,
                      const ReferenceKVCache& cache,
                      std::int32_t layer_id,
                      std::int32_t model_position,
                      std::span<const float> query,
                      std::span<float> output) {
    validate_config(config);
    validate_span_size(query.size(), config.hidden_size, "attention query");
    validate_span_size(output.size(), config.hidden_size, "attention output");
    if (model_position < 0 || model_position >= cache.filled_tokens()) {
        throw std::runtime_error("attention model_position has not been written");
    }
    std::fill(output.begin(), output.end(), 0.0F);

    const std::int32_t group_size = config.query_heads / config.kv_heads;
    const float score_scale = 1.0F / std::sqrt(static_cast<float>(config.head_dim));
    std::vector<float> scores(checked_size(model_position + 1, "score count"),
                              LCQI_SOFTMAX_NEGATIVE_INFINITY);
    std::vector<float> probabilities(scores.size(), 0.0F);

    for (std::int32_t query_head = 0; query_head < config.query_heads; ++query_head) {
        const std::int32_t kv_head = query_head / group_size;
        const std::size_t query_base =
            checked_size(query_head, "query_head") * checked_size(config.head_dim, "head_dim");
        float max_score = LCQI_SOFTMAX_NEGATIVE_INFINITY;
        for (std::int32_t position = 0; position <= model_position; ++position) {
            const std::span<const float> key = cache.key(layer_id, position, kv_head);
            float dot = 0.0F;
            for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
                dot += query[query_base + checked_size(dim, "dim")] *
                       key[checked_size(dim, "dim")];
            }
            const float score = dot * score_scale;
            scores[checked_size(position, "position")] = score;
            max_score = std::max(max_score, score);
        }

        float probability_sum = 0.0F;
        for (std::int32_t position = 0; position <= model_position; ++position) {
            const std::size_t index = checked_size(position, "position");
            const float unnormalized = std::exp(scores[index] - max_score);
            probabilities[index] = unnormalized;
```

```cpp
            probability_sum += unnormalized;
        }
        if (probability_sum <= 0.0F) {
            throw std::runtime_error("attention probability sum is invalid");
        }
        for (std::int32_t position = 0; position <= model_position; ++position) {
            const float probability =
                probabilities[checked_size(position, "position")] / probability_sum;
            const std::span<const float> value = cache.value(layer_id, position, kv_head);
            for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
                output[query_base + checked_size(dim, "dim")] +=
                    probability * value[checked_size(dim, "dim")];
            }
        }
    }
}

ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
                                           std::span<const std::int32_t> token_ids) {
    validate_config(model.config);
    if (token_ids.empty()) {
        throw std::runtime_error("reference decode needs at least one token");
    }
    if (token_ids.size() > checked_size(model.config.max_context, "max_context")) {
        throw std::runtime_error("token_ids exceed max_context");
    }
    if (model.token_embedding.size() !=
        checked_size(model.config.vocab_size, "vocab_size") *
            checked_size(model.config.hidden_size, "hidden_size")) {
        throw std::runtime_error("token embedding size mismatch");
    }
    if (model.final_norm_weight.size() != checked_size(model.config.hidden_size, "hidden_size")) {
        throw std::runtime_error("final norm weight size mismatch");
    }
    validate_linear(model.lm_head, "lm_head");

    ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers.size()));
    std::vector<float> hidden(checked_size(model.config.hidden_size, "hidden_size"), 0.0F);
    for (std::int32_t position = 0; position < static_cast<std::int32_t>(token_ids.size());
         ++position) {
        const std::int32_t token_id = token_ids[checked_size(position, "position")];
        if (token_id < 0 || token_id >= model.config.vocab_size) {
```

```
            throw std::runtime_error("token id out of vocabulary");
        }
        const std::span<const float> embedding =
            row_span(model.token_embedding, token_id, model.config.hidden_size);
        std::copy(embedding.begin(), embedding.end(), hidden.begin());
        for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(model.layers.size());
             ++layer_id) {
            forward_layer(model.config,
                          model.layers[checked_size(layer_id, "layer_id")],
                          layer_id,
                          position,
                          cache,
                          hidden);
        }
    }

    std::vector<float> normed(checked_size(model.config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> logits(checked_size(model.config.vocab_size, "vocab_size"), 0.0F);
    rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed);
    linear_f32(model.lm_head, normed, logits);

    ReferenceDecodeResult result;
    result.predicted_token = argmax(logits);
    result.logits = std::move(logits);
    return result;
}

ReferenceDecoderModel make_tiny_reference_decoder_model() {
    ReferenceDecoderModel model;
    model.config.hidden_size = 4;
    model.config.query_heads = 2;
    model.config.kv_heads = 1;
    model.config.head_dim = 2;
    model.config.intermediate_size = 6;
    model.config.vocab_size = 5;
    model.config.max_context = 8;
    model.config.rms_epsilon = 1.0e-5F;
    model.config.rope_base = 10000.0F;

    model.token_embedding = {
        0.20F, -0.10F, 0.05F, 0.30F,
        -0.40F, 0.10F, 0.25F, -0.20F,
        0.15F, 0.35F, -0.30F, 0.05F,
        0.50F, -0.25F, 0.10F, -0.15F,
        -0.05F, 0.20F, 0.40F, -0.35F,
    };

```

```
    DecoderLayerWeightsF32 layer;
    layer.rms_attention_weight = {1.0F, 0.9F, 1.1F, 1.0F};
    layer.wq = make_linear(4,
                           4,
                           {
                               0.20F, -0.10F, 0.05F, 0.30F,
                               -0.15F, 0.25F, 0.10F, -0.05F,
                               0.05F, 0.10F, -0.20F, 0.15F,
                               0.30F, -0.05F, 0.20F, 0.10F,
                           });
    layer.wk = make_linear(4,
                           2,
                           {
                               0.10F, 0.20F, -0.10F, 0.05F,
                               -0.05F, 0.15F, 0.25F, -0.20F,
                           });
    layer.wv = make_linear(4,
                           2,
                           {
                               0.25F, -0.05F, 0.10F, 0.15F,
                               -0.10F, 0.30F, 0.05F, 0.20F,
                           });
    layer.wo = make_linear(4,
                           4,
                           {
                               0.20F, 0.10F, -0.05F, 0.15F,
                               -0.10F, 0.25F, 0.20F, -0.05F,
                               0.05F, -0.20F, 0.30F, 0.10F,
                               0.15F, 0.05F, -0.10F, 0.25F,
                           });
    layer.rms_mlp_weight = {0.95F, 1.05F, 1.0F, 0.9F};
    layer.w_gate = make_linear(4,
                               6,
                               {
                                   0.20F, -0.10F, 0.05F, 0.10F,
                                   -0.05F, 0.15F, 0.20F, -0.10F,
                                   0.10F, 0.05F, -0.15F, 0.25F,
                                   -0.20F, 0.10F, 0.15F, 0.05F,
                                   0.05F, -0.25F, 0.10F, 0.20F,
                                   0.15F, 0.05F, 0.05F, -0.15F,
                               });
    layer.w_up = make_linear(4,
                             6,
                             {
                                 0.10F, 0.20F, -0.05F, 0.05F,
                                 -0.15F, 0.05F, 0.25F, 0.10F,
                                 0.20F, -0.10F, 0.05F, 0.15F,
                                 0.05F, 0.10F, -0.20F, 0.25F,
                                 -0.05F, 0.15F, 0.10F, -0.10F,
                                 0.25F, -0.05F, 0.05F, 0.10F,
                             });
    layer.w_down = make_linear(6,
```

```cpp
                               4,
                               {
                                   0.10F, -0.05F, 0.15F, 0.20F, -0.10F, 0.05F,
                                   -0.20F, 0.10F, 0.05F, -0.05F, 0.15F, 0.20F,
                                   0.05F, 0.15F, -0.10F, 0.10F, 0.20F, -0.05F,
                                   0.20F, 0.05F, 0.10F, -0.15F, -0.05F, 0.10F,
                               });
    model.layers.push_back(std::move(layer));
    model.final_norm_weight = {1.0F, 0.95F, 1.05F, 1.0F};
    model.lm_head = make_linear(4,
                                5,
                                {
                                    0.20F, -0.10F, 0.05F, 0.15F,
                                    -0.05F, 0.25F, 0.10F, -0.20F,
                                    0.15F, 0.05F, -0.25F, 0.10F,
                                    -0.10F, 0.20F, 0.15F, 0.05F,
                                    0.05F, -0.15F, 0.20F, 0.25F,
                                },
                                zeros(5));
    return model;
}

}  // namespace lcqi
```

Tokenizer 与 Safetensors 实现

`tokenizer.cpp` 把 prompt 变成 token id，并固定 unknown token 行为。读者应检查 BOS/EOS 是否稳定插入，UNK 是否按合同处理。

Listing 10.9 src/tokenizer.cpp

```cpp
#include <lcqi/tokenizer.hpp>

#include <cstddef>
#include <fstream>
#include <stdexcept>
#include <string>
#include <string_view>

namespace lcqi {
namespace {

constexpr std::uint64_t LCQI_FNV_OFFSET_BASIS = 14695981039346656037ULL;
constexpr std::uint64_t LCQI_FNV_PRIME = 1099511628211ULL;
constexpr std::int32_t LCQI_INVALID_TOKEN_ID = -1;

std::string read_key(std::istream& input) {
    std::string key;
    if (!(input >> key)) {
        throw std::runtime_error("unexpected end of tokenizer file");
    }
    return key;
```

```cpp
}

void expect_key(std::istream& input, const std::string& expected) {
    const std::string key = read_key(input);
    if (key != expected) {
        throw std::runtime_error("expected tokenizer key '" + expected + "', got '" + key + "'");
    }
}

std::int32_t read_i32(std::istream& input, const std::string& key) {
    expect_key(input, key);
    std::int32_t value = 0;
    if (!(input >> value)) {
        throw std::runtime_error("invalid tokenizer int32 value for key '" + key + "'");
    }
    return value;
}

std::string read_string(std::istream& input, const std::string& key) {
    expect_key(input, key);
    std::string value;
    if (!(input >> value)) {
        throw std::runtime_error("invalid tokenizer string value for key '" + key + "'");
    }
    return value;
}

std::int32_t lookup_token_id(const TokenizerModel& tokenizer, std::string_view text) {
    for (const TokenEntry& entry : tokenizer.vocab) {
        if (entry.text == text) {
            return entry.id;
        }
    }
    return LCQI_INVALID_TOKEN_ID;
}

void validate_tokenizer(const TokenizerModel& tokenizer) {
    if (tokenizer.vocab.empty()) {
        throw std::runtime_error("tokenizer vocab is empty");
    }
    for (std::size_t i = 0; i < tokenizer.vocab.size(); ++i) {
        const TokenEntry& left = tokenizer.vocab[i];
        if (left.id < 0) {
            throw std::runtime_error("tokenizer token id must be non-negative");
        }
        for (std::size_t j = i + 1; j < tokenizer.vocab.size(); ++j) {
            const TokenEntry& right = tokenizer.vocab[j];
```

```cpp
            if (left.id == right.id) {
                throw std::runtime_error("duplicate tokenizer token id");
            }
            if (left.text == right.text) {
                throw std::runtime_error("duplicate tokenizer token text");
            }
        }
    }
    if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID &&
        lookup_token_id(tokenizer, "<bos>") != tokenizer.bos_id) {
        throw std::runtime_error("tokenizer BOS id does not match vocab");
    }
    if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID &&
        lookup_token_id(tokenizer, "<eos>") != tokenizer.eos_id) {
        throw std::runtime_error("tokenizer EOS id does not match vocab");
    }
}

std::uint64_t hash_byte(std::uint64_t hash, unsigned char value) {
    hash ^= static_cast<std::uint64_t>(value);
    hash *= LCQI_FNV_PRIME;
    return hash;
}

std::uint64_t hash_string(std::uint64_t hash, std::string_view text) {
    for (const unsigned char value : text) {
        hash = hash_byte(hash, value);
    }
    return hash;
}

std::uint64_t hash_i32(std::uint64_t hash, std::int32_t value) {
    const std::uint32_t bits = static_cast<std::uint32_t>(value);
    for (std::int32_t shift = 0; shift < 32; shift += 8) {
        hash = hash_byte(hash, static_cast<unsigned char>((bits >> shift) & 0xFFU));
    }
    return hash;
}

}  // namespace

TokenizerModel load_tokenizer(const std::filesystem::path& path) {
    std::ifstream input(path);
    if (!input) {
        throw std::runtime_error("cannot open tokenizer file: " + path.string());
    }

    expect_key(input, "LCQI_TOKENIZER_V1");

    TokenizerModel tokenizer;
```

```
    tokenizer.bos_id = read_i32(input, "bos_id");
    tokenizer.eos_id = read_i32(input, "eos_id");
    tokenizer.unk_id = read_i32(input, "unk_id");
    tokenizer.chat_template_hash = read_string(input, "chat_template_hash");
    const std::int32_t vocab_size = read_i32(input, "vocab_size");
    if (vocab_size <= 0) {
        throw std::runtime_error("tokenizer vocab_size must be positive");
    }

    tokenizer.vocab.reserve(static_cast<std::size_t>(vocab_size));
    expect_key(input, "vocab");
    for (std::int32_t i = 0; i < vocab_size; ++i) {
        TokenEntry entry;
        if (!(input >> entry.id >> entry.text)) {
            throw std::runtime_error("invalid tokenizer vocab entry");
        }
        tokenizer.vocab.push_back(entry);
    }

    validate_tokenizer(tokenizer);
    return tokenizer;
}

std::vector<std::int32_t> encode_prompt(const TokenizerModel& tokenizer,
                                        std::string_view prompt) {
    std::vector<std::int32_t> ids;
    if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID) {
        ids.push_back(tokenizer.bos_id);
    }

    std::size_t begin = 0;
    while (begin < prompt.size()) {
        while (begin < prompt.size() && prompt[begin] == ' ') {
            ++begin;
        }
        if (begin >= prompt.size()) {
            break;
        }
        std::size_t end = begin;
        while (end < prompt.size() && prompt[end] != ' ') {
            ++end;
        }
        const std::string_view token = prompt.substr(begin, end - begin);
        const std::int32_t token_id = lookup_token_id(tokenizer, token);
        if (token_id == LCQI_INVALID_TOKEN_ID) {
            if (tokenizer.unk_id == LCQI_INVALID_TOKEN_ID) {
                throw std::runtime_error("tokenizer encountered unknown token");
            }
            ids.push_back(tokenizer.unk_id);
        } else {
            ids.push_back(token_id);
```

```
        }
        begin = end;
    }

    if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID) {
        ids.push_back(tokenizer.eos_id);
    }
    return ids;
}

std::uint64_t tokenizer_contract_hash(const TokenizerModel& tokenizer) {
    std::uint64_t hash = LCQI_FNV_OFFSET_BASIS;
    hash = hash_string(hash, tokenizer.chat_template_hash);
    hash = hash_i32(hash, tokenizer.bos_id);
    hash = hash_i32(hash, tokenizer.eos_id);
    hash = hash_i32(hash, tokenizer.unk_id);
    for (const TokenEntry& entry : tokenizer.vocab) {
        hash = hash_string(hash, entry.text);
        hash = hash_i32(hash, entry.id);
    }
    return hash;
}

}  // namespace lcqi
```

`safetensors.cpp` 解析 header、metadata、dtype、shape 和 data offsets。坏 dtype、坏 shape、坏 offset 和 payload 太短都应该被拒绝。

Listing 10.10 src/safetensors.cpp

```
#include <lcqi/safetensors.hpp>

#include <algorithm>
#include <cctype>
#include <cstddef>
#include <cmath>
#include <cstring>
#include <fstream>
#include <limits>
#include <stdexcept>
#include <string>
#include <string_view>
#include <vector>

namespace lcqi {
namespace {

constexpr std::size_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
constexpr std::uint64_t LCQI_SAFETENSORS_MAX_HEADER_BYTES = 16U * 1024U * 1024U;
constexpr std::size_t LCQI_SAFETENSORS_OFFSET_COUNT = 2;
constexpr std::uint64_t LCQI_BYTE_BITS = 8;
constexpr std::uint64_t LCQI_DECIMAL_BASE = 10;
```

```cpp
constexpr std::uint64_t LCQI_BYTE_MASK = 0xFFU;
constexpr unsigned char LCQI_JSON_CONTROL_CHAR_LIMIT = 0x20U;
constexpr std::uint64_t LCQI_DTYPE_BYTES_F64 = 8;
constexpr std::uint64_t LCQI_DTYPE_BYTES_F32 = 4;
constexpr std::uint64_t LCQI_DTYPE_BYTES_F16 = 2;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I64 = 8;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I32 = 4;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I16 = 2;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I8 = 1;
constexpr std::uint64_t LCQI_DTYPE_BYTES_U8 = 1;
constexpr std::uint32_t LCQI_F32_EXPONENT_MASK = 0x7F800000U;
constexpr std::uint32_t LCQI_F16_SIGN_MASK = 0x8000U;
constexpr std::uint32_t LCQI_F16_EXPONENT_MASK = 0x7C00U;
constexpr std::uint32_t LCQI_F16_MANTISSA_MASK = 0x03FFU;
constexpr std::uint32_t LCQI_F16_EXPONENT_BIAS = 15U;
constexpr std::uint32_t LCQI_F32_EXPONENT_BIAS = 127U;
constexpr std::uint32_t LCQI_F32_MANTISSA_BITS = 23U;
constexpr std::uint32_t LCQI_F16_MANTISSA_BITS = 10U;
constexpr std::uint32_t LCQI_F16_TO_F32_SIGN_SHIFT = 16U;

class HeaderCursor {
public:
    explicit HeaderCursor(std::string_view text) : text_(text) {}

    void expect(char expected) {
        this->skip_ws();
        if (this->eof() || this->text_[this->position_] != expected) {
            throw std::runtime_error("invalid safetensors header syntax");
        }
        ++this->position_;
    }

    [[nodiscard]] bool consume(char expected) {
        this->skip_ws();
        if (!this->eof() && this->text_[this->position_] == expected) {
            ++this->position_;
            return true;
        }
        return false;
    }

    [[nodiscard]] std::string parse_string() {
        this->skip_ws();
        this->expect('"');
        std::string result;
        while (!this->eof()) {
            const char ch = this->text_[this->position_++];
            if (ch == '"') {
                return result;
            }
            if (static_cast<unsigned char>(ch) < LCQI_JSON_CONTROL_CHAR_LIMIT) {
```

```
                throw std::runtime_error("control character in safetensors string");
            }
            if (ch == '\\') {
                result.push_back(this->parse_escape());
            } else {
                result.push_back(ch);
            }
        }
        throw std::runtime_error("unterminated safetensors string");
    }

    [[nodiscard]] std::uint64_t parse_u64() {
        this->skip_ws();
        if (this->eof() ||
            !std::isdigit(static_cast<unsigned char>(this->text_[this->position_]))) {
            throw std::runtime_error("expected unsigned integer in safetensors header");
        }
        std::uint64_t value = 0;
        while (!this->eof() &&
               std::isdigit(static_cast<unsigned char>(this->text_[this->position_]))) {
            const std::uint64_t digit =
                static_cast<std::uint64_t>(this->text_[this->position_] - '0');
            if (value >
                (std::numeric_limits<std::uint64_t>::max() - digit) / LCQI_DECIMAL_BASE) {
                throw std::runtime_error("safetensors integer overflow");
            }
            value = value * LCQI_DECIMAL_BASE + digit;
            ++this->position_;
        }
        return value;
    }

    [[nodiscard]] std::vector<std::int64_t> parse_i64_array() {
        std::vector<std::int64_t> values;
        this->expect('[');
        if (this->consume(']')) {
            return values;
        }
        while (true) {
            const std::uint64_t raw = this->parse_u64();
            if (raw > static_cast<std::uint64_t>(std::numeric_limits<std::int64_t>::max())) {
                throw std::runtime_error("safetensors shape value is too large");
            }
            values.push_back(static_cast<std::int64_t>(raw));
            if (this->consume(']')) {
```

```cpp
                return values;
            }
            this->expect(',');
        }
    }

    [[nodiscard]] std::vector<std::uint64_t> parse_u64_array() {
        std::vector<std::uint64_t> values;
        this->expect('[');
        if (this->consume(']')) {
            return values;
        }
        while (true) {
            values.push_back(this->parse_u64());
            if (this->consume(']')) {
                return values;
            }
            this->expect(',');
        }
    }

    [[nodiscard]] bool eof_after_ws() {
        this->skip_ws();
        return this->eof();
    }

private:
    std::string_view text_;
    std::size_t position_ = 0;

    [[nodiscard]] bool eof() const {
        return this->position_ >= this->text_.size();
    }

    void skip_ws() {
        while (!this->eof() &&
               std::isspace(static_cast<unsigned char>(this->text_[this->position_]))) {
            ++this->position_;
        }
    }

    [[nodiscard]] char parse_escape() {
        if (this->eof()) {
            throw std::runtime_error("unterminated safetensors escape");
        }
        const char escaped = this->text_[this->position_++];
        switch (escaped) {
            case '"':
            case '\\':
            case '/':
                return escaped;
```

```cpp
            case 'b':
                return '\b';
            case 'f':
                return '\f';
            case 'n':
                return '\n';
            case 'r':
                return '\r';
            case 't':
                return '\t';
            default:
                throw std::runtime_error("unsupported safetensors string escape");
        }
    }
};

std::uint64_t read_le_u64(std::span<const std::uint8_t> bytes) {
    if (bytes.size() < LCQI_SAFETENSORS_PREFIX_BYTES) {
        throw std::runtime_error("safetensors file is too small");
    }
    std::uint64_t value = 0;
    for (std::size_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
        value |= static_cast<std::uint64_t>(bytes[i]) << (LCQI_BYTE_BITS * i);
    }
    return value;
}

std::string read_file_bytes(const std::filesystem::path& path) {
    std::ifstream input(path, std::ios::binary);
    if (!input) {
        throw std::runtime_error("cannot open safetensors file: " + path.string());
    }
    input.seekg(0, std::ios::end);
    const std::streamoff size = input.tellg();
    if (size < 0) {
        throw std::runtime_error("cannot determine safetensors file size");
    }
    input.seekg(0, std::ios::beg);
    std::string bytes(static_cast<std::size_t>(size), '\0');
    if (!bytes.empty() && !input.read(bytes.data(), static_cast<std::streamsize>(bytes.size()))) {
        throw std::runtime_error("cannot read safetensors file");
    }
    return bytes;
}

std::vector<std::uint8_t> read_file_u8(const std::filesystem::path& path) {
    std::ifstream input(path, std::ios::binary);
    if (!input) {
        throw std::runtime_error("cannot open safetensors file: " + path.string
```

```cpp
        ());
    }
    input.seekg(0, std::ios::end);
    const std::streamoff size = input.tellg();
    if (size < 0) {
        throw std::runtime_error("cannot determine safetensors file size");
    }
    input.seekg(0, std::ios::beg);
    std::vector<std::uint8_t> bytes(static_cast<std::size_t>(size), 0);
    if (!bytes.empty() &&
        !input.read(reinterpret_cast<char*>(bytes.data()),
                    static_cast<std::streamsize>(bytes.size()))) {
        throw std::runtime_error("cannot read safetensors file");
    }
    return bytes;
}

void parse_metadata_value(HeaderCursor& cursor,
                          SafeTensorManifest& manifest,
                          const std::string& key) {
    if (key == "__metadata__") {
        cursor.expect('{');
        if (cursor.consume('}')) {
            return;
        }
        while (true) {
            const std::string metadata_key = cursor.parse_string();
            cursor.expect(':');
            const std::string metadata_value = cursor.parse_string();
            manifest.metadata.emplace_back(metadata_key, metadata_value);
            if (cursor.consume('}')) {
                return;
            }
            cursor.expect(',');
        }
    }
    throw std::runtime_error("unexpected safetensors metadata value");
}

SafeTensorEntry parse_tensor_entry(HeaderCursor& cursor, const std::string& name) {
    SafeTensorEntry entry;
    entry.name = name;
    bool saw_dtype = false;
    bool saw_shape = false;
    bool saw_offsets = false;

    cursor.expect('{');
    if (cursor.consume('}')) {
        throw std::runtime_error("empty safetensors tensor entry");
    }
    while (true) {
```

```cpp
        const std::string field = cursor.parse_string();
        cursor.expect(':');
        if (field == "dtype") {
            entry.dtype = cursor.parse_string();
            saw_dtype = true;
        } else if (field == "shape") {
            entry.shape = cursor.parse_i64_array();
            saw_shape = true;
        } else if (field == "data_offsets") {
            const std::vector<std::uint64_t> offsets = cursor.parse_u64_array();
            if (offsets.size() != LCQI_SAFETENSORS_OFFSET_COUNT) {
                throw std::runtime_error("safetensors data_offsets must contain two values");
            }
            entry.data_begin = offsets[0];
            entry.data_end = offsets[1];
            saw_offsets = true;
        } else {
            throw std::runtime_error("unsupported safetensors tensor field: " + field);
        }
        if (cursor.consume('}')) {
            break;
        }
        cursor.expect(',');
    }

    if (!saw_dtype || !saw_shape || !saw_offsets) {
        throw std::runtime_error("safetensors tensor entry is missing required fields");
    }
    if (entry.data_end < entry.data_begin) {
        throw std::runtime_error("safetensors tensor offset range is invalid");
    }
    return entry;
}

void parse_header(std::string_view header, SafeTensorManifest& manifest) {
    HeaderCursor cursor(header);
    cursor.expect('{');
    if (cursor.consume('}')) {
        throw std::runtime_error("safetensors header is empty");
    }

    while (true) {
        const std::string key = cursor.parse_string();
        cursor.expect(':');
        if (key == "__metadata__") {
            parse_metadata_value(cursor, manifest, key);
        } else {
            manifest.tensors.push_back(parse_tensor_entry(cursor, key));
```

```cpp
        }
        if (cursor.consume('}')) {
            break;
        }
        cursor.expect(',');
    }

    if (!cursor.eof_after_ws()) {
        throw std::runtime_error("trailing data after safetensors header");
    }
}

std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry);

void validate_manifest(const SafeTensorManifest& manifest,
                       std::uint64_t file_size) {
    const std::uint64_t payload_size = file_size - manifest.data_start_offset;
    for (std::size_t i = 0; i < manifest.tensors.size(); ++i) {
        const SafeTensorEntry& left = manifest.tensors[i];
        if (left.name.empty() || left.dtype.empty()) {
            throw std::runtime_error("safetensors tensor has empty name or dtype");
        }
        if (left.data_end > payload_size) {
            throw std::runtime_error("safetensors tensor data_offsets exceed payload size");
        }
        const std::uint64_t expected_bytes = expected_tensor_bytes(left);
        if (left.byte_size() != expected_bytes) {
            throw std::runtime_error("safetensors tensor byte size does not match dtype and shape");
        }
        for (const std::int64_t dim : left.shape) {
            if (dim < 0) {
                throw std::runtime_error("safetensors shape dimension is negative");
            }
        }
        for (std::size_t j = i + 1; j < manifest.tensors.size(); ++j) {
            const SafeTensorEntry& right = manifest.tensors[j];
            if (left.name == right.name) {
                throw std::runtime_error("duplicate safetensors tensor name");
            }
            const bool overlaps =
                left.data_begin < right.data_end && right.data_begin < left.data_end;
            if (overlaps) {
                throw std::runtime_error("overlapping safetensors tensor data range");
            }
        }
    }
```

```cpp
}

std::uint64_t dtype_byte_size(std::string_view dtype) {
    if (dtype == "F64") {
        return LCQI_DTYPE_BYTES_F64;
    }
    if (dtype == "F32") {
        return LCQI_DTYPE_BYTES_F32;
    }
    if (dtype == "F16" || dtype == "BF16") {
        return LCQI_DTYPE_BYTES_F16;
    }
    if (dtype == "I64" || dtype == "U64") {
        return LCQI_DTYPE_BYTES_I64;
    }
    if (dtype == "I32" || dtype == "U32") {
        return LCQI_DTYPE_BYTES_I32;
    }
    if (dtype == "I16" || dtype == "U16") {
        return LCQI_DTYPE_BYTES_I16;
    }
    if (dtype == "I8") {
        return LCQI_DTYPE_BYTES_I8;
    }
    if (dtype == "U8" || dtype == "BOOL") {
        return LCQI_DTYPE_BYTES_U8;
    }
    throw std::runtime_error("unsupported safetensors dtype: " + std::string(dtype));
}

std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry) {
    std::uint64_t element_count = 1;
    for (const std::int64_t dim : entry.shape) {
        if (dim < 0) {
            throw std::runtime_error("safetensors shape dimension is negative");
        }
        const std::uint64_t extent = static_cast<std::uint64_t>(dim);
        if (extent != 0 &&
            element_count > std::numeric_limits<std::uint64_t>::max() / extent) {
            throw std::runtime_error("safetensors shape element count overflow");
        }
        element_count *= extent;
    }
    const std::uint64_t dtype_bytes = dtype_byte_size(entry.dtype);
    if (dtype_bytes != 0 &&
        element_count > std::numeric_limits<std::uint64_t>::max() / dtype_bytes) {
        throw std::runtime_error("safetensors tensor byte size overflow");
```

```cpp
    }
    return element_count * dtype_bytes;
}

SafeTensorManifest parse_manifest_from_bytes(std::span<const std::uint8_t> bytes) {
    const std::uint64_t header_size = read_le_u64(bytes);
    if (header_size == 0 || header_size > LCQI_SAFETENSORS_MAX_HEADER_BYTES) {
        throw std::runtime_error("safetensors header size is invalid");
    }
    const std::uint64_t data_start_offset =
        static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) + header_size;
    if (data_start_offset > bytes.size()) {
        throw std::runtime_error("safetensors header extends beyond file size");
    }

    SafeTensorManifest manifest;
    manifest.header_size = header_size;
    manifest.data_start_offset = data_start_offset;
    const char* header_begin =
        reinterpret_cast<const char*>(bytes.data() + LCQI_SAFETENSORS_PREFIX_BYTES);
    const std::string_view header(header_begin, static_cast<std::size_t>(header_size));
    parse_header(header, manifest);
    validate_manifest(manifest, static_cast<std::uint64_t>(bytes.size()));
    return manifest;
}

std::uint16_t read_le_u16(std::span<const std::uint8_t> bytes, std::size_t offset) {
    return static_cast<std::uint16_t>(
        static_cast<std::uint16_t>(bytes[offset]) |
        (static_cast<std::uint16_t>(bytes[offset + 1]) << LCQI_BYTE_BITS));
}

std::uint32_t read_le_u32(std::span<const std::uint8_t> bytes, std::size_t offset) {
    std::uint32_t value = 0;
    for (std::size_t i = 0; i < LCQI_DTYPE_BYTES_F32; ++i) {
        value |= static_cast<std::uint32_t>(bytes[offset + i]) << (LCQI_BYTE_BITS * i);
    }
    return value;
}

float bits_to_float(std::uint32_t bits) {
    float value = 0.0F;
    std::memcpy(&value, &bits, sizeof(value));
    return value;
```

```cpp
}

float f16_to_f32(std::uint16_t bits) {
    const std::uint32_t sign = (static_cast<std::uint32_t>(bits) & LCQI_F16_SIGN_MASK)
                               << LCQI_F16_TO_F32_SIGN_SHIFT;
    const std::uint32_t exponent = static_cast<std::uint32_t>(bits) & LCQI_F16_EXPONENT_MASK;
    const std::uint32_t mantissa = static_cast<std::uint32_t>(bits) & LCQI_F16_MANTISSA_MASK;

    if (exponent == 0U) {
        if (mantissa == 0U) {
            return bits_to_float(sign);
        }
        float value = static_cast<float>(mantissa) /
                      static_cast<float>(1U << LCQI_F16_MANTISSA_BITS);
        value = std::ldexp(value, 1 - static_cast<int>(LCQI_F16_EXPONENT_BIAS));
        return (sign == 0U) ? value : -value;
    }

    if (exponent == LCQI_F16_EXPONENT_MASK) {
        const std::uint32_t f32_mantissa =
            mantissa << (LCQI_F32_MANTISSA_BITS - LCQI_F16_MANTISSA_BITS);
        return bits_to_float(sign | LCQI_F32_EXPONENT_MASK | f32_mantissa);
    }

    const std::uint32_t f32_exponent =
        ((exponent >> LCQI_F16_MANTISSA_BITS) - LCQI_F16_EXPONENT_BIAS +
         LCQI_F32_EXPONENT_BIAS)
        << LCQI_F32_MANTISSA_BITS;
    const std::uint32_t f32_mantissa =
        mantissa << (LCQI_F32_MANTISSA_BITS - LCQI_F16_MANTISSA_BITS);
    return bits_to_float(sign | f32_exponent | f32_mantissa);
}

float bf16_to_f32(std::uint16_t bits) {
    return bits_to_float(static_cast<std::uint32_t>(bits) << 16U);
}

std::span<const std::uint8_t> tensor_payload_span(const SafeTensorFile& file,
                                                  const SafeTensorEntry& entry) {
    const std::uint64_t begin = file.manifest.data_start_offset + entry.data_begin;
    const std::uint64_t end = file.manifest.data_start_offset + entry.data_end;
    if (begin > end || end > file.bytes.size()) {
        throw std::runtime_error("safetensors tensor byte range is invalid");
    }
    return std::span<const std::uint8_t>(
        file.bytes.data() + static_cast<std::size_t>(begin),
```

```cpp
        static_cast<std::size_t>(end - begin));
}

}  // namespace

std::uint64_t SafeTensorEntry::byte_size() const {
    return this->data_end - this->data_begin;
}

const SafeTensorEntry* SafeTensorManifest::find_tensor(std::string_view name) const {
    const auto found = std::find_if(
        this->tensors.begin(),
        this->tensors.end(),
        [name](const SafeTensorEntry& entry) {
            return entry.name == name;
        });
    if (found == this->tensors.end()) {
        return nullptr;
    }
    return &(*found);
}

SafeTensorManifest load_safetensors_manifest(const std::filesystem::path& path) {
    const std::string bytes = read_file_bytes(path);
    return parse_manifest_from_bytes(
        std::span<const std::uint8_t>(
            reinterpret_cast<const std::uint8_t*>(bytes.data()),
            bytes.size()));
}

SafeTensorFile load_safetensors_file(const std::filesystem::path& path) {
    SafeTensorFile file;
    file.path = path;
    file.bytes = read_file_u8(path);
    file.manifest = parse_manifest_from_bytes(file.bytes);
    return file;
}

std::span<const std::uint8_t> read_safetensor_tensor_bytes(
    const SafeTensorFile& file,
    std::string_view name) {
    const SafeTensorEntry* entry = file.manifest.find_tensor(name);
    if (entry == nullptr) {
        throw std::runtime_error("missing safetensors tensor: " + std::string(name));
    }
    return tensor_payload_span(file, *entry);
}

std::vector<float> read_safetensor_f32_tensor(const SafeTensorFile& file,
```

```cpp
                                             std::string_view name) {
    const SafeTensorEntry* entry = file.manifest.find_tensor(name);
    if (entry == nullptr) {
        throw std::runtime_error("missing safetensors tensor: " + std::string(name));
    }
    const std::span<const std::uint8_t> bytes = tensor_payload_span(file, *entry);
    std::vector<float> values;
    if (entry->dtype == "F32") {
        values.resize(bytes.size() / LCQI_DTYPE_BYTES_F32);
        for (std::size_t index = 0; index < values.size(); ++index) {
            values[index] = bits_to_float(
                read_le_u32(bytes, index * LCQI_DTYPE_BYTES_F32));
        }
        return values;
    }
    if (entry->dtype == "F16") {
        values.resize(bytes.size() / LCQI_DTYPE_BYTES_F16);
        for (std::size_t index = 0; index < values.size(); ++index) {
            values[index] = f16_to_f32(
                read_le_u16(bytes, index * LCQI_DTYPE_BYTES_F16));
        }
        return values;
    }
    if (entry->dtype == "BF16") {
        values.resize(bytes.size() / LCQI_DTYPE_BYTES_F16);
        for (std::size_t index = 0; index < values.size(); ++index) {
            values[index] = bf16_to_f32(
                read_le_u16(bytes, index * LCQI_DTYPE_BYTES_F16));
        }
        return values;
    }
    throw std::runtime_error("safetensors tensor is not float-compatible: " + entry->name);
}

}  // namespace lcqi
```

GPT-2 Reference Path

`gpt2_reference.cpp` 包括 JSON 配置读取、byte-level BPE、safetensors F32/F16/BF16 张量读取、HuggingFace 权重名前缀解析、LayerNorm、packed QKV、causal attention、GELU、tied lm head、full-prefix greedy generation 和 cached-KV generation。它仍然是正确性优先的 reference path，但现在已经开始承接前三册的优化方法：先建立 baseline，再改 hot path，再用测试和 A/B benchmark 决定改动是否能进入书。

一次可审查的 cached-KV 优化

优化不能从“感觉慢”开始。LCQI 先用 `makecpu3-gpt2-benchmark-compare` 证明 full-prefix 每生成一个 token 都重算前缀，而 cached-KV 已经把前缀复用下来；随后再看 cached path 内部。旧实现每处理一个 token、每经过一层 Transformer，都会重新分配 `normed`、`query`、`key`、 `value`、`attention`、`projected`、`mlp_fc`、`mlp_out` 和 attention `scores`。同时，生成路径只需要下一个 greedy token，却仍然每步构造完整 `Gpt2ForwardResult.logits`。这些问题不是算法语义错误，但会在真实 GPT-2 的 decode loop 里形成额外分配、重复 shape validation、重复边界转换和不必要的结果写回。

本轮优化对应三册里的三条原则。第一册要求从热循环和二进制可观察行为出发，所以先量 `lm_head`、MLP、QKV、attention projection 和 attention 本体各占多少，再决定改哪里。第二册要求减少重复分配和内存扰动，所以新增 `Gpt2ForwardWorkspace`，把 hidden、QKV、MLP、scores 和 logits 缓冲区的生命周期提升到整段 cached generation。第三册要求按 decoder runtime 的状态组织代码，所以新增 `Gpt2CachedGreedyDecoder`，让 KV cache、workspace、worker pool 和模型验证跟随一次生成会话，而不是散落在每个 token 的临时对象里。

代码上有五处关键边界。第一，旧的 `run_gpt2_forward_cached` 仍然保留完整 logits API，测试和源码阅读可以继续用它和 full-prefix logits 做逐项对照。第二，CLI 的 `cached_kv` 生成路径改用 `Gpt2CachedGreedyDecoder::step`，只返回 greedy token；需要完整 logits 的测试使用 `step_with_logits`。第三，prompt prefill 的前缀 token 使用 `advance_without_prediction`，只更新 KV cache，不做 final norm 和 `lm_head`，因为这些中间预测会被后一个 prompt token 立即覆盖。第四，KV cache 对外仍保留 checked `key/value` API，内部 attention 在一次地址和 workspace 校验后使用私有 unchecked span 扫描热路径，避免把“不检查”的能力暴露给普通调用者。第五，线程池只属于 cached decoder 会话；`--threads0` 自动选择 worker 数，`--threads1` 强制串行，`--threadsN` 固定线程数，避免把全局可变线程服务塞进 reference model。

正确性证据在 `tests/lcqi_tests.cpp`。tiny GPT-2 同一个 prompt 同时覆盖 full-prefix logits、旧 cached logits、优化 decoder logits、强制两 worker 的并行 decoder logits 和 logits-free greedy token。它要求 predicted token、logits size、逐项 logit 误差、KV cache filled token 数和 greedy generated ids 全部一致。这样优化后的代码不是“跑得快但不知道还对不对”，而是先通过 tiny golden，再进入真实 GPT-2 benchmark。

性能证据不只看一次运行。新增 `tools/run_gpt2_optimization_ab.py` 会从指定 baseline commit 建临时 worktree，baseline 和当前工作区都通过 `books/cpu-volume-3-practice` 的 Release CMake 入口构建，再用同一份 GPT-2 F32 safetensors 模型跑多轮 full-prefix 与 cached-KV。后来远端 `caw` 不能直接复用本机 SSH remote 别名时，本轮采用等价的源码包 A/B：本机用 `gitarchive73f40c7` 导出 baseline，用当前工作区导出 current，二者在 `caw` 上分别 Release 构建并运行同一模型。raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-threaded-manual-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-threaded-optimization-summary.md`。5 轮中位数显示，4 token cached-KV 生成阶段从 `395.397ms` 到 `76.4141ms`，16 token 从 `989.060ms` 到 `185.479ms`；生成吞吐分别到 `52.3464token/s` 和 `86.2633token/s`。

用户看到上一轮提升幅度后，正确反应不是继续猜，而是重新量热点。新增 `--profile-hotspots` 和 `tools/run_gpt2_hotspot_profile.py` 后，可以把 cached decode 的时间拆成 embedding、LayerNorm、QKV projection、KV append、attention、attention projection、MLP fc、GELU、MLP projection 和 lm head。远端 `caw` 使用 Intel Core Ultra 7 265KF、GCC 15.2.1、Release 构建、GPT-2 F32 safetensors，跑 `Hello,mynameis` 这个 prompt，优化前 3 轮中位数保存在 `books/cpu-volume-3/reports/lcqi-gpt2-hotspot-profile-summary.md`：生成 4 个 token 时，`lm_head` 约占 `30.2423%`，MLP 的 `c_fc` 和 `c_proj` 合计约 `46.4228%`，QKV projection 约 `17.2127%`，cached attention 本体只有 `0.0558254%`。这说明慢点不是 attention，而是大量互相独立的行点积。

这一轮优化据此把 `lm_head` greedy argmax 和 GPT-2 F32 `linear` 的输出行切给持久 worker pool。每一行仍然是同一个 dot product；不同线程只负责不同输出行。完整 logits 路径把 logits buffer 分片写入，logits-free greedy 路径每个分片先找本地 best token，再按分片顺序合并，用严格大于比较保留较小 token id 的 tie-breaking。这样并行改变调度，不改变数学定义。

还有一个细节说明为什么优化必须带着 profiler 走。第一次 auto 阈值只按“一百万次乘加”判断是否并行，结果 `768\times768` 的 attention output projection 仍走串行。中间 profile 立即显示 attention projection 从约 `6%` 反弹到约 `25%`，成为新瓶颈。最终阈值改成：输出行数至少 512，且输入列数至少 512 时也允许并行。最终 profile 保存在 `books/cpu-volume-3/reports/`

`lcqi-gpt2-threaded-hotspot-caw.txt`：4 token 生成中位数 `75.9807ms`，16 token `184.780ms`；`lm_head` 约 `20%`，MLP 两个投影合计约 `45%`，QKV 约 `19%`，attention projection 约 `12%`，attention 本体仍低于 `1%`。

用户继续问和 llama.cpp 的差距时，下一步就不能再停留在“行并行”。LCQI 新增 `include/lcqi/f32_kernels.hpp`、`src/f32_kernels.cpp` 和 `src/f32_kernels_avx2.cpp`，把 GPT-2 cached 热路径的单行点积抽成 `dot_f32_unchecked()`。checked `dot_f32()` 负责普通 span 长度检查，unchecked 入口给已经完成 shape validation 的 `linear`、logits 和 logits-free `lm_head` 使用；x86-64/GNU/Clang 构建会编译 AVX2/FMA 文件，运行时再用 `dot_f32_avx2_available()` 判断是否能执行。这样 AVX intrinsics 不进入 GPT-2 层逻辑，后续替换 AVX-512、int8 或 int4 block kernel 时，边界仍然清楚。

`tests/lcqi_tests.cpp` 也随之增加 F32 dot kernel 对照：同一组固定输入先跑 scalar，再跑 dispatch；如果当前 CPU 支持 AVX2/FMA，再显式跑 AVX2 实现，并要求误差落在 `1.0e-4` 内。测试还会传入长度不一致的 span，确认 checked API 会拒绝。这样优化路径不是“编译过就算”，而是有 scalar oracle、运行时能力判断和错误输入边界。

这轮 F32 dot 优化以 `cb4d89f` 为 baseline，在 `caw` 上用同一 GPT-2 F32 模型、Release 构建和 `--threads0` 自动 16 workers 跑 5 轮。raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-dot-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-dot-optimization-summary.md`。4 token 生成中位数从 `74.4247ms` 到 `62.3628ms`，约 `1.19x`；16 token 从 `181.257ms` 到 `157.230ms`，约 `1.15x`。decode 中位数分别从 `26.1887ms` 到 `23.0640ms`，从 `132.201ms` 到 `117.734ms`。这个提升比线程化小，因为它只缩短每一行内部 dot，而没有减少行数、层数、vocab 扫描或内存流量。

继续顺着热点走，下一步不是在 GPT-2 层里散落更多 intrinsics，而是把 kernel 边界从“单行 dot”扩成“row range”。新增的 `linear_f32_rows_unchecked()` 一次计算一段输出行，`max_dot_f32_rows_unchecked()` 一次扫描一段 vocab 行并返回局部最大值；`Gpt2ParallelWorkerPool::parallel_for_rows()` 改成模板入口，去掉每个热路径任务里的 `std::function` 构造和间接调用。AVX2 文件里还增加 4-row row-range kernel：同一组 input 向量 load 后，同时喂给 4 个输出行累加器。这个写法已经接近真实 matvec kernel 的形状：不是每一行各自重新走完整控制流，而是在一个小 tile 里复用输入。

这轮 row-range 优化的 raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-rowrange-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-rowrange-optimization-summary.md`。相对 `ab72ccb` 这个 F32 dot baseline，7 轮中位数显示 4 token 端到端生成从 `60.6291ms` 到 `61.1045ms`，属于噪声内波动；16 token 从 `153.354ms` 到 `152.574ms`，约 `1.01x`。但把 row-range 版本和 4-row AVX2 row-range 版本直接对比，4 token 从 `62.0243ms` 到 `60.3965ms`，16 token 从 `154.087ms` 到 `151.222ms`。所以这段代码可以保留，但结论要诚实：它证明 tile 化 row-range 有正收益，却不是数量级变化。

最终 hotspot 仍然说明差距在哪里。row-range 后，16 token profile 中 `lm_head` 约 `22.3075%`，MLP `c_fc` 约 `22.4147%`，MLP projection 约 `22.448%`，QKV projection 约 `18.6294%`，attention projection 约 `11.1687%`，cached attention 本体只有 `0.653274%`。也就是说，小 kernel 和调度优化已经把 reference path 又往前推了一点，但主要时间仍在 F32 row-major matvec。真实 llama.cpp/ggml 的大优势来自 GGUF、block quantization、packed layout、cache-friendly matmul/matvec、mmap 和成熟线程调度，不是只把裸 F32 行循环写得更精巧。

这个结论必须保守阅读。真实端到端 GPT-2 benchmark 受调度、频率、温度和共享机器状态影响明显，单次结果可能反向波动。因此报告里必须同时看 `median/min/max`、生成文本是否一致、baseline 和 current 是否同一构建入口。更大的差距仍然不在这轮 F32 row-range 里：LCQI 还没有 GGUF 量化导入、packed/quantized lm head、量化 block 权重、分页 KV cache、NUMA/亲和和成熟调度。也就是说，本轮把 reference path 从“行点积有 SIMD 内核边界”推进到“输出行段有可替换 tile kernel”，但还没有把它变成 llama.cpp 级推理引擎。

继续看热点时，有一类优化不在 kernel 内部，而在调用链上。cached generation 的 prompt prefill 要把 prompt 的每个 token 写入 KV cache；但如果 prompt 长度是 `N`，前 `N-1` 个 prompt token 的 next-token prediction 根本不会被输出，也不会作为后续输入。只有最后一个 prompt token 的 prediction 会成为第一个新生成 token。旧代码每个 prompt token 都跑 final norm 和 `lm_head`，等

于把前缀 token 的预测算出来后马上丢掉。

因此新增 `Gpt2CachedGreedyDecoder::advance_without_prediction()`。它复用同一条 cached layer path，照样执行 embedding、每层 QKV、attention、MLP 和 KV append，只是在层结束后直接返回，不进入 final norm 和 `lm_head`。CLI 和 `gpt2_generate_greedy_cached()` 的 prefill 改成：前缀 token 调 `advance_without_prediction()`，最后一个 prompt token 调 `step()`。这不是近似优化，因为被跳过的预测值在旧流程中没有任何 caller 读取。

远端 `caw` 上 7 轮 A/B 证明这条优化有效，raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-prefill-skip-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-prefill-skip-optimization-summary.md`。同一 GPT-2 F32 模型、同一 prompt、同一 `--threads0` 下，4 token 生成中位数从 `60.5154ms` 到 `52.1949ms`，约快 `13.75%`；16 token 从 `153.574ms` 到 `147.424ms`，约快 `4.00%`。收益在短生成里更明显，因为 prompt prefill 占总时间比例更大；decode 段几乎不变，因为真正生成阶段仍然必须预测下一个 token。所有轮次生成文本和 token id 都一致。

这一轮还做过一个没有进入主线的 packed F32 `lm_head` 实验：把 `lm_head` 权重按 8 个输出一组重排，让 AVX2 一次累计 8 个 token score。它能让 `lm_head_ms` 小幅下降，但 `caw` 多轮 A/B 没有稳定端到端收益，甚至会让部分 decode 口径略慢。这个结果本身也要写进工程判断：不是每个看起来更接近 SIMD 的布局都值得进入教材主线；没有稳定证据的优化只能留作实验记录，不能伪装成成功案例。

继续追问"为什么提升还是不够大"时，下一步不是再猜一个更复杂的内核，而是把已经接受的 profile 再拆开看。row-range 之后，GPT-2 cached path 每个 token 会在 12 层里反复调用 QKV projection、attention projection、MLP `c_fc`、MLP projection；再加上需要预测 token 时的 `lm_head`。这些矩阵向量乘每次本身不算巨型 GEMM，却会频繁进入 `parallel_for_rows()`。也就是说，热点不只有"每行点积慢"，还有"每个小并行区唤醒 worker 的成本"。如果 worker pool 每次都拿 mutex、发 condition variable、再等完成信号，那么短生成里会把调度成本支付几百次。

因此这一轮接受的优化没有改数学公式，而是改 worker pool 的交接方式。旧版本用 `std::mutex`、`std::condition_variable` 和 `finished_` 条件变量发布任务；新版本把任务元数据写入原子字段，再用 `generation` 作为发布点。后台 worker 看到新的 generation 后按固定 worker index 取自己的行块，做完后递减 `active_workers`；主线程自己处理最后一个行块，然后用 bounded spin/yield 等后台 worker 归零。代码里特意把这个池限制在 `Gpt2CachedGreedyDecoder` 会话内，没有变成全局忙等线程服务；当 `--threads1` 或模型规模不够并行时，路径仍然直接串行执行。

这个设计有三个边界必须看懂。第一，任务上下文仍然是栈上的 `ParallelTaskContext`，所以所有任务元数据必须先写完，再递增 `generation`；worker 对 `generation` 做 acquire load 后才允许读上下文。第二，后台 worker 不再争抢全局 `next_worker_index`，而是启动时固定自己的 worker index；这样不会出现旧任务里空闲 worker 和下一轮任务元数据交叉的问题。第三，spin 不是无限空转：短时间使用 x86 pause，超过阈值后让出调度；这适合当前 cached decode 的短小并行区，但不应该被复制到长时间等待的 I/O 或服务端队列里。

`caw` 上 9 轮 A/B 证明这条优化是本轮真正站得住的提升。raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-spinpool-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-spinpool-optimization-summary.md`。相对旧 auto 16 worker baseline，4 token 生成中位数从 `53.7887ms` 到 `37.5403ms`，约快 `30.21%`；16 token 从 `146.930ms` 到 `104.107ms`，约快 `29.15%`；64 token 从 `510.650ms` 到 `377.233ms`，约快 `26.13%`。只和 auto8 condition-variable 版本比较，也能看到 4/16/64 token 分别约 `25.67%`、`29.56%`、`20.44%` 的生成阶段下降；固定 `--threads8` 下也有约 `18%` 到 `23%` 的下降。这说明收益不是只来自 worker 数从 16 改成 8，而是高频并行区交接成本确实降下来了。

热点证据也吻合原因。auto 口径下，4 token 的 QKV projection 中位数约快 `33.58%`，attention projection 约快 `63.30%`，MLP `c_fc` 约快 `30.36%`，MLP projection 约快 `29.08%`；而 `lm_head` 只约快 `8.57%`。原因是 QKV 和 MLP 这些线性层在每层每 token 都调用小并行区，调度成本占比高；`lm_head` 是一次更大的 vocab 扫描，仍然主要受内存流量和点积内核影响。这个差异比一句"线程池更快"更重要，因为它告诉读者优化到底打中了哪个成本。

本轮还试过一个 8-row AVX2 row-range 候选：把现有一次处理 4 个输出行的 row-range kernel 扩

成一次处理 8 行。它没有进入主线，因为 `caw` 数据没有稳定收益：auto 口径下 4 token 从 `37.4937ms` 到 `37.7082ms`，16 token 从 `103.714ms` 到 `103.178ms`，64 token 从 `374.439ms` 到 `374.457ms`；固定 `--threads8` 下 64 token 还从 `371.807ms` 退到 `377.053ms`。这说明寄存器压力、load/store 形状和调度噪声抵消了多行复用 input 的设想，不能因为“8 行看起来比 4 行更充分”就合并。

至此可以重新回答和 llama.cpp 的差距。LCQI 现在已经有 cached KV、workspace 复用、prefill 跳过无用 prediction、F32 row-range AVX2、行并行和轻量 worker hand-off；这些能把 reference path 推进很多，但它仍然主要在 F32 row-major matvec 上花时间。llama.cpp/ggml 的优势不只是“也用了线程”，而是 GGUF 模型格式、block quantization、量化权重布局、成熟 matvec microkernel、mmap 加载、NUMA/亲和策略和大量 shape-specific 调参一起工作。LCQI 这轮优化的价值是把调度证据链讲清楚，并把失败候选挡在书外；下一阶段如果要继续接近 llama.cpp，应优先做 GGUF/量化权重路径和量化 matvec，而不是继续在裸 F32 row-major 上堆小修小补。

Listing 10.11 src/f32_kernels.cpp

```cpp
#include <lcqi/f32_kernels.hpp>

#include <limits>
#include <stdexcept>

namespace lcqi {
namespace {

constexpr std::size_t LCQI_F32_AVX2_MIN_DOT_SIZE = 32;
constexpr std::size_t LCQI_F32_AVX2_MIN_ROW_COUNT = 4;

}  // namespace

float dot_f32(std::span<const float> lhs, std::span<const float> rhs) {
    if (lhs.size() != rhs.size()) {
        throw std::runtime_error("dot_f32 size mismatch");
    }
    return dot_f32_unchecked(lhs.data(), rhs.data(), lhs.size());
}

float dot_f32_scalar_unchecked(const float* lhs,
                               const float* rhs,
                               std::size_t size) noexcept {
    float sum = 0.0F;
    for (std::size_t index = 0; index < size; ++index) {
        sum += lhs[index] * rhs[index];
    }
    return sum;
}

float dot_f32_unchecked(const float* lhs,
                        const float* rhs,
                        std::size_t size) noexcept {
    static const bool AVX2_AVAILABLE = dot_f32_avx2_available();
    if (AVX2_AVAILABLE && size >= LCQI_F32_AVX2_MIN_DOT_SIZE) {
        return dot_f32_avx2_unchecked(lhs, rhs, size);
    }
    return dot_f32_scalar_unchecked(lhs, rhs, size);
}
```

```

void linear_f32_rows_scalar_unchecked(const float* weights,
                                      const float* input,
                                      const float* bias,
                                      std::size_t input_size,
                                      std::size_t row_begin,
                                      std::size_t row_end,
                                      float* output) noexcept {
    for (std::size_t row = row_begin; row < row_end; ++row) {
        const float* weight_row = weights + row * input_size;
        output[row] = dot_f32_scalar_unchecked(weight_row, input, input_size) +
                      (bias == nullptr ? 0.0F : bias[row]);
    }
}

void linear_f32_rows_unchecked(const float* weights,
                               const float* input,
                               const float* bias,
                               std::size_t input_size,
                               std::size_t row_begin,
                               std::size_t row_end,
                               float* output) noexcept {
    static const bool AVX2_AVAILABLE = dot_f32_avx2_available();
    if (AVX2_AVAILABLE && input_size >= LCQI_F32_AVX2_MIN_DOT_SIZE &&
        row_end - row_begin >= LCQI_F32_AVX2_MIN_ROW_COUNT) {
        linear_f32_rows_avx2_unchecked(weights,
                                       input,
                                       bias,
                                       input_size,
                                       row_begin,
                                       row_end,
                                       output);
        return;
    }
    linear_f32_rows_scalar_unchecked(weights, input, bias, input_size, row_begin, row_end, output);
}

F32RowMax max_dot_f32_rows_scalar_unchecked(const float* weights,
                                            const float* input,
                                            std::size_t input_size,
                                            std::size_t row_begin,
                                            std::size_t row_end) noexcept {
    F32RowMax best;
    best.row = row_begin;
    best.value = -std::numeric_limits<float>::infinity();
    for (std::size_t row = row_begin; row < row_end; ++row) {
        const float* weight_row = weights + row * input_size;
        const float value = dot_f32_scalar_unchecked(weight_row, input, input_size);
        if (row == row_begin || value > best.value) {
            best.row = row;
```

```
            best.value = value;
        }
    }
    return best;
}

F32RowMax max_dot_f32_rows_unchecked(const float* weights,
                                     const float* input,
                                     std::size_t input_size,
                                     std::size_t row_begin,
                                     std::size_t row_end) noexcept {
    static const bool AVX2_AVAILABLE = dot_f32_avx2_available();
    if (AVX2_AVAILABLE && input_size >= LCQI_F32_AVX2_MIN_DOT_SIZE &&
        row_end - row_begin >= LCQI_F32_AVX2_MIN_ROW_COUNT) {
        return max_dot_f32_rows_avx2_unchecked(weights,
                                               input,
                                               input_size,
                                               row_begin,
                                               row_end);
    }
    return max_dot_f32_rows_scalar_unchecked(weights, input, input_size, row_begin, row_end);
}

}  // namespace lcqi

#if !defined(LCQI_ENABLE_AVX2)
namespace lcqi {

bool dot_f32_avx2_available() noexcept {
    return false;
}

float dot_f32_avx2_unchecked(const float* lhs,
                             const float* rhs,
                             std::size_t size) noexcept {
    return dot_f32_scalar_unchecked(lhs, rhs, size);
}

void linear_f32_rows_avx2_unchecked(const float* weights,
                                    const float* input,
                                    const float* bias,
                                    std::size_t input_size,
                                    std::size_t row_begin,
                                    std::size_t row_end,
                                    float* output) noexcept {
    linear_f32_rows_scalar_unchecked(weights,
                                     input,
                                     bias,
                                     input_size,
                                     row_begin,
                                     row_end,
```

```cpp
                                               output);
}

F32RowMax max_dot_f32_rows_avx2_unchecked(const float* weights,
                                          const float* input,
                                          std::size_t input_size,
                                          std::size_t row_begin,
                                          std::size_t row_end) noexcept {
    return max_dot_f32_rows_scalar_unchecked(weights, input, input_size, row_begin, row_end);
}

}  // namespace lcqi
#endif
```

Listing 10.12 src/f32_kernels_avx2.cpp

```cpp
#include <lcqi/f32_kernels.hpp>

#if defined(LCQI_ENABLE_AVX2)

#include <immintrin.h>

#include <array>
#include <cstddef>
#include <limits>

namespace lcqi {
namespace {

constexpr std::size_t LCQI_F32_AVX2_LANE_COUNT = 8;
constexpr std::size_t LCQI_F32_AVX2_UNROLL_COUNT = 4;
constexpr std::size_t LCQI_F32_AVX2_UNROLLED_FLOATS =
    LCQI_F32_AVX2_LANE_COUNT * LCQI_F32_AVX2_UNROLL_COUNT;

[[nodiscard]] float horizontal_sum(__m256 values) noexcept {
    const __m128 low = _mm256_castps256_ps128(values);
    const __m128 high = _mm256_extractf128_ps(values, 1);
    const __m128 pair_sum = _mm_add_ps(low, high);
    const __m128 first_fold = _mm_hadd_ps(pair_sum, pair_sum);
    const __m128 second_fold = _mm_hadd_ps(first_fold, first_fold);
    return _mm_cvtss_f32(second_fold);
}

void compute_four_row_dot(const float* row0,
                          const float* row1,
                          const float* row2,
                          const float* row3,
                          const float* input,
                          std::size_t input_size,
                          float* sums) noexcept {
    std::size_t index = 0;
```

```
    __m256 accumulator0 = _mm256_setzero_ps();
    __m256 accumulator1 = _mm256_setzero_ps();
    __m256 accumulator2 = _mm256_setzero_ps();
    __m256 accumulator3 = _mm256_setzero_ps();

    for (; index + LCQI_F32_AVX2_LANE_COUNT <= input_size;
         index += LCQI_F32_AVX2_LANE_COUNT) {
        const __m256 input_values = _mm256_loadu_ps(input + index);
        accumulator0 =
            _mm256_fmadd_ps(_mm256_loadu_ps(row0 + index), input_values, accumulator0);
        accumulator1 =
            _mm256_fmadd_ps(_mm256_loadu_ps(row1 + index), input_values, accumulator1);
        accumulator2 =
            _mm256_fmadd_ps(_mm256_loadu_ps(row2 + index), input_values, accumulator2);
        accumulator3 =
            _mm256_fmadd_ps(_mm256_loadu_ps(row3 + index), input_values, accumulator3);
    }

    sums[0] = horizontal_sum(accumulator0);
    sums[1] = horizontal_sum(accumulator1);
    sums[2] = horizontal_sum(accumulator2);
    sums[3] = horizontal_sum(accumulator3);
    for (; index < input_size; ++index) {
        const float input_value = input[index];
        sums[0] += row0[index] * input_value;
        sums[1] += row1[index] * input_value;
        sums[2] += row2[index] * input_value;
        sums[3] += row3[index] * input_value;
    }
}

}  // namespace

bool dot_f32_avx2_available() noexcept {
#if defined(__GNUC__) || defined(__clang__)
    __builtin_cpu_init();
    return __builtin_cpu_supports("avx2") && __builtin_cpu_supports("fma");
#else
    return true;
#endif
}

float dot_f32_avx2_unchecked(const float* lhs,
                             const float* rhs,
                             std::size_t size) noexcept {
    std::size_t index = 0;
    __m256 accumulator0 = _mm256_setzero_ps();
    __m256 accumulator1 = _mm256_setzero_ps();
```

```cpp
    __m256 accumulator2 = _mm256_setzero_ps();
    __m256 accumulator3 = _mm256_setzero_ps();

    for (; index + LCQI_F32_AVX2_UNROLLED_FLOATS <= size;
         index += LCQI_F32_AVX2_UNROLLED_FLOATS) {
        const __m256 lhs0 = _mm256_loadu_ps(lhs + index);
        const __m256 rhs0 = _mm256_loadu_ps(rhs + index);
        const __m256 lhs1 = _mm256_loadu_ps(lhs + index + LCQI_F32_AVX2_LANE_COUNT);
        const __m256 rhs1 = _mm256_loadu_ps(rhs + index + LCQI_F32_AVX2_LANE_COUNT);
        const __m256 lhs2 = _mm256_loadu_ps(lhs + index + LCQI_F32_AVX2_LANE_COUNT * 2);
        const __m256 rhs2 = _mm256_loadu_ps(rhs + index + LCQI_F32_AVX2_LANE_COUNT * 2);
        const __m256 lhs3 = _mm256_loadu_ps(lhs + index + LCQI_F32_AVX2_LANE_COUNT * 3);
        const __m256 rhs3 = _mm256_loadu_ps(rhs + index + LCQI_F32_AVX2_LANE_COUNT * 3);

        accumulator0 = _mm256_fmadd_ps(lhs0, rhs0, accumulator0);
        accumulator1 = _mm256_fmadd_ps(lhs1, rhs1, accumulator1);
        accumulator2 = _mm256_fmadd_ps(lhs2, rhs2, accumulator2);
        accumulator3 = _mm256_fmadd_ps(lhs3, rhs3, accumulator3);
    }

    __m256 accumulator = _mm256_add_ps(
        _mm256_add_ps(accumulator0, accumulator1),
        _mm256_add_ps(accumulator2, accumulator3));
    for (; index + LCQI_F32_AVX2_LANE_COUNT <= size;
         index += LCQI_F32_AVX2_LANE_COUNT) {
        const __m256 lhs_values = _mm256_loadu_ps(lhs + index);
        const __m256 rhs_values = _mm256_loadu_ps(rhs + index);
        accumulator = _mm256_fmadd_ps(lhs_values, rhs_values, accumulator);
    }

    float sum = horizontal_sum(accumulator);
    for (; index < size; ++index) {
        sum += lhs[index] * rhs[index];
    }
    return sum;
}

void linear_f32_rows_avx2_unchecked(const float* weights,
                                    const float* input,
                                    const float* bias,
                                    std::size_t input_size,
                                    std::size_t row_begin,
                                    std::size_t row_end,
                                    float* output) noexcept {
    std::size_t row = row_begin;
    for (; row + LCQI_F32_AVX2_UNROLL_COUNT <= row_end;
```

```
         row += LCQI_F32_AVX2_UNROLL_COUNT) {
        const float* row0 = weights + row * input_size;
        const float* row1 = row0 + input_size;
        const float* row2 = row1 + input_size;
        const float* row3 = row2 + input_size;
        std::array<float, LCQI_F32_AVX2_UNROLL_COUNT> sums{};
        compute_four_row_dot(row0, row1, row2, row3, input, input_size, sums.data());
        for (std::size_t offset = 0; offset < LCQI_F32_AVX2_UNROLL_COUNT; ++offset) {
            output[row + offset] =
                sums[offset] + (bias == nullptr ? 0.0F : bias[row + offset]);
        }
    }
    for (; row < row_end; ++row) {
        const float* weight_row = weights + row * input_size;
        output[row] = dot_f32_avx2_unchecked(weight_row, input, input_size) +
                      (bias == nullptr ? 0.0F : bias[row]);
    }
}

F32RowMax max_dot_f32_rows_avx2_unchecked(const float* weights,
                                          const float* input,
                                          std::size_t input_size,
                                          std::size_t row_begin,
                                          std::size_t row_end) noexcept {
    F32RowMax best;
    best.row = row_begin;
    best.value = -std::numeric_limits<float>::infinity();
    std::size_t row = row_begin;
    for (; row + LCQI_F32_AVX2_UNROLL_COUNT <= row_end;
         row += LCQI_F32_AVX2_UNROLL_COUNT) {
        const float* row0 = weights + row * input_size;
        const float* row1 = row0 + input_size;
        const float* row2 = row1 + input_size;
        const float* row3 = row2 + input_size;
        std::array<float, LCQI_F32_AVX2_UNROLL_COUNT> sums{};
        compute_four_row_dot(row0, row1, row2, row3, input, input_size, sums.data());
        for (std::size_t offset = 0; offset < LCQI_F32_AVX2_UNROLL_COUNT; ++offset) {
            if (row + offset == row_begin || sums[offset] > best.value) {
                best.row = row + offset;
                best.value = sums[offset];
            }
        }
    }
    for (; row < row_end; ++row) {
        const float* weight_row = weights + row * input_size;
        const float value = dot_f32_avx2_unchecked(weight_row, input, input_size);
        if (row == row_begin || value > best.value) {
```

```
                best.row = row;
                best.value = value;
            }
        }
        return best;
}

}  // namespace lcqi

#endif
```

Listing 10.13 src/gpt2_reference.cpp

```
#include <lcqi/gpt2_reference.hpp>

#include <lcqi/f32_kernels.hpp>
#include <lcqi/safetensors.hpp>

#include <algorithm>
#include <array>
#include <atomic>
#include <chrono>
#include <cmath>
#include <cstddef>
#include <fstream>
#include <limits>
#include <optional>
#include <sstream>
#include <stdexcept>
#include <string>
#include <string_view>
#include <thread>
#include <unordered_set>
#include <utility>

namespace lcqi {
namespace {

constexpr std::int32_t LCQI_GPT2_DEFAULT_BOS_EOS = 50256;
constexpr std::int32_t LCQI_GPT2_ATTENTION_PROJECTIONS = 3;
constexpr std::int32_t LCQI_GPT2_PAIR_TOKEN_COUNT = 2;
constexpr std::int32_t LCQI_GPT2_BYTE_COUNT = 256;
constexpr std::int32_t LCQI_GPT2_UNICODE_PRIVATE_BASE = 256;
constexpr std::int32_t LCQI_GPT2_ASCII_EXCLAMATION = 33;
constexpr std::int32_t LCQI_GPT2_ASCII_TILDE = 126;
constexpr std::int32_t LCQI_GPT2_LATIN_INVERTED_EXCLAMATION = 161;
constexpr std::int32_t LCQI_GPT2_LATIN_NOT_SIGN = 172;
constexpr std::int32_t LCQI_GPT2_LATIN_REGISTERED = 174;
constexpr std::int32_t LCQI_GPT2_LATIN_Y_DIAERESIS = 255;
constexpr std::int32_t LCQI_GPT2_UTF8_CONTINUATION_MASK = 0x3F;
constexpr std::int32_t LCQI_GPT2_UTF8_CONTINUATION_BITS = 0x80;
constexpr std::int32_t LCQI_GPT2_UTF8_TWO_BYTE_HEAD = 0xC0;
```

```cpp
constexpr std::int32_t LCQI_GPT2_UTF8_THREE_BYTE_HEAD = 0xE0;
constexpr std::int32_t LCQI_GPT2_UTF8_FOUR_BYTE_HEAD = 0xF0;
constexpr std::int32_t LCQI_GPT2_UTF8_ONE_BYTE_LIMIT = 0x80;
constexpr std::int32_t LCQI_GPT2_UTF8_TWO_BYTE_LIMIT = 0x800;
constexpr std::int32_t LCQI_GPT2_UTF8_THREE_BYTE_LIMIT = 0x10000;
constexpr unsigned char LCQI_GPT2_ASCII_APOSTROPHE = 0x27U;
constexpr float LCQI_GPT2_GELU_SQRT_2_OVER_PI = 0.7978845608028654F;
constexpr float LCQI_GPT2_GELU_CUBIC_COEFF = 0.044715F;
constexpr float LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY =
    -std::numeric_limits<float>::infinity();
constexpr std::int32_t LCQI_GPT2_MAX_AUTO_WORKERS = 8;
constexpr std::int32_t LCQI_GPT2_DEFAULT_PARALLEL_MIN_ROWS = 512;
constexpr std::int32_t LCQI_GPT2_PARALLEL_MIN_COLUMNS = 512;
constexpr std::int64_t LCQI_GPT2_PARALLEL_MIN_OPS = 1'000'000;
constexpr std::size_t LCQI_GPT2_PARALLEL_ROW_BLOCK_MULTIPLIER = 4;
constexpr std::size_t LCQI_GPT2_WORKER_SPIN_PAUSES_BEFORE_YIELD = 4096;

using Gpt2ProfileClock = std::chrono::steady_clock;

void pause_worker_spin(std::size_t& pause_count) noexcept {
#if defined(__GNUC__) && (defined(__x86_64__) || defined(__i386__))
    __builtin_ia32_pause();
#endif
    ++pause_count;
    if (pause_count >= LCQI_GPT2_WORKER_SPIN_PAUSES_BEFORE_YIELD) {
        pause_count = 0;
        std::this_thread::yield();
    }
}

}  // namespace

namespace detail {

class Gpt2ParallelWorkerPool {
public:
    explicit Gpt2ParallelWorkerPool(std::int32_t requested_workers)
        : worker_count_(Gpt2ParallelWorkerPool::normalize_worker_count(requested_workers)) {
        if (this->worker_count_ <= 1) {
            return;
        }
        const std::size_t background_count =
            static_cast<std::size_t>(this->worker_count_ - 1);
        this->threads_.reserve(background_count);
        for (std::size_t index = 0; index < background_count; ++index) {
            this->threads_.emplace_back([this, index]() { this->worker_loop(index); });
        }
    }

    ~Gpt2ParallelWorkerPool() {
```

```cpp
        this->stopping_.store(true, std::memory_order_release);
        this->generation_.fetch_add(1, std::memory_order_release);
        for (std::thread& thread : this->threads_) {
            if (thread.joinable()) {
                thread.join();
            }
        }
    }

    Gpt2ParallelWorkerPool(const Gpt2ParallelWorkerPool&) = delete;
    Gpt2ParallelWorkerPool& operator=(const Gpt2ParallelWorkerPool&) = delete;

    [[nodiscard]] std::int32_t worker_count() const noexcept {
        return this->worker_count_;
    }

    template <typename Function>
    void parallel_for_rows(std::size_t begin,
                           std::size_t end,
                           std::size_t grain,
                           Function&& function) {
        if (end <= begin) {
            return;
        }
        const std::size_t row_count = end - begin;
        if (this->worker_count_ <= 1 || row_count <= grain) {
            function(begin, end);
            return;
        }

        const std::size_t task_count =
            std::min<std::size_t>(static_cast<std::size_t>(this->worker_count_),
                                  (row_count + grain - 1) / grain);
        if (task_count <= 1) {
            function(begin, end);
            return;
        }

        auto task_context = ParallelTaskContext<Function>{function};
        // Publish all task metadata before the generation bump; workers use the
        // generation acquire load as the hand-off point for this stack context.
        this->task_begin_.store(begin, std::memory_order_release);
        this->task_end_.store(end, std::memory_order_release);
        this->task_count_.store(task_count, std::memory_order_release);
        this->task_context_.store(&task_context, std::memory_order_release);
        this->task_invoke_.store(&ParallelTaskContext<Function>::invoke,
                                 std::memory_order_release);
        this->active_workers_.store(task_count - 1, std::memory_order_release);
        this->generation_.fetch_add(1, std::memory_order_release);
```

```

        const std::size_t main_worker = task_count - 1;
        const auto [main_begin, main_end] = this->chunk_bounds(begin, end, task_count, main_worker);
        function(main_begin, main_end);

        std::size_t pause_count = 0;
        while (this->active_workers_.load(std::memory_order_acquire) != 0) {
            pause_worker_spin(pause_count);
        }
        this->task_context_.store(nullptr, std::memory_order_release);
        this->task_invoke_.store(nullptr, std::memory_order_release);
    }

private:
    template <typename Function>
    struct ParallelTaskContext {
        Function& function;

        static void invoke(void* context, std::size_t begin, std::size_t end) {
            auto* typed_context = static_cast<ParallelTaskContext*>(context);
            typed_context->function(begin, end);
        }
    };

    std::int32_t worker_count_ = 1;
    std::vector<std::thread> threads_;
    std::atomic<bool> stopping_ = false;
    std::atomic<std::uint64_t> generation_ = 0;
    std::atomic<std::size_t> task_begin_ = 0;
    std::atomic<std::size_t> task_end_ = 0;
    std::atomic<std::size_t> task_count_ = 0;
    std::atomic<std::size_t> active_workers_ = 0;
    std::atomic<void*> task_context_ = nullptr;
    std::atomic<void (*)(void*, std::size_t, std::size_t)> task_invoke_ = nullptr;

    [[nodiscard]] static std::int32_t normalize_worker_count(std::int32_t requested_workers) {
        if (requested_workers > 0) {
            return requested_workers;
        }
        const unsigned int hardware_workers = std::thread::hardware_concurrency();
        if (hardware_workers == 0) {
            return 1;
        }
        const std::int32_t capped =
            std::min<std::int32_t>(static_cast<std::int32_t>(hardware_workers),
                                   LCQI_GPT2_MAX_AUTO_WORKERS);
        return std::max(capped, 1);
    }
```

```cpp

    [[nodiscard]] std::pair<std::size_t, std::size_t> chunk_bounds(
        std::size_t begin,
        std::size_t end,
        std::size_t task_count,
        std::size_t worker_index) const {
        const std::size_t row_count = end - begin;
        const std::size_t chunk_begin = begin + row_count * worker_index / task_count;
        const std::size_t chunk_end = begin + row_count * (worker_index + 1) / task_count;
        return {chunk_begin, chunk_end};
    }

    void worker_loop(std::size_t worker_index) {
        std::uint64_t observed_generation = 0;
        std::size_t pause_count = 0;
        while (true) {
            const std::uint64_t current_generation =
                this->generation_.load(std::memory_order_acquire);
            if (current_generation == observed_generation) {
                if (this->stopping_.load(std::memory_order_acquire)) {
                    return;
                }
                pause_worker_spin(pause_count);
                continue;
            }

            observed_generation = current_generation;
            pause_count = 0;
            if (this->stopping_.load(std::memory_order_acquire)) {
                return;
            }

            const std::size_t task_count = this->task_count_.load(std::memory_order_acquire);
            if (this->generation_.load(std::memory_order_acquire) != current_generation) {
                continue;
            }
            const std::size_t background_task_count = task_count - 1;
            if (worker_index >= background_task_count) {
                continue;
            }
            void* task_context = this->task_context_.load(std::memory_order_acquire);
            void (*task_invoke)(void*, std::size_t, std::size_t) =
                this->task_invoke_.load(std::memory_order_acquire);
            if (task_context == nullptr || task_invoke == nullptr) {
                continue;
            }
            const std::size_t task_begin = this->task_begin_.load(std::::
```

```
memory_order_acquire);
            const std::size_t task_end = this->task_end_.load(std::memory_order_acquire);
            if (this->generation_.load(std::memory_order_acquire) != current_generation) {
                continue;
            }
            const auto [begin, end] =
                this->chunk_bounds(task_begin,
                                   task_end,
                                   task_count,
                                   worker_index);
            task_invoke(task_context, begin, end);

            this->active_workers_.fetch_sub(1, std::memory_order_acq_rel);
        }
    }
};

}  // namespace detail

namespace {

class JsonCursor {
public:
    explicit JsonCursor(std::string_view text) : text_(text) {}

    void expect(char expected) {
        this->skip_ws();
        if (this->eof() || this->text_[this->position_] != expected) {
            throw std::runtime_error("invalid JSON syntax");
        }
        ++this->position_;
    }

    [[nodiscard]] bool consume(char expected) {
        this->skip_ws();
        if (!this->eof() && this->text_[this->position_] == expected) {
            ++this->position_;
            return true;
        }
        return false;
    }

    [[nodiscard]] std::string parse_string() {
        this->skip_ws();
        this->expect('"');
        std::string result;
        while (!this->eof()) {
            const char ch = this->text_[this->position_++];
            if (ch == '"') {
                return result;
```

```
            }
            if (ch == '\\') {
                result += this->parse_escape();
            } else {
                result.push_back(ch);
            }
        }
        throw std::runtime_error("unterminated JSON string");
    }

    [[nodiscard]] std::int64_t parse_i64() {
        this->skip_ws();
        bool negative = false;
        if (this->consume('-')) {
            negative = true;
        }
        if (this->eof() || this->text_[this->position_] < '0' ||
            this->text_[this->position_] > '9') {
            throw std::runtime_error("expected JSON integer");
        }
        std::int64_t value = 0;
        while (!this->eof() && this->text_[this->position_] >= '0' &&
               this->text_[this->position_] <= '9') {
            const std::int64_t digit = this->text_[this->position_] - '0';
            if (value >
                (std::numeric_limits<std::int64_t>::max() - digit) / 10) {
                throw std::runtime_error("JSON integer overflow");
            }
            value = value * 10 + digit;
            ++this->position_;
        }
        return negative ? -value : value;
    }

    [[nodiscard]] double parse_number() {
        this->skip_ws();
        const std::size_t begin = this->position_;
        if (!this->eof() && this->text_[this->position_] == '-') {
            ++this->position_;
        }
        while (!this->eof() && this->text_[this->position_] >= '0' &&
               this->text_[this->position_] <= '9') {
            ++this->position_;
        }
        if (!this->eof() && this->text_[this->position_] == '.') {
            ++this->position_;
            while (!this->eof() && this->text_[this->position_] >= '0' &&
                   this->text_[this->position_] <= '9') {
                ++this->position_;
            }
        }
        if (!this->eof() &&
```

```
            (this->text_[this->position_] == 'e' || this->text_[this->position_] == 'E')) {
            ++this->position_;
            if (!this->eof() &&
                (this->text_[this->position_] == '+' || this->text_[this->position_] == '-')) {
                ++this->position_;
            }
            while (!this->eof() && this->text_[this->position_] >= '0' &&
                   this->text_[this->position_] <= '9') {
                ++this->position_;
            }
        }
        if (begin == this->position_) {
            throw std::runtime_error("expected JSON number");
        }
        return std::stod(std::string(this->text_.substr(begin, this->position_ - begin)));
    }

    void skip_value() {
        this->skip_ws();
        if (this->eof()) {
            throw std::runtime_error("unexpected end of JSON");
        }
        const char ch = this->text_[this->position_];
        if (ch == '"') {
            static_cast<void>(this->parse_string());
            return;
        }
        if (ch == '{') {
            this->expect('{');
            if (this->consume('}')) {
                return;
            }
            while (true) {
                static_cast<void>(this->parse_string());
                this->expect(':');
                this->skip_value();
                if (this->consume('}')) {
                    return;
                }
                this->expect(',');
            }
        }
        if (ch == '[') {
            this->expect('[');
            if (this->consume(']')) {
                return;
            }
            while (true) {
                this->skip_value();
```

```
                if (this->consume(']')) {
                    return;
                }
                this->expect(',');
            }
        }
        if (ch == 't') {
            this->expect_literal("true");
            return;
        }
        if (ch == 'f') {
            this->expect_literal("false");
            return;
        }
        if (ch == 'n') {
            this->expect_literal("null");
            return;
        }
        static_cast<void>(this->parse_number());
    }

    [[nodiscard]] bool eof_after_ws() {
        this->skip_ws();
        return this->eof();
    }

private:
    std::string_view text_;
    std::size_t position_ = 0;

    [[nodiscard]] bool eof() const {
        return this->position_ >= this->text_.size();
    }

    void skip_ws() {
        while (!this->eof() &&
               (this->text_[this->position_] == ' ' ||
                this->text_[this->position_] == '\n' ||
                this->text_[this->position_] == '\r' ||
                this->text_[this->position_] == '\t')) {
            ++this->position_;
        }
    }

    void expect_literal(std::string_view literal) {
        for (const char expected : literal) {
            if (this->eof() || this->text_[this->position_] != expected) {
                throw std::runtime_error("invalid JSON literal");
            }
            ++this->position_;
        }
    }
```

```

    [[nodiscard]] std::string parse_escape() {
        if (this->eof()) {
            throw std::runtime_error("unterminated JSON escape");
        }
        const char escaped = this->text_[this->position_++];
        switch (escaped) {
            case '"':
            case '\\':
            case '/':
                return std::string(1, escaped);
            case 'b':
                return "\b";
            case 'f':
                return "\f";
            case 'n':
                return "\n";
            case 'r':
                return "\r";
            case 't':
                return "\t";
            case 'u':
                return this->parse_unicode_escape();
            default:
                throw std::runtime_error("unsupported JSON escape");
        }
    }

    [[nodiscard]] std::string parse_unicode_escape() {
        std::uint32_t value = 0;
        for (std::int32_t i = 0; i < 4; ++i) {
            if (this->eof()) {
                throw std::runtime_error("unterminated JSON unicode escape");
            }
            const char ch = this->text_[this->position_++];
            value <<= 4U;
            if (ch >= '0' && ch <= '9') {
                value += static_cast<std::uint32_t>(ch - '0');
            } else if (ch >= 'a' && ch <= 'f') {
                value += static_cast<std::uint32_t>(10 + ch - 'a');
            } else if (ch >= 'A' && ch <= 'F') {
                value += static_cast<std::uint32_t>(10 + ch - 'A');
            } else {
                throw std::runtime_error("invalid JSON unicode escape");
            }
        }
        std::string encoded;
        if (value < LCQI_GPT2_UTF8_ONE_BYTE_LIMIT) {
            encoded.push_back(static_cast<char>(value));
        } else if (value < LCQI_GPT2_UTF8_TWO_BYTE_LIMIT) {
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_TWO_BYTE_HEAD | (value >> 6U)));
```

```
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_CONTINUATION_BITS |
                (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        } else {
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_THREE_BYTE_HEAD | (value >> 12U)));
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_CONTINUATION_BITS |
                ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_CONTINUATION_BITS |
                (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        }
        return encoded;
    }
};

std::string read_text_file(const std::filesystem::path& path) {
    std::ifstream input(path);
    if (!input) {
        throw std::runtime_error("cannot open text file: " + path.string());
    }
    std::ostringstream buffer;
    buffer << input.rdbuf();
    return buffer.str();
}

std::size_t checked_size(std::int64_t value, const char* name) {
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
}

std::string encode_codepoint(std::uint32_t value) {
    std::string encoded;
    if (value < LCQI_GPT2_UTF8_ONE_BYTE_LIMIT) {
        encoded.push_back(static_cast<char>(value));
    } else if (value < LCQI_GPT2_UTF8_TWO_BYTE_LIMIT) {
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_TWO_BYTE_HEAD | (value >> 6U)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
    } else if (value < LCQI_GPT2_UTF8_THREE_BYTE_LIMIT) {
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_THREE_BYTE_HEAD | (value >> 12U)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
```

```
            (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
    } else {
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_FOUR_BYTE_HEAD | (value >> 18U)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            ((value >> 12U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
    }
    return encoded;
}

void require_positive(std::int32_t value, const char* name) {
    if (value <= 0) {
        throw std::runtime_error(std::string(name) + " must be positive");
    }
}

void validate_config(const Gpt2Config& config) {
    require_positive(config.hidden_size, "hidden_size");
    require_positive(config.head_count, "head_count");
    require_positive(config.layer_count, "layer_count");
    require_positive(config.max_positions, "max_positions");
    require_positive(config.vocab_size, "vocab_size");
    require_positive(config.intermediate_size, "intermediate_size");
    if (config.hidden_size % config.head_count != 0) {
        throw std::runtime_error("hidden_size must be divisible by head_count");
    }
    if (config.layer_norm_epsilon <= 0.0F) {
        throw std::runtime_error("layer_norm_epsilon must be positive");
    }
}

std::int32_t head_dim(const Gpt2Config& config) {
    return config.hidden_size / config.head_count;
}

std::size_t matrix_size(std::int32_t rows, std::int32_t columns) {
    return checked_size(rows, "rows") * checked_size(columns, "columns");
}

Gpt2ProfileClock::time_point profile_start(const Gpt2HotspotProfile* profile) {
    if (profile == nullptr) {
        return Gpt2ProfileClock::time_point{};
    }
    return Gpt2ProfileClock::now();
```

```cpp
}

void add_profile_ms(Gpt2HotspotProfile* profile,
                    double Gpt2HotspotProfile::*field,
                    Gpt2ProfileClock::time_point start) {
    if (profile == nullptr) {
        return;
    }
    (*profile).*field +=
        std::chrono::duration<double, std::milli>(Gpt2ProfileClock::now() - start).count();
}

void validate_vector_size(std::size_t actual, std::int32_t expected, const char* name) {
    if (actual != checked_size(expected, name)) {
        throw std::runtime_error(std::string(name) + " size mismatch");
    }
}

void validate_linear(const Gpt2LinearF32& linear, const char* name) {
    require_positive(linear.input_size, "linear input_size");
    require_positive(linear.output_size, "linear output_size");
    if (linear.weights.size() != matrix_size(linear.output_size, linear.input_size)) {
        throw std::runtime_error(std::string(name) + " weight size mismatch");
    }
    if (!linear.bias.empty() &&
        linear.bias.size() != checked_size(linear.output_size, "linear output_size")) {
        throw std::runtime_error(std::string(name) + " bias size mismatch");
    }
}

void validate_model(const Gpt2ReferenceModel& model) {
    validate_config(model.config);
    if (model.layers.size() != checked_size(model.config.layer_count, "layer_count")) {
        throw std::runtime_error("GPT-2 layer count mismatch");
    }
    if (model.token_embedding.size() !=
        matrix_size(model.config.vocab_size, model.config.hidden_size)) {
        throw std::runtime_error("GPT-2 token embedding size mismatch");
    }
    if (model.position_embedding.size() !=
        matrix_size(model.config.max_positions, model.config.hidden_size)) {
        throw std::runtime_error("GPT-2 position embedding size mismatch");
    }
    validate_vector_size(model.final_ln_weight.size(), model.config.hidden_size, "ln_f weight");
    validate_vector_size(model.final_ln_bias.size(), model.config.hidden_size, "ln_f bias");
```

```cpp
    if (model.tie_lm_head_to_embedding) {
        if (!model.lm_head_weight.empty()) {
            throw std::runtime_error("tied GPT-2 lm_head should not carry separate weights");
        }
    } else if (model.lm_head_weight.size() !=
               matrix_size(model.config.vocab_size, model.config.hidden_size)) {
        throw std::runtime_error("GPT-2 lm_head size mismatch");
    }
    for (const Gpt2LayerWeightsF32& layer : model.layers) {
        validate_vector_size(layer.ln_1_weight.size(), model.config.hidden_size, "ln_1 weight");
        validate_vector_size(layer.ln_1_bias.size(), model.config.hidden_size, "ln_1 bias");
        validate_linear(layer.c_attn, "c_attn");
        validate_linear(layer.c_proj, "c_proj");
        validate_vector_size(layer.ln_2_weight.size(), model.config.hidden_size, "ln_2 weight");
        validate_vector_size(layer.ln_2_bias.size(), model.config.hidden_size, "ln_2 bias");
        validate_linear(layer.c_fc, "c_fc");
        validate_linear(layer.mlp_c_proj, "mlp_c_proj");
    }
}

std::span<const float> row_span(std::span<const float> values,
                                std::int32_t row,
                                std::int32_t row_width) {
    return values.subspan(checked_size(row, "row") * checked_size(row_width, "row_width"),
                          checked_size(row_width, "row_width"));
}

void add_inplace(std::span<float> target, std::span<const float> source) {
    if (target.size() != source.size()) {
        throw std::runtime_error("GPT-2 residual add size mismatch");
    }
    for (std::size_t index = 0; index < target.size(); ++index) {
        target[index] += source[index];
    }
}

void layer_norm(std::span<const float> input,
                std::span<const float> weight,
                std::span<const float> bias,
                float epsilon,
                std::span<float> output) {
    if (input.empty() || input.size() != weight.size() || input.size() != bias.size() ||
        input.size() != output.size()) {
        throw std::runtime_error("GPT-2 LayerNorm size mismatch");
```

```cpp
    }
    float mean = 0.0F;
    for (const float value : input) {
        mean += value;
    }
    mean /= static_cast<float>(input.size());

    float variance = 0.0F;
    for (const float value : input) {
        const float centered = value - mean;
        variance += centered * centered;
    }
    variance /= static_cast<float>(input.size());
    const float scale = 1.0F / std::sqrt(variance + epsilon);

    for (std::size_t index = 0; index < input.size(); ++index) {
        output[index] = (input[index] - mean) * scale * weight[index] + bias[index];
    }
}

void linear_f32(const Gpt2LinearF32& linear,
                std::span<const float> input,
                std::span<float> output) {
    validate_linear(linear, "GPT-2 linear");
    if (input.size() != checked_size(linear.input_size, "linear input") ||
        output.size() != checked_size(linear.output_size, "linear output")) {
        throw std::runtime_error("GPT-2 linear input/output size mismatch");
    }
    const std::size_t input_size = checked_size(linear.input_size, "linear input");
    const std::size_t output_size = checked_size(linear.output_size, "linear output");
    for (std::size_t out = 0; out < output_size; ++out) {
        float sum = linear.bias.empty() ? 0.0F : linear.bias[out];
        const std::size_t row_base = out * input_size;
        for (std::size_t in = 0; in < input_size; ++in) {
            sum += linear.weights[row_base + in] * input[in];
        }
        output[out] = sum;
    }
}

void linear_f32_unchecked(const Gpt2LinearF32& linear,
                          std::span<const float> input,
                          std::span<float> output) {
    const std::size_t input_size = static_cast<std::size_t>(linear.input_size);
    const std::size_t output_size = static_cast<std::size_t>(linear.output_size);
    const float* weights = linear.weights.data();
    const float* bias = linear.bias.empty() ? nullptr : linear.bias.data();
    const float* input_data = input.data();
```

```cpp
    linear_f32_rows_unchecked(weights, input_data, bias, input_size, 0, output_size, output.data());
}

[[nodiscard]] bool should_parallelize_rows(const Gpt2ExecutionOptions& options,
                                           std::size_t rows,
                                           std::size_t columns,
                                           const detail::Gpt2ParallelWorkerPool* worker_pool) {
    if (worker_pool == nullptr || worker_pool->worker_count() <= 1) {
        return false;
    }
    const std::int32_t min_rows =
        options.parallel_min_rows > 0 ? options.parallel_min_rows
                                      : LCQI_GPT2_DEFAULT_PARALLEL_MIN_ROWS;
    if (rows < checked_size(min_rows, "parallel_min_rows")) {
        return false;
    }
    if (options.worker_count > 1) {
        return true;
    }
    return columns >= checked_size(LCQI_GPT2_PARALLEL_MIN_COLUMNS, "parallel_min_columns") ||
           rows * columns >= static_cast<std::size_t>(LCQI_GPT2_PARALLEL_MIN_OPS);
}

[[nodiscard]] std::size_t parallel_row_grain(std::size_t rows,
                                             const detail::Gpt2ParallelWorkerPool& worker_pool) {
    const std::size_t workers = checked_size(worker_pool.worker_count(), "worker_count");
    const std::size_t target_tasks = workers * LCQI_GPT2_PARALLEL_ROW_BLOCK_MULTIPLIER;
    return std::max<std::size_t>(1, (rows + target_tasks - 1) / target_tasks);
}

[[nodiscard]] bool model_has_parallel_work(const Gpt2ReferenceModel& model,
                                           const Gpt2ExecutionOptions& options) {
    const std::size_t hidden_size = checked_size(model.config.hidden_size, "hidden_size");
    const std::size_t vocab_size = checked_size(model.config.vocab_size, "vocab_size");
    const std::size_t intermediate_size =
        checked_size(model.config.intermediate_size, "intermediate_size");
    const std::size_t qkv_rows =
        checked_size(model.config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS,
                     "qkv output rows");
    const std::int32_t min_rows =
        options.parallel_min_rows > 0 ? options.parallel_min_rows
```

```cpp
                                      : LCQI_GPT2_DEFAULT_PARALLEL_MIN_ROWS;
    const auto large_enough = [min_rows](std::size_t rows, std::size_t columns) {
        if (rows < checked_size(min_rows, "parallel_min_rows")) {
            return false;
        }
        return columns >= checked_size(LCQI_GPT2_PARALLEL_MIN_COLUMNS,
                                       "parallel_min_columns") ||
               rows * columns >= static_cast<std::size_t>(LCQI_GPT2_PARALLEL_MIN_OPS);
    };
    if (options.worker_count > 1) {
        return vocab_size >= checked_size(min_rows, "parallel_min_rows") ||
               qkv_rows >= checked_size(min_rows, "parallel_min_rows") ||
               intermediate_size >= checked_size(min_rows, "parallel_min_rows") ||
               hidden_size >= checked_size(min_rows, "parallel_min_rows");
    }
    if (options.worker_count == 1) {
        return false;
    }
    return large_enough(vocab_size, hidden_size) ||
           large_enough(qkv_rows, hidden_size) ||
           large_enough(hidden_size, hidden_size) ||
           large_enough(intermediate_size, hidden_size) ||
           large_enough(hidden_size, intermediate_size);
}

[[nodiscard]] std::unique_ptr<detail::Gpt2ParallelWorkerPool> make_worker_pool(
    const Gpt2ReferenceModel& model,
    const Gpt2ExecutionOptions& options) {
    if (!model_has_parallel_work(model, options)) {
        return nullptr;
    }
    return std::make_unique<detail::Gpt2ParallelWorkerPool>(options.worker_count);
}

void linear_f32_unchecked_parallel(const Gpt2LinearF32& linear,
                                   std::span<const float> input,
                                   std::span<float> output,
                                   const Gpt2ExecutionOptions& options,
                                   detail::Gpt2ParallelWorkerPool* worker_pool) {
    const std::size_t input_size = static_cast<std::size_t>(linear.input_size);
    const std::size_t output_size = static_cast<std::size_t>(linear.output_size);
    if (!should_parallelize_rows(options, output_size, input_size, worker_pool)) {
        linear_f32_unchecked(linear, input, output);
        return;
    }
```

```cpp

    const float* weights = linear.weights.data();
    const float* bias = linear.bias.empty() ? nullptr : linear.bias.data();
    const float* input_data = input.data();
    float* output_data = output.data();
    const auto rows_fn =
        [input_size, weights, bias, input_data, output_data](std::size_t begin,
                                                             std::size_t end) {
            linear_f32_rows_unchecked(weights,
                                      input_data,
                                      bias,
                                      input_size,
                                      begin,
                                      end,
                                      output_data);
        };
    worker_pool->parallel_for_rows(0, output_size, parallel_row_grain(output_size, *worker_pool), rows_fn);
}

float gelu_new(float value) {
    const float cubic = value * value * value;
    return 0.5F * value *
           (1.0F + std::tanh(LCQI_GPT2_GELU_SQRT_2_OVER_PI *
                             (value + LCQI_GPT2_GELU_CUBIC_COEFF * cubic)));
}

std::int32_t argmax(std::span<const float> values) {
    if (values.empty()) {
        throw std::runtime_error("cannot take argmax of empty logits");
    }
    std::int32_t best_index = 0;
    float best_value = values.front();
    for (std::int32_t index = 1; index < static_cast<std::int32_t>(values.size()); ++index) {
        const float value = values[checked_size(index, "argmax index")];
        if (value > best_value) {
            best_value = value;
            best_index = index;
        }
    }
    return best_index;
}

void project_qkv(const Gpt2Config& config,
                 const Gpt2LayerWeightsF32& layer,
                 std::span<const float> hidden,
                 std::span<float> q,
                 std::span<float> k,
                 std::span<float> v) {
    std::vector<float> packed(matrix_size(LCQI_GPT2_ATTENTION_PROJECTIONS,
                                          config.hidden_size),
```

```
                              0.0F);
    linear_f32(layer.c_attn, hidden, packed);
    const std::size_t width = checked_size(config.hidden_size, "hidden_size");
    std::copy_n(packed.begin(), config.hidden_size, q.begin());
    std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width),
                config.hidden_size,
                k.begin());
    std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width * 2),
                config.hidden_size,
                v.begin());
}

void project_qkv_reuse_workspace(const Gpt2Config& config,
                                 const Gpt2LayerWeightsF32& layer,
                                 std::span<const float> hidden,
                                 std::span<float> packed,
                                 std::span<float> q,
                                 std::span<float> k,
                                 std::span<float> v,
                                 const Gpt2ExecutionOptions& options,
                                 detail::Gpt2ParallelWorkerPool* worker_pool) {
    linear_f32_unchecked_parallel(layer.c_attn, hidden, packed, options, worker_pool);
    const std::size_t width = checked_size(config.hidden_size, "hidden_size");
    std::copy_n(packed.begin(), config.hidden_size, q.begin());
    std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width),
                config.hidden_size,
                k.begin());
    std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width * 2),
                config.hidden_size,
                v.begin());
}

void attention_full_sequence(const Gpt2Config& config,
                             std::span<const float> q_by_pos,
                             std::span<const float> k_by_pos,
                             std::span<const float> v_by_pos,
                             std::int32_t sequence_length,
                             std::span<float> output_by_pos) {
    const std::int32_t local_head_dim = head_dim(config);
    const float score_scale = 1.0F / std::sqrt(static_cast<float>(local_head_dim));
    std::fill(output_by_pos.begin(), output_by_pos.end(), 0.0F);
    std::vector<float> scores(checked_size(sequence_length, "sequence_length"),
                              LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY);

    for (std::int32_t position = 0; position < sequence_length; ++position) {
        for (std::int32_t head = 0; head < config.head_count; ++head) {
            float max_score = LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY;
            for (std::int32_t past = 0; past <= position; ++past) {
                float dot = 0.0F;
                for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
```

```cpp
                    const std::size_t q_index =
                        matrix_size(position, config.hidden_size) +
                        matrix_size(head, local_head_dim) + checked_size(dim, "head dim");
                    const std::size_t k_index =
                        matrix_size(past, config.hidden_size) +
                        matrix_size(head, local_head_dim) + checked_size(dim, "head dim");
                    dot += q_by_pos[q_index] * k_by_pos[k_index];
                }
                const float score = dot * score_scale;
                scores[checked_size(past, "past")] = score;
                max_score = std::max(max_score, score);
            }

            float denominator = 0.0F;
            for (std::int32_t past = 0; past <= position; ++past) {
                const std::size_t index = checked_size(past, "past");
                scores[index] = std::exp(scores[index] - max_score);
                denominator += scores[index];
            }
            if (denominator <= 0.0F) {
                throw std::runtime_error("GPT-2 attention denominator is invalid");
            }
            for (std::int32_t past = 0; past <= position; ++past) {
                const float probability = scores[checked_size(past, "past")] / denominator;
                for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
                    const std::size_t output_index =
                        matrix_size(position, config.hidden_size) +
                        matrix_size(head, local_head_dim) + checked_size(dim, "head dim");
                    const std::size_t value_index =
                        matrix_size(past, config.hidden_size) +
                        matrix_size(head, local_head_dim) + checked_size(dim, "head dim");
                    output_by_pos[output_index] += probability * v_by_pos[value_index];
                }
            }
        }
    }
}

void forward_gpt2_layer(const Gpt2Config& config,
                        const Gpt2LayerWeightsF32& layer,
                        std::int32_t sequence_length,
                        std::vector<float>& hidden_by_pos) {
    std::vector<float> normed(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> q_by_pos(matrix_size(sequence_length, config.hidden_size
```

```cpp
    ), 0.0F);
    std::vector<float> k_by_pos(q_by_pos.size(), 0.0F);
    std::vector<float> v_by_pos(q_by_pos.size(), 0.0F);
    std::vector<float> attention_by_pos(q_by_pos.size(), 0.0F);
    std::vector<float> projected(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> mlp_fc(checked_size(config.intermediate_size, "intermediate_size"), 0.0F);
    std::vector<float> mlp_out(checked_size(config.hidden_size, "hidden_size"), 0.0F);

    for (std::int32_t position = 0; position < sequence_length; ++position) {
        const std::span<float> hidden = std::span<float>(
            hidden_by_pos.data() + matrix_size(position, config.hidden_size),
            checked_size(config.hidden_size, "hidden_size"));
        layer_norm(hidden,
                   layer.ln_1_weight,
                   layer.ln_1_bias,
                   config.layer_norm_epsilon,
                   normed);
        std::span<float> q = std::span<float>(
            q_by_pos.data() + matrix_size(position, config.hidden_size),
            checked_size(config.hidden_size, "hidden_size"));
        std::span<float> k = std::span<float>(
            k_by_pos.data() + matrix_size(position, config.hidden_size),
            checked_size(config.hidden_size, "hidden_size"));
        std::span<float> v = std::span<float>(
            v_by_pos.data() + matrix_size(position, config.hidden_size),
            checked_size(config.hidden_size, "hidden_size"));
        project_qkv(config, layer, normed, q, k, v);
    }

    attention_full_sequence(config, q_by_pos, k_by_pos, v_by_pos, sequence_length, attention_by_pos);

    for (std::int32_t position = 0; position < sequence_length; ++position) {
        const std::span<float> hidden = std::span<float>(
            hidden_by_pos.data() + matrix_size(position, config.hidden_size),
            checked_size(config.hidden_size, "hidden_size"));
        const std::span<const float> attention = std::span<const float>(
            attention_by_pos.data() + matrix_size(position, config.hidden_size),
            checked_size(config.hidden_size, "hidden_size"));
        linear_f32(layer.c_proj, attention, projected);
        add_inplace(hidden, projected);

        layer_norm(hidden,
                   layer.ln_2_weight,
                   layer.ln_2_bias,
                   config.layer_norm_epsilon,
                   normed);
        linear_f32(layer.c_fc, normed, mlp_fc);
```

```
        for (float& value : mlp_fc) {
            value = gelu_new(value);
        }
        linear_f32(layer.mlp_c_proj, mlp_fc, mlp_out);
        add_inplace(hidden, mlp_out);
    }
}

void forward_gpt2_layer_cached(const Gpt2Config& config,
                               const Gpt2LayerWeightsF32& layer,
                               std::int32_t layer_id,
                               std::int32_t model_position,
                               Gpt2KvCache& cache,
                               Gpt2ForwardWorkspace& workspace,
                               std::span<float> hidden,
                               Gpt2HotspotProfile* hotspot_profile,
                               const Gpt2ExecutionOptions& options,
                               detail::Gpt2ParallelWorkerPool* worker_pool) {
    if (hotspot_profile != nullptr) {
        ++hotspot_profile->layer_steps;
    }
    std::span<float> normed = workspace.normed();
    std::span<float> query = workspace.query();
    std::span<float> key = workspace.key();
    std::span<float> value = workspace.value();
    std::span<float> attention = workspace.attention();
    std::span<float> projected = workspace.projected();
    std::span<float> qkv_packed = workspace.qkv_packed();
    std::span<float> mlp_fc = workspace.mlp_fc();
    std::span<float> mlp_out = workspace.mlp_out();
    std::span<float> scores = workspace.scores_prefix(model_position + 1);

    Gpt2ProfileClock::time_point start =
        profile_start(hotspot_profile);
    layer_norm(hidden,
               layer.ln_1_weight,
               layer.ln_1_bias,
               config.layer_norm_epsilon,
               normed);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::layer_norm_ms, start);

    start = profile_start(hotspot_profile);
    project_qkv_reuse_workspace(config,
                                layer,
                                normed,
                                qkv_packed,
                                query,
                                key,
                                value,
                                options,
                                worker_pool);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::qkv_projection_ms, ↩
```

```
        start);

    start = profile_start(hotspot_profile);
    cache.append(layer_id, model_position, key, value);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::kv_append_ms, start);

    start = profile_start(hotspot_profile);
    detail::attend_cached_position(cache,
                                   layer_id,
                                   model_position,
                                   query,
                                   scores,
                                   attention);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::attention_ms, start);

    start = profile_start(hotspot_profile);
    linear_f32_unchecked_parallel(layer.c_proj, attention, projected, options, worker_pool);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::attention_projection_ms, start);

    start = profile_start(hotspot_profile);
    add_inplace(hidden, projected);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::residual_add_ms, start);

    start = profile_start(hotspot_profile);
    layer_norm(hidden,
               layer.ln_2_weight,
               layer.ln_2_bias,
               config.layer_norm_epsilon,
               normed);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::layer_norm_ms, start);

    start = profile_start(hotspot_profile);
    linear_f32_unchecked_parallel(layer.c_fc, normed, mlp_fc, options, worker_pool);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::mlp_fc_ms, start);

    start = profile_start(hotspot_profile);
    for (float& value_ref : mlp_fc) {
        value_ref = gelu_new(value_ref);
    }
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::gelu_ms, start);

    start = profile_start(hotspot_profile);
    linear_f32_unchecked_parallel(layer.mlp_c_proj, mlp_fc, mlp_out, options, worker_pool);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::mlp_projection_ms, start);

    start = profile_start(hotspot_profile);
```

```
    add_inplace(hidden, mlp_out);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::residual_add_ms, start);
}

void compute_logits(const Gpt2ReferenceModel& model,
                    std::span<const float> hidden,
                    std::span<float> logits,
                    const Gpt2ExecutionOptions& options,
                    detail::Gpt2ParallelWorkerPool* worker_pool) {
    if (logits.size() != checked_size(model.config.vocab_size, "vocab_size")) {
        throw std::runtime_error("GPT-2 logits size mismatch");
    }
    const std::span<const float> weight =
        model.tie_lm_head_to_embedding ? std::span<const float>(model.token_embedding)
                                       : std::span<const float>(model.lm_head_weight);
    const std::size_t hidden_size = checked_size(model.config.hidden_size, "hidden_size");
    const std::size_t vocab_size = checked_size(model.config.vocab_size, "vocab_size");
    const float* weight_data = weight.data();
    const float* hidden_data = hidden.data();
    const auto rows_fn =
        [hidden_size, weight_data, hidden_data, logits_data = logits.data()](
            std::size_t begin,
            std::size_t end) {
            linear_f32_rows_unchecked(weight_data,
                                      hidden_data,
                                      nullptr,
                                      hidden_size,
                                      begin,
                                      end,
                                      logits_data);
        };
    if (should_parallelize_rows(options, vocab_size, hidden_size, worker_pool)) {
        worker_pool->parallel_for_rows(0,
                                       vocab_size,
                                       parallel_row_grain(vocab_size, *worker_pool),
                                       rows_fn);
    } else {
        rows_fn(0, vocab_size);
    }
}

void compute_logits(const Gpt2ReferenceModel& model,
                    std::span<const float> hidden,
                    std::span<float> logits) {
    Gpt2ExecutionOptions options;
```

```
    options.worker_count = 1;
    compute_logits(model, hidden, logits, options, nullptr);
}

struct Gpt2TokenScore {
    std::int32_t token = 0;
    float value = -std::numeric_limits<float>::infinity();
};

enum class Gpt2CachedStepMode {
    advance_only,
    predict_token,
    logits,
};

[[nodiscard]] Gpt2TokenScore compute_predicted_token_range(const float* weight_data,
                                                          const float* hidden_data,
                                                          std::size_t hidden_size,
                                                          std::size_t begin,
                                                          std::size_t end) {
    Gpt2TokenScore best;
    const F32RowMax row_max =
        max_dot_f32_rows_unchecked(weight_data, hidden_data, hidden_size, begin, end);
    best.token = static_cast<std::int32_t>(row_max.row);
    best.value = row_max.value;
    return best;
}

std::int32_t compute_predicted_token(const Gpt2ReferenceModel& model,
                                     std::span<const float> hidden,
                                     const Gpt2ExecutionOptions& options,
                                     detail::Gpt2ParallelWorkerPool* worker_pool) {
    const std::span<const float> weight =
        model.tie_lm_head_to_embedding ? std::span<const float>(model.token_embedding)
                                       : std::span<const float>(model.lm_head_weight);
    const std::size_t hidden_size = checked_size(model.config.hidden_size, "hidden_size");
    const std::size_t vocab_size = checked_size(model.config.vocab_size, "vocab_size");
    const float* weight_data = weight.data();
    const float* hidden_data = hidden.data();
    if (!should_parallelize_rows(options, vocab_size, hidden_size, worker_pool)) {
        return compute_predicted_token_range(weight_data, hidden_data, hidden_size, 0, vocab_size)
```

```cpp
            .token;
    }

    const std::size_t worker_count = checked_size(worker_pool->worker_count(), "worker_count");
    const std::size_t chunk_count = std::min(worker_count, vocab_size);
    std::vector<Gpt2TokenScore> partials(chunk_count);
    const auto rows_fn =
        [chunk_count, hidden_size, vocab_size, weight_data, hidden_data, &partials](
            std::size_t begin,
            std::size_t end) {
            for (std::size_t chunk = begin; chunk < end; ++chunk) {
                const std::size_t token_begin = vocab_size * chunk / chunk_count;
                const std::size_t token_end = vocab_size * (chunk + 1) / chunk_count;
                partials[chunk] = compute_predicted_token_range(
                    weight_data,
                    hidden_data,
                    hidden_size,
                    token_begin,
                    token_end);
            }
        };
    worker_pool->parallel_for_rows(0, chunk_count, 1, rows_fn);

    Gpt2TokenScore best = partials.front();
    for (std::size_t index = 1; index < partials.size(); ++index) {
        if (partials[index].value > best.value) {
            best = partials[index];
        }
    }
    return best.token;
}

Gpt2ForwardResult run_gpt2_cached_step_unchecked(const Gpt2ReferenceModel& model,
                                                 Gpt2KvCache& cache,
                                                 Gpt2ForwardWorkspace& workspace,
                                                 std::int32_t token_id,
                                                 Gpt2CachedStepMode mode,
                                                 Gpt2HotspotProfile* hotspot_profile,
                                                 const Gpt2ExecutionOptions& options,
                                                 detail::Gpt2ParallelWorkerPool* worker_pool) {
    const Gpt2ProfileClock::time_point step_start = profile_start(hotspot_profile);
    if (hotspot_profile != nullptr) {
```

```
        ++hotspot_profile->decoder_steps;
    }
    if (token_id < 0 || token_id >= model.config.vocab_size) {
        throw std::runtime_error("GPT-2 token id out of range");
    }
    const std::int32_t model_position = cache.filled_tokens();
    if (model_position >= model.config.max_positions) {
        throw std::runtime_error("GPT-2 cached forward would exceed max_positions");
    }

    std::span<float> hidden = workspace.hidden();
    const std::span<const float> token =
        row_span(model.token_embedding, token_id, model.config.hidden_size);
    const std::span<const float> position_embedding =
        row_span(model.position_embedding, model_position, model.config.hidden_size);
    Gpt2ProfileClock::time_point start = profile_start(hotspot_profile);
    for (std::int32_t dim = 0; dim < model.config.hidden_size; ++dim) {
        hidden[checked_size(dim, "dim")] =
            token[checked_size(dim, "dim")] + position_embedding[checked_size(dim, "dim")];
    }
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::embedding_ms, start);

    for (std::int32_t layer_id = 0;
         layer_id < static_cast<std::int32_t>(model.layers.size());
         ++layer_id) {
        forward_gpt2_layer_cached(model.config,
                                  model.layers[checked_size(layer_id, "layer_id")],
                                  layer_id,
                                  model_position,
                                  cache,
                                  workspace,
                                  hidden,
                                  hotspot_profile,
                                  options,
                                  worker_pool);
    }

    Gpt2ForwardResult result;
    if (mode == Gpt2CachedStepMode::advance_only) {
        add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::total_step_ms, step_start);
        return result;
    }

    std::span<float> normed = workspace.normed();
    start = profile_start(hotspot_profile);
    layer_norm(hidden,
               model.final_ln_weight,
```

```
               model.final_ln_bias,
               model.config.layer_norm_epsilon,
               normed);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::final_norm_ms, start);

    if (mode == Gpt2CachedStepMode::logits) {
        std::span<float> logits = workspace.logits();
        start = profile_start(hotspot_profile);
        compute_logits(model, normed, logits, options, worker_pool);
        add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::lm_head_ms, start);
        start = profile_start(hotspot_profile);
        result.logits.assign(logits.begin(), logits.end());
        result.predicted_token = argmax(result.logits);
        add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::logits_result_ms, start);
    } else {
        start = profile_start(hotspot_profile);
        result.predicted_token = compute_predicted_token(model, normed, options, worker_pool);
        add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::lm_head_ms, start);
    }
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::total_step_ms, step_start);
    return result;
}

std::vector<std::string> gpt2_bytes_to_unicode() {
    std::vector<std::int32_t> bytes;
    for (std::int32_t value = LCQI_GPT2_ASCII_EXCLAMATION;
         value <= LCQI_GPT2_ASCII_TILDE;
         ++value) {
        bytes.push_back(value);
    }
    for (std::int32_t value = LCQI_GPT2_LATIN_INVERTED_EXCLAMATION;
         value <= LCQI_GPT2_LATIN_NOT_SIGN;
         ++value) {
        bytes.push_back(value);
    }
    for (std::int32_t value = LCQI_GPT2_LATIN_REGISTERED;
         value <= LCQI_GPT2_LATIN_Y_DIAERESIS;
         ++value) {
        bytes.push_back(value);
    }

    std::unordered_set<std::int32_t> byte_set(bytes.begin(), bytes.end());
    std::vector<std::int32_t> chars = bytes;
    std::int32_t private_offset = 0;
    for (std::int32_t byte = 0; byte < LCQI_GPT2_BYTE_COUNT; ++byte) {
        if (!byte_set.contains(byte)) {
            bytes.push_back(byte);
```

```
            chars.push_back(LCQI_GPT2_UNICODE_PRIVATE_BASE + private_offset);
            ++private_offset;
        }
    }

    std::vector<std::string> mapping(LCQI_GPT2_BYTE_COUNT);
    for (std::size_t index = 0; index < bytes.size(); ++index) {
        mapping[checked_size(bytes[index], "byte unicode index")] =
            encode_codepoint(static_cast<std::uint32_t>(chars[index]));
    }
    return mapping;
}

std::unordered_map<std::string, unsigned char> gpt2_unicode_to_bytes(
    const std::vector<std::string>& byte_encoder) {
    std::unordered_map<std::string, unsigned char> result;
    for (std::size_t byte = 0; byte < byte_encoder.size(); ++byte) {
        result.emplace(byte_encoder[byte], static_cast<unsigned char>(byte));
    }
    return result;
}

std::string bytes_to_bpe_alphabet(std::string_view text) {
    static const std::vector<std::string> BYTE_ENCODER = gpt2_bytes_to_unicode();
    std::string result;
    for (const unsigned char byte : text) {
        result += BYTE_ENCODER[byte];
    }
    return result;
}

std::vector<std::string> utf8_symbols(std::string_view text) {
    std::vector<std::string> symbols;
    std::size_t offset = 0;
    while (offset < text.size()) {
        const unsigned char lead = static_cast<unsigned char>(text[offset]);
        std::size_t width = 1;
        if ((lead & 0xE0U) == LCQI_GPT2_UTF8_TWO_BYTE_HEAD) {
            width = 2;
        } else if ((lead & 0xF0U) == LCQI_GPT2_UTF8_THREE_BYTE_HEAD) {
            width = 3;
        } else if ((lead & 0xF8U) == LCQI_GPT2_UTF8_FOUR_BYTE_HEAD) {
            width = 4;
        }
        if (offset + width > text.size()) {
            throw std::runtime_error("invalid UTF-8/BPE symbol boundary");
        }
        symbols.emplace_back(text.substr(offset, width));
        offset += width;
    }
    return symbols;
```

```
}

std::string pair_key(std::string_view left, std::string_view right) {
    std::string key(left);
    key.push_back('\n');
    key += right;
    return key;
}

std::vector<std::string> bpe_apply(const Gpt2Tokenizer& tokenizer, std::string_view token) {
    std::vector<std::string> word = utf8_symbols(token);
    if (word.size() <= 1) {
        return word;
    }

    while (true) {
        std::optional<std::int32_t> best_rank;
        std::string best_left;
        std::string best_right;
        for (std::size_t index = 0; index + 1 < word.size(); ++index) {
            const auto rank = tokenizer.bpe_ranks.find(pair_key(word[index], word[index + 1]));
            if (rank != tokenizer.bpe_ranks.end() &&
                (!best_rank.has_value() || rank->second < *best_rank)) {
                best_rank = rank->second;
                best_left = word[index];
                best_right = word[index + 1];
            }
        }
        if (!best_rank.has_value()) {
            return word;
        }

        std::vector<std::string> merged;
        std::size_t index = 0;
        while (index < word.size()) {
            if (index + 1 < word.size() && word[index] == best_left &&
                word[index + 1] == best_right) {
                merged.push_back(best_left + best_right);
                index += LCQI_GPT2_PAIR_TOKEN_COUNT;
            } else {
                merged.push_back(word[index]);
                ++index;
            }
        }
        word = std::move(merged);
        if (word.size() == 1) {
            return word;
        }
    }
}
```

```cpp

bool is_ascii_letter(unsigned char value) {
    return (value >= 'A' && value <= 'Z') || (value >= 'a' && value <= 'z');
}

bool is_ascii_digit(unsigned char value) {
    return value >= '0' && value <= '9';
}

bool is_ascii_space(unsigned char value) {
    return value == ' ' || value == '\t' || value == '\n' ||
           value == '\r' || value == '\f' || value == '\v';
}

bool is_gpt2_punctuation(unsigned char value) {
    return !is_ascii_letter(value) && !is_ascii_digit(value) &&
           !is_ascii_space(value);
}

std::size_t consume_utf8_codepoint(std::string_view text, std::size_t offset) {
    const unsigned char lead = static_cast<unsigned char>(text[offset]);
    std::size_t width = 1;
    if ((lead & 0xE0U) == LCQI_GPT2_UTF8_TWO_BYTE_HEAD) {
        width = 2;
    } else if ((lead & 0xF0U) == LCQI_GPT2_UTF8_THREE_BYTE_HEAD) {
        width = 3;
    } else if ((lead & 0xF8U) == LCQI_GPT2_UTF8_FOUR_BYTE_HEAD) {
        width = 4;
    }
    if (offset + width > text.size()) {
        throw std::runtime_error("invalid UTF-8 text for GPT-2 tokenizer");
    }
    return offset + width;
}

bool starts_with_contraction(std::string_view text,
                             std::size_t offset,
                             std::string_view contraction) {
    if (offset + contraction.size() > text.size()) {
        return false;
    }
    for (std::size_t index = 0; index < contraction.size(); ++index) {
        char left = text[offset + index];
        const char right = contraction[index];
        if (left >= 'A' && left <= 'Z') {
            left = static_cast<char>(left - 'A' + 'a');
        }
        if (left != right) {
            return false;
        }
    }
    return true;
```

```cpp
}

std::vector<std::string> gpt2_pretokenize(std::string_view text) {
    std::vector<std::string> pieces;
    std::size_t offset = 0;
    const std::array<std::string_view, 7> contractions{
        "'s", "'t", "'re", "'ve", "'m", "'ll", "'d",
    };
    while (offset < text.size()) {
        bool emitted_contraction = false;
        for (const std::string_view contraction : contractions) {
            if (starts_with_contraction(text, offset, contraction)) {
                pieces.emplace_back(text.substr(offset, contraction.size()));
                offset += contraction.size();
                emitted_contraction = true;
                break;
            }
        }
        if (emitted_contraction) {
            continue;
        }

        const std::size_t begin = offset;
        bool has_prefix_space = false;
        if (text[offset] == ' ' && offset + 1 < text.size() &&
            !is_ascii_space(static_cast<unsigned char>(text[offset + 1]))) {
            has_prefix_space = true;
            ++offset;
        }

        if (offset >= text.size()) {
            pieces.emplace_back(text.substr(begin, offset - begin));
            continue;
        }

        const unsigned char ch = static_cast<unsigned char>(text[offset]);
        if (is_ascii_letter(ch)) {
            while (offset < text.size() &&
                   is_ascii_letter(static_cast<unsigned char>(text[offset]))) {
                ++offset;
            }
        } else if (is_ascii_digit(ch)) {
            while (offset < text.size() &&
                   is_ascii_digit(static_cast<unsigned char>(text[offset]))) {
                ++offset;
            }
        } else if (is_gpt2_punctuation(ch) && ch != LCQI_GPT2_ASCII_APOSTROPHE) {
            while (offset < text.size() &&
                   is_gpt2_punctuation(static_cast<unsigned char>(text[offset])) &&
                   static_cast<unsigned char>(text[offset]) !=
```

```cpp
                LCQI_GPT2_ASCII_APOSTROPHE) {
                    offset = consume_utf8_codepoint(text, offset);
                }
            } else if (is_ascii_space(ch)) {
                while (offset < text.size() &&
                       is_ascii_space(static_cast<unsigned char>(text[offset]))) {
                    ++offset;
                }
            } else {
                offset = consume_utf8_codepoint(text, offset);
            }

            if (has_prefix_space || offset > begin) {
                pieces.emplace_back(text.substr(begin, offset - begin));
            } else {
                ++offset;
            }
        }
        return pieces;
}

std::int32_t parse_i32_field(JsonCursor& cursor) {
    const std::int64_t value = cursor.parse_i64();
    if (value < std::numeric_limits<std::int32_t>::min() ||
        value > std::numeric_limits<std::int32_t>::max()) {
        throw std::runtime_error("JSON int32 field out of range");
    }
    return static_cast<std::int32_t>(value);
}

std::unordered_map<std::string, std::int32_t> parse_vocab_json(std::string_view text) {
    JsonCursor cursor(text);
    std::unordered_map<std::string, std::int32_t> vocab;
    cursor.expect('{');
    if (cursor.consume('}')) {
        return vocab;
    }
    while (true) {
        const std::string key = cursor.parse_string();
        cursor.expect(':');
        const std::int32_t id = parse_i32_field(cursor);
        vocab.emplace(key, id);
        if (cursor.consume('}')) {
            break;
        }
        cursor.expect(',');
    }
    if (!cursor.eof_after_ws()) {
        throw std::runtime_error("trailing data after vocab JSON");
    }
    return vocab;
```

```cpp
}

std::vector<std::string> build_id_to_token(
    const std::unordered_map<std::string, std::int32_t>& vocab) {
    std::int32_t max_id = -1;
    for (const auto& [token, id] : vocab) {
        if (id < 0) {
            throw std::runtime_error("GPT-2 vocab id cannot be negative");
        }
        max_id = std::max(max_id, id);
    }
    std::vector<std::string> result(checked_size(max_id + 1, "max vocab id"));
    for (const auto& [token, id] : vocab) {
        std::string& slot = result[checked_size(id, "vocab id")];
        if (!slot.empty()) {
            throw std::runtime_error("duplicate GPT-2 vocab id");
        }
        slot = token;
    }
    return result;
}

std::unordered_map<std::string, std::int32_t> parse_merges(std::string_view ↩
    ↪ text) {
    std::unordered_map<std::string, std::int32_t> ranks;
    std::istringstream input{std::string(text)};
    std::string line;
    std::int32_t rank = 0;
    while (std::getline(input, line)) {
        if (line.empty() || line[0] == '#') {
            continue;
        }
        std::istringstream line_stream(line);
        std::string left;
        std::string right;
        if (!(line_stream >> left >> right)) {
            throw std::runtime_error("invalid GPT-2 merges line");
        }
        ranks.emplace(pair_key(left, right), rank);
        ++rank;
    }
    return ranks;
}

void transpose_hf_conv1d(std::vector<float>& values,
                         std::int32_t input_size,
                         std::int32_t output_size) {
    if (values.size() != matrix_size(input_size, output_size)) {
        throw std::runtime_error("GPT-2 Conv1D tensor size mismatch");
    }
    std::vector<float> transposed(matrix_size(output_size, input_size), 0.0F);
    for (std::int32_t input = 0; input < input_size; ++input) {
```

```
        for (std::int32_t output = 0; output < output_size; ++output) {
            transposed[matrix_size(output, input_size) + checked_size(input, "input")] =
                values[matrix_size(input, output_size) + checked_size(output, "output")];
        }
    }
    values = std::move(transposed);
}

std::vector<float> require_tensor(const SafeTensorFile& file,
                                  std::string_view name,
                                  std::span<const std::int64_t> expected_shape) {
    const SafeTensorEntry* entry = file.manifest.find_tensor(name);
    if (entry == nullptr) {
        throw std::runtime_error("missing GPT-2 tensor: " + std::string(name));
    }
    if (entry->shape.size() != expected_shape.size()) {
        throw std::runtime_error("GPT-2 tensor rank mismatch: " + std::string(name));
    }
    for (std::size_t index = 0; index < expected_shape.size(); ++index) {
        if (entry->shape[index] != expected_shape[index]) {
            throw std::runtime_error("GPT-2 tensor shape mismatch: " + std::string(name));
        }
    }
    return read_safetensor_f32_tensor(file, name);
}

std::vector<float> require_first_tensor(const SafeTensorFile& file,
                                        std::span<const std::string_view> names,
                                        std::span<const std::int64_t> expected_shape) {
    for (const std::string_view name : names) {
        if (file.manifest.find_tensor(name) != nullptr) {
            return require_tensor(file, name, expected_shape);
        }
    }
    throw std::runtime_error("missing GPT-2 tensor: " + std::string(names.front()));
}

Gpt2LinearF32 make_linear(std::int32_t input_size,
                          std::int32_t output_size,
                          std::vector<float> weights,
                          std::vector<float> bias = {}) {
    Gpt2LinearF32 linear;
    linear.input_size = input_size;
    linear.output_size = output_size;
```

```cpp
    linear.weights = std::move(weights);
    linear.bias = std::move(bias);
    validate_linear(linear, "make GPT-2 linear");
    return linear;
}

std::vector<float> zeros(std::int32_t count) {
    return std::vector<float>(checked_size(count, "zero count"), 0.0F);
}

std::vector<float> ones(std::int32_t count) {
    return std::vector<float>(checked_size(count, "one count"), 1.0F);
}

std::string tensor_name(std::int32_t layer, std::string_view suffix) {
    return "transformer.h." + std::to_string(layer) + "." + std::string(suffix);
}

std::string bare_tensor_name(std::int32_t layer, std::string_view suffix) {
    return "h." + std::to_string(layer) + "." + std::string(suffix);
}

std::array<std::string_view, 2> embedding_names(std::string_view bare) {
    if (bare == "wte.weight") {
        return {"transformer.wte.weight", "wte.weight"};
    }
    if (bare == "wpe.weight") {
        return {"transformer.wpe.weight", "wpe.weight"};
    }
    if (bare == "ln_f.weight") {
        return {"transformer.ln_f.weight", "ln_f.weight"};
    }
    if (bare == "ln_f.bias") {
        return {"transformer.ln_f.bias", "ln_f.bias"};
    }
    throw std::runtime_error("unsupported GPT-2 embedding tensor alias");
}

std::array<std::string, 2> layer_names(std::int32_t layer, std::string_view suffix) {
    return {tensor_name(layer, suffix), bare_tensor_name(layer, suffix)};
}

std::vector<float> require_layer_tensor(const SafeTensorFile& file,
                                        std::int32_t layer,
                                        std::string_view suffix,
                                        std::span<const std::int64_t> expected_shape) {
    const std::array<std::string, 2> names = layer_names(layer, suffix);
    const std::array<std::string_view, 2> views{names[0], names[1]};
    return require_first_tensor(file, views, expected_shape);
```

```cpp
}

std::filesystem::path find_gpt2_weight_file(const std::filesystem::path& model_directory) {
    const std::array<const char*, 2> candidates{
        "model.safetensors",
        "pytorch_model.safetensors",
    };
    for (const char* candidate : candidates) {
        const std::filesystem::path path = model_directory / candidate;
        if (std::filesystem::exists(path)) {
            return path;
        }
    }
    throw std::runtime_error("cannot find model.safetensors in GPT-2 directory");
}

}  // namespace

Gpt2Tokenizer load_gpt2_tokenizer(const std::filesystem::path& vocab_json,
                                  const std::filesystem::path& merges_txt,
                                  std::int32_t bos_token_id,
                                  std::int32_t eos_token_id) {
    Gpt2Tokenizer tokenizer;
    tokenizer.bos_token_id = bos_token_id;
    tokenizer.eos_token_id = eos_token_id;
    tokenizer.vocab = parse_vocab_json(read_text_file(vocab_json));
    tokenizer.id_to_token = build_id_to_token(tokenizer.vocab);
    tokenizer.bpe_ranks = parse_merges(read_text_file(merges_txt));
    if (tokenizer.vocab.empty() || tokenizer.bpe_ranks.empty()) {
        throw std::runtime_error("GPT-2 tokenizer assets are empty");
    }
    return tokenizer;
}

std::vector<std::int32_t> gpt2_encode(const Gpt2Tokenizer& tokenizer,
                                      std::string_view text) {
    std::vector<std::int32_t> ids;
    for (const std::string& piece : gpt2_pretokenize(text)) {
        const std::string byte_piece = bytes_to_bpe_alphabet(piece);
        const std::vector<std::string> bpe_tokens = bpe_apply(tokenizer, byte_piece);
        for (const std::string& token : bpe_tokens) {
            const auto found = tokenizer.vocab.find(token);
            if (found == tokenizer.vocab.end()) {
                throw std::runtime_error("GPT-2 BPE token is missing from vocab");
            }
            ids.push_back(found->second);
        }
    }
```

```cpp
    return ids;
}

std::string gpt2_decode(const Gpt2Tokenizer& tokenizer,
                        std::span<const std::int32_t> token_ids) {
    static const std::vector<std::string> BYTE_ENCODER = gpt2_bytes_to_unicode();
    static const std::unordered_map<std::string, unsigned char> BYTE_DECODER =
        gpt2_unicode_to_bytes(BYTE_ENCODER);

    std::string token_text;
    for (const std::int32_t token_id : token_ids) {
        if (token_id < 0 || token_id >= static_cast<std::int32_t>(tokenizer.id_to_token.size())) {
            throw std::runtime_error("GPT-2 token id out of range");
        }
        token_text += tokenizer.id_to_token[checked_size(token_id, "token id")];
    }

    std::string result;
    const std::vector<std::string> symbols = utf8_symbols(token_text);
    for (const std::string& symbol : symbols) {
        const auto found = BYTE_DECODER.find(symbol);
        if (found == BYTE_DECODER.end()) {
            throw std::runtime_error("GPT-2 token cannot be decoded to byte");
        }
        result.push_back(static_cast<char>(found->second));
    }
    return result;
}

Gpt2Config load_gpt2_config(const std::filesystem::path& config_json) {
    const std::string config_text = read_text_file(config_json);
    JsonCursor cursor(config_text);
    Gpt2Config config;
    config.bos_token_id = LCQI_GPT2_DEFAULT_BOS_EOS;
    config.eos_token_id = LCQI_GPT2_DEFAULT_BOS_EOS;

    cursor.expect('{');
    if (cursor.consume('}')) {
        throw std::runtime_error("GPT-2 config is empty");
    }
    while (true) {
        const std::string key = cursor.parse_string();
        cursor.expect(':');
        if (key == "n_embd") {
            config.hidden_size = parse_i32_field(cursor);
        } else if (key == "n_head") {
            config.head_count = parse_i32_field(cursor);
        } else if (key == "n_layer") {
            config.layer_count = parse_i32_field(cursor);
```

```cpp
        } else if (key == "n_positions" || key == "n_ctx") {
            const std::int32_t value = parse_i32_field(cursor);
            config.max_positions = std::max(config.max_positions, value);
        } else if (key == "vocab_size") {
            config.vocab_size = parse_i32_field(cursor);
        } else if (key == "n_inner") {
            config.intermediate_size = parse_i32_field(cursor);
        } else if (key == "layer_norm_epsilon") {
            config.layer_norm_epsilon = static_cast<float>(cursor.parse_number());
        } else if (key == "activation_function") {
            config.activation_function = cursor.parse_string();
        } else if (key == "bos_token_id") {
            config.bos_token_id = parse_i32_field(cursor);
        } else if (key == "eos_token_id") {
            config.eos_token_id = parse_i32_field(cursor);
        } else {
            cursor.skip_value();
        }
        if (cursor.consume('}')) {
            break;
        }
        cursor.expect(',');
    }
    if (!cursor.eof_after_ws()) {
        throw std::runtime_error("trailing data after GPT-2 config JSON");
    }
    if (config.intermediate_size == 0 && config.hidden_size > 0) {
        config.intermediate_size = config.hidden_size * 4;
    }
    validate_config(config);
    return config;
}

Gpt2ReferenceModel load_gpt2_reference_model(const std::filesystem::path& config_json,
                                             const std::filesystem::path& safetensors_path) {
    Gpt2ReferenceModel model;
    model.config = load_gpt2_config(config_json);
    const SafeTensorFile file = load_safetensors_file(safetensors_path);
    const Gpt2Config& config = model.config;
    const std::array<std::int64_t, 2> wte_shape{config.vocab_size, config.hidden_size};
    const std::array<std::int64_t, 2> wpe_shape{config.max_positions, config.hidden_size};
    const std::array<std::int64_t, 1> hidden_shape{config.hidden_size};

    const std::array<std::string_view, 2> wte_names = embedding_names("wte.weight");
    const std::array<std::string_view, 2> wpe_names = embedding_names("wpe.weight");
```

```
    model.token_embedding = require_first_tensor(file, wte_names, wte_shape);
    model.position_embedding = require_first_tensor(file, wpe_names, wpe_shape);
    model.layers.resize(checked_size(config.layer_count, "layer_count"));

    for (std::int32_t layer_id = 0; layer_id < config.layer_count; ++layer_id) {
        Gpt2LayerWeightsF32& layer = model.layers[checked_size(layer_id, "layer_id")];
        layer.ln_1_weight = require_layer_tensor(file, layer_id, "ln_1.weight", hidden_shape);
        layer.ln_1_bias = require_layer_tensor(file, layer_id, "ln_1.bias", hidden_shape);
        layer.ln_2_weight = require_layer_tensor(file, layer_id, "ln_2.weight", hidden_shape);
        layer.ln_2_bias = require_layer_tensor(file, layer_id, "ln_2.bias", hidden_shape);

        const std::array<std::int64_t, 2> c_attn_shape{
            config.hidden_size,
            config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS};
        std::vector<float> c_attn_weight =
            require_layer_tensor(file, layer_id, "attn.c_attn.weight", c_attn_shape);
        transpose_hf_conv1d(c_attn_weight,
                            config.hidden_size,
                            config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS);
        const std::array<std::int64_t, 1> c_attn_bias_shape{
            config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS};
        layer.c_attn = make_linear(config.hidden_size,
                                   config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS,
                                   std::move(c_attn_weight),
                                   require_layer_tensor(file,
                                                        layer_id,
                                                        "attn.c_attn.bias",
                                                        c_attn_bias_shape));

        const std::array<std::int64_t, 2> c_proj_shape{config.hidden_size, config.hidden_size};
        std::vector<float> c_proj_weight =
            require_layer_tensor(file, layer_id, "attn.c_proj.weight", c_proj_shape);
        transpose_hf_conv1d(c_proj_weight, config.hidden_size, config.hidden_size);
        layer.c_proj = make_linear(config.hidden_size,
                                   config.hidden_size,
                                   std::move(c_proj_weight),
                                   require_layer_tensor(file,
                                                        layer_id,
                                                        "attn.c_proj.bias",
```

```
                                                        hidden_shape));

        const std::array<std::int64_t, 2> c_fc_shape{
            config.hidden_size,
            config.intermediate_size};
        std::vector<float> c_fc_weight =
            require_layer_tensor(file, layer_id, "mlp.c_fc.weight", c_fc_shape);
        transpose_hf_conv1d(c_fc_weight, config.hidden_size, config.intermediate_size);
        const std::array<std::int64_t, 1> intermediate_shape{config.intermediate_size};
        layer.c_fc = make_linear(config.hidden_size,
                                 config.intermediate_size,
                                 std::move(c_fc_weight),
                                 require_layer_tensor(file,
                                                      layer_id,
                                                      "mlp.c_fc.bias",
                                                      intermediate_shape));

        const std::array<std::int64_t, 2> mlp_proj_shape{
            config.intermediate_size,
            config.hidden_size};
        std::vector<float> mlp_proj_weight =
            require_layer_tensor(file, layer_id, "mlp.c_proj.weight", mlp_proj_shape);
        transpose_hf_conv1d(mlp_proj_weight, config.intermediate_size, config.hidden_size);
        layer.mlp_c_proj = make_linear(config.intermediate_size,
                                       config.hidden_size,
                                       std::move(mlp_proj_weight),
                                       require_layer_tensor(file,
                                                            layer_id,
                                                            "mlp.c_proj.bias",
                                                            hidden_shape));
    }

    const std::array<std::string_view, 2> ln_weight_names = embedding_names("ln_f.weight");
    const std::array<std::string_view, 2> ln_bias_names = embedding_names("ln_f.bias");
    model.final_ln_weight = require_first_tensor(file, ln_weight_names, hidden_shape);
    model.final_ln_bias = require_first_tensor(file, ln_bias_names, hidden_shape);
    if (file.manifest.find_tensor("lm_head.weight") != nullptr) {
        model.tie_lm_head_to_embedding = false;
        model.lm_head_weight = require_tensor(file, "lm_head.weight", wte_shape);
    } else {
        model.tie_lm_head_to_embedding = true;
    }
```

```cpp
    validate_model(model);
    return model;
}

Gpt2ReferenceModel load_gpt2_from_directory(const std::filesystem::path& model_directory) {
    return load_gpt2_reference_model(model_directory / "config.json",
                                     find_gpt2_weight_file(model_directory));
}

Gpt2ForwardWorkspace::Gpt2ForwardWorkspace(const Gpt2Config& config) : config_(config) {
    this->reset_for_config(config);
}

void Gpt2ForwardWorkspace::reset_for_config(const Gpt2Config& config) {
    validate_config(config);
    this->config_ = config;
    const std::size_t hidden_size = checked_size(this->config_.hidden_size, "hidden_size");
    const std::size_t intermediate_size =
        checked_size(this->config_.intermediate_size, "intermediate_size");
    this->hidden_.assign(hidden_size, 0.0F);
    this->normed_.assign(hidden_size, 0.0F);
    this->query_.assign(hidden_size, 0.0F);
    this->key_.assign(hidden_size, 0.0F);
    this->value_.assign(hidden_size, 0.0F);
    this->attention_.assign(hidden_size, 0.0F);
    this->projected_.assign(hidden_size, 0.0F);
    this->qkv_packed_.assign(
        matrix_size(LCQI_GPT2_ATTENTION_PROJECTIONS, this->config_.hidden_size),
        0.0F);
    this->mlp_fc_.assign(intermediate_size, 0.0F);
    this->mlp_out_.assign(hidden_size, 0.0F);
    this->logits_.assign(checked_size(this->config_.vocab_size, "vocab_size"), 0.0F);
    this->scores_.assign(checked_size(this->config_.max_positions, "max_positions"), 0.0F);
}

std::span<float> Gpt2ForwardWorkspace::hidden() noexcept {
    return this->hidden_;
}

std::span<float> Gpt2ForwardWorkspace::normed() noexcept {
    return this->normed_;
}

std::span<float> Gpt2ForwardWorkspace::query() noexcept {
    return this->query_;
}
```

```cpp

std::span<float> Gpt2ForwardWorkspace::key() noexcept {
    return this->key_;
}

std::span<float> Gpt2ForwardWorkspace::value() noexcept {
    return this->value_;
}

std::span<float> Gpt2ForwardWorkspace::attention() noexcept {
    return this->attention_;
}

std::span<float> Gpt2ForwardWorkspace::projected() noexcept {
    return this->projected_;
}

std::span<float> Gpt2ForwardWorkspace::qkv_packed() noexcept {
    return this->qkv_packed_;
}

std::span<float> Gpt2ForwardWorkspace::mlp_fc() noexcept {
    return this->mlp_fc_;
}

std::span<float> Gpt2ForwardWorkspace::mlp_out() noexcept {
    return this->mlp_out_;
}

std::span<float> Gpt2ForwardWorkspace::logits() noexcept {
    return this->logits_;
}

std::span<float> Gpt2ForwardWorkspace::scores_prefix(std::int32_t count) {
    if (count < 0 || count > this->config_.max_positions) {
        throw std::runtime_error("GPT-2 attention scores count out of range");
    }
    return std::span<float>(this->scores_.data(), checked_size(count, "scores ↩
    ↪ count"));
}

Gpt2KvCache::Gpt2KvCache(const Gpt2Config& config) : config_(config) {
    validate_config(this->config_);
    const std::size_t element_count =
        checked_size(this->config_.layer_count, "layer_count") *
        checked_size(this->config_.max_positions, "max_positions") *
        checked_size(this->config_.head_count, "head_count") *
        checked_size(head_dim(this->config_), "head_dim");
    this->keys_.assign(element_count, 0.0F);
    this->values_.assign(element_count, 0.0F);
    this->written_.assign(
        checked_size(this->config_.layer_count, "layer_count") *
```

```
            checked_size(this->config_.max_positions, "max_positions"),
        0);
}

void Gpt2KvCache::append(std::int32_t layer_id,
                         std::int32_t model_position,
                         std::span<const float> key,
                         std::span<const float> value) {
    const std::size_t hidden_size = checked_size(this->config_.hidden_size, "hidden_size");
    if (key.size() != hidden_size || value.size() != hidden_size) {
        throw std::runtime_error("GPT-2 KV key/value size mismatch");
    }
    this->validate_address(layer_id, model_position, 0);
    const std::int32_t local_head_dim = head_dim(this->config_);
    for (std::int32_t head = 0; head < this->config_.head_count; ++head) {
        const std::size_t source =
            checked_size(head, "head") * checked_size(local_head_dim, "head_dim");
        const std::size_t target = this->base_offset(layer_id, model_position, head);
        std::copy_n(key.begin() + static_cast<std::ptrdiff_t>(source),
                    local_head_dim,
                    this->keys_.begin() + static_cast<std::ptrdiff_t>(target));
        std::copy_n(value.begin() + static_cast<std::ptrdiff_t>(source),
                    local_head_dim,
                    this->values_.begin() + static_cast<std::ptrdiff_t>(target));
    }
    const std::size_t written_index =
        checked_size(layer_id, "layer_id") *
            checked_size(this->config_.max_positions, "max_positions") +
        checked_size(model_position, "model_position");
    this->written_[written_index] = 1;
    this->filled_tokens_ = std::max(this->filled_tokens_, model_position + 1);
}

std::span<const float> Gpt2KvCache::key(std::int32_t layer_id,
                                        std::int32_t model_position,
                                        std::int32_t head) const {
    this->validate_address(layer_id, model_position, head);
    this->validate_written(layer_id, model_position);
    return this->key_unchecked(layer_id, model_position, head);
}

std::span<const float> Gpt2KvCache::value(std::int32_t layer_id,
                                          std::int32_t model_position,
                                          std::int32_t head) const {
    this->validate_address(layer_id, model_position, head);
    this->validate_written(layer_id, model_position);
    return this->value_unchecked(layer_id, model_position, head);
}
```

```cpp

void detail::attend_cached_position(const Gpt2KvCache& cache,
                                    std::int32_t layer_id,
                                    std::int32_t model_position,
                                    std::span<const float> query,
                                    std::span<float> scores,
                                    std::span<float> output) {
    cache.validate_address(layer_id, model_position, 0);
    cache.validate_written(layer_id, model_position);
    const std::int32_t local_head_dim = head_dim(cache.config_);
    const std::size_t hidden_size = checked_size(cache.config_.hidden_size, "hidden_size");
    const std::size_t local_head_dim_size = checked_size(local_head_dim, "head_dim");
    const std::size_t model_position_size =
        checked_size(model_position, "model_position");
    if (query.size() != hidden_size || output.size() != hidden_size ||
        scores.size() < model_position_size + 1) {
        throw std::runtime_error("GPT-2 cached attention workspace size mismatch");
    }

    const float score_scale = 1.0F / std::sqrt(static_cast<float>(local_head_dim));
    std::fill(output.begin(), output.end(), 0.0F);
    for (std::int32_t head = 0; head < cache.config_.head_count; ++head) {
        const std::size_t head_base =
            checked_size(head, "head") * local_head_dim_size;
        float max_score = LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY;
        for (std::size_t past = 0; past <= model_position_size; ++past) {
            const std::int32_t past_i32 = static_cast<std::int32_t>(past);
            const std::span<const float> key =
                cache.key_unchecked(layer_id, past_i32, head);
            float dot = 0.0F;
            for (std::size_t dim = 0; dim < local_head_dim_size; ++dim) {
                dot += query[head_base + dim] * key[dim];
            }
            const float score = dot * score_scale;
            scores[past] = score;
            max_score = std::max(max_score, score);
        }

        float denominator = 0.0F;
        for (std::size_t past = 0; past <= model_position_size; ++past) {
            scores[past] = std::exp(scores[past] - max_score);
            denominator += scores[past];
        }
        if (denominator <= 0.0F) {
            throw std::runtime_error("GPT-2 cached attention denominator is invalid");
        }

```

```cpp
        for (std::size_t past = 0; past <= model_position_size; ++past) {
            const std::int32_t past_i32 = static_cast<std::int32_t>(past);
            const float probability = scores[past] / denominator;
            const std::span<const float> value =
                cache.value_unchecked(layer_id, past_i32, head);
            for (std::size_t dim = 0; dim < local_head_dim_size; ++dim) {
                output[head_base + dim] += probability * value[dim];
            }
        }
    }
}

std::span<const float> Gpt2KvCache::key_unchecked(std::int32_t layer_id,
                                                  std::int32_t model_position,
                                                  std::int32_t head) const noexcept {
    const std::size_t offset = this->base_offset_unchecked(layer_id, model_position, head);
    return std::span<const float>(
        this->keys_.data() + offset,
        static_cast<std::size_t>(head_dim(this->config_)));
}

std::span<const float> Gpt2KvCache::value_unchecked(std::int32_t layer_id,
                                                    std::int32_t model_position,
                                                    std::int32_t head) const noexcept {
    const std::size_t offset = this->base_offset_unchecked(layer_id, model_position, head);
    return std::span<const float>(
        this->values_.data() + offset,
        static_cast<std::size_t>(head_dim(this->config_)));
}

std::int32_t Gpt2KvCache::filled_tokens() const noexcept {
    return this->filled_tokens_;
}

std::size_t Gpt2KvCache::byte_size() const noexcept {
    return (this->keys_.size() + this->values_.size()) * sizeof(float) +
           this->written_.size() * sizeof(std::uint8_t);
}

std::size_t Gpt2KvCache::base_offset(std::int32_t layer_id,
                                     std::int32_t model_position,
                                     std::int32_t head) const {
    return (((checked_size(layer_id, "layer_id") *
              checked_size(this->config_.max_positions, "max_positions")) +
             checked_size(model_position, "model_position")) *
                checked_size(this->config_.head_count, "head_count") +
            checked_size(head, "head")) *
```

```
           checked_size(head_dim(this->config_), "head_dim");
}

std::size_t Gpt2KvCache::base_offset_unchecked(std::int32_t layer_id,
                                               std::int32_t model_position,
                                               std::int32_t head) const noexcept {
    return (((static_cast<std::size_t>(layer_id) *
              static_cast<std::size_t>(this->config_.max_positions)) +
             static_cast<std::size_t>(model_position)) *
                static_cast<std::size_t>(this->config_.head_count) +
            static_cast<std::size_t>(head)) *
           static_cast<std::size_t>(head_dim(this->config_));
}

void Gpt2KvCache::validate_address(std::int32_t layer_id,
                                   std::int32_t model_position,
                                   std::int32_t head) const {
    if (layer_id < 0 || layer_id >= this->config_.layer_count) {
        throw std::runtime_error("GPT-2 KV layer_id out of range");
    }
    if (model_position < 0 || model_position >= this->config_.max_positions) {
        throw std::runtime_error("GPT-2 KV model_position out of range");
    }
    if (head < 0 || head >= this->config_.head_count) {
        throw std::runtime_error("GPT-2 KV head out of range");
    }
}

void Gpt2KvCache::validate_written(std::int32_t layer_id,
                                   std::int32_t model_position) const {
    const std::size_t written_index =
        checked_size(layer_id, "layer_id") *
            checked_size(this->config_.max_positions, "max_positions") +
        checked_size(model_position, "model_position");
    if (written_index >= this->written_.size() || this->written_[written_index] == 0) {
        throw std::runtime_error("GPT-2 KV slot has not been written");
    }
}

Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& model)
    : Gpt2CachedGreedyDecoder(model, nullptr) {}

Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(
    const Gpt2ReferenceModel& model,
    Gpt2HotspotProfile* hotspot_profile)
    : Gpt2CachedGreedyDecoder(model, hotspot_profile, Gpt2ExecutionOptions{}) {}

Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(
```

```cpp
    const Gpt2ReferenceModel& model,
    Gpt2HotspotProfile* hotspot_profile,
    const Gpt2ExecutionOptions& execution_options)
    : model_(&model),
      hotspot_profile_(hotspot_profile),
      execution_options_(execution_options),
      cache_(model.config),
      workspace_(model.config),
      worker_pool_(make_worker_pool(model, execution_options)) {
    validate_model(*this->model_);
}

Gpt2CachedGreedyDecoder::~Gpt2CachedGreedyDecoder() = default;

Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(Gpt2CachedGreedyDecoder&&) noexcept =
    default;

Gpt2CachedGreedyDecoder& Gpt2CachedGreedyDecoder::operator=(
    Gpt2CachedGreedyDecoder&&) noexcept = default;

void Gpt2CachedGreedyDecoder::advance_without_prediction(std::int32_t token_id) {
    static_cast<void>(run_gpt2_cached_step_unchecked(*this->model_,
                                                     this->cache_,
                                                     this->workspace_,
                                                     token_id,
                                                     Gpt2CachedStepMode::advance_only,
                                                     this->hotspot_profile_,
                                                     this->execution_options_,
                                                     this->worker_pool_.get()));
}

std::int32_t Gpt2CachedGreedyDecoder::step(std::int32_t token_id) {
    return run_gpt2_cached_step_unchecked(*this->model_,
                                          this->cache_,
                                          this->workspace_,
                                          token_id,
                                          Gpt2CachedStepMode::predict_token,
                                          this->hotspot_profile_,
                                          this->execution_options_,
                                          this->worker_pool_.get())
        .predicted_token;
}

Gpt2ForwardResult Gpt2CachedGreedyDecoder::step_with_logits(std::int32_t token_id) {
    return run_gpt2_cached_step_unchecked(*this->model_,
                                          this->cache_,
                                          this->workspace_,
                                          token_id,
```

```cpp
                                                  Gpt2CachedStepMode::logits,
                                                  this->hotspot_profile_,
                                                  this->execution_options_,
                                                  this->worker_pool_.get());
}

const Gpt2KvCache& Gpt2CachedGreedyDecoder::cache() const noexcept {
    return this->cache_;
}

std::int32_t Gpt2CachedGreedyDecoder::filled_tokens() const noexcept {
    return this->cache_.filled_tokens();
}

std::size_t Gpt2CachedGreedyDecoder::kv_cache_bytes() const noexcept {
    return this->cache_.byte_size();
}

std::int32_t Gpt2CachedGreedyDecoder::worker_count() const noexcept {
    return this->worker_pool_ == nullptr ? 1 : this->worker_pool_->worker_count();
}

Gpt2ForwardResult run_gpt2_forward(const Gpt2ReferenceModel& model,
                                   std::span<const std::int32_t> token_ids) {
    validate_model(model);
    if (token_ids.empty()) {
        throw std::runtime_error("GPT-2 forward needs at least one token");
    }
    if (token_ids.size() > checked_size(model.config.max_positions, "max_positions")) {
        throw std::runtime_error("GPT-2 token_ids exceed max_positions");
    }

    const std::int32_t sequence_length = static_cast<std::int32_t>(token_ids.size());
    std::vector<float> hidden_by_pos(matrix_size(sequence_length, model.config.hidden_size), 0.0F);
    for (std::int32_t position = 0; position < sequence_length; ++position) {
        const std::int32_t token_id = token_ids[checked_size(position, "position")];
        if (token_id < 0 || token_id >= model.config.vocab_size) {
            throw std::runtime_error("GPT-2 token id out of range");
        }
        const std::span<const float> token =
            row_span(model.token_embedding, token_id, model.config.hidden_size);
        const std::span<const float> position_embedding =
            row_span(model.position_embedding, position, model.config.hidden_size);
        std::span<float> hidden = std::span<float>(
            hidden_by_pos.data() + matrix_size(position, model.config.
```

```cpp
        hidden_size),
            checked_size(model.config.hidden_size, "hidden_size"));
        for (std::int32_t dim = 0; dim < model.config.hidden_size; ++dim) {
            hidden[checked_size(dim, "dim")] =
                token[checked_size(dim, "dim")] + position_embedding[checked_size(dim, "dim")];
        }
    }

    for (const Gpt2LayerWeightsF32& layer : model.layers) {
        forward_gpt2_layer(model.config, layer, sequence_length, hidden_by_pos);
    }

    std::vector<float> normed(checked_size(model.config.hidden_size, "hidden_size"), 0.0F);
    const std::span<const float> final_hidden = std::span<const float>(
        hidden_by_pos.data() + matrix_size(sequence_length - 1, model.config.hidden_size),
        checked_size(model.config.hidden_size, "hidden_size"));
    layer_norm(final_hidden,
               model.final_ln_weight,
               model.final_ln_bias,
               model.config.layer_norm_epsilon,
               normed);

    Gpt2ForwardResult result;
    result.logits.assign(checked_size(model.config.vocab_size, "vocab_size"), 0.0F);
    compute_logits(model, normed, result.logits);
    result.predicted_token = argmax(result.logits);
    return result;
}

Gpt2ForwardResult run_gpt2_forward_cached(const Gpt2ReferenceModel& model,
                                          Gpt2KvCache& cache,
                                          std::int32_t token_id) {
    validate_model(model);
    Gpt2ForwardWorkspace workspace(model.config);
    Gpt2ExecutionOptions options;
    options.worker_count = 1;
    return run_gpt2_cached_step_unchecked(
        model,
        cache,
        workspace,
        token_id,
        Gpt2CachedStepMode::logits,
        nullptr,
        options,
        nullptr);
}

```

```cpp
std::vector<std::int32_t> gpt2_generate_greedy(const Gpt2ReferenceModel& model,
                                               std::span<const std::int32_t> prompt_token_ids,
                                               std::int32_t max_new_tokens) {
    if (max_new_tokens < 0) {
        throw std::runtime_error("max_new_tokens cannot be negative");
    }
    std::vector<std::int32_t> tokens(prompt_token_ids.begin(), prompt_token_ids.end());
    for (std::int32_t step = 0; step < max_new_tokens; ++step) {
        if (tokens.size() >= checked_size(model.config.max_positions, "max_positions")) {
            throw std::runtime_error("GPT-2 generation would exceed max_positions");
        }
        const Gpt2ForwardResult result = run_gpt2_forward(model, tokens);
        tokens.push_back(result.predicted_token);
        if (model.config.eos_token_id >= 0 && result.predicted_token == model.config.eos_token_id) {
            break;
        }
    }
    return tokens;
}

std::vector<std::int32_t> gpt2_generate_greedy_cached(
    const Gpt2ReferenceModel& model,
    std::span<const std::int32_t> prompt_token_ids,
    std::int32_t max_new_tokens) {
    if (prompt_token_ids.empty()) {
        throw std::runtime_error("GPT-2 generation needs at least one prompt token");
    }
    if (max_new_tokens < 0) {
        throw std::runtime_error("max_new_tokens cannot be negative");
    }
    if (prompt_token_ids.size() > checked_size(model.config.max_positions, "max_positions")) {
        throw std::runtime_error("GPT-2 prompt exceeds max_positions");
    }

    Gpt2CachedGreedyDecoder decoder(model);
    std::vector<std::int32_t> tokens(prompt_token_ids.begin(), prompt_token_ids.end());
    if (max_new_tokens == 0) {
        return tokens;
    }
    for (std::size_t index = 0; index + 1 < prompt_token_ids.size(); ++index) {
        decoder.advance_without_prediction(prompt_token_ids[index]);
    }
    std::int32_t predicted_token = decoder.step(prompt_token_ids.back());
    for (std::int32_t step = 0; step < max_new_tokens; ++step) {
```

```cpp
        if (tokens.size() >= checked_size(model.config.max_positions, "max_positions")) {
            throw std::runtime_error("GPT-2 generation would exceed max_positions");
        }
        tokens.push_back(predicted_token);
        if (model.config.eos_token_id >= 0 && predicted_token == model.config.eos_token_id) {
            break;
        }
        if (step + 1 < max_new_tokens) {
            predicted_token = decoder.step(predicted_token);
        }
    }
    return tokens;
}

Gpt2ReferenceModel make_tiny_gpt2_reference_model() {
    Gpt2ReferenceModel model;
    model.config.hidden_size = 4;
    model.config.head_count = 2;
    model.config.layer_count = 1;
    model.config.max_positions = 8;
    model.config.vocab_size = 6;
    model.config.intermediate_size = 8;
    model.config.bos_token_id = 0;
    model.config.eos_token_id = 5;
    model.config.layer_norm_epsilon = 1.0e-5F;
    model.config.activation_function = "gelu_new";

    model.token_embedding = {
        0.20F, -0.10F, 0.00F, 0.10F,
        -0.10F, 0.30F, 0.20F, -0.20F,
        0.05F, 0.10F, -0.30F, 0.25F,
        0.40F, -0.20F, 0.15F, 0.05F,
        -0.30F, 0.05F, 0.35F, -0.15F,
        0.10F, 0.20F, -0.10F, 0.30F,
    };
    model.position_embedding = {
        0.01F, 0.02F, 0.03F, 0.04F,
        -0.02F, 0.01F, 0.00F, 0.03F,
        0.03F, -0.01F, 0.02F, 0.00F,
        0.00F, 0.02F, -0.02F, 0.01F,
        0.02F, 0.00F, 0.01F, -0.01F,
        -0.01F, 0.03F, 0.00F, 0.02F,
        0.01F, -0.02F, 0.03F, 0.00F,
        0.00F, 0.01F, -0.01F, 0.02F,
    };

    Gpt2LayerWeightsF32 layer;
    layer.ln_1_weight = ones(4);
    layer.ln_1_bias = zeros(4);
```

```
    layer.ln_2_weight = {0.9F, 1.1F, 1.0F, 0.95F};
    layer.ln_2_bias = {0.01F, -0.02F, 0.00F, 0.03F};
    layer.c_attn = make_linear(
        4,
        12,
        {
            0.10F, -0.20F, 0.05F, 0.15F,
            -0.05F, 0.12F, 0.20F, -0.10F,
            0.18F, 0.02F, -0.08F, 0.11F,
            -0.12F, 0.06F, 0.14F, 0.04F,
            0.07F, 0.16F, -0.09F, 0.03F,
            -0.15F, 0.05F, 0.13F, 0.10F,
            0.04F, -0.11F, 0.19F, -0.02F,
            0.09F, 0.08F, -0.06F, 0.17F,
            0.20F, -0.04F, 0.01F, 0.06F,
            -0.08F, 0.15F, 0.07F, -0.03F,
            0.05F, 0.10F, -0.14F, 0.12F,
            0.11F, -0.07F, 0.16F, 0.02F,
        },
        {
            0.01F, -0.02F, 0.00F, 0.03F,
            -0.01F, 0.02F, 0.01F, -0.02F,
            0.00F, 0.01F, -0.01F, 0.02F,
        });
    layer.c_proj = make_linear(
        4,
        4,
        {
            0.10F, 0.05F, -0.08F, 0.12F,
            -0.04F, 0.11F, 0.09F, -0.02F,
            0.07F, -0.10F, 0.13F, 0.05F,
            0.02F, 0.14F, -0.06F, 0.08F,
        },
        {0.01F, 0.00F, -0.01F, 0.02F});
    layer.c_fc = make_linear(
        4,
        8,
        {
            0.20F, -0.10F, 0.05F, 0.03F,
            -0.05F, 0.14F, 0.12F, -0.08F,
            0.09F, 0.07F, -0.11F, 0.16F,
            -0.12F, 0.05F, 0.18F, 0.02F,
            0.04F, -0.09F, 0.13F, 0.10F,
            0.11F, 0.03F, -0.07F, 0.15F,
            -0.02F, 0.17F, 0.06F, -0.04F,
            0.08F, -0.06F, 0.10F, 0.12F,
        },
        {0.01F, -0.02F, 0.00F, 0.03F, -0.01F, 0.02F, 0.01F, 0.00F});
    layer.mlp_c_proj = make_linear(
        8,
        4,
        {
```

```
            0.10F, -0.08F, 0.06F, 0.12F, -0.04F, 0.07F, 0.03F, 0.11F,
            -0.05F, 0.13F, 0.02F, -0.09F, 0.15F, 0.04F, -0.03F, 0.08F,
            0.07F, 0.01F, -0.12F, 0.10F, 0.06F, -0.05F, 0.14F, 0.02F,
            0.03F, 0.09F, 0.11F, -0.06F, 0.05F, 0.12F, -0.08F, 0.04F,
        },
        {0.00F, 0.01F, -0.01F, 0.02F});
    model.layers.push_back(std::move(layer));
    model.final_ln_weight = {1.0F, 0.95F, 1.05F, 1.0F};
    model.final_ln_bias = {0.0F, 0.01F, -0.01F, 0.02F};
    model.tie_lm_head_to_embedding = true;
    validate_model(model);
    return model;
}

}  // namespace lcqi
```

gpt2_cli.cpp 负责把模型目录、prompt、max_new_tokens 和 benchmark 选项接到 GPT-2 reference path。无参数时运行 tiny GPT-2 fixture；传入 HuggingFace 模型目录时加载真实资产。

Listing 10.14 src/gpt2_cli.cpp

```
#include <lcqi/gpt2_reference.hpp>

#include <chrono>
#include <cstdlib>
#include <exception>
#include <filesystem>
#include <iostream>
#include <span>
#include <stdexcept>
#include <string>
#include <string_view>
#include <vector>

namespace {

constexpr int LCQI_GPT2_TINY_ARGC = 1;
constexpr int LCQI_GPT2_MIN_REAL_ARGC = 3;
constexpr std::int32_t LCQI_GPT2_DEFAULT_MAX_NEW_TOKENS = 8;
constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_0 = 1;
constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_1 = 2;
constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_2 = 3;

enum class Gpt2EngineMode {
    full_prefix,
    cached_kv,
};

struct Gpt2CliOptions {
    Gpt2EngineMode engine = Gpt2EngineMode::cached_kv;
    bool benchmark = false;
    bool profile_hotspots = false;
    std::filesystem::path model_dir;
```

```cpp
    std::string prompt;
    std::int32_t max_new_tokens = LCQI_GPT2_DEFAULT_MAX_NEW_TOKENS;
    std::int32_t worker_count = 0;
};

struct Gpt2BenchmarkTimings {
    double load_ms = 0.0;
    double tokenize_ms = 0.0;
    double prefill_ms = 0.0;
    double decode_ms = 0.0;
    double generate_ms = 0.0;
    double total_ms = 0.0;
    std::size_t prefill_steps = 0;
    std::size_t decode_steps = 0;
    std::size_t kv_cache_bytes = 0;
    std::int32_t worker_count = 1;
};

using Clock = std::chrono::steady_clock;

void print_ids(std::span<const std::int32_t> ids) {
    for (std::size_t index = 0; index < ids.size(); ++index) {
        if (index != 0) {
            std::cout << ' ';
        }
        std::cout << ids[index];
    }
}

[[nodiscard]] double elapsed_ms(Clock::time_point start, Clock::time_point end) {
    return std::chrono::duration<double, std::milli>(end - start).count();
}

[[nodiscard]] std::size_t checked_size(std::int64_t value, const char* name) {
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
}

[[nodiscard]] const char* engine_name(Gpt2EngineMode engine) {
    switch (engine) {
        case Gpt2EngineMode::full_prefix:
            return "full_prefix";
        case Gpt2EngineMode::cached_kv:
            return "cached_kv";
    }
    return "unknown";
}

[[nodiscard]] Gpt2EngineMode parse_engine(std::string_view value) {
```

```cpp
    if (value == "full" || value == "full_prefix") {
        return Gpt2EngineMode::full_prefix;
    }
    if (value == "cached" || value == "cached_kv") {
        return Gpt2EngineMode::cached_kv;
    }
    throw std::runtime_error("unknown --engine value, expected cached or full");
}

[[nodiscard]] std::int32_t parse_non_negative_i32(const std::string& text,
                                                   const char* name) {
    const std::int32_t value = static_cast<std::int32_t>(std::stoi(text));
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return value;
}

[[nodiscard]] Gpt2CliOptions parse_options(int argc, char** argv) {
    Gpt2CliOptions options;
    std::vector<std::string> positional;
    for (int index = 1; index < argc; ++index) {
        const std::string arg = argv[index];
        if (arg == "--benchmark") {
            options.benchmark = true;
        } else if (arg == "--profile-hotspots") {
            options.profile_hotspots = true;
            options.benchmark = true;
        } else if (arg == "--engine") {
            if (index + 1 >= argc) {
                throw std::runtime_error("--engine requires a value");
            }
            ++index;
            options.engine = parse_engine(argv[index]);
        } else if (arg.rfind("--engine=", 0) == 0) {
            options.engine = parse_engine(std::string_view(arg).substr(std::string("--engine=").size()));
        } else if (arg == "--threads") {
            if (index + 1 >= argc) {
                throw std::runtime_error("--threads requires a value");
            }
            ++index;
            options.worker_count = parse_non_negative_i32(argv[index], "--threads");
        } else if (arg.rfind("--threads=", 0) == 0) {
            options.worker_count =
                parse_non_negative_i32(arg.substr(std::string("--threads=").size()), "--threads");
        } else if (arg == "--full") {
            options.engine = Gpt2EngineMode::full_prefix;
        } else if (arg == "--cached") {
```

```cpp
            options.engine = Gpt2EngineMode::cached_kv;
        } else if (!arg.empty() && arg.front() == '-') {
            throw std::runtime_error("unknown option: " + arg);
        } else {
            positional.push_back(arg);
        }
    }

    if (positional.size() < 2 || positional.size() > 3) {
        throw std::runtime_error(
            "usage: lcqi_gpt2 [--engine cached|full] [--threads N] [--benchmark] "
            "model_dir prompt [max_new_tokens]");
    }
    options.model_dir = positional[0];
    options.prompt = positional[1];
    if (positional.size() == 3) {
        options.max_new_tokens = parse_non_negative_i32(positional[2], "max_new_tokens");
    }
    return options;
}

[[nodiscard]] std::vector<std::int32_t> generate_full_prefix(
    const lcqi::Gpt2ReferenceModel& model,
    std::span<const std::int32_t> prompt_ids,
    std::int32_t max_new_tokens,
    Gpt2BenchmarkTimings& timings) {
    const Clock::time_point generate_start = Clock::now();
    std::vector<std::int32_t> tokens(prompt_ids.begin(), prompt_ids.end());
    for (std::int32_t step = 0; step < max_new_tokens; ++step) {
        if (tokens.size() >= checked_size(model.config.max_positions, "max_positions")) {
            throw std::runtime_error("GPT-2 generation would exceed max_positions");
        }
        const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, tokens);
        ++timings.decode_steps;
        tokens.push_back(result.predicted_token);
        if (model.config.eos_token_id >= 0 && result.predicted_token == model.config.eos_token_id) {
            break;
        }
    }
    const Clock::time_point generate_end = Clock::now();
    timings.generate_ms = elapsed_ms(generate_start, generate_end);
    timings.decode_ms = timings.generate_ms;
    return tokens;
}

[[nodiscard]] std::vector<std::int32_t> generate_cached(
```

```cpp
    const lcqi::Gpt2ReferenceModel& model,
    std::span<const std::int32_t> prompt_ids,
    std::int32_t max_new_tokens,
    Gpt2BenchmarkTimings& timings,
    lcqi::Gpt2HotspotProfile* hotspot_profile,
    std::int32_t worker_count) {
    if (prompt_ids.empty()) {
        throw std::runtime_error("GPT-2 generation needs at least one prompt token");
    }
    lcqi::Gpt2ExecutionOptions execution_options;
    execution_options.worker_count = worker_count;
    lcqi::Gpt2CachedGreedyDecoder decoder(model, hotspot_profile, execution_options);
    timings.kv_cache_bytes = decoder.kv_cache_bytes();
    timings.worker_count = decoder.worker_count();
    std::vector<std::int32_t> tokens(prompt_ids.begin(), prompt_ids.end());
    if (max_new_tokens == 0) {
        return tokens;
    }

    const Clock::time_point generate_start = Clock::now();
    const Clock::time_point prefill_start = Clock::now();
    for (std::size_t index = 0; index + 1 < prompt_ids.size(); ++index) {
        decoder.advance_without_prediction(prompt_ids[index]);
        ++timings.prefill_steps;
    }
    std::int32_t predicted_token = decoder.step(prompt_ids.back());
    ++timings.prefill_steps;
    const Clock::time_point prefill_end = Clock::now();
    timings.prefill_ms = elapsed_ms(prefill_start, prefill_end);

    const Clock::time_point decode_start = Clock::now();
    for (std::int32_t step = 0; step < max_new_tokens; ++step) {
        if (tokens.size() >= checked_size(model.config.max_positions, "max_positions")) {
            throw std::runtime_error("GPT-2 generation would exceed max_positions");
        }
        tokens.push_back(predicted_token);
        if (model.config.eos_token_id >= 0 && predicted_token == model.config.eos_token_id) {
            break;
        }
        if (step + 1 < max_new_tokens) {
            predicted_token = decoder.step(predicted_token);
            ++timings.decode_steps;
        }
    }
    const Clock::time_point decode_end = Clock::now();
    const Clock::time_point generate_end = Clock::now();
    timings.decode_ms = elapsed_ms(decode_start, decode_end);
```

```cpp
    timings.generate_ms = elapsed_ms(generate_start, generate_end);
    return tokens;
}

void print_benchmark(const Gpt2BenchmarkTimings& timings,
                     std::size_t prompt_tokens,
                     std::size_t generated_tokens) {
    const double new_tokens = static_cast<double>(generated_tokens);
    const double generated_tokens_per_second =
        timings.generate_ms > 0.0 ? new_tokens * 1000.0 / timings.generate_ms : 0.0;
    const double decode_tokens_per_second =
        timings.decode_ms > 0.0
            ? static_cast<double>(timings.decode_steps) * 1000.0 / timings.decode_ms
            : 0.0;

    std::cout << "benchmark_load_ms " << timings.load_ms << "\n";
    std::cout << "benchmark_tokenize_ms " << timings.tokenize_ms << "\n";
    std::cout << "benchmark_prefill_ms " << timings.prefill_ms << "\n";
    std::cout << "benchmark_decode_ms " << timings.decode_ms << "\n";
    std::cout << "benchmark_generate_ms " << timings.generate_ms << "\n";
    std::cout << "benchmark_total_ms " << timings.total_ms << "\n";
    std::cout << "benchmark_prompt_tokens " << prompt_tokens << "\n";
    std::cout << "benchmark_generated_tokens " << generated_tokens << "\n";
    std::cout << "benchmark_prefill_steps " << timings.prefill_steps << "\n";
    std::cout << "benchmark_decode_steps " << timings.decode_steps << "\n";
    std::cout << "benchmark_worker_count " << timings.worker_count << "\n";
    std::cout << "benchmark_generate_tokens_per_second "
              << generated_tokens_per_second << "\n";
    std::cout << "benchmark_decode_tokens_per_second "
              << decode_tokens_per_second << "\n";
    std::cout << "benchmark_kv_cache_bytes " << timings.kv_cache_bytes << "\n";
}

void print_hotspot_profile(const lcqi::Gpt2HotspotProfile& profile) {
    const double total = profile.total_step_ms > 0.0 ? profile.total_step_ms : 1.0;
    const auto print_ms = [total](std::string_view name, double value) {
        std::cout << "hotspot_" << name << "_ms " << value << "\n";
        std::cout << "hotspot_" << name << "_pct " << (value * 100.0 / total) << "\n";
    };
    std::cout << "hotspot_decoder_steps " << profile.decoder_steps << "\n";
    std::cout << "hotspot_layer_steps " << profile.layer_steps << "\n";
    print_ms("total_step", profile.total_step_ms);
    print_ms("embedding", profile.embedding_ms);
    print_ms("layer_norm", profile.layer_norm_ms);
    print_ms("final_norm", profile.final_norm_ms);
    print_ms("qkv_projection", profile.qkv_projection_ms);
    print_ms("kv_append", profile.kv_append_ms);
    print_ms("attention", profile.attention_ms);
```

```
    print_ms("attention_projection", profile.attention_projection_ms);
    print_ms("residual_add", profile.residual_add_ms);
    print_ms("mlp_fc", profile.mlp_fc_ms);
    print_ms("gelu", profile.gelu_ms);
    print_ms("mlp_projection", profile.mlp_projection_ms);
    print_ms("lm_head", profile.lm_head_ms);
    print_ms("logits_result", profile.logits_result_ms);
}

int run_tiny_mode() {
    const lcqi::Gpt2ReferenceModel model = lcqi::make_tiny_gpt2_reference_model();
    const std::vector<std::int32_t> prompt{
        LCQI_GPT2_TINY_PROMPT_0,
        LCQI_GPT2_TINY_PROMPT_1,
        LCQI_GPT2_TINY_PROMPT_2,
    };
    const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, prompt);
    const std::vector<std::int32_t> generated =
        lcqi::gpt2_generate_greedy(model, prompt, 2);
    std::cout << "mode tiny\n";
    std::cout << "prompt_ids ";
    print_ids(prompt);
    std::cout << "\n";
    std::cout << "predicted_token " << result.predicted_token << "\n";
    std::cout << "logits";
    for (const float value : result.logits) {
        std::cout << ' ' << value;
    }
    std::cout << "\n";
    std::cout << "generated_ids ";
    print_ids(generated);
    std::cout << "\n";
    return EXIT_SUCCESS;
}

int run_real_model_mode(int argc, char** argv) {
    if (argc < LCQI_GPT2_MIN_REAL_ARGC) {
        throw std::runtime_error(
            "usage: lcqi_gpt2 [--engine cached|full] [--threads N] [--benchmark] "
            "model_dir prompt [max_new_tokens]");
    }
    const Clock::time_point total_start = Clock::now();
    const Gpt2CliOptions options = parse_options(argc, argv);

    Gpt2BenchmarkTimings timings;
    const Clock::time_point load_start = Clock::now();
    const lcqi::Gpt2ReferenceModel model = lcqi::load_gpt2_from_directory(options.model_dir);
    const lcqi::Gpt2Tokenizer tokenizer = lcqi::load_gpt2_tokenizer(
```

```cpp
        options.model_dir / "vocab.json",
        options.model_dir / "merges.txt",
        model.config.bos_token_id,
        model.config.eos_token_id);
    timings.load_ms = elapsed_ms(load_start, Clock::now());

    const Clock::time_point tokenize_start = Clock::now();
    const std::vector<std::int32_t> prompt_ids = lcqi::gpt2_encode(tokenizer, options.prompt);
    timings.tokenize_ms = elapsed_ms(tokenize_start, Clock::now());

    std::vector<std::int32_t> generated;
    lcqi::Gpt2HotspotProfile hotspot_profile;
    if (options.engine == Gpt2EngineMode::full_prefix) {
        if (options.profile_hotspots) {
            throw std::runtime_error("--profile-hotspots currently supports cached engine only");
        }
        generated = generate_full_prefix(model, prompt_ids, options.max_new_tokens, timings);
    } else {
        generated = generate_cached(model,
                                    prompt_ids,
                                    options.max_new_tokens,
                                    timings,
                                    options.profile_hotspots ? &hotspot_profile : nullptr,
                                    options.worker_count);
    }
    timings.total_ms = elapsed_ms(total_start, Clock::now());

    std::cout << "mode gpt2_directory\n";
    std::cout << "engine " << engine_name(options.engine) << "\n";
    std::cout << "model_dir " << options.model_dir.string() << "\n";
    std::cout << "prompt_ids ";
    print_ids(prompt_ids);
    std::cout << "\n";
    std::cout << "generated_ids ";
    print_ids(generated);
    std::cout << "\n";
    std::cout << "generated_text " << lcqi::gpt2_decode(tokenizer, generated) << "\n";
    if (options.benchmark) {
        const std::size_t generated_tokens = generated.size() - prompt_ids.size();
        print_benchmark(timings, prompt_ids.size(), generated_tokens);
    }
    if (options.profile_hotspots) {
        print_hotspot_profile(hotspot_profile);
    }
    return EXIT_SUCCESS;
}
```

```cpp

}  // namespace

int main(int argc, char** argv) {
    try {
        if (argc == LCQI_GPT2_TINY_ARGC) {
            return run_tiny_mode();
        }
        return run_real_model_mode(argc, argv);
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << "\n";
        return EXIT_FAILURE;
    }
}
```

第 11 章实践：运行时内存计划、liveness 与 Workspace

本章对应原书中的第三册“推理运行时与内存规划”。不要把它当作概念复述，本章要把 activation arena、workspace、liveness、state edge、debug dump、buffer reuse 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `layout`，核心命令或测试入口是 `lcqi_trace`。

Listing 11.1 第 11 章实践：运行时内存计划、liveness 与 Workspace 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / layout
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：比较普通 activation 和 KV cache 的生命周期。读者要先手写预期结果，再运行项目观察输出。动作是：说明为什么 KV cache 不能被 activation planner 当作临时缓冲复用。如果输出里没有 `checksum` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：内存复用只在生命周期不重叠时成立；debug/oracle/materialization 会延长生命周期这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 liveness 表：每个 checkpoint 的 `defined_at`、`last_used_at`、是否 state、是否允许复用。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 12 章实践：IR、Pass、Lowering 与 Executable Plan

本章对应原书中的第三册“AI 编译器细化：IR、pass、lowering 与 executable plan”。不要把它当作概念复述，本章要把 typed graph、pass、fusion、layout propagation、backend capability、fallback reason 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `run_inference`，核心命令或测试入口是 `lcqi_cli`。

Listing 12.1 第 12 章实践：IR、Pass、Lowering 与 Executable Plan 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_cli / run_inference
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把 tiny MLP 写成三节点 IR：linear、relu、linear。读者要先手写预期结果，再运行项目观察输出。动作是：说明哪些 pass 可以融合，哪些必须保留为 oracle checkpoint。如果输出里没有 `backend` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：IR 的价值不是画图，而是让 shape、layout、量化、后端选择和 fallback 诊断有稳定承载物这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 executable plan 草图：节点、输入输出 tensor、layout、kernel、fallback reason 和 profiler event。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第零部分

后端优化、工业专项与验收

第 13 章实践：CPU 后端、微内核与性能证据

本章对应原书中的第三册“CPU 后端优化细讲”和“CPU decode 实战”。不要把它当作概念复述，本章要把 scalar baseline、packed weight、AVX2、objdump、perf boundary、shape sweep 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `avx2`、`GGUF`、`Q4_K` 和 `Q8_K`，核心命令或测试入口是 `lcqi_bench` 与 `lcqi_gguf_q4_bench`。

Listing 13.1 第 13 章实践：CPU 后端、微内核与性能证据的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_bench / lcqi_gguf_q4_bench / avx2 / Q4_K
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：对同一 kernel 保存 benchmark CSV 和反汇编摘要。读者要先手写预期结果，再运行项目观察输出。动作是：解释速度差来自地址流、SIMD lane、packing、load 或 reduction 哪一项。如果输出里没有 `average_us` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

第二个特例必须对齐真实模型：`lcqi_gguf_q4_bench` 读取 llama.cpp smoke 使用的同一个 `SmolLM2-135M-Instruct-Q4_K_M.gguf`，从 GGUF tensor 表中选出真实 `Q4_K` 矩阵，比较三条路径：预先解成 f32 权重再 matvec、`Q4_K` 权重直接乘 f32 input、`Q4_K` 权重乘临时 `Q8_K` input。报告必须同时写 tensor 名称、shape、原始量化字节、解码后 f32 字节、重复轮次、最大误差和 checksum。

从这个特例推出的规律是：CPU 后端优化必须同时有 oracle、反汇编、shape sweep 和降级边界；只有耗时表不够这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA、GGUF 模型缓存或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。真实模型量化切片用下面命令生成报告：

Listing 13.2 SmolLM2 Q4_K 单算子证据入口

```
python3 books/cpu-volume-3/tools/run_smollm2_q4_k_benchmark.py --no-download --↩
    ↪ repeat 20
```

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系

统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 CPU kernel evidence：命令、机器、编译器、输入 shape、backend、checksum、反汇编摘要和未验证 PMU 边界。若使用真实 GGUF 模型，还要给出模型 repo、文件名、SHA-256、tensor 名、量化格式、同模型 llama.cpp 参考和“为什么不能把单 tensor us 直接等价为端到端 tokens/s”。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 14 章实践：GPU 后端边界、copy 与同步成本

本章对应原书中的第三册“GPU 硬件基础”和“GPU 后端优化细讲”。不要把它当作概念复述，本章要把 device buffer、stream、event、kernel launch、copy、sync、fallback 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `StepTrace`，核心命令或测试入口是 `lcqi_trace`。

Listing 14.1 第 14 章实践：GPU 后端边界、copy 与同步成本的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / StepTrace
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：当前项目没有 GPU kernel，也要写清 adapter 合同。读者要先手写预期结果，再运行项目观察输出。动作是：把 CPU trace 字段扩展成 GPU StepTrace 需要哪些字段。如果输出里没有 `copy_time_ns` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：GPU 快不快不能只看 kernel；copy、sync、sampler、fallback 和 batch wait 必须进入端到端 trace 这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 GPU adapter 设计：buffer owner、stream、event、H2D/D2H 字节、sync 点、fallback 和 oracle 对照。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 15 章实践：工业 LLM 专项能力对照

本章对应原书中的第三册“工业 LLM 专项：投机解码、MoE、LoRA、多 GPU 与源码对照”。不要把它当作概念复述，本章要把 speculative decoding、MoE routing、LoRA adapter、multi-GPU、source reading 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `bandwidth_limit`，核心命令或测试入口是 `lcqi_ledger`。

Listing 15.1 第 15 章实践：工业 LLM 专项能力对照的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_ledger / bandwidth_limit
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：选择一个专项，不写泛泛介绍。读者要先手写预期结果，再运行项目观察输出。动作是：把它接到现有 trace、memory、backend 或 serving 字段上。如果输出里没有 `tokens/s` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：工业专项不是功能清单；必须说明它购买什么能力、增加什么状态、破坏哪些假设这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交专项设计卡：动机、状态新增、trace 字段、回归测试、性能账本和不能验证的部分。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 16 章实践：工程验收、回归门禁与最终项目

本章对应原书中的第三册“工程验收与回归体系”。不要把它当作概念复述，本章要把 reference、unit test、trace golden、benchmark gate、SLO gate、coverage、CI 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `lcqi_tests`，核心命令或测试入口是 `ctest`。

Listing 16.1 第 16 章实践：工程验收、回归门禁与最终项目的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: ctest / lcqi_tests
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把所有前面报告合成最终验收包。读者要先手写预期结果，再运行项目观察输出。动作是：运行 `ctest` 并说明每个 test 保护哪条合同。如果输出里没有 `100% tests passed` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：实践卷最终目标是长期维护：结果正确、trace 稳定、性能可解释、服务指标可回归、失败边界不伪装这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交最终项目：README、命令记录、报告索引、测试结果、benchmark 表、trace 摘要、风险清单和下一阶段计划。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

代码走读：工具、Smoke、Benchmark 与回归测试

工具不是附属品

LCQI 的工程证据不只在 C++ 源码里。smoke 脚本决定真实模型怎样下载、校验和运行；benchmark 程序决定性能口径；trace、ledger 和 serving probe 决定报告字段；CTest 决定哪些行为能长期回归。读者必须把这些文件当作工程合同的一部分。

Benchmark、Trace、Ledger 与 Serving Probe

`bench.cpp` 输出 int8 kernel benchmark。它服务性能观察，但必须和 max diff 对照一起看。

Listing 16.2 src/bench.cpp

```cpp
#include <lcqi/int8_kernels.hpp>

#include <algorithm>
#include <cstdlib>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

namespace {

constexpr const char* LCQI_TARGET_ARCH_X86_64 = "x86_64";
constexpr const char* LCQI_TARGET_ARCH_I386 = "i386";
constexpr const char* LCQI_TARGET_ARCH_UNKNOWN = "unknown";
constexpr std::int32_t LCQI_DEFAULT_REPEAT = 200;
constexpr int LCQI_REPEAT_ARG_INDEX = 1;
constexpr std::int32_t LCQI_LARGE_CASE_REPEAT_DIVISOR = 4;
constexpr std::int32_t LCQI_MIN_LARGE_CASE_REPEAT = 1;
constexpr std::int32_t LCQI_DECODE_SHAPE_SIZE = 4096;

const char* target_arch() noexcept {
#if defined(__x86_64__) || defined(_M_X64)
    return LCQI_TARGET_ARCH_X86_64;
#elif defined(__i386__) || defined(_M_IX86)
    return LCQI_TARGET_ARCH_I386;
#else
    return LCQI_TARGET_ARCH_UNKNOWN;
#endif
}

std::int32_t parse_repeat(int argc, char** argv) {
    if (argc <= LCQI_REPEAT_ARG_INDEX) {
        return LCQI_DEFAULT_REPEAT;
    }
    const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[
    ↪ LCQI_REPEAT_ARG_INDEX]));
    if (repeat <= 0) {
```

```
        throw std::runtime_error("repeat must be positive");
    }
    return repeat;
}

std::vector<lcqi::KernelBenchmarkCase> make_cases(std::int32_t repeat) {
    return {
        {128, 128, 16, repeat},
        {256, 256, 16, repeat},
        {512, 512, 16, repeat},
        {513, 257, 16, repeat},
        {1024, 4096, 16, std::max(LCQI_MIN_LARGE_CASE_REPEAT,
                                   repeat / LCQI_LARGE_CASE_REPEAT_DIVISOR)},
        {LCQI_DECODE_SHAPE_SIZE, LCQI_DECODE_SHAPE_SIZE, 16,
         std::max(LCQI_MIN_LARGE_CASE_REPEAT, repeat / LCQI_LARGE_CASE_REPEAT_DIVISOR)},
    };
}

}  // namespace

int main(int argc, char** argv) {
    try {
        const std::int32_t repeat = parse_repeat(argc, argv);
        std::cout
            << "target_arch,input_size,output_size,output_block,repeat,backend,available,"
               "average_us,max_abs_diff,checksum\n";
        for (const lcqi::KernelBenchmarkCase& benchmark_case : make_cases(repeat)) {
            const lcqi::KernelBenchmarkResult result =
                lcqi::benchmark_linear_i8_case(benchmark_case);
            for (const lcqi::KernelBackendBenchmark& backend : result.backends) {
                std::cout << target_arch() << ','
                          << result.benchmark_case.input_size << ','
                          << result.benchmark_case.output_size << ','
                          << result.benchmark_case.output_block_size << ','
                          << result.benchmark_case.repeat << ','
                          << backend.name << ','
                          << (backend.available ? 1 : 0) << ','
                          << backend.average_us << ','
                          << backend.max_abs_diff << ','
                          << backend.checksum << '\n';
            }
        }
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

`trace.cpp` 输出 reference decoder checkpoint。它用于定位错误边界，不应该被简化成只打印最终 token。

Listing 16.3 src/trace.cpp

```cpp
#include <lcqi/reference_decoder.hpp>
#include <lcqi/tokenizer.hpp>

#include <algorithm>
#include <array>
#include <cstdlib>
#include <exception>
#include <filesystem>
#include <iostream>
#include <span>
#include <sstream>
#include <string>
#include <string_view>

namespace {

constexpr std::array<std::int32_t, 3> LCQI_TRACE_TOKEN_IDS{1, 2, 3};
constexpr std::string_view LCQI_TRACE_PROMPT = "tok1 tok2 tok3";
constexpr std::int32_t LCQI_TRACE_METADATA_LAYER = -1;
constexpr std::int32_t LCQI_TRACE_BOS_ID = 0;
constexpr std::int32_t LCQI_TRACE_EOS_ID = 4;
constexpr std::size_t LCQI_TRACE_VALUE_LIMIT = 8;

std::filesystem::path tokenizer_fixture_path() {
    return std::filesystem::path(__FILE__).parent_path().parent_path() /
              "models" / "tiny_tokenizer.txt";
}

std::vector<std::int32_t> decoder_tokens_from_prompt(const lcqi::TokenizerModel& tokenizer,
                                                     std::string_view prompt) {
    std::vector<std::int32_t> full_tokens = lcqi::encode_prompt(tokenizer, prompt);
    if (full_tokens.size() != LCQI_TRACE_TOKEN_IDS.size() + 2 ||
        full_tokens.front() != LCQI_TRACE_BOS_ID ||
        full_tokens.back() != LCQI_TRACE_EOS_ID) {
        throw std::runtime_error("unexpected tokenizer fixture ids");
    }

    std::vector<std::int32_t> decoder_tokens(
        full_tokens.begin() + 1,
        full_tokens.end() - 1);
    if (!std::equal(decoder_tokens.begin(),
                    decoder_tokens.end(),
                    LCQI_TRACE_TOKEN_IDS.begin(),
                    LCQI_TRACE_TOKEN_IDS.end())) {
        throw std::runtime_error("unexpected decoder token ids");
    }
```

```cpp
    return decoder_tokens;
}

std::string join_ints(std::span<const std::int32_t> values) {
    std::ostringstream stream;
    for (std::size_t index = 0; index < values.size(); ++index) {
        if (index != 0) {
            stream << 'x';
        }
        stream << values[index];
    }
    return stream.str();
}

std::string join_int_values(std::span<const std::int32_t> values) {
    std::ostringstream stream;
    for (std::size_t index = 0; index < values.size(); ++index) {
        if (index != 0) {
            stream << ';';
        }
        stream << values[index];
    }
    return stream.str();
}

std::int32_t checksum_ints(std::span<const std::int32_t> values) {
    std::int32_t result = 0;
    for (const std::int32_t value : values) {
        result += value;
    }
    return result;
}

std::int32_t max_abs_ints(std::span<const std::int32_t> values) {
    std::int32_t result = 0;
    for (const std::int32_t value : values) {
        result = std::max(result, std::abs(value));
    }
    return result;
}

std::string join_floats(std::span<const float> values) {
    std::ostringstream stream;
    const std::size_t count = std::min(values.size(), LCQI_TRACE_VALUE_LIMIT);
    for (std::size_t index = 0; index < count; ++index) {
        if (index != 0) {
            stream << ';';
        }
        stream << values[index];
    }
    if (values.size() > LCQI_TRACE_VALUE_LIMIT) {
        stream << ";...";
```

```cpp
    }
    return stream.str();
}

void print_trace_csv(const lcqi::ReferenceDecodeTraceResult& trace,
                     const lcqi::TokenizerModel& tokenizer,
                     std::span<const std::int32_t> full_tokens) {
    std::cout << "name,token_position,layer_id,shape,stride,dtype,layout,↩
    ↪ checksum,max_abs,values\n";
    std::cout << "prompt,0," << LCQI_TRACE_METADATA_LAYER
              << ",scalar,scalar,utf8,text,0,0," << LCQI_TRACE_PROMPT << '\n';
    std::cout << "tokenizer,0," << LCQI_TRACE_METADATA_LAYER
              << ',' << full_tokens.size() << ",1,i32,contiguous,"
              << checksum_ints(full_tokens) << ','
              << max_abs_ints(full_tokens) << ','
              << join_int_values(full_tokens) << '\n';
    std::cout << "tokenizer_contract,0," << LCQI_TRACE_METADATA_LAYER
              << ",scalar,scalar,u64,fnv1a,"
              << tokenizer_contract_hash(tokenizer) << ",0,"
              << tokenizer.chat_template_hash << '\n';
    for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints)↩
    ↪  {
        std::cout << checkpoint.name << ','
                  << checkpoint.token_position << ','
                  << checkpoint.layer_id << ','
                  << join_ints(checkpoint.shape) << ','
                  << join_ints(checkpoint.stride) << ','
                  << checkpoint.dtype << ','
                  << checkpoint.layout << ','
                  << checkpoint.checksum << ','
                  << checkpoint.max_abs << ','
                  << join_floats(checkpoint.values) << '\n';
    }
    std::cout << "sampler_argmax,"
              << LCQI_TRACE_TOKEN_IDS.size() - 1 << ','
              << LCQI_TRACE_METADATA_LAYER
              << ',' << trace.result.logits.size() << ",1,f32,argmax,"
              << "0,0,token=" << trace.result.predicted_token << '\n';
    std::cout << "generated_token,"
              << LCQI_TRACE_TOKEN_IDS.size() << ','
              << LCQI_TRACE_METADATA_LAYER
              << ",scalar,scalar,i32,generated,"
              << trace.result.predicted_token << ','
              << trace.result.predicted_token << ','
              << trace.result.predicted_token << '\n';
}

}  // namespace

int main() {
    try {
        const lcqi::TokenizerModel tokenizer =
```

```
            lcqi::load_tokenizer(tokenizer_fixture_path());
        const std::vector<std::int32_t> full_tokens =
            lcqi::encode_prompt(tokenizer, LCQI_TRACE_PROMPT);
        const std::vector<std::int32_t> decoder_tokens =
            decoder_tokens_from_prompt(tokenizer, LCQI_TRACE_PROMPT);
        const lcqi::ReferenceDecoderModel model = lcqi::make_tiny_reference_decoder_model();
        const lcqi::ReferenceDecodeTraceResult trace =
            lcqi::run_reference_decode_with_trace(model, decoder_tokens);
        print_trace_csv(trace, tokenizer, full_tokens);
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

`ledger.cpp` 输出 7B 规模账本。它是估算，不是实测吞吐；读者要区分 FLOPs、KV 容量、权重/KV 字节和 bandwidth-only tokens/s 上限。

Listing 16.4 src/ledger.cpp

```
#include <cstdlib>
#include <cstdint>
#include <exception>
#include <iomanip>
#include <iostream>
#include <string_view>

namespace {

constexpr double LCQI_BYTES_PER_GB = 1000.0 * 1000.0 * 1000.0;
constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
constexpr double LCQI_FLOPS_PER_MAC = 2.0;
constexpr std::int32_t LCQI_SEVENB_LAYER_COUNT = 32;
constexpr std::int32_t LCQI_SEVENB_HIDDEN_SIZE = 4096;
constexpr std::int32_t LCQI_SEVENB_QUERY_HEADS = 32;
constexpr std::int32_t LCQI_SEVENB_KV_HEADS = 8;
constexpr std::int32_t LCQI_SEVENB_HEAD_DIM = 128;
constexpr std::int32_t LCQI_SEVENB_INTERMEDIATE_SIZE = 11008;
constexpr std::int32_t LCQI_SEVENB_CONTEXT_LENGTH = 4096;

struct DTypeLedger {
    std::string_view name;
    double weight_gb_per_token = 0.0;
    double kv_element_bytes = 0.0;
};

constexpr DTypeLedger LCQI_DTYPE_LEDGERS[] = {
    {"fp16", 14.0, 2.0},
    {"int8", 7.0, 2.0},
    {"int4", 3.7, 2.0},
};
```

```
constexpr double LCQI_BANDWIDTH_GB_PER_S[] = {
    50.0,
    100.0,
    200.0,
    600.0,
    1000.0,
    1600.0,
};

double linear_macs_per_layer() noexcept {
    const double hidden = LCQI_SEVENB_HIDDEN_SIZE;
    const double kv_width = LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM;
    const double qkv = hidden * (hidden + 2.0 * kv_width);
    const double output_projection = hidden * hidden;
    const double mlp = 3.0 * hidden * LCQI_SEVENB_INTERMEDIATE_SIZE;
    return qkv + output_projection + mlp;
}

double attention_macs_per_layer() noexcept {
    return 2.0 * LCQI_SEVENB_QUERY_HEADS * LCQI_SEVENB_CONTEXT_LENGTH *
           LCQI_SEVENB_HEAD_DIM;
}

double kv_bytes_per_token_all_layers(double element_bytes) noexcept {
    return static_cast<double>(LCQI_SEVENB_LAYER_COUNT) * 2.0 *
           LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM * element_bytes;
}

double kv_read_gb_per_decode_token(double element_bytes) noexcept {
    const double bytes = static_cast<double>(LCQI_SEVENB_CONTEXT_LENGTH) *
                         kv_bytes_per_token_all_layers(element_bytes);
    return bytes / LCQI_BYTES_PER_GB;
}

double kv_capacity_mib(double element_bytes, std::int32_t context_length) noexcept {
    const double bytes = static_cast<double>(context_length) *
                         kv_bytes_per_token_all_layers(element_bytes);
    return bytes / LCQI_BYTES_PER_MIB;
}

void print_model_rows() {
    const double linear_macs = linear_macs_per_layer();
    const double attention_macs = attention_macs_per_layer();
    const double total_flops =
        (linear_macs + attention_macs) * LCQI_SEVENB_LAYER_COUNT * LCQI_FLOPS_PER_MAC;
    std::cout << "section,item,value,unit,notes\n";
    std::cout << "model,layers," << LCQI_SEVENB_LAYER_COUNT << ",count,7B fixture\n";
    std::cout << "model,hidden," << LCQI_SEVENB_HIDDEN_SIZE << ",elements,H\n";
```

```
    std::cout << "model,query_heads," << LCQI_SEVENB_QUERY_HEADS << ",count,GQA query heads\n";
    std::cout << "model,kv_heads," << LCQI_SEVENB_KV_HEADS << ",count,GQA KV heads\n";
    std::cout << "model,head_dim," << LCQI_SEVENB_HEAD_DIM << ",elements,per head\n";
    std::cout << "compute,linear_macs_per_layer," << linear_macs << ",macs,decode token\n";
    std::cout << "compute,attention_macs_per_layer," << attention_macs
              << ",macs,context=4096\n";
    std::cout << "compute,total_decode_flops_per_token," << total_flops
              << ",flops,linear plus attention\n";
}

void print_kv_rows() {
    for (const std::int32_t context : {1024, 2048, 4096, 8192}) {
        std::cout << "kv,fp16_capacity_context_" << context << ','
                  << kv_capacity_mib(2.0, context)
                  << ",MiB,single session\n";
    }
}

void print_dtype_rows() {
    for (const DTypeLedger& dtype : LCQI_DTYPE_LEDGERS) {
        const double kv_read_gb = kv_read_gb_per_decode_token(dtype.kv_element_bytes);
        const double total_gb = dtype.weight_gb_per_token + kv_read_gb;
        std::cout << "decode_bytes," << dtype.name << "_weight_gb,"
                  << dtype.weight_gb_per_token << ",GB/token,weight stream\n";
        std::cout << "decode_bytes," << dtype.name << "_kv_read_gb,"
                  << kv_read_gb << ",GB/token,context=4096\n";
        std::cout << "decode_bytes," << dtype.name << "_total_gb,"
                  << total_gb << ",GB/token,weight plus KV\n";
        for (const double bandwidth : LCQI_BANDWIDTH_GB_PER_S) {
            std::cout << "bandwidth_limit," << dtype.name << "_at_"
                      << static_cast<std::int32_t>(bandwidth) << "_gbps,"
                      << bandwidth / total_gb
                      << ",tokens/s,bandwidth-only upper bound\n";
        }
    }
}

}  // namespace

int main() {
    try {
        std::cout << std::fixed << std::setprecision(3);
        print_model_rows();
        print_kv_rows();
        print_dtype_rows();
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
```

```
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

`serving_probe.cpp` 固定短请求、RAG 请求、取消请求和超长请求。它把 TTFT、TPOT、拒绝和取消回收变成可观察字段。

Listing 16.5 src/serving_probe.cpp

```cpp
#include <algorithm>
#include <cstdlib>
#include <cstdint>
#include <exception>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <string_view>
#include <vector>

namespace {

constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
constexpr std::int32_t LCQI_PROBE_LAYER_COUNT = 32;
constexpr std::int32_t LCQI_PROBE_KV_HEADS = 8;
constexpr std::int32_t LCQI_PROBE_HEAD_DIM = 128;
constexpr double LCQI_PROBE_KV_ELEMENT_BYTES = 2.0;
constexpr double LCQI_PROBE_KV_BUDGET_MIB = 512.0;
constexpr double LCQI_PROBE_WORKSPACE_MIB = 64.0;
constexpr double LCQI_PROBE_ARENA_MIB = 32.0;
constexpr double LCQI_PROBE_TRACE_MIB = 4.0;
constexpr double LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS = 8.0;
constexpr double LCQI_PROBE_PREFILL_MS_PER_TOKEN = 0.08;
constexpr double LCQI_PROBE_DECODE_STEP_MS = 14.0;
constexpr double LCQI_PROBE_CANCEL_RECLAIM_MS = 2.0;
constexpr std::size_t LCQI_PERCENTILE_MIN_INDEX = 1;

struct ProbeRequest {
    std::string_view request_id;
    std::int32_t prompt_tokens = 0;
    std::int32_t max_new_tokens = 0;
    bool cancel_after_prefill = false;
};

struct ProbeResult {
    std::string_view request_id;
    std::int32_t accepted = 0;
    std::string_view rejection_reason;
    std::int32_t prompt_tokens = 0;
    std::int32_t reserved_tokens = 0;
    double kv_reserved_mib = 0.0;
    double queue_wait_ms = 0.0;
    double prefill_ms = 0.0;
```

```cpp
    double ttft_ms = 0.0;
    double decode_step_p95_ms = 0.0;
    double tpot_ms = 0.0;
    std::int32_t completion_tokens = 0;
    std::string_view finish_reason;
    double cancel_reclaim_ms = 0.0;
};

const std::vector<ProbeRequest> LCQI_PROBE_REQUESTS{
    {"req_short", 32, 4, false},
    {"req_rag", 512, 8, false},
    {"req_cancel", 128, 16, true},
    {"req_too_long", 4096, 512, false},
};

double kv_mib_for_tokens(std::int32_t tokens) noexcept {
    const double bytes = static_cast<double>(tokens) * LCQI_PROBE_LAYER_COUNT * 2.0 *
                         LCQI_PROBE_KV_HEADS * LCQI_PROBE_HEAD_DIM *
                         LCQI_PROBE_KV_ELEMENT_BYTES;
    return bytes / LCQI_BYTES_PER_MIB;
}

double percentile(std::vector<double> values, double fraction) {
    if (values.empty()) {
        return 0.0;
    }
    std::sort(values.begin(), values.end());
    const std::size_t max_index = values.size() - LCQI_PERCENTILE_MIN_INDEX;
    const std::size_t index =
        std::min(max_index,
                 static_cast<std::size_t>(
                     std::ceil(fraction * static_cast<double>(max_index))));
    return values[index];
}

std::vector<ProbeResult> run_probe() {
    std::vector<ProbeResult> results;
    double reserved_kv_mib = 0.0;
    std::int32_t accepted_before = 0;
    for (const ProbeRequest& request : LCQI_PROBE_REQUESTS) {
        ProbeResult result;
        result.request_id = request.request_id;
        result.prompt_tokens = request.prompt_tokens;
        result.reserved_tokens = request.prompt_tokens + request.max_new_tokens;
        result.kv_reserved_mib = kv_mib_for_tokens(result.reserved_tokens);
        const double static_session_mib =
            result.kv_reserved_mib + LCQI_PROBE_WORKSPACE_MIB +
            LCQI_PROBE_ARENA_MIB + LCQI_PROBE_TRACE_MIB;
        if (reserved_kv_mib + result.kv_reserved_mib > LCQI_PROBE_KV_BUDGET_MIB) {
```

```
            result.accepted = 0;
            result.rejection_reason = "memory_budget_exceeded";
            result.finish_reason = "rejected";
            results.push_back(result);
            continue;
        }

        reserved_kv_mib += result.kv_reserved_mib;
        result.accepted = 1;
        result.rejection_reason = "none";
        result.queue_wait_ms = static_cast<double>(accepted_before) *
                               LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS;
        result.prefill_ms = static_cast<double>(request.prompt_tokens) *
                            LCQI_PROBE_PREFILL_MS_PER_TOKEN;
        result.ttft_ms = result.queue_wait_ms + result.prefill_ms + LCQI_PROBE_DECODE_STEP_MS;
        result.decode_step_p95_ms = LCQI_PROBE_DECODE_STEP_MS +
                                    static_session_mib / 512.0;
        result.tpot_ms = result.decode_step_p95_ms;
        if (request.cancel_after_prefill) {
            result.completion_tokens = 0;
            result.finish_reason = "cancelled";
            result.cancel_reclaim_ms = LCQI_PROBE_CANCEL_RECLAIM_MS;
            reserved_kv_mib -= result.kv_reserved_mib;
        } else {
            result.completion_tokens = request.max_new_tokens;
            result.finish_reason = "max_tokens";
        }
        ++accepted_before;
        results.push_back(result);
    }
    return results;
}

void print_request_rows(const std::vector<ProbeResult>& results) {
    std::cout << "row_type,request_id,accepted,rejection_reason,prompt_tokens,reserved_tokens,"
                 "kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,decode_step_p95_ms,"
                 "tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,metric_name,"
                 "metric_value\n";
    for (const ProbeResult& result : results) {
        std::cout << "request," << result.request_id << ','
                  << result.accepted << ','
                  << result.rejection_reason << ','
                  << result.prompt_tokens << ','
                  << result.reserved_tokens << ','
                  << result.kv_reserved_mib << ','
                  << result.queue_wait_ms << ','
                  << result.prefill_ms << ','
                  << result.ttft_ms << ','
```

```
                  << result.decode_step_p95_ms << ','
                  << result.tpot_ms << ','
                  << result.completion_tokens << ','
                  << result.finish_reason << ','
                  << result.cancel_reclaim_ms << ",,\n";
    }
}

void print_summary_row(const std::vector<ProbeResult>& results) {
    std::vector<double> ttft_values;
    std::vector<double> queue_values;
    double accepted = 0.0;
    double rejected = 0.0;
    double cancelled = 0.0;
    double reserved_peak_mib = 0.0;
    double running_reserved_mib = 0.0;
    for (const ProbeResult& result : results) {
        if (result.accepted == 0) {
            rejected += 1.0;
            continue;
        }
        accepted += 1.0;
        ttft_values.push_back(result.ttft_ms);
        queue_values.push_back(result.queue_wait_ms);
        running_reserved_mib += result.kv_reserved_mib;
        reserved_peak_mib = std::max(reserved_peak_mib, running_reserved_mib);
        if (result.finish_reason == "cancelled") {
            cancelled += 1.0;
            running_reserved_mib -= result.kv_reserved_mib;
        }
    }
    const double total = accepted + rejected;
    const double rejection_rate = total == 0.0 ? 0.0 : rejected / total;
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "accepted_count," << accepted << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,"
              << LCQI_PROBE_CANCEL_RECLAIM_MS << ",cancel_count," << cancelled << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "rejected_count," << rejected << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "rejection_rate," << rejection_rate << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "kv_reserved_peak_mib," << reserved_peak_mib << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "queue_wait_p95_ms," << percentile(queue_values, 0.95) << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "ttft_p95_ms," << percentile(ttft_values, 0.95) << '\n';
}

}  // namespace
```

```cpp

int main() {
    try {
        std::cout << std::fixed << std::setprecision(3);
        const std::vector<ProbeResult> results = run_probe();
        print_request_rows(results);
        print_summary_row(results);
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

`onednn_bench.cpp` 是可选外部库对标入口。它只在环境安装 oneDNN 时构建，不能把不可用平台写成已验证性能。

Listing 16.6 src/onednn_bench.cpp

```cpp
#include <dnnl.hpp>

#include <chrono>
#include <cstdint>
#include <cstdlib>
#include <iostream>
#include <numeric>
#include <stdexcept>
#include <string>
#include <unordered_map>
#include <vector>

namespace {

constexpr std::int64_t LCQI_ONEDNN_BATCH = 1;
constexpr std::int64_t LCQI_ONEDNN_K = 4096;
constexpr std::int64_t LCQI_ONEDNN_O = 4096;
constexpr std::int32_t LCQI_ONEDNN_DEFAULT_REPEAT = 20;
constexpr std::int32_t LCQI_ONEDNN_WARMUP = 3;
constexpr int LCQI_REPEAT_ARG_INDEX = 1;
constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
constexpr float LCQI_INPUT_SCALE = 0.125F;
constexpr std::int32_t LCQI_INPUT_MODULUS = 17;
constexpr std::int32_t LCQI_INPUT_CENTER = 8;
constexpr std::int32_t LCQI_WEIGHT_MODULUS = 23;
constexpr std::int32_t LCQI_WEIGHT_CENTER = 11;
constexpr float LCQI_WEIGHT_SCALE = 0.03125F;

std::int32_t parse_repeat(int argc, char** argv) {
    if (argc <= LCQI_REPEAT_ARG_INDEX) {
        return LCQI_ONEDNN_DEFAULT_REPEAT;
    }
    const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[LCQI_REPEAT_ARG_INDEX]));
```

```cpp
    if (repeat <= 0) {
        throw std::runtime_error("repeat must be positive");
    }
    return repeat;
}

std::vector<float> make_input() {
    std::vector<float> input(static_cast<std::size_t>(LCQI_ONEDNN_K));
    for (std::int64_t index = 0; index < LCQI_ONEDNN_K; ++index) {
        input[static_cast<std::size_t>(index)] =
            static_cast<float>((index % LCQI_INPUT_MODULUS) - LCQI_INPUT_CENTER) *
            LCQI_INPUT_SCALE;
    }
    return input;
}

std::vector<float> make_weights_kn() {
    std::vector<float> weights(static_cast<std::size_t>(LCQI_ONEDNN_K * LCQI_ONEDNN_O));
    for (std::int64_t k = 0; k < LCQI_ONEDNN_K; ++k) {
        for (std::int64_t out = 0; out < LCQI_ONEDNN_O; ++out) {
            const std::int64_t row_major_out_k = out * LCQI_ONEDNN_K + k;
            const float value =
                static_cast<float>((row_major_out_k % LCQI_WEIGHT_MODULUS) -
                                   LCQI_WEIGHT_CENTER) *
                LCQI_WEIGHT_SCALE;
            weights[static_cast<std::size_t>(k * LCQI_ONEDNN_O + out)] = value;
        }
    }
    return weights;
}

float checksum(const std::vector<float>& values) {
    return std::accumulate(values.begin(), values.end(), 0.0F);
}

}  // namespace

int main(int argc, char** argv) {
    try {
        const std::int32_t repeat = parse_repeat(argc, argv);
        std::vector<float> input = make_input();
        std::vector<float> weights = make_weights_kn();
        std::vector<float> output(static_cast<std::size_t>(LCQI_ONEDNN_BATCH * LCQI_ONEDNN_O),
                                  0.0F);

        dnnl::engine engine(dnnl::engine::kind::cpu, 0);
        dnnl::stream stream(engine);

        const dnnl::memory::dims source_dims = {LCQI_ONEDNN_BATCH,
```

```cpp
LCQI_ONEDNN_K};
        const dnnl::memory::dims weight_dims = {LCQI_ONEDNN_K, LCQI_ONEDNN_O};
        const dnnl::memory::dims output_dims = {LCQI_ONEDNN_BATCH, LCQI_ONEDNN_O};

        const dnnl::memory::desc source_desc(
            source_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag::ab);
        const dnnl::memory::desc weight_desc(
            weight_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag::ab);
        const dnnl::memory::desc output_desc(
            output_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag::ab);

        dnnl::memory source_memory(source_desc, engine, input.data());
        dnnl::memory weight_memory(weight_desc, engine, weights.data());
        dnnl::memory output_memory(output_desc, engine, output.data());

        dnnl::matmul::primitive_desc matmul_desc(engine, source_desc, weight_desc, output_desc);
        dnnl::matmul matmul(matmul_desc);
        const std::unordered_map<int, dnnl::memory> arguments = {
            {DNNL_ARG_SRC, source_memory},
            {DNNL_ARG_WEIGHTS, weight_memory},
            {DNNL_ARG_DST, output_memory},
        };

        for (std::int32_t index = 0; index < LCQI_ONEDNN_WARMUP; ++index) {
            matmul.execute(stream, arguments);
            stream.wait();
        }

        const auto begin = std::chrono::steady_clock::now();
        for (std::int32_t index = 0; index < repeat; ++index) {
            matmul.execute(stream, arguments);
            stream.wait();
        }
        const auto end = std::chrono::steady_clock::now();
        const std::chrono::duration<double> elapsed = end - begin;
        const double average_us =
            elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>(repeat);

        std::cout << "library,input_size,output_size,dtype,repeat,average_us,checksum\n";
        std::cout << "onednn," << LCQI_ONEDNN_K << ',' << LCQI_ONEDNN_O
                  << ",f32," << repeat << ',' << average_us << ','
                  << checksum(output) << '\n';
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
```

```
        return EXIT_FAILURE;
    }
}
```

gguf_q4_bench.cpp 是真实 GGUF 量化 tensor 的单算子 benchmark。它读取同一个 SmolLM2 Q4_K_M GGUF，选择真实 Q4_K 矩阵，分别测 f32 解码基线、Q4_Kxf32 和 Q4_KxQ8_K。它输出的 speedup 只适用于这个单 tensor matvec，不能写成完整模型 tokens/s。

Listing 16.7 src/gguf_q4_bench.cpp

```cpp
#include <lcqi/gguf.hpp>
#include <lcqi/q4_k.hpp>

#include <algorithm>
#include <chrono>
#include <cmath>
#include <cstdlib>
#include <filesystem>
#include <iomanip>
#include <iostream>
#include <span>
#include <stdexcept>
#include <string>
#include <string_view>
#include <vector>

namespace {

constexpr int LCQI_ARG_MODEL = 1;
constexpr int LCQI_ARG_TENSOR = 2;
constexpr int LCQI_ARG_REPEAT = 3;
constexpr int LCQI_DEFAULT_REPEAT = 20;
constexpr std::int64_t LCQI_MAX_AUTO_ROWS = 4096;
constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
constexpr float LCQI_INPUT_PATTERN_SCALE = 0.015625F;
constexpr int LCQI_INPUT_PATTERN_MODULUS = 31;
constexpr int LCQI_INPUT_PATTERN_CENTER = 15;

struct SelectedTensor {
    const lcqi::GgufTensorInfo* tensor = nullptr;
    std::int64_t rows = 0;
    std::int64_t columns = 0;
};

std::int32_t parse_repeat(int argc, char** argv) {
    if (argc <= LCQI_ARG_REPEAT) {
        return LCQI_DEFAULT_REPEAT;
    }
    const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[LCQI_ARG_REPEAT]));
    if (repeat <= 0) {
        throw std::runtime_error("repeat must be positive");
    }
```

```cpp
    return repeat;
}

bool is_q4_k_matrix(const lcqi::GgufTensorInfo& tensor) {
    return tensor.type == lcqi::GgmlType::q4_k && tensor.shape.size() == 2 &&
           tensor.shape[0] > 0 && tensor.shape[1] > 0 &&
           tensor.shape[0] % lcqi::LCQI_QK_K_BLOCK_VALUES == 0;
}

bool preferred_tensor_name(std::string_view name) {
    return name.find("ffn_up.weight") != std::string_view::npos ||
           name.find("ffn_gate.weight") != std::string_view::npos ||
           name.find("ffn_down.weight") != std::string_view::npos;
}

SelectedTensor select_tensor(const lcqi::GgufManifest& manifest, std::string_view requested) {
    if (!requested.empty() && requested != "auto") {
        const lcqi::GgufTensorInfo* tensor = manifest.find_tensor(requested);
        if (tensor == nullptr) {
            throw std::runtime_error("requested GGUF tensor not found");
        }
        if (!is_q4_k_matrix(*tensor)) {
            throw std::runtime_error("requested GGUF tensor is not a rank-2 Q4_K matrix");
        }
        return {tensor, tensor->shape[1], tensor->shape[0]};
    }

    const lcqi::GgufTensorInfo* best = nullptr;
    for (const lcqi::GgufTensorInfo& tensor : manifest.tensors) {
        if (!is_q4_k_matrix(tensor) || tensor.shape[1] > LCQI_MAX_AUTO_ROWS) {
            continue;
        }
        if (best == nullptr) {
            best = &tensor;
            continue;
        }
        const bool tensor_preferred = preferred_tensor_name(tensor.name);
        const bool best_preferred = preferred_tensor_name(best->name);
        if ((tensor_preferred && !best_preferred) ||
            (tensor_preferred == best_preferred &&
             tensor.element_count() > best->element_count())) {
            best = &tensor;
        }
    }
    if (best == nullptr) {
        throw std::runtime_error("no suitable rank-2 Q4_K tensor found in GGUF model");
    }
    return {best, best->shape[1], best->shape[0]};
}
```

```cpp

std::vector<float> make_input(std::int64_t columns) {
    std::vector<float> input(static_cast<std::size_t>(columns), 0.0F);
    for (std::int64_t index = 0; index < columns; ++index) {
        input[static_cast<std::size_t>(index)] =
            static_cast<float>((index % LCQI_INPUT_PATTERN_MODULUS) -
                               LCQI_INPUT_PATTERN_CENTER) *
            LCQI_INPUT_PATTERN_SCALE;
    }
    return input;
}

float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs) {
    if (lhs.size() != rhs.size()) {
        throw std::runtime_error("max_abs_diff size mismatch");
    }
    float diff = 0.0F;
    for (std::size_t index = 0; index < lhs.size(); ++index) {
        diff = std::max(diff, std::fabs(lhs[index] - rhs[index]));
    }
    return diff;
}

float checksum(std::span<const float> values) {
    float sum = 0.0F;
    for (const float value : values) {
        sum += value;
    }
    return sum;
}

void matvec_f32(std::span<const float> weights,
                std::int64_t rows,
                std::int64_t columns,
                std::span<const float> input,
                std::span<float> output) {
    if (weights.size() != static_cast<std::size_t>(rows * columns) ||
        input.size() != static_cast<std::size_t>(columns) ||
        output.size() != static_cast<std::size_t>(rows)) {
        throw std::runtime_error("F32 matvec shape mismatch");
    }
    for (std::int64_t row = 0; row < rows; ++row) {
        const float* row_weights = weights.data() + row * columns;
        float sum = 0.0F;
        for (std::int64_t column = 0; column < columns; ++column) {
            sum += row_weights[column] * input[static_cast<std::size_t>(column)
    ];
        }
        output[static_cast<std::size_t>(row)] = sum;
    }
}

```

```cpp
template <typename Callable>
double measure_average_us(std::int32_t repeat, Callable&& callable) {
    const auto begin = std::chrono::steady_clock::now();
    for (std::int32_t index = 0; index < repeat; ++index) {
        callable();
    }
    const auto end = std::chrono::steady_clock::now();
    const std::chrono::duration<double> elapsed = end - begin;
    return elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>(repeat);
}

void print_usage(const char* program) {
    std::cerr << "usage: " << program << " <model.gguf> [tensor-name|auto] [repeat]\n";
}

}  // namespace

int main(int argc, char** argv) {
    try {
        if (argc <= LCQI_ARG_MODEL) {
            print_usage(argv[0]);
            return EXIT_FAILURE;
        }
        const std::filesystem::path model_path = argv[LCQI_ARG_MODEL];
        const std::string tensor_request =
            argc > LCQI_ARG_TENSOR ? argv[LCQI_ARG_TENSOR] : "auto";
        const std::int32_t repeat = parse_repeat(argc, argv);

        const lcqi::GgufManifest manifest = lcqi::load_gguf_manifest(model_path);
        const SelectedTensor selected = select_tensor(manifest, tensor_request);
        const std::vector<std::uint8_t> tensor_bytes =
            lcqi::read_gguf_tensor_bytes(model_path, *selected.tensor);
        const std::vector<float> input = make_input(selected.columns);
        const std::vector<lcqi::Q8KBlock> q8_input = lcqi::quantize_q8_k_input(input);
        std::vector<float> q4_f32_output(static_cast<std::size_t>(selected.rows), 0.0F);
        std::vector<float> q4_q8_output(q4_f32_output.size(), 0.0F);
        std::vector<float> f32_output(q4_f32_output.size(), 0.0F);

        lcqi::matvec_q4_k_f32(tensor_bytes,
                              selected.rows,
                              selected.columns,
                              input,
                              q4_f32_output);
        lcqi::matvec_q4_k_q8(tensor_bytes,
                             selected.rows,
                             selected.columns,
```

```
                         q8_input,
                         q4_q8_output);

    const std::vector<float> f32_weights =
        lcqi::dequantize_q4_k(tensor_bytes, selected.rows * selected.columns);
    matvec_f32(f32_weights, selected.rows, selected.columns, input, f32_output);

    const double f32_us = measure_average_us(
        repeat,
        [&]() { matvec_f32(f32_weights, selected.rows, selected.columns, input, f32_output); });
    const double q4_f32_us = measure_average_us(
        repeat,
        [&]() {
            lcqi::matvec_q4_k_f32(tensor_bytes,
                                  selected.rows,
                                  selected.columns,
                                  input,
                                  q4_f32_output);
        });
    const double q8_quantize_us = measure_average_us(
        repeat,
        [&]() {
            const std::vector<lcqi::Q8KBlock> temporary =
                lcqi::quantize_q8_k_input(input);
            static_cast<void>(temporary);
        });
    const double q4_q8_us = measure_average_us(
        repeat,
        [&]() {
            lcqi::matvec_q4_k_q8(tensor_bytes,
                                 selected.rows,
                                 selected.columns,
                                 q8_input,
                                 q4_q8_output);
        });

    const double q4_q8_total_us = q8_quantize_us + q4_q8_us;
    std::cout << std::fixed << std::setprecision(3);
    std::cout << "lcqi_smollm2_q4_k_benchmark\n";
    std::cout << "model_path=" << model_path.string() << '\n';
    std::cout << "gguf_version=" << manifest.version << '\n';
    std::cout << "gguf_alignment=" << manifest.alignment << '\n';
    std::cout << "gguf_tensors=" << manifest.tensors.size() << '\n';
    std::cout << "tensor_name=" << selected.tensor->name << '\n';
    std::cout << "tensor_type=" << lcqi::ggml_type_name(selected.tensor->type) << '\n';
    std::cout << "tensor_shape_ne0_ne1=" << selected.columns << 'x' << selected.rows << '\n';
    std::cout << "tensor_bytes=" << selected.tensor->byte_size << '\n';
```

```
        std::cout << "f32_dequantized_bytes=" << f32_weights.size() * sizeof(float) << '\n';
        std::cout << "repeat=" << repeat << '\n';
        std::cout << "f32_matvec_average_us=" << f32_us << '\n';
        std::cout << "q4_f32_input_average_us=" << q4_f32_us << '\n';
        std::cout << "q8_quantize_input_average_us=" << q8_quantize_us << '\n';
        std::cout << "q4_q8_backend=" << lcqi::q4_k_q8_active_backend() << '\n';
        std::cout << "q4_q8_matvec_average_us=" << q4_q8_us << '\n';
        std::cout << "q4_q8_total_average_us=" << q4_q8_total_us << '\n';
        std::cout << "speedup_q4_q8_total_vs_f32="
                  << (q4_q8_total_us > 0.0 ? f32_us / q4_q8_total_us : 0.0) << '\n';
        std::cout << "speedup_q4_f32_input_vs_f32="
                  << (q4_f32_us > 0.0 ? f32_us / q4_f32_us : 0.0) << '\n';
        std::cout << "q4_f32_vs_f32_max_abs_diff="
                  << max_abs_diff(q4_f32_output, f32_output) << '\n';
        std::cout << "q4_q8_vs_q4_f32_max_abs_diff="
                  << max_abs_diff(q4_q8_output, q4_f32_output) << '\n';
        std::cout << "q4_q8_checksum=" << checksum(q4_q8_output) << '\n';
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

`ggml_tensors.cpp` 是真实 GGUF 混合量化进入 LCQI 的格式闸门。它按 GGML block 布局解码 `Q8_0`、`Q5_0`、`Q6_K` 和已有 `Q4_K`，也读取 `F32` 标量张量。测试用手造 block 固定每个格式的位布局，真实 SmolLM2 smoke 再验证同一个解码层能读完 272 个 tensor。

Listing 16.8 src/ggml_tensors.cpp

```
#include <lcqi/ggml_tensors.hpp>

#include <lcqi/q4_k.hpp>

#include <cmath>
#include <cstring>
#include <limits>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr std::uint32_t LCQI_GGML_F32_EXPONENT_MASK = 0x7F800000U;
constexpr std::uint32_t LCQI_GGML_F16_SIGN_MASK = 0x8000U;
constexpr std::uint32_t LCQI_GGML_F16_EXPONENT_MASK = 0x7C00U;
constexpr std::uint32_t LCQI_GGML_F16_MANTISSA_MASK = 0x03FFU;
constexpr std::uint32_t LCQI_GGML_F16_EXPONENT_BIAS = 15U;
constexpr std::uint32_t LCQI_GGML_F32_EXPONENT_BIAS = 127U;
constexpr std::uint32_t LCQI_GGML_F32_MANTISSA_BITS = 23U;
```

```cpp
constexpr std::uint32_t LCQI_GGML_F16_MANTISSA_BITS = 10U;
constexpr std::uint32_t LCQI_GGML_F16_TO_F32_SIGN_SHIFT = 16U;
constexpr std::uint32_t LCQI_GGML_BYTE_BITS = 8U;
constexpr std::uint8_t LCQI_GGML_LOW_NIBBLE_MASK = 0x0FU;
constexpr std::uint8_t LCQI_GGML_Q5_HIGH_BIT_MASK = 0x10U;
constexpr std::uint32_t LCQI_GGML_Q5_HIGH_BIT_SHIFT = 4U;
constexpr std::int32_t LCQI_GGML_Q5_SECOND_HALF_BIT_OFFSET = 16;
constexpr std::uint8_t LCQI_GGML_Q6_HIGH_TWO_BITS_MASK = 0x03U;
constexpr std::int32_t LCQI_GGML_Q5_ZERO_POINT = 16;
constexpr std::int32_t LCQI_GGML_Q6_ZERO_POINT = 32;
constexpr std::int32_t LCQI_GGML_Q6_HALF_BLOCK_VALUES = 128;
constexpr std::int32_t LCQI_GGML_Q6_QUARTER_BLOCK_VALUES = 64;
constexpr std::int32_t LCQI_GGML_Q6_SUBBLOCK_VALUES = 32;
constexpr std::int32_t LCQI_GGML_Q6_SCALE_STRIDE = 8;
constexpr std::int32_t LCQI_GGML_F32_BYTES = 4;

[[nodiscard]] std::size_t checked_size(std::int64_t value, const char* name) {
    if (value < 0 ||
        static_cast<std::uint64_t>(value) >
            static_cast<std::uint64_t>(std::numeric_limits<std::size_t>::max()) ) {
        throw std::runtime_error(std::string(name) + " is outside size_t range");
    }
    return static_cast<std::size_t>(value);
}

void require_multiple(std::int64_t value, std::int64_t divisor, const char* name) {
    if (value <= 0 || value % divisor != 0) {
        throw std::runtime_error(std::string(name) + " must be a positive multiple of " +
                                 std::to_string(divisor));
    }
}

[[nodiscard]] std::uint16_t read_le_u16(const std::uint8_t* bytes) {
    return static_cast<std::uint16_t>(
        static_cast<std::uint16_t>(bytes[0]) |
        (static_cast<std::uint16_t>(bytes[1]) << LCQI_GGML_BYTE_BITS));
}

[[nodiscard]] std::uint32_t read_le_u32(const std::uint8_t* bytes) {
    std::uint32_t value = 0;
    for (std::uint32_t index = 0; index < LCQI_GGML_F32_BYTES; ++index) {
        value |= static_cast<std::uint32_t>(bytes[index]) <<
                 (LCQI_GGML_BYTE_BITS * index);
    }
    return value;
}

[[nodiscard]] float bits_to_float(std::uint32_t bits) {
```

```
    float value = 0.0F;
    std::memcpy(&value, &bits, sizeof(value));
    return value;
}

void validate_quantized_bytes(std::span<const std::uint8_t> bytes,
                              std::int64_t element_count,
                              std::int64_t block_values,
                              std::int64_t block_bytes,
                              const char* name) {
    require_multiple(element_count, block_values, name);
    const std::int64_t expected_bytes = (element_count / block_values) * block_bytes;
    if (bytes.size() != checked_size(expected_bytes, name)) {
        throw std::runtime_error(std::string(name) + " byte count does not match element count");
    }
}

void dequantize_f32_to(std::span<const std::uint8_t> bytes, std::span<float> output) {
    const std::size_t expected_bytes = output.size() * LCQI_GGML_F32_BYTES;
    if (bytes.size() != expected_bytes) {
        throw std::runtime_error("F32 byte count does not match element count");
    }
    for (std::size_t index = 0; index < output.size(); ++index) {
        output[index] = bits_to_float(
            read_le_u32(bytes.data() + index * LCQI_GGML_F32_BYTES));
    }
}

void dequantize_q8_0_to(std::span<const std::uint8_t> bytes, std::span<float> output) {
    validate_quantized_bytes(bytes,
                             static_cast<std::int64_t>(output.size()),
                             LCQI_Q8_0_BLOCK_VALUES,
                             LCQI_Q8_0_BLOCK_BYTES,
                             "Q8_0");
    const std::int64_t block_count =
        static_cast<std::int64_t>(output.size()) / LCQI_Q8_0_BLOCK_VALUES;
    for (std::int64_t block_index = 0; block_index < block_count; ++block_index) {
        const std::uint8_t* block =
            bytes.data() + checked_size(block_index * LCQI_Q8_0_BLOCK_BYTES, "Q8_0 offset");
        const float d = ggml_f16_to_f32(read_le_u16(block));
        const std::uint8_t* qs = block + sizeof(std::uint16_t);
        float* target =
            output.data() + checked_size(block_index * LCQI_Q8_0_BLOCK_VALUES, "Q8_0 target");
        for (std::int32_t index = 0; index < LCQI_Q8_0_BLOCK_VALUES; ++index) {
```

```
            target[index] = static_cast<float>(static_cast<std::int8_t>(qs[index])) * d;
        }
    }
}

void dequantize_q5_0_to(std::span<const std::uint8_t> bytes, std::span<float> output) {
    validate_quantized_bytes(bytes,
                             static_cast<std::int64_t>(output.size()),
                             LCQI_Q5_0_BLOCK_VALUES,
                             LCQI_Q5_0_BLOCK_BYTES,
                             "Q5_0");
    const std::int64_t block_count =
        static_cast<std::int64_t>(output.size()) / LCQI_Q5_0_BLOCK_VALUES;
    for (std::int64_t block_index = 0; block_index < block_count; ++block_index) {
        const std::uint8_t* block =
            bytes.data() + checked_size(block_index * LCQI_Q5_0_BLOCK_BYTES, "Q5_0 offset");
        const float d = ggml_f16_to_f32(read_le_u16(block));
        const std::uint32_t qh = read_le_u32(block + sizeof(std::uint16_t));
        const std::uint8_t* qs = block + sizeof(std::uint16_t) + sizeof(std::uint32_t);
        float* target =
            output.data() + checked_size(block_index * LCQI_Q5_0_BLOCK_VALUES, "Q5_0 target");
        for (std::int32_t index = 0; index < LCQI_Q5_0_BLOCK_VALUES / 2; ++index) {
            const std::uint8_t high_0 =
                static_cast<std::uint8_t>(((qh >> index) & 1U)
                                          << LCQI_GGML_Q5_HIGH_BIT_SHIFT);
            const std::uint8_t high_1 =
                static_cast<std::uint8_t>(((qh >> (index + LCQI_GGML_Q5_SECOND_HALF_BIT_OFFSET)) &
                                           1U)
                                          << LCQI_GGML_Q5_HIGH_BIT_SHIFT);
            const std::int32_t low =
                static_cast<std::int32_t>((qs[index] & LCQI_GGML_LOW_NIBBLE_MASK) | high_0) -
                LCQI_GGML_Q5_ZERO_POINT;
            const std::int32_t high =
                static_cast<std::int32_t>((qs[index] >> 4U) | high_1) -
                LCQI_GGML_Q5_ZERO_POINT;
            target[index] = static_cast<float>(low) * d;
            target[index + LCQI_Q5_0_BLOCK_VALUES / 2] = static_cast<float>(high) * d;
        }
    }
}

void dequantize_q6_k_to(std::span<const std::uint8_t> bytes, std::span<float>
```

```
    output) {
    validate_quantized_bytes(bytes,
                             static_cast<std::int64_t>(output.size()),
                             LCQI_Q6_K_BLOCK_VALUES,
                             LCQI_Q6_K_BLOCK_BYTES,
                             "Q6_K");
    const std::int64_t block_count =
        static_cast<std::int64_t>(output.size()) / LCQI_Q6_K_BLOCK_VALUES;
    for (std::int64_t block_index = 0; block_index < block_count; ++block_index) {
        const std::uint8_t* block =
            bytes.data() + checked_size(block_index * LCQI_Q6_K_BLOCK_BYTES, "Q6_K offset");
        const std::uint8_t* ql = block;
        const std::uint8_t* qh = ql + LCQI_Q6_K_BLOCK_VALUES / 2;
        const auto* scales =
            reinterpret_cast<const std::int8_t*>(qh + LCQI_Q6_K_BLOCK_VALUES / 4);
        const float d = ggml_f16_to_f32(
            read_le_u16(block + LCQI_Q6_K_BLOCK_BYTES - sizeof(std::uint16_t)));
        float* y =
            output.data() + checked_size(block_index * LCQI_Q6_K_BLOCK_VALUES, "Q6_K target");

        for (std::int32_t half = 0; half < LCQI_Q6_K_BLOCK_VALUES;
             half += LCQI_GGML_Q6_HALF_BLOCK_VALUES) {
            for (std::int32_t lane = 0; lane < LCQI_GGML_Q6_SUBBLOCK_VALUES; ++lane) {
                const std::int32_t scale_index = lane / (LCQI_GGML_Q6_SUBBLOCK_VALUES / 2);
                const std::int32_t q1 =
                    static_cast<std::int32_t>((ql[lane] & LCQI_GGML_LOW_NIBBLE_MASK) |
                                              (((qh[lane] >> 0U) &
                                                LCQI_GGML_Q6_HIGH_TWO_BITS_MASK)
                                               << 4U)) -
                    LCQI_GGML_Q6_ZERO_POINT;
                const std::int32_t q2 =
                    static_cast<std::int32_t>((ql[lane + LCQI_GGML_Q6_SUBBLOCK_VALUES] &
                                               LCQI_GGML_LOW_NIBBLE_MASK) |
                                              (((qh[lane] >> 2U) &
                                                LCQI_GGML_Q6_HIGH_TWO_BITS_MASK)
                                               << 4U)) -
                    LCQI_GGML_Q6_ZERO_POINT;
                const std::int32_t q3 =
                    static_cast<std::int32_t>((ql[lane] >> 4U) |
                                              (((qh[lane] >> 4U) &
                                                LCQI_GGML_Q6_HIGH_TWO_BITS_MASK
```

```cpp
)
                                               << 4U)) -
                    LCQI_GGML_Q6_ZERO_POINT;
                const std::int32_t q4 =
                    static_cast<std::int32_t>((ql[lane + LCQI_GGML_Q6_SUBBLOCK_VALUES] >> 4U) |
                                              (((qh[lane] >> 6U) &
                                                LCQI_GGML_Q6_HIGH_TWO_BITS_MASK)
                                               << 4U)) -
                    LCQI_GGML_Q6_ZERO_POINT;

                y[lane] = d * static_cast<float>(scales[scale_index + 0]) *
                          static_cast<float>(q1);
                y[lane + LCQI_GGML_Q6_SUBBLOCK_VALUES] =
                    d * static_cast<float>(scales[scale_index + 2]) *
                    static_cast<float>(q2);
                y[lane + LCQI_GGML_Q6_QUARTER_BLOCK_VALUES] =
                    d * static_cast<float>(scales[scale_index + 4]) *
                    static_cast<float>(q3);
                y[lane + LCQI_GGML_Q6_QUARTER_BLOCK_VALUES +
                  LCQI_GGML_Q6_SUBBLOCK_VALUES] =
                    d * static_cast<float>(scales[scale_index + 6]) *
                    static_cast<float>(q4);
            }
            y += LCQI_GGML_Q6_HALF_BLOCK_VALUES;
            ql += LCQI_GGML_Q6_QUARTER_BLOCK_VALUES;
            qh += LCQI_GGML_Q6_SUBBLOCK_VALUES;
            scales += LCQI_GGML_Q6_SCALE_STRIDE;
        }
    }
}

}  // namespace

float ggml_f16_to_f32(std::uint16_t bits) {
    const std::uint32_t sign =
        (static_cast<std::uint32_t>(bits) & LCQI_GGML_F16_SIGN_MASK)
        << LCQI_GGML_F16_TO_F32_SIGN_SHIFT;
    const std::uint32_t exponent =
        static_cast<std::uint32_t>(bits) & LCQI_GGML_F16_EXPONENT_MASK;
    const std::uint32_t mantissa =
        static_cast<std::uint32_t>(bits) & LCQI_GGML_F16_MANTISSA_MASK;

    if (exponent == 0U) {
        if (mantissa == 0U) {
            return bits_to_float(sign);
        }
        float value = static_cast<float>(mantissa) /
                      static_cast<float>(1U << LCQI_GGML_F16_MANTISSA_BITS);
        value = std::ldexp(value, 1 - static_cast<int>(LCQI_GGML_F16_EXPONENT_BIAS));
```

```
        return sign == 0U ? value : -value;
    }

    if (exponent == LCQI_GGML_F16_EXPONENT_MASK) {
        const std::uint32_t f32_mantissa =
            mantissa << (LCQI_GGML_F32_MANTISSA_BITS - LCQI_GGML_F16_MANTISSA_BITS);
        return bits_to_float(sign | LCQI_GGML_F32_EXPONENT_MASK | f32_mantissa);
    }

    const std::uint32_t f32_exponent =
        ((exponent >> LCQI_GGML_F16_MANTISSA_BITS) - LCQI_GGML_F16_EXPONENT_BIAS +
         LCQI_GGML_F32_EXPONENT_BIAS)
        << LCQI_GGML_F32_MANTISSA_BITS;
    const std::uint32_t f32_mantissa =
        mantissa << (LCQI_GGML_F32_MANTISSA_BITS - LCQI_GGML_F16_MANTISSA_BITS);
    return bits_to_float(sign | f32_exponent | f32_mantissa);
}

std::vector<float> dequantize_ggml_tensor(GgmlType type,
                                          std::span<const std::uint8_t> bytes,
                                          std::int64_t element_count) {
    if (element_count < 0) {
        throw std::runtime_error("GGML element_count cannot be negative");
    }
    std::vector<float> output(checked_size(element_count, "GGML element_count"), 0.0F);
    dequantize_ggml_tensor_to(type, bytes, output);
    return output;
}

void dequantize_ggml_tensor_to(GgmlType type,
                               std::span<const std::uint8_t> bytes,
                               std::span<float> output) {
    switch (type) {
        case GgmlType::f32:
            dequantize_f32_to(bytes, output);
            return;
        case GgmlType::q8_0:
            dequantize_q8_0_to(bytes, output);
            return;
        case GgmlType::q5_0:
            dequantize_q5_0_to(bytes, output);
            return;
        case GgmlType::q6_k:
            dequantize_q6_k_to(bytes, output);
            return;
        case GgmlType::q4_k:
            dequantize_q4_k_to(bytes, output);
```

```
            return;
        default:
            throw std::runtime_error(std::string("LCQI cannot dequantize GGML type ") +
                                     ggml_type_name(type));
    }
}

} // namespace lcqi
```

`ggml_matvec.cpp` 是 `Q5_0/Q6_K/Q8_0` direct matvec 的实验实现。它有两条 oracle：一条是 `matvec_ggml_quantized_f32`，直接从 GGUF block bytes 和 f32 input 做点积，用来对照“先解成 f32 权重再算”；另一条是 `matvec_ggml_quantized_q8_0`，先把 input 临时量化成 `Q8_0InputBlock`，再做低比特权重点积。`Q5_0` 需要用 input block 的 `sum` 减掉 zero-point 贡献，`Q6_K` 需要按 256 元素 block 的 scale lane 解包。测试会同时比较 exact path 和 Q8_0 近似 path，避免把 block 布局写错后直接进入真实模型。

Listing 16.9 src/ggml_matvec.cpp

```
#include <lcqi/ggml_matvec.hpp>

#include <lcqi/ggml_tensors.hpp>

#include <algorithm>
#include <cmath>
#include <limits>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr std::uint32_t LCQI_GGML_MATVEC_BYTE_BITS = 8U;
constexpr std::uint8_t LCQI_GGML_MATVEC_LOW_NIBBLE_MASK = 0x0FU;
constexpr std::uint8_t LCQI_GGML_MATVEC_Q6_HIGH_TWO_BITS_MASK = 0x03U;
constexpr std::int32_t LCQI_GGML_MATVEC_Q5_ZERO_POINT = 16;
constexpr std::int32_t LCQI_GGML_MATVEC_Q6_ZERO_POINT = 32;
constexpr std::int32_t LCQI_GGML_MATVEC_Q6_HALF_BLOCK_VALUES = 128;
constexpr std::int32_t LCQI_GGML_MATVEC_Q6_QUARTER_BLOCK_VALUES = 64;
constexpr std::int32_t LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES = 32;
constexpr std::int32_t LCQI_GGML_MATVEC_Q6_SCALE_STRIDE = 8;
constexpr std::int32_t LCQI_GGML_MATVEC_Q8_QUANT_MAX = 127;

[[nodiscard]] std::size_t checked_size(std::int64_t value, const char* name) {
    if (value < 0 ||
        static_cast<std::uint64_t>(value) >
            static_cast<std::uint64_t>(std::numeric_limits<std::size_t>::max())) {
        throw std::runtime_error(std::string(name) + " is outside size_t range");
    }
    return static_cast<std::size_t>(value);
```

```cpp
}

void require_positive(std::int64_t value, const char* name) {
    if (value <= 0) {
        throw std::runtime_error(std::string(name) + " must be positive");
    }
}

void require_multiple(std::int64_t value, std::int64_t divisor, const char* name) {
    if (value <= 0 || value % divisor != 0) {
        throw std::runtime_error(std::string(name) + " must be a positive multiple of " +
                                 std::to_string(divisor));
    }
}

[[nodiscard]] std::uint16_t read_le_u16(const std::uint8_t* bytes) {
    return static_cast<std::uint16_t>(
        static_cast<std::uint16_t>(bytes[0]) |
        (static_cast<std::uint16_t>(bytes[1]) << LCQI_GGML_MATVEC_BYTE_BITS));
}

[[nodiscard]] std::uint32_t read_le_u32(const std::uint8_t* bytes) {
    std::uint32_t value = 0;
    for (std::uint32_t index = 0; index < sizeof(std::uint32_t); ++index) {
        value |= static_cast<std::uint32_t>(bytes[index]) <<
                 (LCQI_GGML_MATVEC_BYTE_BITS * index);
    }
    return value;
}

[[nodiscard]] float dot_q8_0_block_f32(const std::uint8_t* block, const float* input) {
    const float d = ggml_f16_to_f32(read_le_u16(block));
    const auto* qs = reinterpret_cast<const std::int8_t*>(block + sizeof(std::uint16_t));
    float sum = 0.0F;
    for (std::int32_t index = 0; index < LCQI_Q8_0_BLOCK_VALUES; ++index) {
        sum += static_cast<float>(qs[index]) * input[index];
    }
    return d * sum;
}

[[nodiscard]] float dot_q8_0_block_q8_0(const std::uint8_t* block,
                                        const Q8_0InputBlock& input) {
    const float d = ggml_f16_to_f32(read_le_u16(block));
    const auto* qs = reinterpret_cast<const std::int8_t*>(block + sizeof(std::uint16_t));
    std::int32_t sum = 0;
    for (std::int32_t index = 0; index < LCQI_Q8_0_BLOCK_VALUES; ++index) {
        sum += static_cast<std::int32_t>(qs[index]) *
```

```cpp
                static_cast<std::int32_t>(input.qs[index]);
    }
    return d * input.d * static_cast<float>(sum);
}

[[nodiscard]] float dot_q5_0_block_f32(const std::uint8_t* block, const float* input) {
    const float d = ggml_f16_to_f32(read_le_u16(block));
    const std::uint32_t qh = read_le_u32(block + sizeof(std::uint16_t));
    const std::uint8_t* qs = block + sizeof(std::uint16_t) + sizeof(std::uint32_t);
    float sum = 0.0F;
    for (std::int32_t index = 0; index < LCQI_Q5_0_BLOCK_VALUES / 2; ++index) {
        const std::uint8_t high_0 =
            static_cast<std::uint8_t>(((qh >> index) & 1U) << 4U);
        const std::uint8_t high_1 =
            static_cast<std::uint8_t>(((qh >> (index + LCQI_Q5_0_BLOCK_VALUES / 2)) & 1U)
                                      << 4U);
        const std::int32_t first =
            static_cast<std::int32_t>((qs[index] & LCQI_GGML_MATVEC_LOW_NIBBLE_MASK) |
                                      high_0) -
            LCQI_GGML_MATVEC_Q5_ZERO_POINT;
        const std::int32_t second =
            static_cast<std::int32_t>((qs[index] >> 4U) | high_1) -
            LCQI_GGML_MATVEC_Q5_ZERO_POINT;
        sum += static_cast<float>(first) * input[index];
        sum += static_cast<float>(second) * input[index + LCQI_Q5_0_BLOCK_VALUES / 2];
    }
    return d * sum;
}

[[nodiscard]] float dot_q5_0_block_q8_0(const std::uint8_t* block,
                                        const Q8_0InputBlock& input) {
    const float d = ggml_f16_to_f32(read_le_u16(block));
    const std::uint32_t qh = read_le_u32(block + sizeof(std::uint16_t));
    const std::uint8_t* qs = block + sizeof(std::uint16_t) + sizeof(std::uint32_t);
    std::int32_t weighted_sum = 0;
    for (std::int32_t index = 0; index < LCQI_Q5_0_BLOCK_VALUES / 2; ++index) {
        const std::uint8_t high_0 =
            static_cast<std::uint8_t>(((qh >> index) & 1U) << 4U);
        const std::uint8_t high_1 =
            static_cast<std::uint8_t>(((qh >> (index + LCQI_Q5_0_BLOCK_VALUES / 2)) & 1U)
                                      << 4U);
        const std::int32_t first =
            static_cast<std::int32_t>((qs[index] & LCQI_GGML_MATVEC_LOW_NIBBLE_MASK) |
                                      high_0);
```

```cpp
        const std::int32_t second =
            static_cast<std::int32_t>((qs[index] >> 4U) | high_1);
        weighted_sum += first * static_cast<std::int32_t>(input.qs[index]);
        weighted_sum += second *
                        static_cast<std::int32_t>(input.qs[index + LCQI_Q5_0_BLOCK_VALUES / 2]);
    }
    weighted_sum -= LCQI_GGML_MATVEC_Q5_ZERO_POINT * static_cast<std::int32_t>(input.sum);
    return d * input.d * static_cast<float>(weighted_sum);
}

[[nodiscard]] float dot_q6_k_block_f32(const std::uint8_t* block, const float* input) {
    const std::uint8_t* ql = block;
    const std::uint8_t* qh = ql + LCQI_Q6_K_BLOCK_VALUES / 2;
    const auto* scales =
        reinterpret_cast<const std::int8_t*>(qh + LCQI_Q6_K_BLOCK_VALUES / 4);
    const float d = ggml_f16_to_f32(
        read_le_u16(block + LCQI_Q6_K_BLOCK_BYTES - sizeof(std::uint16_t)));
    float sum = 0.0F;

    for (std::int32_t half = 0; half < LCQI_Q6_K_BLOCK_VALUES;
         half += LCQI_GGML_MATVEC_Q6_HALF_BLOCK_VALUES) {
        const float* x = input + half;
        for (std::int32_t lane = 0; lane < LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES; ++lane) {
            const std::int32_t scale_index =
                lane / (LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES / 2);
            const std::int32_t q1 =
                static_cast<std::int32_t>((ql[lane] & LCQI_GGML_MATVEC_LOW_NIBBLE_MASK) |
                                          (((qh[lane] >> 0U) &
                                            LCQI_GGML_MATVEC_Q6_HIGH_TWO_BITS_MASK)
                                           << 4U)) -
                LCQI_GGML_MATVEC_Q6_ZERO_POINT;
            const std::int32_t q2 =
                static_cast<std::int32_t>((ql[lane + LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES] &
                                           LCQI_GGML_MATVEC_LOW_NIBBLE_MASK) |
                                          (((qh[lane] >> 2U) &
                                            LCQI_GGML_MATVEC_Q6_HIGH_TWO_BITS_MASK)
                                           << 4U)) -
                LCQI_GGML_MATVEC_Q6_ZERO_POINT;
            const std::int32_t q3 =
                static_cast<std::int32_t>((ql[lane] >> 4U) |
                                          (((qh[lane] >> 4U) &
                                            LCQI_GGML_MATVEC_Q6_HIGH_TWO_BITS_MASK)
                                           << 4U)) -
```

```
                LCQI_GGML_MATVEC_Q6_ZERO_POINT;
            const std::int32_t q4 =
                static_cast<std::int32_t>((ql[lane + LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES] >>
                                          4U) |
                                         (((qh[lane] >> 6U) &
                                           LCQI_GGML_MATVEC_Q6_HIGH_TWO_BITS_MASK)
                                          << 4U)) -
                LCQI_GGML_MATVEC_Q6_ZERO_POINT;

            sum += static_cast<float>(scales[scale_index + 0]) *
                   static_cast<float>(q1) * x[lane];
            sum += static_cast<float>(scales[scale_index + 2]) *
                   static_cast<float>(q2) * x[lane + LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES];
            sum += static_cast<float>(scales[scale_index + 4]) *
                   static_cast<float>(q3) * x[lane + LCQI_GGML_MATVEC_Q6_QUARTER_BLOCK_VALUES];
            sum += static_cast<float>(scales[scale_index + 6]) *
                   static_cast<float>(q4) *
                   x[lane + LCQI_GGML_MATVEC_Q6_QUARTER_BLOCK_VALUES +
                     LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES];
        }
        ql += LCQI_GGML_MATVEC_Q6_QUARTER_BLOCK_VALUES;
        qh += LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES;
        scales += LCQI_GGML_MATVEC_Q6_SCALE_STRIDE;
    }
    return d * sum;
}

[[nodiscard]] float dot_q6_k_block_q8_0(const std::uint8_t* block,
                                        const Q8_0InputBlock* input) {
    const std::uint8_t* ql = block;
    const std::uint8_t* qh = ql + LCQI_Q6_K_BLOCK_VALUES / 2;
    const auto* scales =
        reinterpret_cast<const std::int8_t*>(qh + LCQI_Q6_K_BLOCK_VALUES / 4);
    const float d = ggml_f16_to_f32(
        read_le_u16(block + LCQI_Q6_K_BLOCK_BYTES - sizeof(std::uint16_t)));
    float sum = 0.0F;

    for (std::int32_t half = 0; half < LCQI_Q6_K_BLOCK_VALUES;
         half += LCQI_GGML_MATVEC_Q6_HALF_BLOCK_VALUES) {
        const std::int32_t input_half_block = half / LCQI_Q8_0_INPUT_BLOCK_VALUES;
        for (std::int32_t lane = 0; lane < LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES; ++lane) {
            const std::int32_t scale_index =
                lane / (LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES / 2);
            const std::int32_t q1 =
                static_cast<std::int32_t>((ql[lane] & LCQI_GGML_MATVEC_LOW_NIBBLE_MASK) |
```

```
                                               (((qh[lane] >> 0U) &
                                                 ↩
    ↪ LCQI_GGML_MATVEC_Q6_HIGH_TWO_BITS_MASK)
                                                << 4U)) -
                    LCQI_GGML_MATVEC_Q6_ZERO_POINT;
                const std::int32_t q2 =
                    static_cast<std::int32_t>((ql[lane + ↩
    ↪ LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES] &
                                                LCQI_GGML_MATVEC_LOW_NIBBLE_MASK) |
                                               (((qh[lane] >> 2U) &
                                                 ↩
    ↪ LCQI_GGML_MATVEC_Q6_HIGH_TWO_BITS_MASK)
                                                << 4U)) -
                    LCQI_GGML_MATVEC_Q6_ZERO_POINT;
                const std::int32_t q3 =
                    static_cast<std::int32_t>((ql[lane] >> 4U) |
                                               (((qh[lane] >> 4U) &
                                                 ↩
    ↪ LCQI_GGML_MATVEC_Q6_HIGH_TWO_BITS_MASK)
                                                << 4U)) -
                    LCQI_GGML_MATVEC_Q6_ZERO_POINT;
                const std::int32_t q4 =
                    static_cast<std::int32_t>((ql[lane + ↩
    ↪ LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES] >>
                                                4U) |
                                               (((qh[lane] >> 6U) &
                                                 ↩
    ↪ LCQI_GGML_MATVEC_Q6_HIGH_TWO_BITS_MASK)
                                                << 4U)) -
                    LCQI_GGML_MATVEC_Q6_ZERO_POINT;

                const Q8_0InputBlock& x1 = input[input_half_block + 0];
                const Q8_0InputBlock& x2 = input[input_half_block + 1];
                const Q8_0InputBlock& x3 = input[input_half_block + 2];
                const Q8_0InputBlock& x4 = input[input_half_block + 3];
                sum += x1.d * static_cast<float>(scales[scale_index + 0]) *
                       static_cast<float>(q1) * static_cast<float>(x1.qs[lane]);
                sum += x2.d * static_cast<float>(scales[scale_index + 2]) *
                       static_cast<float>(q2) * static_cast<float>(x2.qs[lane]);
                sum += x3.d * static_cast<float>(scales[scale_index + 4]) *
                       static_cast<float>(q3) * static_cast<float>(x3.qs[lane]);
                sum += x4.d * static_cast<float>(scales[scale_index + 6]) *
                       static_cast<float>(q4) * static_cast<float>(x4.qs[lane]);
            }
            ql += LCQI_GGML_MATVEC_Q6_QUARTER_BLOCK_VALUES;
            qh += LCQI_GGML_MATVEC_Q6_SUBBLOCK_VALUES;
            scales += LCQI_GGML_MATVEC_Q6_SCALE_STRIDE;
        }
    return d * sum;
}

[[nodiscard]] float dot_quantized_block_f32(GgmlType type,
```

```
                                            const std::uint8_t* block,
                                            const float* input) {
    switch (type) {
        case GgmlType::q8_0:
            return dot_q8_0_block_f32(block, input);
        case GgmlType::q5_0:
            return dot_q5_0_block_f32(block, input);
        case GgmlType::q6_k:
            return dot_q6_k_block_f32(block, input);
        default:
            throw std::runtime_error(std::string("LCQI has no direct F32 matvec for ") +
                                     ggml_type_name(type));
    }
}

[[nodiscard]] float dot_quantized_block_q8_0(GgmlType type,
                                             const std::uint8_t* block,
                                             const Q8_0InputBlock* input) {
    switch (type) {
        case GgmlType::q8_0:
            return dot_q8_0_block_q8_0(block, input[0]);
        case GgmlType::q5_0:
            return dot_q5_0_block_q8_0(block, input[0]);
        case GgmlType::q6_k:
            return dot_q6_k_block_q8_0(block, input);
        default:
            throw std::runtime_error(std::string("LCQI has no direct Q8_0 matvec for ") +
                                     ggml_type_name(type));
    }
}

void validate_q8_0_blocks(std::span<const Q8_0InputBlock> input, std::int64_t column_count) {
    require_multiple(column_count, LCQI_Q8_0_INPUT_BLOCK_VALUES, "Q8_0 input column count");
    if (input.size() !=
        checked_size(column_count / LCQI_Q8_0_INPUT_BLOCK_VALUES, "Q8_0 input block count")) {
        throw std::runtime_error("Q8_0 input block count mismatch");
    }
}

}  // namespace

bool ggml_type_has_f32_direct_matvec(GgmlType type) noexcept {
    return type == GgmlType::q8_0 || type == GgmlType::q5_0 || type == GgmlType::q6_k;
}

bool ggml_type_has_q8_0_direct_matvec(GgmlType type) noexcept {
```

```
    return type == GgmlType::q8_0 || type == GgmlType::q5_0 || type == GgmlType::q6_k;
}

#if defined(LCQI_ENABLE_AVX2)
bool ggml_q8_0_avx2_available() noexcept;

void matvec_ggml_quantized_q8_0_avx2_unchecked(GgmlType type,
                                               std::span<const std::uint8_t> rows,
                                               std::int64_t row_count,
                                               std::int64_t column_count,
                                               std::span<const Q8_0InputBlock> input,
                                               std::span<float> output);
#else
bool ggml_q8_0_avx2_available() noexcept {
    return false;
}

void matvec_ggml_quantized_q8_0_avx2_unchecked(GgmlType,
                                               std::span<const std::uint8_t>,
                                               std::int64_t,
                                               std::int64_t,
                                               std::span<const Q8_0InputBlock>,
                                               std::span<float>) {}
#endif

std::vector<Q8_0InputBlock> quantize_q8_0_input(std::span<const float> input) {
    require_multiple(static_cast<std::int64_t>(input.size()),
                     LCQI_Q8_0_INPUT_BLOCK_VALUES,
                     "Q8_0 input size");
    std::vector<Q8_0InputBlock> output(
        checked_size(static_cast<std::int64_t>(input.size()) / LCQI_Q8_0_INPUT_BLOCK_VALUES,
                     "Q8_0 input block count"));
    quantize_q8_0_input_to(input, output);
    return output;
}

void quantize_q8_0_input_to(std::span<const float> input,
                            std::span<Q8_0InputBlock> output) {
    require_multiple(static_cast<std::int64_t>(input.size()),
                     LCQI_Q8_0_INPUT_BLOCK_VALUES,
                     "Q8_0 input size");
    if (output.size() != checked_size(static_cast<std::int64_t>(input.size()) /
                                          LCQI_Q8_0_INPUT_BLOCK_VALUES,
                                      "Q8_0 input block count")) {
        throw std::runtime_error("Q8_0 output block count mismatch");
    }
    for (std::size_t block_index = 0; block_index < output.size(); ++block_index) {
```

```cpp
        Q8_0InputBlock& block = output[block_index];
        const float* values =
            input.data() + block_index * checked_size(LCQI_Q8_0_INPUT_BLOCK_VALUES,
                                                      "Q8_0 input block values");
        float max_value = 0.0F;
        float max_abs = 0.0F;
        for (std::int32_t index = 0; index < LCQI_Q8_0_INPUT_BLOCK_VALUES; ++index) {
            const float abs_value = std::fabs(values[index]);
            if (abs_value > max_abs) {
                max_abs = abs_value;
                max_value = values[index];
            }
        }
        if (max_abs == 0.0F) {
            block.d = 0.0F;
            block.qs.fill(0);
            block.sum = 0;
            continue;
        }
        const float inverse_scale =
            -static_cast<float>(LCQI_GGML_MATVEC_Q8_QUANT_MAX) / max_value;
        std::int32_t sum = 0;
        for (std::int32_t index = 0; index < LCQI_Q8_0_INPUT_BLOCK_VALUES; ++index) {
            const int quantized = static_cast<int>(std::nearbyint(inverse_scale * values[index]));
            const int clamped = std::clamp(quantized,
                                           -LCQI_GGML_MATVEC_Q8_QUANT_MAX,
                                           LCQI_GGML_MATVEC_Q8_QUANT_MAX);
            block.qs[index] = static_cast<std::int8_t>(clamped);
            sum += clamped;
        }
        block.sum = static_cast<std::int16_t>(sum);
        block.d = 1.0F / inverse_scale;
    }
}

void matvec_ggml_quantized_f32(GgmlType type,
                               std::span<const std::uint8_t> rows,
                               std::int64_t row_count,
                               std::int64_t column_count,
                               std::span<const float> input,
                               std::span<float> output) {
    require_positive(row_count, "GGML matvec row count");
    const GgmlTypeLayout layout = ggml_type_layout(type);
    if (!ggml_type_has_f32_direct_matvec(type)) {
        throw std::runtime_error(std::string("LCQI has no direct F32 matvec for ") +
                                 ggml_type_name(type));
```

```cpp
    }
    require_multiple(column_count, layout.block_size, "GGML matvec column count");
    if (input.size() != checked_size(column_count, "GGML matvec column count")) {
        throw std::runtime_error("GGML matvec input size mismatch");
    }
    if (output.size() != checked_size(row_count, "GGML matvec row count")) {
        throw std::runtime_error("GGML matvec output size mismatch");
    }
    const std::int64_t row_bytes =
        (column_count / layout.block_size) * static_cast<std::int64_t>(layout.type_size);
    if (rows.size() != checked_size(row_bytes * row_count, "GGML matvec matrix bytes")) {
        throw std::runtime_error("GGML matvec matrix byte size mismatch");
    }

    for (std::int64_t row = 0; row < row_count; ++row) {
        const std::uint8_t* row_bytes_begin =
            rows.data() + checked_size(row * row_bytes, "GGML matvec row offset");
        float sum = 0.0F;
        for (std::int64_t block = 0; block < column_count / layout.block_size; ++block) {
            sum += dot_quantized_block_f32(
                type,
                row_bytes_begin +
                    checked_size(block * static_cast<std::int64_t>(layout.type_size),
                                 "GGML matvec block offset"),
                input.data() + checked_size(block * layout.block_size,
                                            "GGML matvec input block offset"));
        }
        output[checked_size(row, "GGML matvec row")] = sum;
    }
}

void matvec_ggml_quantized_q8_0(GgmlType type,
                                std::span<const std::uint8_t> rows,
                                std::int64_t row_count,
                                std::int64_t column_count,
                                std::span<const Q8_0InputBlock> input,
                                std::span<float> output) {
    require_positive(row_count, "GGML Q8_0 matvec row count");
    const GgmlTypeLayout layout = ggml_type_layout(type);
    if (!ggml_type_has_q8_0_direct_matvec(type)) {
        throw std::runtime_error(std::string("LCQI has no direct Q8_0 matvec for ") +
                                 ggml_type_name(type));
    }
    require_multiple(column_count, layout.block_size, "GGML Q8_0 matvec column
```

```cpp
count");
    validate_q8_0_blocks(input, column_count);
    if (output.size() != checked_size(row_count, "GGML Q8_0 matvec row count")) {
        throw std::runtime_error("GGML Q8_0 matvec output size mismatch");
    }
    const std::int64_t row_bytes =
        (column_count / layout.block_size) * static_cast<std::int64_t>(layout.type_size);
    if (rows.size() != checked_size(row_bytes * row_count, "GGML Q8_0 matvec matrix bytes")) {
        throw std::runtime_error("GGML Q8_0 matvec matrix byte size mismatch");
    }
#if defined(LCQI_ENABLE_AVX2)
    if ((type == GgmlType::q8_0 || type == GgmlType::q5_0) && ggml_q8_0_avx2_available()) {
        matvec_ggml_quantized_q8_0_avx2_unchecked(type,
                                                  rows,
                                                  row_count,
                                                  column_count,
                                                  input,
                                                  output);
        return;
    }
#endif
    const std::int64_t input_blocks_per_weight_block =
        layout.block_size / LCQI_Q8_0_INPUT_BLOCK_VALUES;

    for (std::int64_t row = 0; row < row_count; ++row) {
        const std::uint8_t* row_bytes_begin =
            rows.data() + checked_size(row * row_bytes, "GGML Q8_0 matvec row offset");
        float sum = 0.0F;
        for (std::int64_t block = 0; block < column_count / layout.block_size; ++block) {
            sum += dot_quantized_block_q8_0(
                type,
                row_bytes_begin +
                    checked_size(block * static_cast<std::int64_t>(layout.type_size),
                                 "GGML Q8_0 matvec block offset"),
                input.data() +
                    checked_size(block * input_blocks_per_weight_block,
                                 "GGML Q8_0 matvec input block offset"));
        }
        output[checked_size(row, "GGML Q8_0 matvec row")] = sum;
    }
}

}  // namespace lcqi
```

ggml_matvec_avx2.cpp 是一次负结果驱动的优化尝试。它给 Q8_0xQ8_0 与 Q5_0xQ8_0

的 32 元素块点积加了 AVX2 `maddubs` 归约；但 caw 端到端报告显示，实验路径仍然比默认 f32 fallback 慢，因为主要成本已经转移到逐 block 解包、逐 row 调度和 `Q6_K` 标量路径。源码卷保留它，是为了说明内核优化必须接受端到端证据约束：覆盖率提高不等于性能提高。

Listing 16.10 src/ggml_matvec_avx2.cpp

```cpp
#include <lcqi/ggml_matvec.hpp>

#if defined(LCQI_ENABLE_AVX2)

#include <lcqi/ggml_tensors.hpp>

#include <immintrin.h>

#include <array>
#include <cstdint>
#include <span>

namespace lcqi {
namespace {

constexpr std::uint32_t LCQI_GGML_AVX2_BYTE_BITS = 8U;
constexpr std::uint8_t LCQI_GGML_AVX2_LOW_NIBBLE_MASK = 0x0FU;
constexpr std::int32_t LCQI_GGML_AVX2_Q5_ZERO_POINT = 16;
constexpr std::int64_t LCQI_GGML_AVX2_Q8_0_BLOCK_BYTES = 34;
constexpr std::int64_t LCQI_GGML_AVX2_Q5_0_BLOCK_BYTES = 22;

[[nodiscard]] std::uint16_t read_le_u16(const std::uint8_t* bytes) noexcept {
    return static_cast<std::uint16_t>(
        static_cast<std::uint16_t>(bytes[0]) |
        (static_cast<std::uint16_t>(bytes[1]) << LCQI_GGML_AVX2_BYTE_BITS));
}

[[nodiscard]] std::uint32_t read_le_u32(const std::uint8_t* bytes) noexcept {
    std::uint32_t value = 0;
    for (std::uint32_t index = 0; index < sizeof(std::uint32_t); ++index) {
        value |= static_cast<std::uint32_t>(bytes[index]) <<
                 (LCQI_GGML_AVX2_BYTE_BITS * index);
    }
    return value;
}

[[nodiscard]] std::int32_t horizontal_sum_i32(__m256i values) noexcept {
    const __m128i low = _mm256_castsi256_si128(values);
    const __m128i high = _mm256_extracti128_si256(values, 1);
    __m128i sum = _mm_add_epi32(low, high);
    sum = _mm_add_epi32(sum, _mm_shuffle_epi32(sum, _MM_SHUFFLE(1, 0, 3, 2)));
    sum = _mm_add_epi32(sum, _mm_shuffle_epi32(sum, _MM_SHUFFLE(2, 3, 0, 1)));
    return _mm_cvtsi128_si32(sum);
}

[[nodiscard]] std::int32_t dot_32_i8_i8_avx2(const std::int8_t* lhs,
                                             const std::int8_t* rhs) noexcept {
```

```
    const __m256i lhs_values = _mm256_loadu_si256(reinterpret_cast<const __m256i*>(lhs));
    const __m256i rhs_values = _mm256_loadu_si256(reinterpret_cast<const __m256i*>(rhs));
    const __m256i absolute_lhs = _mm256_sign_epi8(lhs_values, lhs_values);
    const __m256i signed_rhs = _mm256_sign_epi8(rhs_values, lhs_values);
    const __m256i products_i16 = _mm256_maddubs_epi16(absolute_lhs, signed_rhs);
    const __m256i ones = _mm256_set1_epi16(1);
    const __m256i partial_i32 = _mm256_madd_epi16(products_i16, ones);
    return horizontal_sum_i32(partial_i32);
}

[[nodiscard]] std::int32_t dot_32_u8_i8_avx2(const std::uint8_t* lhs,
                                             const std::int8_t* rhs) noexcept {
    const __m256i lhs_values = _mm256_loadu_si256(reinterpret_cast<const __m256i*>(lhs));
    const __m256i rhs_values = _mm256_loadu_si256(reinterpret_cast<const __m256i*>(rhs));
    const __m256i products_i16 = _mm256_maddubs_epi16(lhs_values, rhs_values);
    const __m256i ones = _mm256_set1_epi16(1);
    const __m256i partial_i32 = _mm256_madd_epi16(products_i16, ones);
    return horizontal_sum_i32(partial_i32);
}

[[nodiscard]] float dot_q8_0_block_q8_0_avx2(const std::uint8_t* block,
                                             const Q8_0InputBlock& input) noexcept {
    const float d = ggml_f16_to_f32(read_le_u16(block));
    const auto* qs = reinterpret_cast<const std::int8_t*>(block + sizeof(std::uint16_t));
    return d * input.d * static_cast<float>(dot_32_i8_i8_avx2(qs, input.qs.data()));
}

[[nodiscard]] float dot_q5_0_block_q8_0_avx2(const std::uint8_t* block,
                                             const Q8_0InputBlock& input) noexcept {
    const float d = ggml_f16_to_f32(read_le_u16(block));
    const std::uint32_t qh = read_le_u32(block + sizeof(std::uint16_t));
    const std::uint8_t* qs = block + sizeof(std::uint16_t) + sizeof(std::uint32_t);

    alignas(32) std::array<std::uint8_t, LCQI_Q8_0_INPUT_BLOCK_VALUES> values{};
    for (std::int32_t index = 0; index < LCQI_Q8_0_INPUT_BLOCK_VALUES / 2; ++index) {
        const std::uint8_t high_0 =
            static_cast<std::uint8_t>(((qh >> index) & 1U) << 4U);
        const std::uint8_t high_1 = static_cast<std::uint8_t>(
            ((qh >> (index + LCQI_Q8_0_INPUT_BLOCK_VALUES / 2)) & 1U) << 4U);
        values[static_cast<std::size_t>(index)] =
```

```
            static_cast<std::uint8_t>((qs[index] & LCQI_GGML_AVX2_LOW_NIBBLE_MASK) | high_0);
        values[static_cast<std::size_t>(index + LCQI_Q8_0_INPUT_BLOCK_VALUES / 2)] =
            static_cast<std::uint8_t>((qs[index] >> 4U) | high_1);
    }

    const std::int32_t weighted_sum =
        dot_32_u8_i8_avx2(values.data(), input.qs.data()) -
        LCQI_GGML_AVX2_Q5_ZERO_POINT * static_cast<std::int32_t>(input.sum);
    return d * input.d * static_cast<float>(weighted_sum);
}

void matvec_q8_0_q8_0_avx2_unchecked(std::span<const std::uint8_t> rows,
                                     std::int64_t row_count,
                                     std::int64_t column_count,
                                     std::span<const Q8_0InputBlock> input,
                                     std::span<float> output) {
    const std::int64_t row_blocks = column_count / LCQI_Q8_0_INPUT_BLOCK_VALUES;
    const std::int64_t row_bytes = row_blocks * LCQI_GGML_AVX2_Q8_0_BLOCK_BYTES;
    for (std::int64_t row = 0; row < row_count; ++row) {
        float sum = 0.0F;
        const std::uint8_t* row_data = rows.data() + row * row_bytes;
        for (std::int64_t block = 0; block < row_blocks; ++block) {
            sum += dot_q8_0_block_q8_0_avx2(
                row_data + block * LCQI_GGML_AVX2_Q8_0_BLOCK_BYTES,
                input[static_cast<std::size_t>(block)]);
        }
        output[static_cast<std::size_t>(row)] = sum;
    }
}

void matvec_q5_0_q8_0_avx2_unchecked(std::span<const std::uint8_t> rows,
                                     std::int64_t row_count,
                                     std::int64_t column_count,
                                     std::span<const Q8_0InputBlock> input,
                                     std::span<float> output) {
    const std::int64_t row_blocks = column_count / LCQI_Q8_0_INPUT_BLOCK_VALUES;
    const std::int64_t row_bytes = row_blocks * LCQI_GGML_AVX2_Q5_0_BLOCK_BYTES;
    for (std::int64_t row = 0; row < row_count; ++row) {
        float sum = 0.0F;
        const std::uint8_t* row_data = rows.data() + row * row_bytes;
        for (std::int64_t block = 0; block < row_blocks; ++block) {
            sum += dot_q5_0_block_q8_0_avx2(
                row_data + block * LCQI_GGML_AVX2_Q5_0_BLOCK_BYTES,
                input[static_cast<std::size_t>(block)]);
        }
        output[static_cast<std::size_t>(row)] = sum;
```

```cpp
    }
}

}  // namespace

bool ggml_q8_0_avx2_available() noexcept {
#if defined(__GNUC__) || defined(__clang__)
    __builtin_cpu_init();
    return __builtin_cpu_supports("avx2");
#else
    return true;
#endif
}

void matvec_ggml_quantized_q8_0_avx2_unchecked(GgmlType type,
                                                std::span<const std::uint8_t> rows,
                                                std::int64_t row_count,
                                                std::int64_t column_count,
                                                std::span<const Q8_0InputBlock> input,
                                                std::span<float> output) {
    if (type == GgmlType::q8_0) {
        matvec_q8_0_q8_0_avx2_unchecked(rows, row_count, column_count, input, output);
        return;
    }
    if (type == GgmlType::q5_0) {
        matvec_q5_0_q8_0_avx2_unchecked(rows, row_count, column_count, input, output);
    }
}

}  // namespace lcqi

#endif
```

`llama_gguf.cpp` 是自研 SmolLM2/GGUF decode path。它读取 GGUF metadata 得到 hidden size、层数、GQA 头数、RoPE base、RMSNorm epsilon 和 tokenizer id；再读取 `token_embd.weight`、每层 attention/FFN 权重和 `output_norm.weight`。最新实现把权重表示分成三类：默认启用的 shape 兼容 `Q4_K` linear tensor 保留 GGUF 原始 block bytes，前向时通过 `Q8_K` scratch 调用 `matvec_q4_k_q8`；`LCQI_LLAMA_GGML_DIRECT=1` 打开的 `Q5_0/Q6_K/Q8_0` 实验 tensor 通过 `Q8_0` scratch 调用 GGML direct matvec；其余 tensor materialize 成 f32 fallback。读这份源码时要看三条线：loader 如何记录 `q4_k_direct_tensors`、`ggml_direct_tensors` 与 fallback 字节，`linear_gguf` 如何在三类路径之间分派，`LlamaGgufHotspotReport` 如何把 direct/fallback call 和 per-op hotspot 输出到报告。它的意义不是性能已经追上 llama.cpp，而是 LCQI 有了同一个真实 GGUF 文件上的自研端到端 correctness baseline，并且知道哪些 direct path 可以进入默认，哪些必须保留为实验。

Listing 16.11 src/llama_gguf.cpp

```cpp
#include <lcqi/llama_gguf.hpp>
```

```cpp

#include <lcqi/f32_kernels.hpp>
#include <lcqi/ggml_matvec.hpp>
#include <lcqi/ggml_tensors.hpp>
#include <lcqi/gguf.hpp>
#include <lcqi/q4_k.hpp>

#include <algorithm>
#include <array>
#include <chrono>
#include <cmath>
#include <cstdlib>
#include <limits>
#include <sstream>
#include <stdexcept>
#include <string>
#include <unordered_map>
#include <utility>

namespace lcqi {
namespace {

constexpr std::int32_t LCQI_LLAMA_ROPE_PAIR_WIDTH = 2;
constexpr std::int32_t LCQI_LLAMA_DEFAULT_BOS_TOKEN = 1;
constexpr std::int32_t LCQI_LLAMA_DEFAULT_EOS_TOKEN = 2;
constexpr std::int32_t LCQI_LLAMA_DEFAULT_UNK_TOKEN = 0;
constexpr const char* LCQI_LLAMA_ARCHITECTURE_KEY = "general.architecture";
constexpr const char* LCQI_LLAMA_NAME_KEY = "general.name";
constexpr const char* LCQI_LLAMA_EMBEDDING_LENGTH_KEY = "llama.embedding_length";
constexpr const char* LCQI_LLAMA_BLOCK_COUNT_KEY = "llama.block_count";
constexpr const char* LCQI_LLAMA_FEED_FORWARD_LENGTH_KEY = "llama.feed_forward_length";
constexpr const char* LCQI_LLAMA_HEAD_COUNT_KEY = "llama.attention.head_count";
constexpr const char* LCQI_LLAMA_KV_HEAD_COUNT_KEY = "llama.attention.head_count_kv";
constexpr const char* LCQI_LLAMA_ROPE_DIMENSION_KEY = "llama.rope.dimension_count";
constexpr const char* LCQI_LLAMA_ROPE_BASE_KEY = "llama.rope.freq_base";
constexpr const char* LCQI_LLAMA_CONTEXT_LENGTH_KEY = "llama.context_length";
constexpr const char* LCQI_LLAMA_VOCAB_SIZE_KEY = "llama.vocab_size";
constexpr const char* LCQI_LLAMA_RMS_EPSILON_KEY = "llama.attention.layer_norm_rms_epsilon";
constexpr const char* LCQI_LLAMA_TOKENIZER_TOKENS_KEY = "tokenizer.ggml.tokens";
constexpr const char* LCQI_LLAMA_TOKENIZER_MERGES_KEY = "tokenizer.ggml.merges";
constexpr const char* LCQI_LLAMA_BOS_TOKEN_KEY = "tokenizer.ggml.bos_token_id";
constexpr const char* LCQI_LLAMA_EOS_TOKEN_KEY = "tokenizer.ggml.eos_token_id";
constexpr const char* LCQI_LLAMA_UNKNOWN_TOKEN_KEY = "tokenizer.ggml.unknown_token_id";
constexpr const char* LCQI_LLAMA_Q4_DIRECT_ENV = "LCQI_LLAMA_Q4_DIRECT";
```

```cpp
constexpr const char* LCQI_LLAMA_GGML_DIRECT_ENV = "LCQI_LLAMA_GGML_DIRECT";
constexpr const char* LCQI_LLAMA_FALSE_ENV = "0";
constexpr const char* LCQI_LLAMA_TRUE_ENV = "1";

using Clock = std::chrono::steady_clock;

enum class LlamaGgufLinearOp : std::uint8_t {
    wq,
    wk,
    wv,
    wo,
    w_gate,
    w_up,
    w_down,
    lm_head,
};

[[nodiscard]] double elapsed_ms(Clock::time_point begin, Clock::time_point end) {
    return std::chrono::duration<double, std::milli>(end - begin).count();
}

[[nodiscard]] std::size_t checked_size(std::int64_t value, const char* name) {
    if (value < 0 ||
        static_cast<std::uint64_t>(value) >
            static_cast<std::uint64_t>(std::numeric_limits<std::size_t>::max())) {
        throw std::runtime_error(std::string(name) + " is outside size_t range");
    }
    return static_cast<std::size_t>(value);
}

void require_positive(std::int32_t value, const char* name) {
    if (value <= 0) {
        throw std::runtime_error(std::string(name) + " must be positive");
    }
}

[[nodiscard]] const GgufMetadataEntry& require_metadata(const GgufManifest& manifest,
                                                        std::string_view key) {
    const GgufMetadataEntry* entry = manifest.find_metadata(key);
    if (entry == nullptr) {
        throw std::runtime_error("missing GGUF metadata key: " + std::string(key));
    }
    return *entry;
}

[[nodiscard]] std::string metadata_string(const GgufManifest& manifest, std::string_view key) {
```

```cpp
    return require_metadata(manifest, key).value_preview;
}

[[nodiscard]] std::int32_t metadata_i32(const GgufManifest& manifest, std::string_view key) {
    const std::int64_t value = std::stoll(require_metadata(manifest, key).value_preview);
    if (value < std::numeric_limits<std::int32_t>::min() ||
        value > std::numeric_limits<std::int32_t>::max()) {
        throw std::runtime_error("GGUF metadata int32 out of range: " + std::string(key));
    }
    return static_cast<std::int32_t>(value);
}

[[nodiscard]] std::int32_t optional_metadata_i32(const GgufManifest& manifest,
                                                 std::string_view key,
                                                 std::int32_t fallback) {
    const GgufMetadataEntry* entry = manifest.find_metadata(key);
    if (entry == nullptr) {
        return fallback;
    }
    const std::int64_t value = std::stoll(entry->value_preview);
    if (value < std::numeric_limits<std::int32_t>::min() ||
        value > std::numeric_limits<std::int32_t>::max()) {
        throw std::runtime_error("GGUF metadata int32 out of range: " + std::string(key));
    }
    return static_cast<std::int32_t>(value);
}

[[nodiscard]] float metadata_f32(const GgufManifest& manifest, std::string_view key) {
    return std::stof(require_metadata(manifest, key).value_preview);
}

void validate_shape(const GgufTensorInfo& tensor,
                    std::span<const std::int64_t> expected_shape) {
    if (tensor.shape.size() != expected_shape.size()) {
        throw std::runtime_error("GGUF tensor rank mismatch: " + tensor.name);
    }
    for (std::size_t index = 0; index < expected_shape.size(); ++index) {
        if (tensor.shape[index] != expected_shape[index]) {
            throw std::runtime_error("GGUF tensor shape mismatch: " + tensor.name);
        }
    }
}

[[nodiscard]] bool can_use_q4_k_direct(const GgufTensorInfo& tensor) {
    return tensor.type == GgmlType::q4_k && tensor.shape.size() == 2 &&
           tensor.shape[0] > 0 && tensor.shape[1] > 0 &&
```

```cpp
           tensor.shape[0] % LCQI_QK_K_BLOCK_VALUES == 0;
}

[[nodiscard]] bool can_use_ggml_direct(const GgufTensorInfo& tensor) {
    if (tensor.shape.size() != 2 || tensor.shape[0] <= 0 || tensor.shape[1] <= 0 ||
        !ggml_type_has_q8_0_direct_matvec(tensor.type)) {
        return false;
    }
    const GgmlTypeLayout layout = ggml_type_layout(tensor.type);
    return tensor.shape[0] % layout.block_size == 0 &&
           tensor.shape[0] % LCQI_Q8_0_INPUT_BLOCK_VALUES == 0;
}

[[nodiscard]] bool q4_k_direct_enabled() noexcept {
    const char* enabled = std::getenv(LCQI_LLAMA_Q4_DIRECT_ENV);
    return enabled == nullptr || std::string_view(enabled) != LCQI_LLAMA_FALSE_ENV;
}

[[nodiscard]] bool ggml_direct_enabled() noexcept {
    const char* enabled = std::getenv(LCQI_LLAMA_GGML_DIRECT_ENV);
    return enabled != nullptr && std::string_view(enabled) == LCQI_LLAMA_TRUE_ENV;
}

void account_f32_tensor_load(const GgufTensorInfo& tensor,
                             std::span<const float> values,
                             LlamaGgufLoadReport& report) {
    report.quantized_weight_bytes += tensor.byte_size;
    const std::uint64_t f32_bytes =
        static_cast<std::uint64_t>(values.size()) * static_cast<std::uint64_t>(sizeof(float));
    report.f32_weight_bytes += f32_bytes;
    report.fallback_dequantized_weight_bytes += f32_bytes;
    ++report.tensors_loaded;
    ++report.f32_fallback_tensors;
}

[[nodiscard]] std::vector<float> read_tensor_f32(const std::filesystem::path& gguf_path,
                                                 const GgufManifest& manifest,
                                                 std::string_view name,
                                                 std::span<const std::int64_t> expected_shape,
                                                 LlamaGgufLoadReport& report) {
    const GgufTensorInfo* tensor = manifest.find_tensor(name);
    if (tensor == nullptr) {
        throw std::runtime_error("missing LLaMA GGUF tensor: " + std::string(name));
    }
    validate_shape(*tensor, expected_shape);
```

```cpp
    const std::vector<std::uint8_t> bytes = read_gguf_tensor_bytes(gguf_path, *tensor);
    std::vector<float> values =
        dequantize_ggml_tensor(tensor->type,
                               bytes,
                               static_cast<std::int64_t>(tensor->element_count()));
    account_f32_tensor_load(*tensor, values, report);
    return values;
}

void account_quantized_direct_tensor_load(const GgufTensorInfo& tensor,
                                          LlamaGgufLoadReport& report) {
    report.quantized_weight_bytes += tensor.byte_size;
    report.direct_quantized_weight_bytes += tensor.byte_size;
    ++report.tensors_loaded;
    if (tensor.type == GgmlType::q4_k) {
        ++report.q4_k_direct_tensors;
    } else {
        ++report.ggml_direct_tensors;
    }
}

[[nodiscard]] LlamaGgufLinearWeights make_linear_from_gguf(
    const std::filesystem::path& gguf_path,
    const GgufManifest& manifest,
    std::string_view name,
    std::int32_t input_size,
    std::int32_t output_size,
    LlamaGgufLoadReport& report) {
    const std::array<std::int64_t, 2> shape{input_size, output_size};
    const GgufTensorInfo* tensor = manifest.find_tensor(name);
    if (tensor == nullptr) {
        throw std::runtime_error("missing LLaMA GGUF tensor: " + std::string(name));
    }
    validate_shape(*tensor, shape);

    LlamaGgufLinearWeights linear;
    linear.input_size = input_size;
    linear.output_size = output_size;
    linear.source_type = tensor->type;
    const std::vector<std::uint8_t> bytes = read_gguf_tensor_bytes(gguf_path, *tensor);
    if (q4_k_direct_enabled() && can_use_q4_k_direct(*tensor)) {
        linear.storage = LlamaGgufLinearStorage::q4_k;
        linear.quantized_bytes = bytes;
        account_quantized_direct_tensor_load(*tensor, report);
        return linear;
    }
    if (ggml_direct_enabled() && can_use_ggml_direct(*tensor)) {
        linear.storage = LlamaGgufLinearStorage::ggml_quantized;
```

```
        linear.quantized_bytes = bytes;
        account_quantized_direct_tensor_load(*tensor, report);
        return linear;
    }

    linear.storage = LlamaGgufLinearStorage::f32;
    linear.f32_weights =
        dequantize_ggml_tensor(tensor->type,
                               bytes,
                               static_cast<std::int64_t>(tensor->element_count());
    account_f32_tensor_load(*tensor, linear.f32_weights, report);
    return linear;
}

[[nodiscard]] std::string layer_tensor_name(std::int32_t layer_id, std::string_view suffix) {
    return "blk." + std::to_string(layer_id) + "." + std::string(suffix);
}

void validate_llama_config(const DecoderConfig& config, std::int32_t layer_count) {
    require_positive(layer_count, "layer_count");
    require_positive(config.hidden_size, "hidden_size");
    require_positive(config.query_heads, "query_heads");
    require_positive(config.kv_heads, "kv_heads");
    require_positive(config.head_dim, "head_dim");
    require_positive(config.intermediate_size, "intermediate_size");
    require_positive(config.vocab_size, "vocab_size");
    require_positive(config.max_context, "max_context");
    if (config.query_heads % config.kv_heads != 0) {
        throw std::runtime_error("LLaMA query_heads must be divisible by kv_heads");
    }
    if (config.hidden_size != config.query_heads * config.head_dim) {
        throw std::runtime_error("LLaMA hidden_size must equal query_heads * head_dim");
    }
    if (config.head_dim % LCQI_LLAMA_ROPE_PAIR_WIDTH != 0) {
        throw std::runtime_error("LLaMA head_dim must be even for RoPE");
    }
}

[[nodiscard]] std::span<const float> row_span(std::span<const float> values,
                                              std::int32_t row,
                                              std::int32_t row_width) {
    return values.subspan(checked_size(row, "row") * checked_size(row_width, "row_width"),
                          checked_size(row_width, "row_width"));
}

void add_inplace(std::span<float> target, std::span<const float> source) {
```

```cpp
    if (target.size() != source.size()) {
        throw std::runtime_error("LLaMA residual add size mismatch");
    }
    for (std::size_t index = 0; index < target.size(); ++index) {
        target[index] += source[index];
    }
}

[[nodiscard]] float silu(float value) {
    return value / (1.0F + std::exp(-value));
}

template <typename Func>
double measure_ms(Func&& func) {
    const Clock::time_point begin = Clock::now();
    std::forward<Func>(func)();
    return elapsed_ms(begin, Clock::now());
}

[[nodiscard]] double& linear_hotspot_field(LlamaGgufHotspotReport& hotspots,
                                           LlamaGgufLinearOp operation) {
    switch (operation) {
        case LlamaGgufLinearOp::wq:
            return hotspots.wq_ms;
        case LlamaGgufLinearOp::wk:
            return hotspots.wk_ms;
        case LlamaGgufLinearOp::wv:
            return hotspots.wv_ms;
        case LlamaGgufLinearOp::wo:
            return hotspots.wo_ms;
        case LlamaGgufLinearOp::w_gate:
            return hotspots.w_gate_ms;
        case LlamaGgufLinearOp::w_up:
            return hotspots.w_up_ms;
        case LlamaGgufLinearOp::w_down:
            return hotspots.w_down_ms;
        case LlamaGgufLinearOp::lm_head:
            return hotspots.lm_head_ms;
    }
    throw std::runtime_error("unknown LLaMA linear hotspot operation");
}

void validate_linear_shape(const LlamaGgufLinearWeights& layer,
                           std::span<const float> input,
                           std::span<float> output) {
    if (input.size() != checked_size(layer.input_size, "linear input size") ||
        output.size() != checked_size(layer.output_size, "linear output size")) {
        throw std::runtime_error("LLaMA GGUF linear input/output size mismatch");
    }
}
```

```cpp

void linear_f32_fallback(const LlamaGgufLinearWeights& layer,
                         std::span<const float> input,
                         std::span<float> output) {
    if (layer.f32_weights.size() !=
        checked_size(layer.input_size, "linear input size") *
            checked_size(layer.output_size, "linear output size")) {
        throw std::runtime_error("LLaMA GGUF F32 linear weight size mismatch");
    }
    linear_f32_rows_unchecked(layer.f32_weights.data(),
                              input.data(),
                              nullptr,
                              checked_size(layer.input_size, "linear input size"),
                              0,
                              checked_size(layer.output_size, "linear output size"),
                              output.data());
}

void linear_q4_k_direct(const LlamaGgufLinearWeights& layer,
                        std::span<const Q8KBlock> q8_input,
                        std::span<float> output) {
    if (q8_input.size() !=
        checked_size(layer.input_size / LCQI_QK_K_BLOCK_VALUES, "Q8_K input block count")) {
        throw std::runtime_error("LLaMA GGUF Q4_K scratch block count mismatch");
    }
    matvec_q4_k_q8(layer.quantized_bytes,
                   layer.output_size,
                   layer.input_size,
                   q8_input,
                   output);
}

void linear_ggml_quantized_direct(const LlamaGgufLinearWeights& layer,
                                  std::span<const Q8_0InputBlock> q8_0_input,
                                  std::span<float> output) {
    if (q8_0_input.size() != checked_size(layer.input_size / LCQI_Q8_0_INPUT_BLOCK_VALUES,
                                          "Q8_0 input block count")) {
        throw std::runtime_error("LLaMA GGUF Q8_0 scratch block count mismatch");
    }
    matvec_ggml_quantized_q8_0(layer.source_type,
                               layer.quantized_bytes,
                               layer.output_size,
                               layer.input_size,
                               q8_0_input,
                               output);
}
```

```cpp

void linear_gguf(const LlamaGgufLinearWeights& layer,
                 std::span<const float> input,
                 std::span<const Q8KBlock> q8_input,
                 std::span<const Q8_0InputBlock> q8_0_input,
                 std::span<float> output,
                 LlamaGgufLinearOp operation,
                 LlamaGgufHotspotReport& hotspots) {
    validate_linear_shape(layer, input, output);
    double elapsed = 0.0;
    switch (layer.storage) {
        case LlamaGgufLinearStorage::q4_k:
            elapsed = measure_ms([&]() { linear_q4_k_direct(layer, q8_input, output); });
            hotspots.q4_k_direct_ms += elapsed;
            ++hotspots.q4_k_direct_calls;
            break;
        case LlamaGgufLinearStorage::ggml_quantized:
            elapsed = measure_ms([&]() {
                linear_ggml_quantized_direct(layer, q8_0_input, output);
            });
            hotspots.ggml_direct_ms += elapsed;
            ++hotspots.ggml_direct_calls;
            break;
        case LlamaGgufLinearStorage::f32:
            elapsed = measure_ms([&]() { linear_f32_fallback(layer, input, output); });
            hotspots.f32_fallback_ms += elapsed;
            ++hotspots.f32_fallback_calls;
            break;
    }
    linear_hotspot_field(hotspots, operation) += elapsed;
}

[[nodiscard]] bool uses_q4_k_direct(const LlamaGgufLinearWeights& layer) noexcept {
    return layer.storage == LlamaGgufLinearStorage::q4_k;
}

[[nodiscard]] bool uses_ggml_direct(const LlamaGgufLinearWeights& layer) noexcept {
    return layer.storage == LlamaGgufLinearStorage::ggml_quantized;
}

[[nodiscard]] std::size_t q8_scratch_blocks(std::int32_t input_size) {
    if (input_size <= 0 || input_size % LCQI_QK_K_BLOCK_VALUES != 0) {
        return 0;
    }
    return checked_size(input_size / LCQI_QK_K_BLOCK_VALUES, "Q8_K scratch blocks");
}

```

```
void prepare_q8_if_needed(std::span<const float> input,
                          std::span<Q8KBlock> scratch,
                          bool needed) {
    if (!needed) {
        return;
    }
    quantize_q8_k_input_to(input, scratch);
}

[[nodiscard]] std::size_t q8_0_scratch_blocks(std::int32_t input_size) {
    if (input_size <= 0 || input_size % LCQI_Q8_0_INPUT_BLOCK_VALUES != 0) {
        return 0;
    }
    return checked_size(input_size / LCQI_Q8_0_INPUT_BLOCK_VALUES, "Q8_0 scratch blocks");
}

void prepare_q8_0_if_needed(std::span<const float> input,
                            std::span<Q8_0InputBlock> scratch,
                            bool needed) {
    if (!needed) {
        return;
    }
    quantize_q8_0_input_to(input, scratch);
}

struct LlamaWorkspace {
    std::vector<float> hidden;
    std::vector<float> normed;
    std::vector<float> query;
    std::vector<float> key;
    std::vector<float> value;
    std::vector<float> attention;
    std::vector<float> projected;
    std::vector<float> gate;
    std::vector<float> up;
    std::vector<float> mlp_hidden;
    std::vector<float> mlp_out;
    std::vector<float> final_norm;
    std::vector<Q8KBlock> normed_q8;
    std::vector<Q8KBlock> attention_q8;
    std::vector<Q8KBlock> mlp_hidden_q8;
    std::vector<Q8_0InputBlock> normed_q8_0;
    std::vector<Q8_0InputBlock> attention_q8_0;
    std::vector<Q8_0InputBlock> mlp_hidden_q8_0;

    explicit LlamaWorkspace(const DecoderConfig& config)
        : hidden(checked_size(config.hidden_size, "hidden_size"), 0.0F),
          normed(hidden.size(), 0.0F),
          query(hidden.size(), 0.0F),
          key(checked_size(config.kv_heads * config.head_dim, "kv_width"), 0.0F),
```

```
          value(key.size(), 0.0F),
          attention(hidden.size(), 0.0F),
          projected(hidden.size(), 0.0F),
          gate(checked_size(config.intermediate_size, "intermediate_size"), 0.0↩
    ↪ F),
          up(gate.size(), 0.0F),
          mlp_hidden(gate.size(), 0.0F),
          mlp_out(hidden.size(), 0.0F),
          final_norm(hidden.size(), 0.0F),
          normed_q8(q8_scratch_blocks(config.hidden_size)),
          attention_q8(normed_q8.size()),
          mlp_hidden_q8(q8_scratch_blocks(config.intermediate_size)),
          normed_q8_0(q8_0_scratch_blocks(config.hidden_size)),
          attention_q8_0(normed_q8_0.size()),
          mlp_hidden_q8_0(q8_0_scratch_blocks(config.intermediate_size)) {}
};

void forward_llama_layer(const DecoderConfig& config,
                         const LlamaGgufDecoderLayerWeights& layer,
                         std::int32_t layer_id,
                         std::int32_t model_position,
                         ReferenceKVCache& cache,
                         LlamaWorkspace& workspace,
                         LlamaGgufHotspotReport& hotspots) {
    hotspots.rms_norm_ms += measure_ms([&]() {
        rms_norm(workspace.hidden,
                 layer.rms_attention_weight,
                 config.rms_epsilon,
                 workspace.normed);
    });
    prepare_q8_if_needed(workspace.normed,
                         workspace.normed_q8,
                         uses_q4_k_direct(layer.wq) || uses_q4_k_direct(layer.↩
    ↪ wk) ||
                             uses_q4_k_direct(layer.wv));
    prepare_q8_0_if_needed(workspace.normed,
                           workspace.normed_q8_0,
                           uses_ggml_direct(layer.wq) || uses_ggml_direct(layer↩
    ↪ .wk) ||
                               uses_ggml_direct(layer.wv));
    linear_gguf(layer.wq,
                workspace.normed,
                workspace.normed_q8,
                workspace.normed_q8_0,
                workspace.query,
                LlamaGgufLinearOp::wq,
                hotspots);
    linear_gguf(layer.wk,
                workspace.normed,
                workspace.normed_q8,
                workspace.normed_q8_0,
                workspace.key,
```

```
                LlamaGgufLinearOp::wk,
                hotspots);
    linear_gguf(layer.wv,
                workspace.normed,
                workspace.normed_q8,
                workspace.normed_q8_0,
                workspace.value,
                LlamaGgufLinearOp::wv,
                hotspots);
    hotspots.rope_ms += measure_ms([&]() {
        apply_rope(workspace.query,
                   config.query_heads,
                   config.head_dim,
                   model_position,
                   config.rope_base);
        apply_rope(workspace.key,
                   config.kv_heads,
                   config.head_dim,
                   model_position,
                   config.rope_base);
    });
    cache.append(layer_id, model_position, workspace.key, workspace.value);
    hotspots.attention_ms += measure_ms([&]() {
        attention_decode(config,
                         cache,
                         layer_id,
                         model_position,
                         workspace.query,
                         workspace.attention);
    });
    prepare_q8_if_needed(workspace.attention,
                         workspace.attention_q8,
                         uses_q4_k_direct(layer.wo));
    prepare_q8_0_if_needed(workspace.attention,
                           workspace.attention_q8_0,
                           uses_ggml_direct(layer.wo));
    linear_gguf(layer.wo,
                workspace.attention,
                workspace.attention_q8,
                workspace.attention_q8_0,
                workspace.projected,
                LlamaGgufLinearOp::wo,
                hotspots);
    add_inplace(workspace.hidden, workspace.projected);

    hotspots.rms_norm_ms += measure_ms([&]() {
        rms_norm(workspace.hidden, layer.rms_mlp_weight, config.rms_epsilon, workspace.normed);
    });
    prepare_q8_if_needed(workspace.normed,
                         workspace.normed_q8,
                         uses_q4_k_direct(layer.w_gate) || uses_q4_k_direct(
```

```
    layer.w_up));
    prepare_q8_0_if_needed(workspace.normed,
                           workspace.normed_q8_0,
                           uses_ggml_direct(layer.w_gate) || uses_ggml_direct(layer.w_up));
    linear_gguf(layer.w_gate,
                workspace.normed,
                workspace.normed_q8,
                workspace.normed_q8_0,
                workspace.gate,
                LlamaGgufLinearOp::w_gate,
                hotspots);
    linear_gguf(layer.w_up,
                workspace.normed,
                workspace.normed_q8,
                workspace.normed_q8_0,
                workspace.up,
                LlamaGgufLinearOp::w_up,
                hotspots);
    for (std::size_t index = 0; index < workspace.mlp_hidden.size(); ++index) {
        workspace.mlp_hidden[index] = silu(workspace.gate[index]) * workspace.up[index];
    }
    prepare_q8_if_needed(workspace.mlp_hidden,
                         workspace.mlp_hidden_q8,
                         uses_q4_k_direct(layer.w_down));
    prepare_q8_0_if_needed(workspace.mlp_hidden,
                           workspace.mlp_hidden_q8_0,
                           uses_ggml_direct(layer.w_down));
    linear_gguf(layer.w_down,
                workspace.mlp_hidden,
                workspace.mlp_hidden_q8,
                workspace.mlp_hidden_q8_0,
                workspace.mlp_out,
                LlamaGgufLinearOp::w_down,
                hotspots);
    add_inplace(workspace.hidden, workspace.mlp_out);
}

[[nodiscard]] std::int32_t predict_next_token(const LlamaGgufModel& model,
                                              std::span<const float> final_hidden,
                                              LlamaGgufHotspotReport& hotspots) {
    const DecoderConfig& config = model.decoder.config;
    F32RowMax best;
    if (model.lm_head_tied_to_embedding) {
        const std::span<const float> weights = model.decoder.token_embedding;
        hotspots.lm_head_ms += measure_ms([&]() {
            best = max_dot_f32_rows_unchecked(weights.data(),
                                              final_hidden.data(),
                                              checked_size(config.hidden_size, 
```

```cpp
"hidden_size"),
                                              0,
                                              checked_size(config.vocab_size, "vocab_size"));
        });
        return static_cast<std::int32_t>(best.row);
    }

    std::vector<float> logits(checked_size(config.vocab_size, "vocab_size"), 0.0F);
    const std::vector<Q8KBlock> final_hidden_q8 =
        uses_q4_k_direct(model.decoder.lm_head)
            ? quantize_q8_k_input(final_hidden)
            : std::vector<Q8KBlock>{};
    const std::vector<Q8_0InputBlock> final_hidden_q8_0 =
        uses_ggml_direct(model.decoder.lm_head)
            ? quantize_q8_0_input(final_hidden)
            : std::vector<Q8_0InputBlock>{};
    linear_gguf(model.decoder.lm_head,
                final_hidden,
                final_hidden_q8,
                final_hidden_q8_0,
                logits,
                LlamaGgufLinearOp::lm_head,
                hotspots);
    best.row = 0;
    best.value = logits.front();
    for (std::size_t row = 1; row < logits.size(); ++row) {
        if (logits[row] > best.value) {
            best.row = row;
            best.value = logits[row];
        }
    }
    return static_cast<std::int32_t>(best.row);
}

[[nodiscard]] std::int32_t step_llama_decoder(const LlamaGgufModel& model,
                                              const DecoderConfig& active_config,
                                              ReferenceKVCache& cache,
                                              LlamaWorkspace& workspace,
                                              std::int32_t token_id,
                                              bool predict,
                                              LlamaGgufHotspotReport& hotspots) {
    const DecoderConfig& original_config = model.decoder.config;
    if (token_id < 0 || token_id >= original_config.vocab_size) {
        throw std::runtime_error("LLaMA token id out of vocabulary");
    }
    const std::int32_t model_position = cache.filled_tokens();
    if (model_position >= active_config.max_context) {
        throw std::runtime_error("LLaMA generation exceeded active context");
```

```cpp
    }

    const std::span<const float> embedding =
        row_span(model.decoder.token_embedding, token_id, original_config.hidden_size);
    std::copy(embedding.begin(), embedding.end(), workspace.hidden.begin());
    for (std::int32_t layer_id = 0;
         layer_id < static_cast<std::int32_t>(model.decoder.layers.size());
         ++layer_id) {
        forward_llama_layer(active_config,
                            model.decoder.layers[checked_size(layer_id, "layer_id")],
                            layer_id,
                            model_position,
                            cache,
                            workspace,
                            hotspots);
    }
    if (!predict) {
        return -1;
    }
    hotspots.rms_norm_ms += measure_ms([&]() {
        rms_norm(workspace.hidden,
                 model.decoder.final_norm_weight,
                 active_config.rms_epsilon,
                 workspace.final_norm);
    });
    return predict_next_token(model, workspace.final_norm, hotspots);
}

[[nodiscard]] std::size_t kv_cache_bytes(const DecoderConfig& config, std::int32_t layer_count) {
    return checked_size(layer_count, "layer_count") *
           checked_size(config.max_context, "max_context") *
           checked_size(config.kv_heads, "kv_heads") *
           checked_size(config.head_dim, "head_dim") * sizeof(float) * 2U;
}

[[nodiscard]] std::unordered_map<std::string, std::int32_t> build_vocab_from_tokens(
    std::span<const std::string> tokens) {
    std::unordered_map<std::string, std::int32_t> vocab;
    for (std::size_t index = 0; index < tokens.size(); ++index) {
        vocab.emplace(tokens[index], static_cast<std::int32_t>(index));
    }
    return vocab;
}

[[nodiscard]] std::unordered_map<std::string, std::int32_t> build_merge_ranks(
    std::span<const std::string> merges) {
    std::unordered_map<std::string, std::int32_t> ranks;
    for (std::size_t index = 0; index < merges.size(); ++index) {
```

```
        std::istringstream line_stream(merges[index]);
        std::string left;
        std::string right;
        if (!(line_stream >> left >> right)) {
            throw std::runtime_error("invalid GGUF tokenizer merge: " + merges[↩
    ↪ index]);
        }
        ranks.emplace(left + "\n" + right, static_cast<std::int32_t>(index));
    }
    return ranks;
}

[[nodiscard]] bool starts_with(std::string_view text,
                               std::size_t offset,
                               std::string_view needle) {
    return offset + needle.size() <= text.size() &&
           text.substr(offset, needle.size()) == needle;
}

[[nodiscard]] std::vector<std::pair<std::string, std::int32_t>> special_tokens(
    const Gpt2Tokenizer& tokenizer) {
    std::vector<std::pair<std::string, std::int32_t>> result;
    for (const auto& [token, id] : tokenizer.vocab) {
        if (token.size() >= 4 && token.front() == '<' && token.find('|') != std↩
    ↪ ::string::npos) {
            result.emplace_back(token, id);
        }
    }
    std::sort(result.begin(),
              result.end(),
              [](const auto& lhs, const auto& rhs) {
                  return lhs.first.size() > rhs.first.size();
              });
    return result;
}

[[nodiscard]] std::vector<std::int32_t> encode_with_special_tokens(
    const Gpt2Tokenizer& tokenizer,
    std::string_view text) {
    const std::vector<std::pair<std::string, std::int32_t>> specials = ↩
    ↪ special_tokens(tokenizer);
    std::vector<std::int32_t> ids;
    std::size_t offset = 0;
    while (offset < text.size()) {
        bool matched_special = false;
        for (const auto& [token, id] : specials) {
            if (starts_with(text, offset, token)) {
                ids.push_back(id);
                offset += token.size();
                matched_special = true;
                break;
            }
```

```cpp
        }
        if (matched_special) {
            continue;
        }

        std::size_t next_special = text.size();
        for (const auto& [token, id] : specials) {
            (void)id;
            const std::size_t found = text.find(token, offset);
            if (found != std::string_view::npos) {
                next_special = std::min(next_special, found);
            }
        }
        const std::vector<std::int32_t> chunk =
            gpt2_encode(tokenizer, text.substr(offset, next_special - offset));
        ids.insert(ids.end(), chunk.begin(), chunk.end());
        offset = next_special;
    }
    return ids;
}

}  // namespace

LlamaGgufLoadedModel load_llama_gguf_reference_model(const std::filesystem::path& gguf_path) {
    LlamaGgufLoadedModel loaded;
    const Clock::time_point manifest_begin = Clock::now();
    const GgufManifest manifest = load_gguf_manifest(gguf_path);
    loaded.report.manifest_ms = elapsed_ms(manifest_begin, Clock::now());

    const Clock::time_point weights_begin = Clock::now();
    loaded.model.architecture = metadata_string(manifest, LCQI_LLAMA_ARCHITECTURE_KEY);
    loaded.model.name = metadata_string(manifest, LCQI_LLAMA_NAME_KEY);
    DecoderConfig& config = loaded.model.decoder.config;
    config.hidden_size = metadata_i32(manifest, LCQI_LLAMA_EMBEDDING_LENGTH_KEY);
    const std::int32_t layer_count = metadata_i32(manifest, LCQI_LLAMA_BLOCK_COUNT_KEY);
    config.intermediate_size = metadata_i32(manifest, LCQI_LLAMA_FEED_FORWARD_LENGTH_KEY);
    config.query_heads = metadata_i32(manifest, LCQI_LLAMA_HEAD_COUNT_KEY);
    config.kv_heads = metadata_i32(manifest, LCQI_LLAMA_KV_HEAD_COUNT_KEY);
    config.head_dim = config.hidden_size / config.query_heads;
    const std::int32_t rope_dimension = metadata_i32(manifest, LCQI_LLAMA_ROPE_DIMENSION_KEY);
    if (rope_dimension != config.head_dim) {
        throw std::runtime_error("LLaMA rope dimension does not match head_dim");
    }
    config.rope_base = metadata_f32(manifest, LCQI_LLAMA_ROPE_BASE_KEY);
    config.max_context = metadata_i32(manifest, LCQI_LLAMA_CONTEXT_LENGTH_KEY);
```

```
    config.vocab_size = metadata_i32(manifest, LCQI_LLAMA_VOCAB_SIZE_KEY);
    config.rms_epsilon = metadata_f32(manifest, LCQI_LLAMA_RMS_EPSILON_KEY);
    loaded.model.bos_token_id =
        optional_metadata_i32(manifest, LCQI_LLAMA_BOS_TOKEN_KEY, LCQI_LLAMA_DEFAULT_BOS_TOKEN);
    loaded.model.eos_token_id =
        optional_metadata_i32(manifest, LCQI_LLAMA_EOS_TOKEN_KEY, LCQI_LLAMA_DEFAULT_EOS_TOKEN);
    loaded.model.unknown_token_id = optional_metadata_i32(
        manifest,
        LCQI_LLAMA_UNKNOWN_TOKEN_KEY,
        LCQI_LLAMA_DEFAULT_UNK_TOKEN);
    validate_llama_config(config, layer_count);

    const std::array<std::int64_t, 2> embedding_shape{config.hidden_size, config.vocab_size};
    loaded.model.decoder.token_embedding = read_tensor_f32(gguf_path,
                                                           manifest,
                                                           "token_embd.weight",
                                                           embedding_shape,
                                                           loaded.report);
    const std::array<std::int64_t, 1> hidden_shape{config.hidden_size};
    loaded.model.decoder.final_norm_weight = read_tensor_f32(gguf_path,
                                                             manifest,
                                                             "output_norm.weight",
                                                             hidden_shape,
                                                             loaded.report);
    loaded.model.decoder.layers.resize(checked_size(layer_count, "layer_count"));
    const std::int32_t kv_width = config.kv_heads * config.head_dim;
    for (std::int32_t layer_id = 0; layer_id < layer_count; ++layer_id) {
        LlamaGgufDecoderLayerWeights& layer =
            loaded.model.decoder.layers[checked_size(layer_id, "layer_id")];
        layer.rms_attention_weight = read_tensor_f32(
            gguf_path,
            manifest,
            layer_tensor_name(layer_id, "attn_norm.weight"),
            hidden_shape,
            loaded.report);
        layer.rms_mlp_weight = read_tensor_f32(gguf_path,
                                               manifest,
                                               layer_tensor_name(layer_id, "ffn_norm.weight"),
                                               hidden_shape,
                                               loaded.report);
        layer.wq = make_linear_from_gguf(gguf_path,
                                         manifest,
                                         layer_tensor_name(layer_id, "attn_q.weight"),
                                         config.hidden_size,
                                         config.hidden_size,
```

```
                                         loaded.report);
        layer.wk = make_linear_from_gguf(gguf_path,
                                         manifest,
                                         layer_tensor_name(layer_id, "attn_k.weight"),
                                         config.hidden_size,
                                         kv_width,
                                         loaded.report);
        layer.wv = make_linear_from_gguf(gguf_path,
                                         manifest,
                                         layer_tensor_name(layer_id, "attn_v.weight"),
                                         config.hidden_size,
                                         kv_width,
                                         loaded.report);
        layer.wo = make_linear_from_gguf(gguf_path,
                                         manifest,
                                         layer_tensor_name(layer_id, "attn_output.weight"),
                                         config.hidden_size,
                                         config.hidden_size,
                                         loaded.report);
        layer.w_gate = make_linear_from_gguf(gguf_path,
                                             manifest,
                                             layer_tensor_name(layer_id, "ffn_gate.weight"),
                                             config.hidden_size,
                                             config.intermediate_size,
                                             loaded.report);
        layer.w_up = make_linear_from_gguf(gguf_path,
                                           manifest,
                                           layer_tensor_name(layer_id, "ffn_up.weight"),
                                           config.hidden_size,
                                           config.intermediate_size,
                                           loaded.report);
        layer.w_down = make_linear_from_gguf(gguf_path,
                                             manifest,
                                             layer_tensor_name(layer_id, "ffn_down.weight"),
                                             config.intermediate_size,
                                             config.hidden_size,
                                             loaded.report);
    }

    if (manifest.find_tensor("output.weight") != nullptr) {
        loaded.model.lm_head_tied_to_embedding = false;
        loaded.model.decoder.lm_head = make_linear_from_gguf(gguf_path,
                                                             manifest,
                                                             "output.weight",
                                                             config.hidden_size,
```

```cpp
                                                          config.vocab_size,
                                                          loaded.report);
    } else {
        loaded.model.lm_head_tied_to_embedding = true;
        loaded.model.decoder.lm_head.input_size = config.hidden_size;
        loaded.model.decoder.lm_head.output_size = config.vocab_size;
    }
    loaded.report.weights_ms = elapsed_ms(weights_begin, Clock::now());
    return loaded;
}

Gpt2Tokenizer load_gpt2_tokenizer_from_gguf(const std::filesystem::path& gguf_path) {
    const GgufManifest manifest = load_gguf_manifest(gguf_path);
    const GgufMetadataEntry& tokens_entry = require_metadata(manifest,
                                                          LCQI_LLAMA_TOKENIZER_TOKENS_KEY);
    const GgufMetadataEntry& merges_entry = require_metadata(manifest,
                                                          LCQI_LLAMA_TOKENIZER_MERGES_KEY);
    if (tokens_entry.string_values.empty() || merges_entry.string_values.empty()) {
        throw std::runtime_error("GGUF tokenizer tokens or merges are empty");
    }
    Gpt2Tokenizer tokenizer;
    tokenizer.bos_token_id =
        optional_metadata_i32(manifest, LCQI_LLAMA_BOS_TOKEN_KEY, LCQI_LLAMA_DEFAULT_BOS_TOKEN);
    tokenizer.eos_token_id =
        optional_metadata_i32(manifest, LCQI_LLAMA_EOS_TOKEN_KEY, LCQI_LLAMA_DEFAULT_EOS_TOKEN);
    tokenizer.id_to_token = tokens_entry.string_values;
    tokenizer.vocab = build_vocab_from_tokens(tokenizer.id_to_token);
    tokenizer.bpe_ranks = build_merge_ranks(merges_entry.string_values);
    return tokenizer;
}

std::vector<std::int32_t> llama_gguf_chat_prompt_ids(const Gpt2Tokenizer& tokenizer,
                                                     std::string_view user_prompt,
                                                     std::int32_t bos_token_id) {
    std::string prompt;
    prompt += "<|im_start|>system\n";
    prompt += "You are a helpful AI assistant named SmolLM, trained by Hugging Face";
    prompt += "<|im_end|>\n";
    prompt += "<|im_start|>user\n";
    prompt += user_prompt;
    prompt += "<|im_end|>\n";
    prompt += "<|im_start|>assistant\n";
```

```cpp
    std::vector<std::int32_t> ids = encode_with_special_tokens(tokenizer, prompt);
    if (ids.empty() && bos_token_id >= 0) {
        ids.push_back(bos_token_id);
    }
    return ids;
}

LlamaGgufGenerationResult llama_gguf_generate_greedy(const LlamaGgufModel& model,
                                                     std::span<const std::int32_t> prompt_ids,
                                                     std::int32_t max_new_tokens) {
    if (prompt_ids.empty()) {
        throw std::runtime_error("LLaMA generation requires at least one prompt token");
    }
    if (max_new_tokens < 0) {
        throw std::runtime_error("max_new_tokens cannot be negative");
    }
    const std::int32_t layer_count = static_cast<std::int32_t>(model.decoder.layers.size());
    validate_llama_config(model.decoder.config, layer_count);
    const std::int32_t active_context =
        static_cast<std::int32_t>(prompt_ids.size()) + std::max(max_new_tokens, 1);
    if (active_context > model.decoder.config.max_context) {
        throw std::runtime_error("LLaMA prompt plus generation exceeds model context");
    }

    DecoderConfig active_config = model.decoder.config;
    active_config.max_context = active_context;
    ReferenceKVCache cache(active_config, layer_count);
    LlamaWorkspace workspace(active_config);

    LlamaGgufGenerationResult result;
    result.prompt_ids.assign(prompt_ids.begin(), prompt_ids.end());
    result.generated_ids.assign(prompt_ids.begin(), prompt_ids.end());
    result.kv_cache_bytes = kv_cache_bytes(active_config, layer_count);
    if (max_new_tokens == 0) {
        return result;
    }

    const Clock::time_point prefill_begin = Clock::now();
    for (std::size_t index = 0; index + 1 < prompt_ids.size(); ++index) {
        static_cast<void>(step_llama_decoder(model,
                                             active_config,
                                             cache,
                                             workspace,
                                             prompt_ids[index],
```

```
                                             false,
                                             result.hotspots));
    }
    std::int32_t predicted = step_llama_decoder(model,
                                                active_config,
                                                cache,
                                                workspace,
                                                prompt_ids.back(),
                                                true,
                                                result.hotspots);
    result.predicted_first_token = predicted;
    result.prefill_steps = prompt_ids.size();
    result.prefill_ms = elapsed_ms(prefill_begin, Clock::now());

    const Clock::time_point decode_begin = Clock::now();
    for (std::int32_t step = 0; step < max_new_tokens; ++step) {
        result.generated_ids.push_back(predicted);
        if (predicted == model.eos_token_id) {
            break;
        }
        if (step + 1 < max_new_tokens) {
            predicted = step_llama_decoder(model,
                                           active_config,
                                           cache,
                                           workspace,
                                           predicted,
                                           true,
                                           result.hotspots);
            ++result.decode_steps;
        }
    }
    result.decode_ms = elapsed_ms(decode_begin, Clock::now());
    result.total_ms = result.prefill_ms + result.decode_ms;
    return result;
}

}  // namespace lcqi
```

`llama_cli.cpp` 是 `lcqi_llama_gguf` 的命令行入口。它支持 `--ids` 直接给 token id，也支持 `--prompt` 从 GGUF tokenizer arrays 构造 chat prompt；`--benchmark` 输出 manifest、权重加载、prefill、decode、KV cache、量化权重字节、direct `Q4_K` 字节、实验 `ggml_direct` 字节、fallback f32 字节、direct/fallback tensor 数、per-op hotspot 和 direct/fallback call 数。环境变量 `LCQI_LLAMA_Q4_DIRECT=0` 会关闭真实 decoder 里的 Q4 direct path，用于同二进制 A/B；`LCQI_LLAMA_GGML_DIRECT=1` 会打开 `Q5_0/Q6_K/Q8_0` 实验 direct path，用来证明覆盖率和速度必须分开验收。报告中 `decode_tokens_per_second` 只统计真正喂入新 token 后再次预测的 decode step，不能把首 token prefill 的预测误算成续 decode。

Listing 16.12 src/llama_cli.cpp

```
#include <lcqi/llama_gguf.hpp>

#include <chrono>
#include <cstdlib>
```

```cpp
#include <exception>
#include <filesystem>
#include <iostream>
#include <sstream>
#include <stdexcept>
#include <string>
#include <string_view>
#include <vector>

namespace {

constexpr std::int32_t LCQI_LLAMA_DEFAULT_MAX_NEW_TOKENS = 1;
constexpr std::string_view LCQI_LLAMA_IDS_PREFIX = "--ids=";
constexpr std::string_view LCQI_LLAMA_PROMPT_PREFIX = "--prompt=";
constexpr std::string_view LCQI_LLAMA_MAX_NEW_PREFIX = "--max-new=";
constexpr const char* LCQI_LLAMA_Q4_DIRECT_ENV = "LCQI_LLAMA_Q4_DIRECT";
constexpr const char* LCQI_LLAMA_GGML_DIRECT_ENV = "LCQI_LLAMA_GGML_DIRECT";
constexpr std::string_view LCQI_LLAMA_FALSE_ENV = "0";
constexpr std::string_view LCQI_LLAMA_TRUE_ENV = "1";

using Clock = std::chrono::steady_clock;

struct LlamaCliOptions {
    std::filesystem::path gguf_path;
    std::string prompt;
    std::vector<std::int32_t> ids;
    std::int32_t max_new_tokens = LCQI_LLAMA_DEFAULT_MAX_NEW_TOKENS;
    bool use_ids = false;
    bool benchmark = false;
    bool decode_text = false;
};

[[nodiscard]] double elapsed_ms(Clock::time_point begin, Clock::time_point end) {
    return std::chrono::duration<double, std::milli>(end - begin).count();
}

void print_ids(std::string_view label, const std::vector<std::int32_t>& ids) {
    std::cout << label;
    for (const std::int32_t id : ids) {
        std::cout << ' ' << id;
    }
    std::cout << "\n";
}

[[nodiscard]] std::int32_t parse_non_negative_i32(const std::string& text,
                                                   const char* name) {
    const int value = std::stoi(text);
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::int32_t>(value);
```

```cpp
}

[[nodiscard]] std::vector<std::int32_t> parse_ids(std::string_view text) {
    std::vector<std::int32_t> ids;
    std::string normalized(text);
    for (char& ch : normalized) {
        if (ch == ',') {
            ch = ' ';
        }
    }
    std::istringstream input(normalized);
    std::int32_t id = 0;
    while (input >> id) {
        if (id < 0) {
            throw std::runtime_error("token ids cannot be negative");
        }
        ids.push_back(id);
    }
    if (ids.empty()) {
        throw std::runtime_error("--ids requires at least one token id");
    }
    return ids;
}

[[nodiscard]] bool consume_prefix(std::string_view arg,
                                  std::string_view prefix,
                                  std::string_view& value) {
    if (arg.substr(0, prefix.size()) != prefix) {
        return false;
    }
    value = arg.substr(prefix.size());
    return true;
}

[[nodiscard]] LlamaCliOptions parse_options(int argc, char** argv) {
    if (argc < 2) {
        throw std::runtime_error(
            "usage: lcqi_llama_gguf model.gguf [--prompt TEXT|--ids 1,2,3] "
            "[--max-new N] [--benchmark] [--decode-text]");
    }
    LlamaCliOptions options;
    options.gguf_path = argv[1];
    for (int index = 2; index < argc; ++index) {
        const std::string arg = argv[index];
        std::string_view value;
        if (arg == "--benchmark") {
            options.benchmark = true;
        } else if (arg == "--decode-text") {
            options.decode_text = true;
        } else if (arg == "--ids") {
            if (index + 1 >= argc) {
                throw std::runtime_error("--ids requires a value");
```

```
            }
            ++index;
            options.ids = parse_ids(argv[index]);
            options.use_ids = true;
        } else if (consume_prefix(arg, LCQI_LLAMA_IDS_PREFIX, value)) {
            options.ids = parse_ids(value);
            options.use_ids = true;
        } else if (arg == "--prompt") {
            if (index + 1 >= argc) {
                throw std::runtime_error("--prompt requires a value");
            }
            ++index;
            options.prompt = argv[index];
            options.use_ids = false;
        } else if (consume_prefix(arg, LCQI_LLAMA_PROMPT_PREFIX, value)) {
            options.prompt = std::string(value);
            options.use_ids = false;
        } else if (arg == "--max-new") {
            if (index + 1 >= argc) {
                throw std::runtime_error("--max-new requires a value");
            }
            ++index;
            options.max_new_tokens = parse_non_negative_i32(argv[index], "--max-new");
        } else if (consume_prefix(arg, LCQI_LLAMA_MAX_NEW_PREFIX, value)) {
            options.max_new_tokens =
                parse_non_negative_i32(std::string(value), "--max-new");
        } else {
            throw std::runtime_error("unknown option: " + arg);
        }
    }
    if (!options.use_ids && options.prompt.empty()) {
        options.prompt = "Hello";
    }
    return options;
}

[[nodiscard]] std::vector<std::int32_t> make_prompt_ids(const LlamaCliOptions& options,
                                                        lcqi::Gpt2Tokenizer* tokenizer,
                                                        std::int32_t bos_token_id) {
    if (options.use_ids) {
        return options.ids;
    }
    return lcqi::llama_gguf_chat_prompt_ids(*tokenizer, options.prompt, bos_token_id);
}

void print_benchmark(const lcqi::LlamaGgufLoadedModel& loaded,
                     const lcqi::LlamaGgufGenerationResult& result,
```

```cpp
                       std::size_t prompt_size,
                       std::size_t generated_new_tokens) {
    const double first_token_tokens_per_second =
        result.prefill_ms > 0.0 ? 1000.0 / result.prefill_ms : 0.0;
    const double decode_tokens_per_second =
        result.decode_ms > 0.0
            ? static_cast<double>(result.decode_steps) * 1000.0 / result.decode_ms
            : 0.0;
    std::cout << "benchmark_manifest_ms " << loaded.report.manifest_ms << "\n";
    std::cout << "benchmark_weight_load_ms " << loaded.report.weights_ms << "\n";
    std::cout << "benchmark_load_ms " << result.load_ms << "\n";
    std::cout << "benchmark_prefill_ms " << result.prefill_ms << "\n";
    std::cout << "benchmark_decode_ms " << result.decode_ms << "\n";
    std::cout << "benchmark_total_ms " << result.total_ms << "\n";
    std::cout << "benchmark_prompt_tokens " << prompt_size << "\n";
    std::cout << "benchmark_generated_tokens " << generated_new_tokens << "\n";
    std::cout << "benchmark_prefill_steps " << result.prefill_steps << "\n";
    std::cout << "benchmark_decode_steps " << result.decode_steps << "\n";
    std::cout << "benchmark_first_token_tokens_per_second "
              << first_token_tokens_per_second << "\n";
    std::cout << "benchmark_decode_tokens_per_second " << decode_tokens_per_second << "\n";
    std::cout << "benchmark_kv_cache_bytes " << result.kv_cache_bytes << "\n";
    std::cout << "benchmark_quantized_weight_bytes "
              << loaded.report.quantized_weight_bytes << "\n";
    std::cout << "benchmark_f32_weight_bytes " << loaded.report.f32_weight_bytes << "\n";
    std::cout << "benchmark_direct_quantized_weight_bytes "
              << loaded.report.direct_quantized_weight_bytes << "\n";
    std::cout << "benchmark_fallback_dequantized_weight_bytes "
              << loaded.report.fallback_dequantized_weight_bytes << "\n";
    std::cout << "benchmark_tensors_loaded " << loaded.report.tensors_loaded << "\n";
    std::cout << "benchmark_q4_k_direct_tensors "
              << loaded.report.q4_k_direct_tensors << "\n";
    std::cout << "benchmark_ggml_direct_tensors "
              << loaded.report.ggml_direct_tensors << "\n";
    std::cout << "benchmark_f32_fallback_tensors "
              << loaded.report.f32_fallback_tensors << "\n";
    std::cout << "benchmark_hotspot_rms_norm_ms " << result.hotspots.rms_norm_ms << "\n";
    std::cout << "benchmark_hotspot_attention_ms " << result.hotspots.attention_ms << "\n";
    std::cout << "benchmark_hotspot_rope_ms " << result.hotspots.rope_ms << "\n";
    std::cout << "benchmark_hotspot_wq_ms " << result.hotspots.wq_ms << "\n";
    std::cout << "benchmark_hotspot_wk_ms " << result.hotspots.wk_ms << "\n";
    std::cout << "benchmark_hotspot_wv_ms " << result.hotspots.wv_ms << "\n";
    std::cout << "benchmark_hotspot_wo_ms " << result.hotspots.wo_ms << "\n";
    std::cout << "benchmark_hotspot_w_gate_ms " << result.hotspots.w_gate_ms <<
```

```cpp
 "\n";
    std::cout << "benchmark_hotspot_w_up_ms " << result.hotspots.w_up_ms << "\n";
    std::cout << "benchmark_hotspot_w_down_ms " << result.hotspots.w_down_ms << "\n";
    std::cout << "benchmark_hotspot_lm_head_ms " << result.hotspots.lm_head_ms << "\n";
    std::cout << "benchmark_hotspot_q4_k_direct_ms "
              << result.hotspots.q4_k_direct_ms << "\n";
    std::cout << "benchmark_hotspot_ggml_direct_ms "
              << result.hotspots.ggml_direct_ms << "\n";
    std::cout << "benchmark_hotspot_f32_fallback_ms "
              << result.hotspots.f32_fallback_ms << "\n";
    std::cout << "benchmark_hotspot_q4_k_direct_calls "
              << result.hotspots.q4_k_direct_calls << "\n";
    std::cout << "benchmark_hotspot_ggml_direct_calls "
              << result.hotspots.ggml_direct_calls << "\n";
    std::cout << "benchmark_hotspot_f32_fallback_calls "
              << result.hotspots.f32_fallback_calls << "\n";
}

[[nodiscard]] bool q4_direct_enabled() noexcept {
    const char* enabled = std::getenv(LCQI_LLAMA_Q4_DIRECT_ENV);
    return enabled == nullptr || std::string_view(enabled) != LCQI_LLAMA_FALSE_ENV;
}

[[nodiscard]] bool ggml_direct_enabled() noexcept {
    const char* enabled = std::getenv(LCQI_LLAMA_GGML_DIRECT_ENV);
    return enabled != nullptr && std::string_view(enabled) == LCQI_LLAMA_TRUE_ENV;
}

[[nodiscard]] const char* weight_execution_mode() noexcept {
    if (ggml_direct_enabled()) {
        return "gguf_mixed_quantized_direct_experimental";
    }
    if (!q4_direct_enabled()) {
        return "f32_dequantized_reference";
    }
    return "gguf_mixed_q4_k_direct";
}

}  // namespace

int main(int argc, char** argv) {
    try {
        const Clock::time_point total_begin = Clock::now();
        const LlamaCliOptions options = parse_options(argc, argv);
        const Clock::time_point load_begin = Clock::now();
        lcqi::LlamaGgufLoadedModel loaded =
            lcqi::load_llama_gguf_reference_model(options.gguf_path);
```

```cpp
        const double load_ms = elapsed_ms(load_begin, Clock::now());

        lcqi::Gpt2Tokenizer tokenizer;
        if (!options.use_ids || options.decode_text) {
            tokenizer = lcqi::load_gpt2_tokenizer_from_gguf(options.gguf_path);
        }
        const std::vector<std::int32_t> prompt_ids =
            make_prompt_ids(options, &tokenizer, loaded.model.bos_token_id);
        lcqi::LlamaGgufGenerationResult result =
            lcqi::llama_gguf_generate_greedy(loaded.model,
                                             prompt_ids,
                                             options.max_new_tokens);
        result.load_ms = load_ms;
        result.total_ms = elapsed_ms(total_begin, Clock::now());

        std::cout << "mode llama_gguf_reference\n";
        std::cout << "weight_execution " << weight_execution_mode() << "\n";
        std::cout << "model_path " << options.gguf_path.string() << "\n";
        std::cout << "architecture " << loaded.model.architecture << "\n";
        std::cout << "model_name " << loaded.model.name << "\n";
        std::cout << "layers " << loaded.model.decoder.layers.size() << "\n";
        std::cout << "hidden_size " << loaded.model.decoder.config.hidden_size << "\n";
        std::cout << "query_heads " << loaded.model.decoder.config.query_heads << "\n";
        std::cout << "kv_heads " << loaded.model.decoder.config.kv_heads << "\n";
        std::cout << "head_dim " << loaded.model.decoder.config.head_dim << "\n";
        std::cout << "vocab_size " << loaded.model.decoder.config.vocab_size << "\n";
        std::cout << "tie_lm_head_to_embedding "
                  << (loaded.model.lm_head_tied_to_embedding ? 1 : 0) << "\n";
        print_ids("prompt_ids", result.prompt_ids);
        print_ids("generated_ids", result.generated_ids);
        std::cout << "predicted_first_token " << result.predicted_first_token << "\n";
        if (options.decode_text) {
            std::cout << "generated_text "
                      << lcqi::gpt2_decode(tokenizer, result.generated_ids) << "\n";
        }
        if (options.benchmark) {
            const std::size_t generated_new_tokens =
                result.generated_ids.size() >= result.prompt_ids.size()
                    ? result.generated_ids.size() - result.prompt_ids.size()
                    : 0;
            print_benchmark(loaded, result, result.prompt_ids.size(), generated_new_tokens);
        }
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
```

```
        std::cerr << "error: " << error.what() << "\n";
        return EXIT_FAILURE;
    }
}
```

`q4_k_avx2.cpp` 是本轮 Q4_K 优化入口。它不是完整 llama.cpp 后端，只把 `Q4_KxQ8_K` 的 32 元素子块点积换成 AVX2 `maddubs` 归约。读这份源码时要同时看 fallback：环境变量 `LCQI_Q4K_BACKEND=scalar` 可以强制走标量路径，报告脚本用它生成同机同模型的 A/B 证据。

Listing 16.13 src/q4_k_avx2.cpp

```cpp
#include <lcqi/q4_k.hpp>

#if defined(LCQI_ENABLE_AVX2)

#include <immintrin.h>

#include <array>
#include <cmath>
#include <cstddef>
#include <cstdint>
#include <cstdlib>
#include <cstring>

namespace lcqi {
namespace {

constexpr std::uint32_t LCQI_Q4K_F32_EXPONENT_MASK = 0x7F800000U;
constexpr std::uint32_t LCQI_Q4K_F16_SIGN_MASK = 0x8000U;
constexpr std::uint32_t LCQI_Q4K_F16_EXPONENT_MASK = 0x7C00U;
constexpr std::uint32_t LCQI_Q4K_F16_MANTISSA_MASK = 0x03FFU;
constexpr std::uint32_t LCQI_Q4K_F16_EXPONENT_BIAS = 15U;
constexpr std::uint32_t LCQI_Q4K_F32_EXPONENT_BIAS = 127U;
constexpr std::uint32_t LCQI_Q4K_F32_MANTISSA_BITS = 23U;
constexpr std::uint32_t LCQI_Q4K_F16_MANTISSA_BITS = 10U;
constexpr std::uint32_t LCQI_Q4K_F16_TO_F32_SIGN_SHIFT = 16U;
constexpr std::uint32_t LCQI_Q4K_BYTE_BITS = 8U;
constexpr std::uint8_t LCQI_Q4K_LOW_NIBBLE_MASK = 0x0FU;
constexpr std::uint8_t LCQI_Q4K_SIX_BIT_MASK = 63U;
constexpr std::int64_t LCQI_Q4K_QS_OFFSET = 16;
constexpr std::int64_t LCQI_Q4K_QS_BYTES_PER_PAIR = 32;
constexpr std::int64_t LCQI_Q4K_SUBBLOCK_PAIR_COUNT = 4;
constexpr const char* LCQI_Q4K_BACKEND_ENV = "LCQI_Q4K_BACKEND";
constexpr const char* LCQI_Q4K_FORCE_SCALAR_ENV = "LCQI_Q4K_FORCE_SCALAR";
constexpr const char* LCQI_Q4K_BACKEND_SCALAR = "scalar";
constexpr const char* LCQI_Q4K_FORCE_TRUE = "1";

std::uint16_t read_le_u16(const std::uint8_t* bytes) noexcept {
    return static_cast<std::uint16_t>(
        static_cast<std::uint16_t>(bytes[0]) |
        (static_cast<std::uint16_t>(bytes[1]) << LCQI_Q4K_BYTE_BITS));
}

```

```
float bits_to_float(std::uint32_t bits) noexcept {
    float value = 0.0F;
    std::memcpy(&value, &bits, sizeof(value));
    return value;
}

float f16_to_f32(std::uint16_t bits) noexcept {
    const std::uint32_t sign =
        (static_cast<std::uint32_t>(bits) & LCQI_Q4K_F16_SIGN_MASK)
        << LCQI_Q4K_F16_TO_F32_SIGN_SHIFT;
    const std::uint32_t exponent =
        static_cast<std::uint32_t>(bits) & LCQI_Q4K_F16_EXPONENT_MASK;
    const std::uint32_t mantissa =
        static_cast<std::uint32_t>(bits) & LCQI_Q4K_F16_MANTISSA_MASK;

    if (exponent == 0U) {
        if (mantissa == 0U) {
            return bits_to_float(sign);
        }
        float value = static_cast<float>(mantissa) /
                      static_cast<float>(1U << LCQI_Q4K_F16_MANTISSA_BITS);
        value = std::ldexp(value, 1 - static_cast<int>(LCQI_Q4K_F16_EXPONENT_BIAS));
        return sign == 0U ? value : -value;
    }

    if (exponent == LCQI_Q4K_F16_EXPONENT_MASK) {
        const std::uint32_t f32_mantissa =
            mantissa << (LCQI_Q4K_F32_MANTISSA_BITS - LCQI_Q4K_F16_MANTISSA_BITS);
        return bits_to_float(sign | LCQI_Q4K_F32_EXPONENT_MASK | f32_mantissa);
    }

    const std::uint32_t f32_exponent =
        ((exponent >> LCQI_Q4K_F16_MANTISSA_BITS) - LCQI_Q4K_F16_EXPONENT_BIAS +
         LCQI_Q4K_F32_EXPONENT_BIAS)
        << LCQI_Q4K_F32_MANTISSA_BITS;
    const std::uint32_t f32_mantissa =
        mantissa << (LCQI_Q4K_F32_MANTISSA_BITS - LCQI_Q4K_F16_MANTISSA_BITS);
    return bits_to_float(sign | f32_exponent | f32_mantissa);
}

void get_scale_min_k4(std::int32_t index,
                      const std::uint8_t* packed,
                      std::uint8_t& scale,
                      std::uint8_t& min) noexcept {
    if (index < LCQI_Q4K_SUBBLOCK_PAIR_COUNT) {
        scale = packed[index] & LCQI_Q4K_SIX_BIT_MASK;
        min = packed[index + LCQI_Q4K_SUBBLOCK_PAIR_COUNT] & LCQI_Q4K_SIX_BIT_MASK;
        return;
```

```
    }
    scale = static_cast<std::uint8_t>((packed[index + LCQI_Q4K_SUBBLOCK_PAIR_COUNT] &
                                       LCQI_Q4K_LOW_NIBBLE_MASK) |
                                      ((packed[index - LCQI_Q4K_SUBBLOCK_PAIR_COUNT] >> 6U)
                                       << 4U));
    min = static_cast<std::uint8_t>((packed[index + LCQI_Q4K_SUBBLOCK_PAIR_COUNT] >> 4U) |
                                    ((packed[index] >> 6U) << 4U));
}

std::int32_t horizontal_sum_i32(__m256i values) noexcept {
    const __m128i low = _mm256_castsi256_si128(values);
    const __m128i high = _mm256_extracti128_si256(values, 1);
    __m128i sum = _mm_add_epi32(low, high);
    sum = _mm_add_epi32(sum, _mm_shuffle_epi32(sum, _MM_SHUFFLE(1, 0, 3, 2)));
    sum = _mm_add_epi32(sum, _mm_shuffle_epi32(sum, _MM_SHUFFLE(2, 3, 0, 1)));
    return _mm_cvtsi128_si32(sum);
}

std::int32_t dot_32_u4_i8_avx2(__m256i q4_values, const std::int8_t* q8_values) noexcept {
    const __m256i q8 = _mm256_loadu_si256(reinterpret_cast<const __m256i*>(q8_values));
    const __m256i pair_sums_i16 = _mm256_maddubs_epi16(q4_values, q8);
    const __m256i ones = _mm256_set1_epi16(1);
    const __m256i partial_i32 = _mm256_madd_epi16(pair_sums_i16, ones);
    return horizontal_sum_i32(partial_i32);
}

float dot_block_q4_k_q8_avx2(const std::uint8_t* block, const Q8KBlock& input) noexcept {
    const float d = f16_to_f32(read_le_u16(block));
    const float dmin = f16_to_f32(read_le_u16(block + 2));
    const std::uint8_t* scales = block + 4;
    const std::uint8_t* qs = block + LCQI_Q4K_QS_OFFSET;
    const __m256i nibble_mask = _mm256_set1_epi8(static_cast<char>(LCQI_Q4K_LOW_NIBBLE_MASK));
    std::int32_t weighted_sum = 0;
    std::int32_t min_sum = 0;

    for (std::int32_t pair = 0; pair < LCQI_Q4K_SUBBLOCK_PAIR_COUNT; ++pair) {
        const __m256i packed =
            _mm256_loadu_si256(reinterpret_cast<const __m256i*>(
                qs + pair * LCQI_Q4K_QS_BYTES_PER_PAIR));
        const __m256i low_nibbles = _mm256_and_si256(packed, nibble_mask);
        const __m256i high_nibbles =
            _mm256_and_si256(_mm256_srli_epi16(packed, 4), nibble_mask);

        const std::int32_t low_subblock = pair * 2;
        const std::int32_t high_subblock = low_subblock + 1;
```

```
        std::uint8_t low_scale = 0;
        std::uint8_t low_min = 0;
        std::uint8_t high_scale = 0;
        std::uint8_t high_min = 0;
        get_scale_min_k4(low_subblock, scales, low_scale, low_min);
        get_scale_min_k4(high_subblock, scales, high_scale, high_min);

        const std::int32_t low_dot =
            dot_32_u4_i8_avx2(low_nibbles,
                              input.qs.data() + low_subblock * LCQI_Q4_K_SUBBLOCK_VALUES);
        const std::int32_t high_dot =
            dot_32_u4_i8_avx2(high_nibbles,
                              input.qs.data() + high_subblock * LCQI_Q4_K_SUBBLOCK_VALUES);
        weighted_sum += static_cast<std::int32_t>(low_scale) * low_dot;
        weighted_sum += static_cast<std::int32_t>(high_scale) * high_dot;

        min_sum += static_cast<std::int32_t>(low_min) *
                   (static_cast<std::int32_t>(input.bsums[low_subblock * 2]) +
                    static_cast<std::int32_t>(input.bsums[low_subblock * 2 + 1]));
        min_sum += static_cast<std::int32_t>(high_min) *
                   (static_cast<std::int32_t>(input.bsums[high_subblock * 2]) +
                    static_cast<std::int32_t>(input.bsums[high_subblock * 2 + 1]));
    }

    return (d * input.d) * static_cast<float>(weighted_sum) -
           (dmin * input.d) * static_cast<float>(min_sum);
}

bool scalar_backend_forced() noexcept {
    const char* backend = std::getenv(LCQI_Q4K_BACKEND_ENV);
    if (backend != nullptr && std::strcmp(backend, LCQI_Q4K_BACKEND_SCALAR) == 0) {
        return true;
    }
    const char* force_scalar = std::getenv(LCQI_Q4K_FORCE_SCALAR_ENV);
    return force_scalar != nullptr && std::strcmp(force_scalar, LCQI_Q4K_FORCE_TRUE) == 0;
}

}  // namespace

bool q4_k_q8_avx2_available() noexcept {
    if (scalar_backend_forced()) {
        return false;
    }
#if defined(__GNUC__) || defined(__clang__)
    __builtin_cpu_init();
    return __builtin_cpu_supports("avx2");
```

```
#else
    return true;
#endif
}

const char* q4_k_q8_active_backend() noexcept {
    return q4_k_q8_avx2_available() ? "avx2_maddubs" : "scalar";
}

void matvec_q4_k_q8_avx2_unchecked(std::span<const std::uint8_t> q4_rows,
                                   std::int64_t row_count,
                                   std::int64_t column_count,
                                   std::span<const Q8KBlock> input,
                                   std::span<float> output) {
    const std::int64_t row_blocks = column_count / LCQI_QK_K_BLOCK_VALUES;
    const std::int64_t row_bytes = row_blocks * LCQI_Q4_K_BLOCK_BYTES;
    for (std::int64_t row = 0; row < row_count; ++row) {
        float sum = 0.0F;
        const std::uint8_t* row_data = q4_rows.data() + row * row_bytes;
        for (std::int64_t block = 0; block < row_blocks; ++block) {
            sum += dot_block_q4_k_q8_avx2(
                row_data + block * LCQI_Q4_K_BLOCK_BYTES,
                input[static_cast<std::size_t>(block)]);
        }
        output[static_cast<std::size_t>(row)] = sum;
    }
}

}  // namespace lcqi

#endif
```

外部模型 Smoke 与对比脚本

`run_smollm2_small_smoke.py` 下载并校验 SmolLM2-135M-Instruct 的 GGUF 量化文件，构建固定 commit 的 `llama-simple-chat`，在 CPU 上运行一次真实开源 small 模型生成，并输出可复查报告。它是外部真实模型 smoke，不替代 LCQI tiny golden trace。

Listing 16.14 tools/run_smollm2_small_smoke.py

```python
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import datetime as dt
import hashlib
import os
import re
import shlex
import shutil
import subprocess
import sys
```

```python
from dataclasses import dataclass
from pathlib import Path


MODEL_SOURCE_ID = "HuggingFaceTB/SmolLM2-135M-Instruct"
MODEL_GGUF_REPO_ID = "bartowski/SmolLM2-135M-Instruct-GGUF"
MODEL_FILE_NAME = "SmolLM2-135M-Instruct-Q4_K_M.gguf"
MODEL_URL = (
    "https://huggingface.co/bartowski/SmolLM2-135M-Instruct-GGUF"
    f"/resolve/main/{MODEL_FILE_NAME}"
)
MODEL_EXPECTED_BYTES = 105454432
MODEL_EXPECTED_SHA256 = (
    "2e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d"
)
LLAMA_CPP_REPO_URL = "https://github.com/ggml-org/llama.cpp.git"
LLAMA_CPP_COMMIT = "b3fed31b99f9bd37725833674252bccb429bb183"
LLAMA_TARGET = "llama-simple-chat"
DEFAULT_PROMPT = "What is 2+3? Answer in one short sentence."
DEFAULT_CONTEXT_TOKENS = 512
DEFAULT_TIMEOUT_SECONDS = 180
DEFAULT_BUILD_JOBS = 2


@dataclass(frozen=True)
class CommandResult:
    args: list[str]
    returncode: int
    stdout: str
    stderr: str


def script_path() -> Path:
    return Path(__file__).resolve()


def volume_dir() -> Path:
    return script_path().parents[1]


def repo_dir() -> Path:
    return script_path().parents[3]


def default_cache_dir() -> Path:
    explicit_cache = os.environ.get("LCQI_SMOLLM2_CACHE_DIR")
    if explicit_cache:
        return Path(explicit_cache).expanduser()
    xdg_cache = os.environ.get("XDG_CACHE_HOME")
    cache_root = Path(xdg_cache).expanduser() if xdg_cache else Path.home() / ".cache"
    return cache_root / "lcqi-smollm2-small-smoke"
```

```python


def default_report_path() -> Path:
    return volume_dir() / "results" / "lcqi-smollm2-small-model-smoke.txt"


def command_line(args: list[str]) -> str:
    return " ".join(shlex.quote(arg) for arg in args)


def require_tool(name: str) -> str:
    path = shutil.which(name)
    if path is None:
        raise RuntimeError(f"required tool not found in PATH: {name}")
    return path


def run_command(
    args: list[str],
    *,
    cwd: Path | None = None,
    input_text: str | None = None,
    timeout_seconds: int | None = None,
) -> CommandResult:
    try:
        completed = subprocess.run(
            args,
            cwd=str(cwd) if cwd else None,
            input=input_text,
            text=True,
            capture_output=True,
            timeout=timeout_seconds,
            check=False,
        )
    except subprocess.TimeoutExpired as exc:
        stdout = exc.stdout if isinstance(exc.stdout, str) else ""
        stderr = exc.stderr if isinstance(exc.stderr, str) else ""
        return CommandResult(args=args, returncode=124, stdout=stdout, stderr=stderr)

    return CommandResult(
        args=args,
        returncode=completed.returncode,
        stdout=completed.stdout,
        stderr=completed.stderr,
    )


def ensure_success(result: CommandResult, action: str) -> None:
    if result.returncode == 0:
        return
    raise RuntimeError(
```

```
        f"{action} failed with exit code {result.returncode}\n"
        f"command: {command_line(result.args)}\n"
        f"stdout tail:\n{tail(result.stdout)}\n"
        f"stderr tail:\n{tail(result.stderr)}"
    )


def tail(text: str, max_chars: int = 4000) -> str:
    if len(text) <= max_chars:
        return text
    return text[-max_chars:]


def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for block in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def verify_model_file(path: Path) -> tuple[bool, str, int]:
    if not path.exists():
        return False, "", 0
    actual_bytes = path.stat().st_size
    actual_sha = sha256_file(path)
    return (
        actual_bytes == MODEL_EXPECTED_BYTES
        and actual_sha == MODEL_EXPECTED_SHA256,
        actual_sha,
        actual_bytes,
    )


def download_model(model_path: Path, *, force_download: bool, no_download: bool
    ↪ ) -> None:
    model_path.parent.mkdir(parents=True, exist_ok=True)

    if force_download and model_path.exists():
        model_path.unlink()

    is_valid, actual_sha, actual_bytes = verify_model_file(model_path)
    if is_valid:
        print(f"[lcqi-smoke] model cache hit: {model_path}")
        return

    if model_path.exists():
        print(
            "[lcqi-smoke] cached model hash mismatch, redownloading: "
            f"bytes={actual_bytes}, sha256={actual_sha}"
        )
        model_path.unlink()
```

```python

    if no_download:
        raise RuntimeError(
            f"model file is missing or invalid and --no-download was set: {model_path}"
        )

    require_tool("curl")
    part_path = model_path.with_suffix(model_path.suffix + ".part")
    if part_path.exists():
        part_path.unlink()

    print(f"[lcqi-smoke] downloading small GGUF model: {MODEL_GGUF_REPO_ID}/{MODEL_FILE_NAME}")
    result = run_command(
        [
            "curl",
            "-L",
            "--fail",
            "--retry",
            "3",
            "-o",
            str(part_path),
            MODEL_URL,
        ]
    )
    ensure_success(result, "download SmolLM2 GGUF")

    downloaded_ok, downloaded_sha, downloaded_bytes = verify_model_file(part_path)
    if not downloaded_ok:
        raise RuntimeError(
            "downloaded model did not match expected identity: "
            f"bytes={downloaded_bytes}, sha256={downloaded_sha}"
        )
    part_path.replace(model_path)
    print(f"[lcqi-smoke] model verified: sha256={MODEL_EXPECTED_SHA256}")


def ensure_llama_simple_chat(cache_dir: Path, *, jobs: int, skip_build: bool) -> Path:
    cached_binary = cache_dir / "llama.cpp" / "build" / "bin" / LLAMA_TARGET
    source_dir = cache_dir / "llama.cpp"
    build_dir = source_dir / "build"
    if cached_binary.exists() and os.access(cached_binary, os.X_OK) and is_pinned_llama_checkout(source_dir):
        print(f"[lcqi-smoke] llama.cpp binary cache hit: {cached_binary}")
        return cached_binary

    if skip_build:
        raise RuntimeError(f"llama.cpp binary is missing and --skip-build was set: {cached_binary}")
```

```python

    require_tool("git")
    require_tool("cmake")

    cache_dir.mkdir(parents=True, exist_ok=True)

    if not (source_dir / ".git").exists():
        print(f"[lcqi-smoke] cloning llama.cpp into cache: {source_dir}")
        result = run_command(
            [
                "git",
                "clone",
                "--filter=blob:none",
                "--no-checkout",
                LLAMA_CPP_REPO_URL,
                str(source_dir),
            ]
        )
        ensure_success(result, "clone llama.cpp")

    print(f"[lcqi-smoke] checking out pinned llama.cpp commit: {LLAMA_CPP_COMMIT}")
    fetch = run_command(
        ["git", "-C", str(source_dir), "fetch", "--depth", "1", "origin", LLAMA_CPP_COMMIT]
    )
    ensure_success(fetch, "fetch pinned llama.cpp commit")
    checkout = run_command(["git", "-C", str(source_dir), "checkout", "--detach", LLAMA_CPP_COMMIT])
    ensure_success(checkout, "checkout pinned llama.cpp commit")

    print(f"[lcqi-smoke] configuring llama.cpp build: {build_dir}")
    configure = run_command(
        [
            "cmake",
            "-S",
            str(source_dir),
            "-B",
            str(build_dir),
            "-DCMAKE_BUILD_TYPE=Release",
            "-DLLAMA_BUILD_TESTS=OFF",
            "-DLLAMA_BUILD_EXAMPLES=ON",
            "-DLLAMA_BUILD_SERVER=OFF",
            "-DGGML_NATIVE=OFF",
        ]
    )
    ensure_success(configure, "configure llama.cpp")

    print(f"[lcqi-smoke] building {LLAMA_TARGET} with -j{jobs}")
    build = run_command(
        ["cmake", "--build", str(build_dir), "--target", LLAMA_TARGET, "-j", str(jobs)]
```

```python
    )
    ensure_success(build, f"build {LLAMA_TARGET}")

    if not cached_binary.exists():
        raise RuntimeError(f"expected binary was not produced: {cached_binary}")
    return cached_binary


def is_pinned_llama_checkout(source_dir: Path) -> bool:
    if not (source_dir / ".git").exists():
        return False
    result = run_command(["git", "-C", str(source_dir), "rev-parse", "HEAD"])
    if result.returncode != 0:
        return False
    return result.stdout.strip() == LLAMA_CPP_COMMIT


def strip_ansi(text: str) -> str:
    return re.sub(r"\x1b\[[0-?]*[ -/]*[@-~]", "", text)


def extract_response(stdout: str) -> str:
    cleaned = strip_ansi(stdout).replace("\r", "")
    chunks = [chunk.strip() for chunk in re.split(r"(?:^|\n)> ?", cleaned) if chunk.strip()]
    if not chunks:
        return ""
    return chunks[0]


def run_model_smoke(
    llama_binary: Path,
    model_path: Path,
    *,
    prompt: str,
    context_tokens: int,
    timeout_seconds: int,
) -> tuple[CommandResult, str]:
    command = [
        str(llama_binary),
        "-m",
        str(model_path),
        "-ngl",
        "0",
        "-c",
        str(context_tokens),
    ]
    print("[lcqi-smoke] running real small model generation")
    result = run_command(
        command,
        input_text=prompt.rstrip("\n") + "\n\n",
```

```python
        timeout_seconds=timeout_seconds,
    )
    ensure_success(result, "run small model smoke")

    response = extract_response(result.stdout)
    if not response:
        raise RuntimeError(
            "small model smoke exited successfully but produced no assistant text\n"
            f"stdout:\n{result.stdout}\n"
            f"stderr:\n{result.stderr}"
        )
    return result, response


def current_repo_commit() -> str:
    result = run_command(["git", "-C", str(repo_dir()), "rev-parse", "--short", "HEAD"])
    if result.returncode != 0:
        return "unknown"
    return result.stdout.strip()


def current_uname() -> str:
    result = run_command(["uname", "-a"])
    if result.returncode != 0:
        return "unknown"
    return result.stdout.strip()


def write_report(
    report_path: Path,
    *,
    cache_dir: Path,
    llama_binary: Path,
    model_path: Path,
    prompt: str,
    result: CommandResult,
    response: str,
) -> None:
    report_path.parent.mkdir(parents=True, exist_ok=True)
    clean_stdout = strip_ansi(result.stdout).strip()
    report = "\n".join(
        [
            "LCQI SmolLM2 Small Model Smoke",
            "",
            f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
            f"repo_commit={current_repo_commit()}",
            f"host={current_uname()}",
            f"python={sys.version.split()[0]}",
            f"cache_dir={cache_dir}",
            "",
```

```
            f"model_source_id={MODEL_SOURCE_ID}",
            f"model_gguf_repo_id={MODEL_GGUF_REPO_ID}",
            f"model_file={MODEL_FILE_NAME}",
            f"model_url={MODEL_URL}",
            f"model_expected_bytes={MODEL_EXPECTED_BYTES}",
            f"model_expected_sha256={MODEL_EXPECTED_SHA256}",
            f"model_path={model_path}",
            "",
            f"llama_cpp_repo={LLAMA_CPP_REPO_URL}",
            f"llama_cpp_commit={LLAMA_CPP_COMMIT}",
            f"llama_target={LLAMA_TARGET}",
            f"llama_binary={llama_binary}",
            "",
            f"prompt={prompt}",
            f"command={command_line(result.args)}",
            f"exit_code={result.returncode}",
            "",
            "validation=PASS: model hash matched, llama-simple-chat exited 0, "
            "assistant response was non-empty",
            "",
            "assistant_response:",
            response,
            "",
            "stdout_clean_tail:",
            tail(clean_stdout, 2000),
            "",
            "stderr_tail:",
            tail(result.stderr.strip(), 2000),
            "",
        ]
    )
    report_path.write_text(report, encoding="utf-8")


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description=(
            "Run a real local small open-source model smoke for CPU Volume 3. "
            "The script caches llama.cpp and the GGUF model outside the Git repository."
        )
    )
    parser.add_argument("--cache-dir", type=Path, default=default_cache_dir())
    parser.add_argument("--model-path", type=Path, default=None)
    parser.add_argument("--llama-bin", type=Path, default=None)
    parser.add_argument("--report", type=Path, default=default_report_path())
    parser.add_argument("--prompt", default=DEFAULT_PROMPT)
    parser.add_argument("--ctx", type=int, default=DEFAULT_CONTEXT_TOKENS)
    parser.add_argument("--timeout", type=int, default=DEFAULT_TIMEOUT_SECONDS)
    parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
    parser.add_argument("--force-download", action="store_true")
    parser.add_argument("--no-download", action="store_true")
```

```python
    parser.add_argument("--skip-build", action="store_true")
    return parser.parse_args()


def main() -> int:
    args = parse_args()
    cache_dir = args.cache_dir.expanduser().resolve()
    model_path = (
        args.model_path.expanduser().resolve()
        if args.model_path
        else cache_dir / "models" / MODEL_FILE_NAME
    )
    report_path = args.report.expanduser().resolve()

    try:
        download_model(
            model_path,
            force_download=args.force_download,
            no_download=args.no_download,
        )
        if args.llama_bin:
            llama_binary = args.llama_bin.expanduser().resolve()
            if not llama_binary.exists() or not os.access(llama_binary, os.X_OK):
                raise RuntimeError(f"--llama-bin is not executable: {llama_binary}")
        else:
            llama_binary = ensure_llama_simple_chat(
                cache_dir,
                jobs=args.jobs,
                skip_build=args.skip_build,
            )

        result, response = run_model_smoke(
            llama_binary,
            model_path,
            prompt=args.prompt,
            context_tokens=args.ctx,
            timeout_seconds=args.timeout,
        )
        write_report(
            report_path,
            cache_dir=cache_dir,
            llama_binary=llama_binary,
            model_path=model_path,
            prompt=args.prompt,
            result=result,
            response=response,
        )
    except RuntimeError as exc:
        print(f"[lcqi-smoke] ERROR: {exc}", file=sys.stderr)
        return 1
```

```python

    print(f"report={report_path}")
    print(f"model_sha256={MODEL_EXPECTED_SHA256}")
    print(f"assistant_response={response}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

`run_smollm2_q4_k_benchmark.py` 复用同一个 SmolLM2 GGUF 缓存，构建 LCQI 的 `lcqi_gguf_q4_bench`，并可附带 llama.cpp `llama-bench` 端到端参考。它会先用 `LCQI_Q4K_BACKEND=scalar` 跑标量路径，再跑 auto 后端，因此报告里能直接看到 `avx2_maddubs` 相对 scalar 的提升。报告要同时说明模型 SHA-256、tensor 名、shape、量化字节、f32 解码字节、误差、checksum 和成熟引擎参考，避免把不同层级数字混在一起。

Listing 16.15 tools/run_smollm2_q4_k_benchmark.py

```python
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import datetime as dt
import os
import re
import shlex
import subprocess
import sys
from dataclasses import dataclass
from pathlib import Path

import run_smollm2_small_smoke


DEFAULT_BUILD_JOBS = 2
DEFAULT_REPEAT = 20
DEFAULT_TIMEOUT_SECONDS = 300
BENCH_TARGET = "lcqi_gguf_q4_bench"


@dataclass(frozen=True)
class CommandResult:
    name: str
    args: list[str]
    returncode: int
    stdout: str
    stderr: str
    metrics: dict[str, str]


def volume_dir() -> Path:
    return Path(__file__).resolve().parents[1]

```

```python

def repo_dir() -> Path:
    return Path(__file__).resolve().parents[3]


def default_build_dir() -> Path:
    return volume_dir() / "build" / "lcqi-release"


def default_report_path() -> Path:
    return volume_dir() / "results" / "lcqi-smollm2-q4-k-benchmark.txt"


def command_line(args: list[str]) -> str:
    return " ".join(shlex.quote(arg) for arg in args)


def run_command(
    name: str,
    args: list[str],
    *,
    timeout_seconds: int,
    cwd: Path | None = None,
    env: dict[str, str] | None = None,
) -> CommandResult:
    command_env = os.environ.copy()
    if env:
        command_env.update(env)
    try:
        completed = subprocess.run(
            args,
            cwd=str(cwd) if cwd else str(repo_dir()),
            env=command_env,
            text=True,
            capture_output=True,
            timeout=timeout_seconds,
            check=False,
        )
    except subprocess.TimeoutExpired as exc:
        stdout = exc.stdout if isinstance(exc.stdout, str) else ""
        stderr = exc.stderr if isinstance(exc.stderr, str) else ""
        return CommandResult(name, args, 124, stdout, stderr, {})
    return CommandResult(
        name=name,
        args=args,
        returncode=completed.returncode,
        stdout=completed.stdout,
        stderr=completed.stderr,
        metrics=parse_key_value_metrics(completed.stdout),
    )


```

```python
def ensure_success(result: CommandResult, action: str) -> None:
    if result.returncode == 0:
        return
    raise RuntimeError(
        f"{action} failed with exit code {result.returncode}\n"
        f"command: {command_line(result.args)}\n"
        f"stdout tail:\n{tail(result.stdout)}\n"
        f"stderr tail:\n{tail(result.stderr)}"
    )


def tail(text: str, max_chars: int = 4000) -> str:
    return text if len(text) <= max_chars else text[-max_chars:]


def parse_key_value_metrics(stdout: str) -> dict[str, str]:
    metrics: dict[str, str] = {}
    for line in stdout.splitlines():
        key, sep, value = line.partition("=")
        if sep:
            metrics[key.strip()] = value.strip()
    return metrics


def parse_llama_bench_metrics(stdout: str) -> dict[str, str]:
    metrics: dict[str, str] = {}
    for line in stdout.splitlines():
        stripped = line.strip()
        if not stripped.startswith("|"):
            continue
        columns = [column.strip() for column in stripped.strip("|").split("|")]
        if len(columns) != 7 or columns[0] == "model" or columns[0].startswith("-"):
            continue
        metrics["llama_model"] = columns[0]
        metrics["llama_size"] = columns[1]
        metrics["llama_params"] = columns[2]
        metrics["llama_backend"] = columns[3]
        metrics["llama_threads"] = columns[4]
        tokens_per_second = re.sub(r"\s+.*$", "", columns[6])
        if columns[5].startswith("pp"):
            metrics["llama_prefill_test"] = columns[5]
            metrics["llama_prefill_tps"] = tokens_per_second
        if columns[5].startswith("tg"):
            metrics["llama_decode_test"] = columns[5]
            metrics["llama_decode_tps"] = tokens_per_second
    return metrics


def build_lcqi(build_dir: Path, jobs: int, timeout_seconds: int) -> Path:
    configure = run_command(
        "cmake_configure",
```

```
        [
            "cmake",
            "-S",
            str(volume_dir()),
            "-B",
            str(build_dir),
            "-DCMAKE_BUILD_TYPE=Release",
        ],
        timeout_seconds=timeout_seconds,
    )
    ensure_success(configure, "configure LCQI")
    build = run_command(
        "cmake_build_lcqi_gguf_q4_bench",
        [
            "cmake",
            "--build",
            str(build_dir),
            "--target",
            BENCH_TARGET,
            "-j",
            str(jobs),
        ],
        timeout_seconds=timeout_seconds,
    )
    ensure_success(build, "build LCQI Q4_K benchmark")
    binary = build_dir / "labs" / "linux_cpu_inference" / BENCH_TARGET
    if not binary.exists() or not os.access(binary, os.X_OK):
        raise RuntimeError(f"LCQI Q4_K benchmark binary not found: {binary}")
    return binary


def llama_bench_binary(cache_dir: Path) -> Path:
    return cache_dir / "llama.cpp" / "build" / "bin" / "llama-bench"


def model_path(cache_dir: Path) -> Path:
    return cache_dir / "models" / run_smollm2_small_smoke.MODEL_FILE_NAME


def run_lcqi_benchmark(
    binary: Path,
    model: Path,
    *,
    name: str,
    tensor: str,
    repeat: int,
    timeout_seconds: int,
    env: dict[str, str] | None = None,
) -> CommandResult:
    return run_command(
        name,
        [str(binary), str(model), tensor, str(repeat)],
```

```
        timeout_seconds=timeout_seconds,
        env=env,
    )


def run_llama_bench(cache_dir: Path, timeout_seconds: int) -> CommandResult | None:
    binary = llama_bench_binary(cache_dir)
    model = model_path(cache_dir)
    if not binary.exists() or not os.access(binary, os.X_OK) or not model.exists():
        return None
    result = run_command(
        "llama_cpp_smollm2_q4_k_m",
        [
            str(binary),
            "-m",
            str(model),
            "-ngl",
            "0",
            "-p",
            "5",
            "-n",
            "4",
            "-r",
            "1",
        ],
        timeout_seconds=timeout_seconds,
    )
    return CommandResult(
        result.name,
        result.args,
        result.returncode,
        result.stdout,
        result.stderr,
        parse_llama_bench_metrics(result.stdout),
    )


def current_commit() -> str:
    result = subprocess.run(
        ["git", "-C", str(repo_dir()), "rev-parse", "--short", "HEAD"],
        text=True,
        capture_output=True,
        check=False,
    )
    return result.stdout.strip() if result.returncode == 0 else "unknown"


def working_tree_dirty() -> str:
    result = subprocess.run(
        ["git", "-C", str(repo_dir()), "status", "--porcelain"],
```

```python
        text=True,
        capture_output=True,
        check=False,
    )
    if result.returncode != 0:
        return "unknown"
    return "true" if result.stdout.strip() else "false"


def write_report(path: Path, runs: list[CommandResult], *, model: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    model_matches, actual_sha256, actual_bytes = run_smollm2_small_smoke.verify_model_file(model)
    lines = [
        "LCQI SmolLM2 Q4_K Benchmark",
        "",
        f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
        f"repo_commit={current_commit()}",
        f"working_tree_dirty={working_tree_dirty()}",
        f"model={model}",
        f"model_source={run_smollm2_small_smoke.MODEL_GGUF_REPO_ID}",
        f"model_file={run_smollm2_small_smoke.MODEL_FILE_NAME}",
        f"model_expected_bytes={run_smollm2_small_smoke.MODEL_EXPECTED_BYTES}",
        f"model_actual_bytes={actual_bytes}",
        f"model_expected_sha256={run_smollm2_small_smoke.MODEL_EXPECTED_SHA256}",
        f"model_actual_sha256={actual_sha256}",
        f"model_identity_match={str(model_matches).lower()}",
        f"llama_cpp_commit={run_smollm2_small_smoke.LLAMA_CPP_COMMIT}",
        "",
        "scope=LCQI reads the same SmolLM2 Q4_K_M GGUF used by llama.cpp and runs a "
        "single real Q4_K tensor matvec slice. llama.cpp numbers are end-to-end mature "
        "engine context, not same-function microkernel parity.",
        "",
    ]
    for run in runs:
        lines.extend(
            [
                f"[{run.name}]",
                f"returncode={run.returncode}",
                f"command={command_line(run.args)}",
            ]
        )
        for key in sorted(run.metrics):
            lines.append(f"{key}={run.metrics[key]}")
        lines.extend(
            [
                "stdout_tail:",
                tail(run.stdout.strip(), 2400),
```

```python
                "stderr_tail:",
                tail(run.stderr.strip(), 2400),
                "",
            ]
        )
    scalar = next((run for run in runs if run.name == "lcqi_smollm2_q4_k_scalar"), None)
    auto = next((run for run in runs if run.name == "lcqi_smollm2_q4_k_auto"), None)
    if scalar is not None and auto is not None:
        scalar_matvec = metric_as_float(scalar.metrics, "q4_q8_matvec_average_us")
        auto_matvec = metric_as_float(auto.metrics, "q4_q8_matvec_average_us")
        scalar_total = metric_as_float(scalar.metrics, "q4_q8_total_average_us")
        auto_total = metric_as_float(auto.metrics, "q4_q8_total_average_us")
        if scalar_matvec is not None and auto_matvec is not None and auto_matvec > 0.0:
            lines.append("[lcqi_auto_vs_scalar]")
            lines.append(f"q4_q8_matvec_speedup_auto_vs_scalar={scalar_matvec / auto_matvec:.3f}")
            if scalar_total is not None and auto_total is not None and auto_total > 0.0:
                lines.append(f"q4_q8_total_speedup_auto_vs_scalar={scalar_total / auto_total:.3f}")
            lines.append("")
    while lines and lines[-1] == "":
        lines.pop()
    path.write_text("\n".join(lines) + "\n", encoding="utf-8")


def metric_as_float(metrics: dict[str, str], key: str) -> float | None:
    value = metrics.get(key)
    if value is None:
        return None
    try:
        return float(value)
    except ValueError:
        return None


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Benchmark LCQI Q4_K matvec on the same SmolLM2 GGUF used by llama.cpp."
    )
    parser.add_argument("--cache-dir", type=Path, default=run_smollm2_small_smoke.default_cache_dir())
    parser.add_argument("--build-dir", type=Path, default=default_build_dir())
    parser.add_argument("--report", type=Path, default=default_report_path())
    parser.add_argument("--tensor", default="auto")
    parser.add_argument("--repeat", type=int, default=DEFAULT_REPEAT)
```

```python
    parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
    parser.add_argument("--timeout-seconds", type=int, default=DEFAULT_TIMEOUT_SECONDS)
    parser.add_argument("--no-download", action="store_true")
    parser.add_argument("--skip-llama", action="store_true")
    return parser.parse_args()


def main() -> int:
    args = parse_args()
    if args.repeat <= 0:
        print("[lcqi-smollm2-q4] --repeat must be positive", file=sys.stderr)
        return 1
    try:
        cache_dir = args.cache_dir.expanduser().resolve()
        model = model_path(cache_dir)
        run_smollm2_small_smoke.download_model(
            model,
            force_download=False,
            no_download=args.no_download,
        )
        binary = build_lcqi(args.build_dir, args.jobs, args.timeout_seconds)
        runs = [
            run_lcqi_benchmark(
                binary,
                model,
                name="lcqi_smollm2_q4_k_scalar",
                tensor=args.tensor,
                repeat=args.repeat,
                timeout_seconds=args.timeout_seconds,
                env={"LCQI_Q4K_BACKEND": "scalar"},
            ),
            run_lcqi_benchmark(
                binary,
                model,
                name="lcqi_smollm2_q4_k_auto",
                tensor=args.tensor,
                repeat=args.repeat,
                timeout_seconds=args.timeout_seconds,
            )
        ]
        if not args.skip_llama:
            llama = run_llama_bench(cache_dir, args.timeout_seconds)
            if llama is not None:
                runs.append(llama)
        write_report(args.report, runs, model=model)
    except Exception as error:
        print(f"[lcqi-smollm2-q4] error: {error}", file=sys.stderr)
        return 1

    for run in runs:
        print(f"{run.name}: returncode={run.returncode}")
```

```python
        for key in sorted(run.metrics):
            print(f"  {key}={run.metrics[key]}")
    print(f"report={args.report}")
    return 0 if all(run.returncode == 0 for run in runs) else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

`run_smollm2_same_input_compare.py` 是 SmolLM2 同输入对齐脚本。它构建 LCQI 和固定 commit 的 llama.cpp，先用 `LCQI_LLAMA_Q4_DIRECT=0` 跑 LCQI baseline，再跑默认 Q4 direct 路径，再用 `LCQI_LLAMA_GGML_DIRECT=1` 跑 `Q5_0/Q6_K/Q8_0` 实验 direct 路径，最后跑 `llama-simple`、`llama-tokenize` 和 `llama-bench`。报告必须同时看 `same_input_prompt_token_ids_match`、LCQI 自身 A/B、llama.cpp 对比和 hotspot 字段。caw 最新报告显示，默认 Q4 direct 把 `w_down` 热点从 `91.68ms` 降到 `63.54ms`，端到端 prefill 提升 `1.062x`；实验 GGML direct 把 direct 量化字节覆盖率从 `7.68%` 提高到 `70.85%`，但 prefill 从 `370.847ms` 退化到 `1211.710ms`。这正是源码卷要训练的工程判断：覆盖率、内存、局部 kernel 和端到端速度必须分开证明。

Listing 16.16 tools/run_smollm2_same_input_compare.py

```python
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import datetime as dt
import os
import re
import shlex
import statistics
import subprocess
import sys
from dataclasses import dataclass
from pathlib import Path

import run_smollm2_small_smoke


DEFAULT_PROMPT = "Hello"
DEFAULT_MAX_NEW_TOKENS = 2
DEFAULT_ROUNDS = 5
DEFAULT_BUILD_JOBS = 2
DEFAULT_TIMEOUT_SECONDS = 300
LCQI_TARGET = "lcqi_llama_gguf"
LLAMA_SIMPLE_TARGET = "llama-simple"
LLAMA_BENCH_TARGET = "llama-bench"
LLAMA_TOKENIZE_TARGET = "llama-tokenize"
LCQI_Q4_DIRECT_ENV = "LCQI_LLAMA_Q4_DIRECT"
LCQI_GGML_DIRECT_ENV = "LCQI_LLAMA_GGML_DIRECT"
LCQI_Q4_DIRECT_OFF = "0"
LCQI_GGML_DIRECT_ON = "1"
LCQI_SUMMARY_KEYS = [
    "benchmark_load_ms",
    "benchmark_prefill_ms",
```

```python
    "derived_prefill_ms_per_prompt_token",
    "derived_prefill_prompt_tokens_per_second",
    "benchmark_decode_ms",
    "derived_decode_ms_per_step",
    "benchmark_decode_tokens_per_second",
    "derived_f32_weight_inflation_ratio",
    "derived_direct_quantized_weight_byte_share",
    "benchmark_hotspot_rms_norm_ms",
    "benchmark_hotspot_attention_ms",
    "benchmark_hotspot_rope_ms",
    "benchmark_hotspot_wq_ms",
    "benchmark_hotspot_wk_ms",
    "benchmark_hotspot_wv_ms",
    "benchmark_hotspot_wo_ms",
    "benchmark_hotspot_w_gate_ms",
    "benchmark_hotspot_w_up_ms",
    "benchmark_hotspot_w_down_ms",
    "benchmark_hotspot_lm_head_ms",
    "benchmark_hotspot_q4_k_direct_ms",
    "benchmark_hotspot_ggml_direct_ms",
    "benchmark_hotspot_f32_fallback_ms",
    "benchmark_hotspot_q4_k_direct_calls",
    "benchmark_hotspot_ggml_direct_calls",
    "benchmark_hotspot_f32_fallback_calls",
    "benchmark_q4_k_direct_tensors",
    "benchmark_ggml_direct_tensors",
    "benchmark_f32_fallback_tensors",
]


@dataclass(frozen=True)
class CommandResult:
    name: str
    args: list[str]
    returncode: int
    stdout: str
    stderr: str
    metrics: dict[str, str]


def volume_dir() -> Path:
    return Path(__file__).resolve().parents[1]


def repo_dir() -> Path:
    return Path(__file__).resolve().parents[3]


def default_build_dir() -> Path:
    return volume_dir() / "build" / "lcqi-release"


```

```python
def default_report_path() -> Path:
    return volume_dir() / "results" / "lcqi-smollm2-same-input-compare-caw.txt"


def command_line(args: list[str]) -> str:
    return " ".join(shlex.quote(arg) for arg in args)


def tail(text: str, max_chars: int = 4000) -> str:
    return text if len(text) <= max_chars else text[-max_chars:]


def run_command(
    name: str,
    args: list[str],
    *,
    timeout_seconds: int,
    cwd: Path | None = None,
    env: dict[str, str] | None = None,
) -> CommandResult:
    command_env = os.environ.copy()
    if env:
        command_env.update(env)
    try:
        completed = subprocess.run(
            args,
            cwd=str(cwd) if cwd else str(repo_dir()),
            env=command_env,
            text=True,
            capture_output=True,
            timeout=timeout_seconds,
            check=False,
        )
    except subprocess.TimeoutExpired as exc:
        stdout = exc.stdout if isinstance(exc.stdout, str) else ""
        stderr = exc.stderr if isinstance(exc.stderr, str) else ""
        return CommandResult(name, args, 124, stdout, stderr, {})
    return CommandResult(
        name=name,
        args=args,
        returncode=completed.returncode,
        stdout=completed.stdout,
        stderr=completed.stderr,
        metrics={},
    )


def ensure_success(result: CommandResult, action: str) -> None:
    if result.returncode == 0:
        return
    raise RuntimeError(
        f"{action} failed with exit code {result.returncode}\n"
```

```python
        f"command: {command_line(result.args)}\n"
        f"stdout tail:\n{tail(result.stdout)}\n"
        f"stderr tail:\n{tail(result.stderr)}"
    )


def smollm2_chat_prompt(user_prompt: str) -> str:
    return (
        "<|im_start|>system\n"
        "You are a helpful AI assistant named SmolLM, trained by Hugging Face"
        "<|im_end|>\n"
        "<|im_start|>user\n"
        f"{user_prompt}"
        "<|im_end|>\n"
        "<|im_start|>assistant\n"
    )


def escape_newlines(text: str) -> str:
    return text.replace("\\", "\\\\").replace("\n", "\\n")


def parse_int_list(value: str) -> list[int]:
    ids: list[int] = []
    for item in value.replace(",", " ").split():
        ids.append(int(item))
    return ids


def parse_lcqi_metrics(stdout: str) -> dict[str, str]:
    metrics: dict[str, str] = {}
    generated_ids: list[int] = []
    prompt_ids: list[int] = []
    for line in stdout.splitlines():
        key, sep, value = line.partition(" ")
        if not sep:
            continue
        if key in {
            "mode",
            "weight_execution",
            "architecture",
            "model_name",
            "layers",
            "hidden_size",
            "query_heads",
            "kv_heads",
            "head_dim",
            "vocab_size",
            "tie_lm_head_to_embedding",
            "predicted_first_token",
            "benchmark_manifest_ms",
            "benchmark_weight_load_ms",
```

```
            "benchmark_load_ms",
            "benchmark_prefill_ms",
            "benchmark_decode_ms",
            "benchmark_total_ms",
            "benchmark_prompt_tokens",
            "benchmark_generated_tokens",
            "benchmark_prefill_steps",
            "benchmark_decode_steps",
            "benchmark_first_token_tokens_per_second",
            "benchmark_decode_tokens_per_second",
            "benchmark_kv_cache_bytes",
            "benchmark_quantized_weight_bytes",
            "benchmark_f32_weight_bytes",
            "benchmark_direct_quantized_weight_bytes",
            "benchmark_fallback_dequantized_weight_bytes",
            "benchmark_tensors_loaded",
            "benchmark_q4_k_direct_tensors",
            "benchmark_ggml_direct_tensors",
            "benchmark_f32_fallback_tensors",
            "benchmark_hotspot_rms_norm_ms",
            "benchmark_hotspot_attention_ms",
            "benchmark_hotspot_rope_ms",
            "benchmark_hotspot_wq_ms",
            "benchmark_hotspot_wk_ms",
            "benchmark_hotspot_wv_ms",
            "benchmark_hotspot_wo_ms",
            "benchmark_hotspot_w_gate_ms",
            "benchmark_hotspot_w_up_ms",
            "benchmark_hotspot_w_down_ms",
            "benchmark_hotspot_lm_head_ms",
            "benchmark_hotspot_q4_k_direct_ms",
            "benchmark_hotspot_ggml_direct_ms",
            "benchmark_hotspot_f32_fallback_ms",
            "benchmark_hotspot_q4_k_direct_calls",
            "benchmark_hotspot_ggml_direct_calls",
            "benchmark_hotspot_f32_fallback_calls",
        }:
            metrics[key] = value.strip()
        elif key == "prompt_ids":
            prompt_ids = parse_int_list(value)
            metrics["prompt_ids"] = " ".join(str(item) for item in prompt_ids)
            metrics["prompt_token_count_from_ids"] = str(len(prompt_ids))
        elif key == "generated_ids":
            generated_ids = parse_int_list(value)
            metrics["generated_ids"] = " ".join(str(item) for item in generated_ids)
            metrics["generated_token_count_from_ids"] = str(len(generated_ids))
    if prompt_ids and generated_ids and len(generated_ids) >= len(prompt_ids):
        new_ids = generated_ids[len(prompt_ids):]
        metrics["generated_new_ids"] = " ".join(str(item) for item in new_ids)
    add_lcqi_derived_metrics(metrics)
    return metrics
```

```python


def add_lcqi_derived_metrics(metrics: dict[str, str]) -> None:
    prefill_ms = metric_as_float(metrics, "benchmark_prefill_ms")
    prompt_tokens = metric_as_float(metrics, "benchmark_prompt_tokens")
    decode_ms = metric_as_float(metrics, "benchmark_decode_ms")
    decode_steps = metric_as_float(metrics, "benchmark_decode_steps")
    quantized_bytes = metric_as_float(metrics, "benchmark_quantized_weight_bytes")
    f32_bytes = metric_as_float(metrics, "benchmark_f32_weight_bytes")
    direct_bytes = metric_as_float(metrics, "benchmark_direct_quantized_weight_bytes")
    if prefill_ms is not None and prefill_ms > 0.0 and prompt_tokens is not None:
        metrics["derived_prefill_prompt_tokens_per_second"] = (
            f"{prompt_tokens * 1000.0 / prefill_ms:.6f}"
        )
        metrics["derived_prefill_ms_per_prompt_token"] = (
            f"{prefill_ms / prompt_tokens:.6f}"
        )
    if decode_ms is not None and decode_ms > 0.0 and decode_steps is not None:
        metrics["derived_decode_steps_per_second"] = f"{decode_steps * 1000.0 / decode_ms:.6f}"
        metrics["derived_decode_ms_per_step"] = f"{decode_ms / max(decode_steps, 1.0):.6f}"
    if quantized_bytes is not None and quantized_bytes > 0.0 and f32_bytes is not None:
        metrics["derived_f32_weight_inflation_ratio"] = f"{f32_bytes / quantized_bytes:.6f}"
    if quantized_bytes is not None and quantized_bytes > 0.0 and direct_bytes is not None:
        metrics["derived_direct_quantized_weight_byte_share"] = (
            f"{direct_bytes / quantized_bytes:.6f}"
        )


def parse_llama_simple_metrics(stderr: str, stdout: str, full_prompt: str) -> dict[str, str]:
    metrics: dict[str, str] = {}
    parse_llama_perf_metrics(stderr, metrics)
    clean_stdout = strip_ansi(stdout).replace("\r", "")
    metrics["stdout_prompt_prefix_match"] = str(clean_stdout.startswith(full_prompt)).lower()
    if clean_stdout.startswith(full_prompt):
        generated = clean_stdout[len(full_prompt):].strip()
        metrics["generated_suffix"] = escape_newlines(generated)
    decoded_match = re.search(
        r"main:\s+decoded\s+(?P<count>\d+)\s+tokens\s+in\s+(?P<seconds>[0-9.]+)\s+s,"
        r"\s+speed:\s+(?P<tps>[0-9.]+)\s+t/s",
        stderr,
    )
```

```python
    if decoded_match:
        metrics["llama_simple_decoded_tokens"] = decoded_match.group("count")
        metrics["llama_simple_main_seconds"] = decoded_match.group("seconds")
        metrics["llama_simple_main_tokens_per_second"] = decoded_match.group("tps")
    return metrics


def parse_llama_perf_metrics(text: str, metrics: dict[str, str]) -> None:
    load_match = re.search(
        r"llama_(?:perf_context_print|print_timings):\s+load time\s*=\s*"
        r"(?P<ms>[0-9.]+)\s*ms",
        text,
    )
    if load_match:
        metrics["llama_load_ms"] = load_match.group("ms")

    prompt_match = re.search(
        r"llama_(?:perf_context_print|print_timings):\s+prompt eval time\s*=\s*"
        r"(?P<ms>[0-9.]+)\s*ms\s*/\s*(?P<count>\d+)\s+tokens\s*"
        r"\(\s*(?P<ms_per>[0-9.]+)\s*ms per token,\s*"
        r"(?P<tps>[0-9.]+)\s*tokens per second\s*\)",
        text,
    )
    if prompt_match:
        metrics["llama_prompt_eval_ms"] = prompt_match.group("ms")
        metrics["llama_prompt_eval_tokens"] = prompt_match.group("count")
        metrics["llama_prompt_eval_ms_per_token"] = prompt_match.group("ms_per")
        metrics["llama_prompt_eval_tokens_per_second"] = prompt_match.group("tps")

    eval_match = re.search(
        r"llama_(?:perf_context_print|print_timings):\s+eval time\s*=\s*"
        r"(?P<ms>[0-9.]+)\s*ms\s*/\s*(?P<count>\d+)\s+runs\s*"
        r"\(\s*(?P<ms_per>[0-9.]+)\s*ms per token,\s*"
        r"(?P<tps>[0-9.]+)\s*tokens per second\s*\)",
        text,
    )
    if eval_match:
        metrics["llama_eval_ms"] = eval_match.group("ms")
        metrics["llama_eval_runs"] = eval_match.group("count")
        metrics["llama_eval_ms_per_run"] = eval_match.group("ms_per")
        metrics["llama_eval_tokens_per_second"] = eval_match.group("tps")

    total_match = re.search(
        r"llama_(?:perf_context_print|print_timings):\s+total time\s*=\s*"
        r"(?P<ms>[0-9.]+)\s*ms(?:\s*/\s*(?P<count>\d+)\s+tokens)?",
        text,
    )
    if total_match:
```

```python
        metrics["llama_total_ms"] = total_match.group("ms")
        if total_match.group("count"):
            metrics["llama_total_tokens"] = total_match.group("count")


def parse_llama_bench_metrics(stdout: str) -> dict[str, str]:
    metrics: dict[str, str] = {}
    for line in stdout.splitlines():
        stripped = line.strip()
        if not stripped.startswith("|"):
            continue
        columns = [column.strip() for column in stripped.strip("|").split("|")]
        if len(columns) != 7 or columns[0] == "model" or columns[0].startswith("-"):
            continue
        metrics["llama_bench_model"] = columns[0]
        metrics["llama_bench_size"] = columns[1]
        metrics["llama_bench_params"] = columns[2]
        metrics["llama_bench_backend"] = columns[3]
        metrics["llama_bench_threads"] = columns[4]
        test_name = columns[5]
        tokens_per_second = re.sub(r"\s+.*$", "", columns[6])
        if test_name.startswith("pp"):
            metrics["llama_bench_prefill_test"] = test_name
            metrics["llama_bench_prefill_tokens_per_second"] = tokens_per_second
        elif test_name.startswith("tg"):
            metrics["llama_bench_decode_test"] = test_name
            metrics["llama_bench_decode_tokens_per_second"] = tokens_per_second
    return metrics


def parse_llama_tokenize_metrics(stdout: str) -> dict[str, str]:
    metrics: dict[str, str] = {}
    ids_match = re.search(r"\[(?P<ids>[0-9,\s-]+)\]", stdout)
    if ids_match:
        ids = parse_int_list(ids_match.group("ids"))
        metrics["llama_tokenize_ids"] = " ".join(str(item) for item in ids)
        metrics["llama_tokenize_token_count_from_ids"] = str(len(ids))
    count_match = re.search(r"Total number of tokens:\s*(?P<count>\d+)", stdout)
    if count_match:
        metrics["llama_tokenize_show_count"] = count_match.group("count")
    return metrics


def strip_ansi(text: str) -> str:
    return re.sub(r"\x1b\[[0-?]*[ -/]*[@-~]", "", text)


def metric_as_float(metrics: dict[str, str], key: str) -> float | None:
    value = metrics.get(key)
```

```python
    if value is None:
        return None
    try:
        return float(value)
    except ValueError:
        return None


def metric_values(runs: list[CommandResult], key: str) -> list[float]:
    values: list[float] = []
    for run in runs:
        value = metric_as_float(run.metrics, key)
        if value is not None:
            values.append(value)
    return values


def median_metric(runs: list[CommandResult], key: str) -> float | None:
    values = metric_values(runs, key)
    return statistics.median(values) if values else None


def format_summary(name: str, runs: list[CommandResult], keys: list[str]) -> ↩
    ↪ list[str]:
    lines = [f"[summary:{name}]", f"rounds={len(runs)}"]
    for key in keys:
        values = metric_values(runs, key)
        if not values:
            continue
        lines.append(
            f"{key}_median={statistics.median(values):.6f} "
            f"{key}_min={min(values):.6f} "
            f"{key}_max={max(values):.6f}"
        )
    return lines


def build_lcqi(build_dir: Path, jobs: int, timeout_seconds: int) -> Path:
    configure = run_command(
        "cmake_configure_lcqi",
        [
            "cmake",
            "-S",
            str(volume_dir()),
            "-B",
            str(build_dir),
            "-DCMAKE_BUILD_TYPE=Release",
        ],
        timeout_seconds=timeout_seconds,
    )
    ensure_success(configure, "configure LCQI")
    build = run_command(
```

```python
        "cmake_build_lcqi_llama_gguf",
        ["cmake", "--build", str(build_dir), "--target", LCQI_TARGET, "-j", str(jobs)],
        timeout_seconds=timeout_seconds,
    )
    ensure_success(build, "build LCQI llama GGUF target")
    binary = build_dir / "labs" / "linux_cpu_inference" / LCQI_TARGET
    if not binary.exists() or not os.access(binary, os.X_OK):
        raise RuntimeError(f"LCQI binary not found or not executable: {binary}")
    return binary


def ensure_llama_targets(
    cache_dir: Path,
    *,
    jobs: int,
    skip_build: bool,
    timeout_seconds: int,
) -> tuple[Path, Path, Path]:
    run_smollm2_small_smoke.ensure_llama_simple_chat(
        cache_dir,
        jobs=jobs,
        skip_build=skip_build,
    )
    build_dir = cache_dir / "llama.cpp" / "build"
    binaries = {
        LLAMA_SIMPLE_TARGET: build_dir / "bin" / LLAMA_SIMPLE_TARGET,
        LLAMA_BENCH_TARGET: build_dir / "bin" / LLAMA_BENCH_TARGET,
        LLAMA_TOKENIZE_TARGET: build_dir / "bin" / LLAMA_TOKENIZE_TARGET,
    }
    for target, binary in binaries.items():
        if binary.exists() and os.access(binary, os.X_OK):
            continue
        if skip_build:
            raise RuntimeError(f"{target} is missing and --skip-llama-build was set: {binary}")
        build = run_command(
            f"cmake_build_{target}",
            ["cmake", "--build", str(build_dir), "--target", target, "-j", str(jobs)],
            timeout_seconds=timeout_seconds,
        )
        ensure_success(build, f"build llama.cpp target {target}")
        if not binary.exists() or not os.access(binary, os.X_OK):
            raise RuntimeError(f"{target} was not produced: {binary}")
    return (
        binaries[LLAMA_SIMPLE_TARGET],
        binaries[LLAMA_BENCH_TARGET],
        binaries[LLAMA_TOKENIZE_TARGET],
    )
```

```

def run_lcqi_round(
    binary: Path,
    model_path: Path,
    *,
    user_prompt: str,
    max_new_tokens: int,
    round_index: int,
    timeout_seconds: int,
    name_prefix: str = "lcqi_round",
    env: dict[str, str] | None = None,
) -> CommandResult:
    command = [
        str(binary),
        str(model_path),
        "--prompt",
        user_prompt,
        "--max-new",
        str(max_new_tokens),
        "--benchmark",
        "--decode-text",
    ]
    result = run_command(
        f"{name_prefix}_{round_index}",
        command,
        timeout_seconds=timeout_seconds,
        env=env,
    )
    return CommandResult(
        result.name,
        result.args,
        result.returncode,
        result.stdout,
        result.stderr,
        parse_lcqi_metrics(result.stdout),
    )


def run_llama_simple_round(
    binary: Path,
    model_path: Path,
    *,
    full_prompt: str,
    max_new_tokens: int,
    round_index: int,
    timeout_seconds: int,
) -> CommandResult:
    command = [
        str(binary),
        "-m",
        str(model_path),
        "-ngl",
```

```
            "0",
            "-n",
            str(max_new_tokens),
            full_prompt,
        ]
        result = run_command(
            f"llama_simple_round_{round_index}",
            command,
            timeout_seconds=timeout_seconds,
            env={"LC_ALL": "C"},
        )
        return CommandResult(
            result.name,
            result.args,
            result.returncode,
            result.stdout,
            result.stderr,
            parse_llama_simple_metrics(result.stderr, result.stdout, full_prompt),
        )


def run_llama_tokenize(
    binary: Path,
    model_path: Path,
    *,
    full_prompt: str,
    timeout_seconds: int,
) -> CommandResult:
    command = [
        str(binary),
        "-m",
        str(model_path),
        "--ids",
        "--show-count",
        "--log-disable",
        "-p",
        full_prompt,
    ]
    result = run_command(
        "llama_tokenize_same_prompt",
        command,
        timeout_seconds=timeout_seconds,
        env={"LC_ALL": "C"},
    )
    return CommandResult(
        result.name,
        result.args,
        result.returncode,
        result.stdout,
        result.stderr,
        parse_llama_tokenize_metrics(result.stdout),
    )
```

```
def run_llama_bench(
    binary: Path,
    model_path: Path,
    *,
    prompt_tokens: int,
    max_new_tokens: int,
    rounds: int,
    timeout_seconds: int,
) -> CommandResult:
    command = [
        str(binary),
        "-m",
        str(model_path),
        "-ngl",
        "0",
        "-p",
        str(prompt_tokens),
        "-n",
        str(max_new_tokens),
        "-r",
        str(rounds),
    ]
    result = run_command(
        "llama_bench_same_token_count",
        command,
        timeout_seconds=timeout_seconds,
        env={"LC_ALL": "C"},
    )
    return CommandResult(
        result.name,
        result.args,
        result.returncode,
        result.stdout,
        result.stderr,
        parse_llama_bench_metrics(result.stdout),
    )


def current_commit() -> str:
    result = subprocess.run(
        ["git", "-C", str(repo_dir()), "rev-parse", "--short", "HEAD"],
        text=True,
        capture_output=True,
        check=False,
    )
    return result.stdout.strip() if result.returncode == 0 else "unknown"


def working_tree_dirty() -> str:
    result = subprocess.run(
```

```python
        ["git", "-C", str(repo_dir()), "status", "--porcelain"],
        text=True,
        capture_output=True,
        check=False,
    )
    if result.returncode != 0:
        return "unknown"
    return "true" if result.stdout.strip() else "false"


def command_output(args: list[str]) -> str:
    completed = subprocess.run(args, text=True, capture_output=True, check=False)
    if completed.returncode != 0:
        return "unknown"
    return completed.stdout.strip()


def write_report(
    report_path: Path,
    *,
    cache_dir: Path,
    model_path: Path,
    user_prompt: str,
    full_prompt: str,
    max_new_tokens: int,
    lcqi_runs: list[CommandResult],
    lcqi_q4_direct_off_runs: list[CommandResult],
    lcqi_ggml_direct_runs: list[CommandResult],
    llama_tokenize: CommandResult,
    llama_simple_runs: list[CommandResult],
    llama_bench: CommandResult,
) -> None:
    report_path.parent.mkdir(parents=True, exist_ok=True)
    model_matches, actual_sha256, actual_bytes = run_smollm2_small_smoke.verify_model_file(
        model_path
    )
    lines = [
        "LCQI vs llama.cpp SmolLM2 Same Input Compare",
        "",
        f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
        f"repo_commit={current_commit()}",
        f"working_tree_dirty={working_tree_dirty()}",
        f"host={command_output(['hostname'])}",
        f"uname={command_output(['uname', '-a'])}",
        f"nproc={command_output(['nproc'])}",
        f"python={sys.version.split()[0]}",
        f"cache_dir={cache_dir}",
        "",
        f"model_source={run_smollm2_small_smoke.MODEL_GGUF_REPO_ID}",
        f"model_file={run_smollm2_small_smoke.MODEL_FILE_NAME}",
```

```
        f"model_path={model_path}",
        f"model_expected_bytes={run_smollm2_small_smoke.MODEL_EXPECTED_BYTES}",
        f"model_actual_bytes={actual_bytes}",
        f"model_expected_sha256={run_smollm2_small_smoke.MODEL_EXPECTED_SHA256}",
        f"model_actual_sha256={actual_sha256}",
        f"model_identity_match={str(model_matches).lower()}",
        f"llama_cpp_commit={run_smollm2_small_smoke.LLAMA_CPP_COMMIT}",
        "",
        f"user_prompt={escape_newlines(user_prompt)}",
        f"max_new_tokens={max_new_tokens}",
        f"expanded_chat_prompt={escape_newlines(full_prompt)}",
        "",
        "scope=LCQI uses its internal SmolLM2 chat-prompt builder from the same user "
        "prompt. llama-simple receives the expanded chat prompt string directly. "
        "The same-input check is prompt-token-count equality plus matching visible "
        "generated suffix; llama-bench is reported separately as same-token-count "
        "synthetic throughput, not same text.",
        "",
    ]

    if lcqi_q4_direct_off_runs:
        lines.extend(format_summary(
            "lcqi_q4_direct_off_same_input",
            lcqi_q4_direct_off_runs,
            LCQI_SUMMARY_KEYS,
        ))
        lines.append("")

    lines.extend(format_summary(
        "lcqi_same_input",
        lcqi_runs,
        LCQI_SUMMARY_KEYS,
    ))
    lines.append("")
    if lcqi_ggml_direct_runs:
        lines.extend(format_summary(
            "lcqi_ggml_direct_experimental_same_input",
            lcqi_ggml_direct_runs,
            LCQI_SUMMARY_KEYS,
        ))
        lines.append("")
    lines.extend(format_summary(
        "llama_simple_same_input",
        llama_simple_runs,
        [
            "llama_load_ms",
            "llama_prompt_eval_ms",
```

```
            "llama_prompt_eval_ms_per_token",
            "llama_prompt_eval_tokens_per_second",
            "llama_eval_ms",
            "llama_eval_ms_per_run",
            "llama_eval_tokens_per_second",
            "llama_total_ms",
        ],
    ))
    lines.append("")

    add_ratio_summary(
        lines,
        lcqi_runs,
        llama_tokenize,
        llama_simple_runs,
        llama_bench,
        lcqi_q4_direct_off_runs,
        lcqi_ggml_direct_runs,
    )

    for run in (
        lcqi_q4_direct_off_runs +
        lcqi_runs +
        lcqi_ggml_direct_runs +
        [llama_tokenize] +
        llama_simple_runs +
        [llama_bench]
    ):
        lines.extend(
            [
                "",
                f"[{run.name}]",
                f"returncode={run.returncode}",
                f"command={command_line(run.args)}",
            ]
        )
        for key in sorted(run.metrics):
            lines.append(f"{key}={run.metrics[key]}")
        lines.extend(
            [
                "stdout_tail:",
                tail(strip_ansi(run.stdout).strip(), 3000),
                "stderr_tail:",
                tail(strip_ansi(run.stderr).strip(), 3000),
            ]
        )

    while lines and lines[-1] == "":
        lines.pop()
    report_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
```

```python
def add_ratio_summary(
    lines: list[str],
    lcqi_runs: list[CommandResult],
    llama_tokenize: CommandResult,
    llama_simple_runs: list[CommandResult],
    llama_bench: CommandResult,
    lcqi_q4_direct_off_runs: list[CommandResult],
    lcqi_ggml_direct_runs: list[CommandResult],
) -> None:
    lcqi_prompt_tokens = median_metric(lcqi_runs, "benchmark_prompt_tokens")
    lcqi_prompt_ids = lcqi_runs[0].metrics.get("prompt_ids") if lcqi_runs else None
    llama_tokenize_ids = llama_tokenize.metrics.get("llama_tokenize_ids")
    llama_tokenize_count = metric_as_float(llama_tokenize.metrics, "llama_tokenize_show_count")
    llama_prompt_tokens = median_metric(llama_simple_runs, "llama_prompt_eval_tokens")
    lcqi_prefill = median_metric(lcqi_runs, "benchmark_prefill_ms")
    llama_prompt = median_metric(llama_simple_runs, "llama_prompt_eval_ms")
    lcqi_decode_step = median_metric(lcqi_runs, "derived_decode_ms_per_step")
    llama_eval_step = median_metric(llama_simple_runs, "llama_eval_ms_per_run")
    lcqi_weight_ratio = median_metric(lcqi_runs, "derived_f32_weight_inflation_ratio")
    lcqi_direct_share = median_metric(lcqi_runs, "derived_direct_quantized_weight_byte_share")
    bench_pp_tps = metric_as_float(llama_bench.metrics, "llama_bench_prefill_tokens_per_second")
    bench_tg_tps = metric_as_float(llama_bench.metrics, "llama_bench_decode_tokens_per_second")

    lines.extend(["[interpretation]", f"prompt_tokens_lcqi_median={lcqi_prompt_tokens}"])
    lines.append(f"prompt_tokens_llama_tokenize={llama_tokenize_count}")
    lines.append(f"prompt_tokens_llama_simple_median={llama_prompt_tokens}")
    if lcqi_prompt_tokens == llama_prompt_tokens == llama_tokenize_count:
        lines.append("same_input_prompt_token_count_match=true")
    else:
        lines.append("same_input_prompt_token_count_match=false")
    lines.append(
        "same_input_prompt_token_ids_match="
        f"{str(lcqi_prompt_ids == llama_tokenize_ids).lower()}"
    )
    if lcqi_prefill is not None and llama_prompt is not None and llama_prompt > 0.0:
        lines.append(f"prefill_ms_ratio_lcqi_over_llama_simple={lcqi_prefill / llama_prompt:.6f}")
    if lcqi_decode_step is not None and llama_eval_step is not None and llama_eval_step > 0.0:
        lines.append(f"decode_step_ms_ratio_lcqi_over_llama_simple={lcqi_decode_step / llama_eval_step:.6f}")
    if lcqi_weight_ratio is not None:
        lines.append(f"lcqi_f32_dequantized_weight_inflation_ratio={
```

```
lcqi_weight_ratio:.6f}")
    if lcqi_direct_share is not None:
        lines.append(f"lcqi_direct_quantized_weight_byte_share={lcqi_direct_share:.6f}")
    add_lcqi_ab_ratios(lines, lcqi_q4_direct_off_runs, lcqi_runs)
    add_lcqi_ggml_experimental_ratios(lines, lcqi_runs, lcqi_ggml_direct_runs)
    if bench_pp_tps is not None:
        lines.append(f"llama_bench_same_token_count_prefill_tps={bench_pp_tps:.6f}")
    if bench_tg_tps is not None:
        lines.append(f"llama_bench_same_token_count_decode_tps={bench_tg_tps:.6f}")
    lines.append(
        "root_cause=LCQI keeps shape-compatible Q4_K matrices in the default direct "
        "quantized path. The experimental Q5_0/Q6_K/Q8_0 direct path increases "
        "quantized-byte coverage, but its current scalar block kernels are slower "
        "than the dequantized f32 AVX2 fallback, so it is not enabled by default. "
        "llama.cpp batches prompt evaluation, uses ggml graph scheduling, tuned "
        "low-bit CPU kernels, prefetch/thread scheduling, and more compact "
        "KV/cache execution."
    )


def add_lcqi_ab_ratios(
    lines: list[str],
    lcqi_q4_direct_off_runs: list[CommandResult],
    lcqi_runs: list[CommandResult],
) -> None:
    if not lcqi_q4_direct_off_runs:
        return
    off_prefill = median_metric(lcqi_q4_direct_off_runs, "benchmark_prefill_ms")
    on_prefill = median_metric(lcqi_runs, "benchmark_prefill_ms")
    off_decode = median_metric(lcqi_q4_direct_off_runs, "derived_decode_ms_per_step")
    on_decode = median_metric(lcqi_runs, "derived_decode_ms_per_step")
    off_down = median_metric(lcqi_q4_direct_off_runs, "benchmark_hotspot_w_down_ms")
    on_down = median_metric(lcqi_runs, "benchmark_hotspot_w_down_ms")
    off_f32_bytes = median_metric(lcqi_q4_direct_off_runs, "benchmark_f32_weight_bytes")
    on_f32_bytes = median_metric(lcqi_runs, "benchmark_f32_weight_bytes")
    if off_prefill is not None and on_prefill is not None and on_prefill > 0.0:
        lines.append(f"lcqi_q4_direct_prefill_speedup_off_over_on={off_prefill / on_prefill:.6f}")
    if off_decode is not None and on_decode is not None and on_decode > 0.0:
        lines.append(f"lcqi_q4_direct_decode_step_speedup_off_over_on={
```

```python
off_decode / on_decode:.6f}")
    if off_down is not None and on_down is not None and on_down > 0.0:
        lines.append(f"lcqi_q4_direct_w_down_speedup_off_over_on={off_down / on_down:.6f}")
    if off_f32_bytes is not None and on_f32_bytes is not None:
        lines.append(f"lcqi_q4_direct_f32_weight_bytes_saved={off_f32_bytes - on_f32_bytes:.0f}")


def add_lcqi_ggml_experimental_ratios(
    lines: list[str],
    lcqi_runs: list[CommandResult],
    lcqi_ggml_direct_runs: list[CommandResult],
) -> None:
    if not lcqi_ggml_direct_runs:
        return
    default_prefill = median_metric(lcqi_runs, "benchmark_prefill_ms")
    ggml_prefill = median_metric(lcqi_ggml_direct_runs, "benchmark_prefill_ms")
    default_decode = median_metric(lcqi_runs, "derived_decode_ms_per_step")
    ggml_decode = median_metric(lcqi_ggml_direct_runs, "derived_decode_ms_per_step")
    default_direct_share = median_metric(lcqi_runs, "derived_direct_quantized_weight_byte_share")
    ggml_direct_share = median_metric(lcqi_ggml_direct_runs, "derived_direct_quantized_weight_byte_share")
    ggml_hotspot = median_metric(lcqi_ggml_direct_runs, "benchmark_hotspot_ggml_direct_ms")
    if default_prefill is not None and ggml_prefill is not None and ggml_prefill > 0.0:
        lines.append(
            "lcqi_ggml_experimental_prefill_speedup_default_over_experimental="
            f"{default_prefill / ggml_prefill:.6f}"
        )
    if default_decode is not None and ggml_decode is not None and ggml_decode > 0.0:
        lines.append(
            "lcqi_ggml_experimental_decode_step_speedup_default_over_experimental="
            f"{default_decode / ggml_decode:.6f}"
        )
    if default_direct_share is not None and ggml_direct_share is not None:
        lines.append(
            "lcqi_ggml_experimental_direct_share_delta="
            f"{ggml_direct_share - default_direct_share:.6f}"
        )
    if ggml_hotspot is not None:
        lines.append(f"lcqi_ggml_experimental_hotspot_ms={ggml_hotspot:.6f}")


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Run same-input SmolLM2 comparison between LCQI and llama.
```

```python
cpp."
    )
    parser.add_argument("--cache-dir", type=Path, default=run_smollm2_small_smoke.default_cache_dir())
    parser.add_argument("--model-path", type=Path, default=None)
    parser.add_argument("--build-dir", type=Path, default=default_build_dir())
    parser.add_argument("--report", type=Path, default=default_report_path())
    parser.add_argument("--prompt", default=DEFAULT_PROMPT)
    parser.add_argument("--max-new", type=int, default=DEFAULT_MAX_NEW_TOKENS)
    parser.add_argument("--rounds", type=int, default=DEFAULT_ROUNDS)
    parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
    parser.add_argument("--timeout-seconds", type=int, default=DEFAULT_TIMEOUT_SECONDS)
    parser.add_argument("--force-download", action="store_true")
    parser.add_argument("--no-download", action="store_true")
    parser.add_argument("--skip-llama-build", action="store_true")
    parser.add_argument("--include-lcqi-q4-direct-off", action=argparse.BooleanOptionalAction, default=True)
    parser.add_argument("--include-lcqi-ggml-direct", action=argparse.BooleanOptionalAction, default=True)
    return parser.parse_args()


def main() -> int:
    args = parse_args()
    if args.rounds <= 0:
        print("[lcqi-same-input] --rounds must be positive", file=sys.stderr)
        return 1
    if args.max_new < 0:
        print("[lcqi-same-input] --max-new cannot be negative", file=sys.stderr)
        return 1

    cache_dir = args.cache_dir.expanduser().resolve()
    model_path = (
        args.model_path.expanduser().resolve()
        if args.model_path
        else cache_dir / "models" / run_smollm2_small_smoke.MODEL_FILE_NAME
    )
    build_dir = args.build_dir.expanduser().resolve()
    report_path = args.report.expanduser().resolve()
    full_prompt = smollm2_chat_prompt(args.prompt)

    try:
        run_smollm2_small_smoke.download_model(
            model_path,
            force_download=args.force_download,
            no_download=args.no_download,
        )
        lcqi_binary = build_lcqi(build_dir, args.jobs, args.timeout_seconds)
        llama_simple_binary, llama_bench_binary, llama_tokenize_binary = ensure_llama_targets(
```

```python
        cache_dir,
        jobs=args.jobs,
        skip_build=args.skip_llama_build,
        timeout_seconds=args.timeout_seconds,
    )

    lcqi_q4_direct_off_runs: list[CommandResult] = []
    lcqi_runs: list[CommandResult] = []
    lcqi_ggml_direct_runs: list[CommandResult] = []
    llama_simple_runs: list[CommandResult] = []
    for round_index in range(1, args.rounds + 1):
        if args.include_lcqi_q4_direct_off:
            print(f"[lcqi-same-input] LCQI Q4 direct off round {round_index}/{args.rounds}")
            lcqi_q4_direct_off_runs.append(
                run_lcqi_round(
                    lcqi_binary,
                    model_path,
                    user_prompt=args.prompt,
                    max_new_tokens=args.max_new,
                    round_index=round_index,
                    timeout_seconds=args.timeout_seconds,
                    name_prefix="lcqi_q4_direct_off_round",
                    env={LCQI_Q4_DIRECT_ENV: LCQI_Q4_DIRECT_OFF},
                )
            )
            ensure_success(
                lcqi_q4_direct_off_runs[-1],
                "run LCQI Q4 direct off same-input round",
            )
        print(f"[lcqi-same-input] LCQI round {round_index}/{args.rounds}")
        lcqi_runs.append(
            run_lcqi_round(
                lcqi_binary,
                model_path,
                user_prompt=args.prompt,
                max_new_tokens=args.max_new,
                round_index=round_index,
                timeout_seconds=args.timeout_seconds,
            )
        )
        ensure_success(lcqi_runs[-1], "run LCQI same-input round")
        if args.include_lcqi_ggml_direct:
            print(f"[lcqi-same-input] LCQI GGML direct experimental round {round_index}/{args.rounds}")
            lcqi_ggml_direct_runs.append(
                run_lcqi_round(
                    lcqi_binary,
                    model_path,
                    user_prompt=args.prompt,
                    max_new_tokens=args.max_new,
                    round_index=round_index,
```

```
                    timeout_seconds=args.timeout_seconds,
                    name_prefix="lcqi_ggml_direct_experimental_round",
                    env={LCQI_GGML_DIRECT_ENV: LCQI_GGML_DIRECT_ON},
                )
            )
            ensure_success(
                lcqi_ggml_direct_runs[-1],
                "run LCQI GGML direct experimental same-input round",
            )
        print(f"[lcqi-same-input] llama-simple round {round_index}/{args.rounds}")
        llama_simple_runs.append(
            run_llama_simple_round(
                llama_simple_binary,
                model_path,
                full_prompt=full_prompt,
                max_new_tokens=args.max_new,
                round_index=round_index,
                timeout_seconds=args.timeout_seconds,
            )
        )
        ensure_success(llama_simple_runs[-1], "run llama-simple same-input round")

    prompt_tokens = int(lcqi_runs[0].metrics["benchmark_prompt_tokens"])
    print("[lcqi-same-input] llama-tokenize prompt identity check")
    llama_tokenize = run_llama_tokenize(
        llama_tokenize_binary,
        model_path,
        full_prompt=full_prompt,
        timeout_seconds=args.timeout_seconds,
    )
    ensure_success(llama_tokenize, "run llama-tokenize same-input check")
    print("[lcqi-same-input] llama-bench same-token-count reference")
    llama_bench = run_llama_bench(
        llama_bench_binary,
        model_path,
        prompt_tokens=prompt_tokens,
        max_new_tokens=args.max_new,
        rounds=args.rounds,
        timeout_seconds=args.timeout_seconds,
    )
    ensure_success(llama_bench, "run llama-bench same-token-count reference")

    write_report(
        report_path,
        cache_dir=cache_dir,
        model_path=model_path,
        user_prompt=args.prompt,
        full_prompt=full_prompt,
        max_new_tokens=args.max_new,
```

```python
            lcqi_runs=lcqi_runs,
            lcqi_q4_direct_off_runs=lcqi_q4_direct_off_runs,
            lcqi_ggml_direct_runs=lcqi_ggml_direct_runs,
            llama_tokenize=llama_tokenize,
            llama_simple_runs=llama_simple_runs,
            llama_bench=llama_bench,
        )
    except Exception as error:
        print(f"[lcqi-same-input] ERROR: {error}", file=sys.stderr)
        return 1

    print(f"report={report_path}")
    for line in format_summary(
        "lcqi_same_input",
        lcqi_runs,
        ["benchmark_prefill_ms", "derived_prefill_prompt_tokens_per_second", "derived_decode_ms_per_step"],
    ):
        print(line)
    for line in format_summary(
        "llama_simple_same_input",
        llama_simple_runs,
        ["llama_prompt_eval_ms", "llama_prompt_eval_tokens_per_second", "llama_eval_ms_per_run"],
    ):
        print(line)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

`run_gpt2_smoke.py` 下载标准 GPT-2 的 `config.json`、`vocab.json`、`merges.txt` 和 `model.safetensors`，再调用 LCQI 自研 `lcqi_gpt2` reference path 生成 token。它证明自研 safetensors、BPE、GPT-2 forward、KV cache decode 和 greedy generation 已连成闭环。

Listing 16.17 tools/run_gpt2_smoke.py

```python
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import datetime as dt
import hashlib
import os
import shlex
import subprocess
import sys
import urllib.request
from dataclasses import dataclass
from pathlib import Path


```

```
MODEL_REPO_ID = "openai-community/gpt2"
MODEL_REVISION = "main"
MODEL_FILE_IDENTITIES = {
    "config.json": (
        665,
        "0daed7749b4f02b8f76240d5444551d7b08712dab4d0adb8239c56ba823bb7b4",
    ),
    "vocab.json": (
        1042301,
        "196139668be63f3b5d6574427317ae82f612a97c5d1cdaf36ed2256dbf636783",
    ),
    "merges.txt": (
        456318,
        "1ce1664773c50f3e0cc8842619a93edc4624525b728b188a9e0be33b7726adc5",
    ),
    "model.safetensors": (
        548105171,
        "248dfc3911869ec493c76e65bf2fcf7f615828b0254c12b473182f0f81d3a707",
    ),
}
DEFAULT_PROMPT = "Hello, my name is"
DEFAULT_MAX_NEW_TOKENS = 1
DEFAULT_TIMEOUT_SECONDS = 300
DEFAULT_BUILD_JOBS = 2


@dataclass(frozen=True)
class CommandResult:
    args: list[str]
    returncode: int
    stdout: str
    stderr: str


def script_path() -> Path:
    return Path(__file__).resolve()


def volume_dir() -> Path:
    return script_path().parents[1]


def repo_dir() -> Path:
    return script_path().parents[3]


def default_cache_dir() -> Path:
    explicit_cache = os.environ.get("LCQI_GPT2_CACHE_DIR")
    if explicit_cache:
        return Path(explicit_cache).expanduser()
    xdg_cache = os.environ.get("XDG_CACHE_HOME")
    cache_root = Path(xdg_cache).expanduser() if xdg_cache else Path.home() / "↩
```

```
    ↪ .cache"
    return cache_root / "lcqi-gpt2-smoke" / MODEL_REPO_ID.replace("/", "--")


def default_build_dir() -> Path:
    return volume_dir() / "build" / "lcqi-release"


def default_report_path() -> Path:
    return volume_dir() / "results" / "lcqi-gpt2-smoke.txt"


def command_line(args: list[str]) -> str:
    return " ".join(shlex.quote(arg) for arg in args)


def run_command(
    args: list[str],
    *,
    cwd: Path | None = None,
    timeout_seconds: int | None = None,
) -> CommandResult:
    try:
        completed = subprocess.run(
            args,
            cwd=str(cwd) if cwd else None,
            text=True,
            capture_output=True,
            timeout=timeout_seconds,
            check=False,
        )
    except subprocess.TimeoutExpired as exc:
        stdout = exc.stdout if isinstance(exc.stdout, str) else ""
        stderr = exc.stderr if isinstance(exc.stderr, str) else ""
        return CommandResult(args=args, returncode=124, stdout=stdout, stderr=↩
    ↪ stderr)
    return CommandResult(
        args=args,
        returncode=completed.returncode,
        stdout=completed.stdout,
        stderr=completed.stderr,
    )


def ensure_success(result: CommandResult, action: str) -> None:
    if result.returncode == 0:
        return
    raise RuntimeError(
        f"{action} failed with exit code {result.returncode}\n"
        f"command: {command_line(result.args)}\n"
        f"stdout tail:\n{tail(result.stdout)}\n"
        f"stderr tail:\n{tail(result.stderr)}"
```

```python
    )


def tail(text: str, max_chars: int = 4000) -> str:
    if len(text) <= max_chars:
        return text
    return text[-max_chars:]


def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for block in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def model_url(file_name: str) -> str:
    return (
        f"https://huggingface.co/{MODEL_REPO_ID}/resolve/"
        f"{MODEL_REVISION}/{file_name}"
    )


def download_file(url: str, target: Path, timeout_seconds: int) -> None:
    part_path = target.with_suffix(target.suffix + ".part")
    if part_path.exists():
        part_path.unlink()
    request = urllib.request.Request(url, headers={"User-Agent": "lcqi-gpt2-smoke"})
    with urllib.request.urlopen(request, timeout=timeout_seconds) as response:
        with part_path.open("wb") as output:
            while True:
                block = response.read(1024 * 1024)
                if not block:
                    break
                output.write(block)
    part_path.replace(target)


def file_identity_matches(path: Path, expected_bytes: int, expected_sha256: str) -> bool:
    return (
        path.exists()
        and path.stat().st_size == expected_bytes
        and sha256_file(path) == expected_sha256
    )


def ensure_model_files(cache_dir: Path, *, no_download: bool, timeout_seconds: int) -> None:
    cache_dir.mkdir(parents=True, exist_ok=True)
```

```python
    missing = [
        name
        for name, (expected_bytes, expected_sha256) in MODEL_FILE_IDENTITIES.items()
        if not file_identity_matches(cache_dir / name, expected_bytes, expected_sha256)
    ]
    if not missing:
        print(f"[lcqi-gpt2] model cache hit: {cache_dir}")
        return
    if no_download:
        raise RuntimeError(
            f"missing GPT-2 model files and --no-download was set: {missing}"
        )
    for file_name in missing:
        target = cache_dir / file_name
        if target.exists():
            target.unlink()
        print(f"[lcqi-gpt2] downloading {MODEL_REPO_ID}/{file_name}")
        download_file(model_url(file_name), target, timeout_seconds)
        expected_bytes, expected_sha256 = MODEL_FILE_IDENTITIES[file_name]
        if not file_identity_matches(target, expected_bytes, expected_sha256):
            raise RuntimeError(
                f"downloaded GPT-2 file identity mismatch: {file_name}"
            )


def build_lcqi_gpt2(build_dir: Path, jobs: int) -> Path:
    configure = run_command(
        [
            "cmake",
            "-S",
            str(volume_dir()),
            "-B",
            str(build_dir),
            "-DCMAKE_BUILD_TYPE=Release",
        ],
        timeout_seconds=DEFAULT_TIMEOUT_SECONDS,
    )
    ensure_success(configure, "configure LCQI")
    build = run_command(
        [
            "cmake",
            "--build",
            str(build_dir),
            "--target",
            "lcqi_gpt2",
            "-j",
            str(jobs),
        ],
        timeout_seconds=DEFAULT_TIMEOUT_SECONDS,
    )
```

```python
    ensure_success(build, "build lcqi_gpt2")
    binary = build_dir / "labs" / "linux_cpu_inference" / "lcqi_gpt2"
    if not binary.exists():
        raise RuntimeError(f"lcqi_gpt2 binary not found: {binary}")
    return binary


def write_report(
    report_path: Path,
    *,
    cache_dir: Path,
    binary: Path,
    prompt: str,
    max_new_tokens: int,
    engine: str,
    result: CommandResult,
) -> None:
    report_path.parent.mkdir(parents=True, exist_ok=True)
    lines = [
        f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
        f"model_repo_id={MODEL_REPO_ID}",
        f"model_revision={MODEL_REVISION}",
        f"model_cache_dir={cache_dir}",
    ]
    for file_name, (expected_bytes, expected_sha256) in MODEL_FILE_IDENTITIES.items():
        path = cache_dir / file_name
        lines.append(f"file.{file_name}.bytes={path.stat().st_size}")
        lines.append(f"file.{file_name}.sha256={sha256_file(path)}")
        lines.append(f"file.{file_name}.expected_bytes={expected_bytes}")
        lines.append(f"file.{file_name}.expected_sha256={expected_sha256}")
    lines.extend(
        [
            f"binary={binary}",
            f"prompt={prompt}",
            f"max_new_tokens={max_new_tokens}",
            f"engine={engine}",
            f"command={command_line(result.args)}",
            f"exit_code={result.returncode}",
            "stdout_begin",
            result.stdout.rstrip(),
            "stdout_end",
            "stderr_begin",
            tail(result.stderr).rstrip(),
            "stderr_end",
        ]
    )
    passed = result.returncode == 0 and "generated_text " in result.stdout
    lines.append(f"validation={'PASS' if passed else 'FAIL'}")
    report_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
```

```python
def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Run LCQI self-owned GPT-2 smoke on a HuggingFace safetensors directory."
    )
    parser.add_argument("--cache-dir", type=Path, default=default_cache_dir())
    parser.add_argument("--build-dir", type=Path, default=default_build_dir())
    parser.add_argument("--report", type=Path, default=default_report_path())
    parser.add_argument("--prompt", default=DEFAULT_PROMPT)
    parser.add_argument("--max-new-tokens", type=int, default=DEFAULT_MAX_NEW_TOKENS)
    parser.add_argument("--engine", choices=("cached", "full"), default="cached")
    parser.add_argument("--benchmark", action="store_true")
    parser.add_argument("--timeout-seconds", type=int, default=DEFAULT_TIMEOUT_SECONDS)
    parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
    parser.add_argument("--no-download", action="store_true")
    return parser.parse_args()


def main() -> int:
    args = parse_args()
    try:
        if args.max_new_tokens < 0:
            raise RuntimeError("--max-new-tokens cannot be negative")
        ensure_model_files(
            args.cache_dir,
            no_download=args.no_download,
            timeout_seconds=args.timeout_seconds,
        )
        binary = build_lcqi_gpt2(args.build_dir, args.jobs)
        command = [
            str(binary),
            "--engine",
            args.engine,
            str(args.cache_dir),
            args.prompt,
            str(args.max_new_tokens),
        ]
        if args.benchmark:
            command.insert(1, "--benchmark")
        result = run_command(command, cwd=repo_dir(), timeout_seconds=args.timeout_seconds)
        write_report(
            args.report,
            cache_dir=args.cache_dir,
            binary=binary,
            prompt=args.prompt,
            max_new_tokens=args.max_new_tokens,
            engine=args.engine,
            result=result,
```

```
        )
        print(f"[lcqi-gpt2] report: {args.report}")
        print(tail(result.stdout, max_chars=2000))
        return 0 if result.returncode == 0 else result.returncode
    except Exception as error:
        print(f"[lcqi-gpt2] error: {error}", file=sys.stderr)
        return 1


if __name__ == "__main__":
    raise SystemExit(main())
```

`run_gpt2_benchmark_compare.py` 用同一个 GPT-2 F32 safetensors 模型分别运行 LCQI full-prefix 与 cached-KV 路径，并在本机存在 llama.cpp/SmolLM2 缓存时附带成熟引擎参考。它的报告说明算法差距、kernel 差距和模型口径差异。

Listing 16.18 tools/run_gpt2_benchmark_compare.py

```
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import datetime as dt
import os
import re
import shlex
import subprocess
import sys
from dataclasses import dataclass
from pathlib import Path

import run_gpt2_smoke


DEFAULT_PROMPT = "Hello, my name is"
DEFAULT_MAX_NEW_TOKENS = 4
DEFAULT_BUILD_JOBS = 2
DEFAULT_TIMEOUT_SECONDS = 300
DEFAULT_SMOLLM2_CACHE_DIR = Path.home() / ".cache" / "lcqi-smollm2-small-smoke"
SMOLLM2_GGUF = "SmolLM2-135M-Instruct-Q4_K_M.gguf"


@dataclass(frozen=True)
class EngineRun:
    name: str
    command: list[str]
    returncode: int
    stdout: str
    stderr: str
    metrics: dict[str, str]


def volume_dir() -> Path:
```

```python
    return Path(__file__).resolve().parents[1]


def repo_dir() -> Path:
    return Path(__file__).resolve().parents[3]


def default_report_path() -> Path:
    return volume_dir() / "results" / "lcqi-gpt2-benchmark-compare.txt"


def default_build_dir() -> Path:
    return volume_dir() / "build" / "lcqi-release"


def command_line(args: list[str]) -> str:
    return " ".join(shlex.quote(arg) for arg in args)


def tail(text: str, max_chars: int = 4000) -> str:
    return text if len(text) <= max_chars else text[-max_chars:]


def run_command(args: list[str], *, timeout_seconds: int) -> tuple[int, str, str]:
    try:
        completed = subprocess.run(
            args,
            cwd=repo_dir(),
            text=True,
            capture_output=True,
            timeout=timeout_seconds,
            check=False,
        )
    except subprocess.TimeoutExpired as exc:
        stdout = exc.stdout if isinstance(exc.stdout, str) else ""
        stderr = exc.stderr if isinstance(exc.stderr, str) else ""
        return 124, stdout, stderr
    return completed.returncode, completed.stdout, completed.stderr


def parse_lcqi_metrics(stdout: str) -> dict[str, str]:
    metrics: dict[str, str] = {}
    for line in stdout.splitlines():
        if not line:
            continue
        key, _, value = line.partition(" ")
        if key in {
            "engine",
            "generated_text",
            "benchmark_load_ms",
            "benchmark_tokenize_ms",
```

```python
            "benchmark_prefill_ms",
            "benchmark_decode_ms",
            "benchmark_generate_ms",
            "benchmark_total_ms",
            "benchmark_prompt_tokens",
            "benchmark_generated_tokens",
            "benchmark_prefill_steps",
            "benchmark_decode_steps",
            "benchmark_generate_tokens_per_second",
            "benchmark_decode_tokens_per_second",
            "benchmark_kv_cache_bytes",
        }:
            metrics[key] = value
    return metrics


def run_lcqi(
    binary: Path,
    model_dir: Path,
    *,
    engine: str,
    prompt: str,
    max_new_tokens: int,
    timeout_seconds: int,
) -> EngineRun:
    command = [
        str(binary),
        "--benchmark",
        "--engine",
        engine,
        str(model_dir),
        prompt,
        str(max_new_tokens),
    ]
    returncode, stdout, stderr = run_command(command, timeout_seconds=timeout_seconds)
    return EngineRun(
        name=f"lcqi_{engine}",
        command=command,
        returncode=returncode,
        stdout=stdout,
        stderr=stderr,
        metrics=parse_lcqi_metrics(stdout),
    )


def llama_bench_binary(cache_dir: Path) -> Path:
    return cache_dir / "llama.cpp" / "build" / "bin" / "llama-bench"


def smollm2_model_path(cache_dir: Path) -> Path:
    return cache_dir / "models" / SMOLLM2_GGUF
```

```python
def parse_llama_bench_metrics(stdout: str) -> dict[str, str]:
    metrics: dict[str, str] = {}
    lines = [line.strip() for line in stdout.splitlines() if line.strip()]
    for line in lines:
        if not line.startswith("|"):
            continue
        columns = [column.strip() for column in line.strip("|").split("|")]
        if len(columns) != 7 or columns[0] == "model" or columns[0].startswith("-"):
            continue
        metrics["llama_model"] = columns[0]
        metrics["llama_size"] = columns[1]
        metrics["llama_params"] = columns[2]
        metrics["llama_backend"] = columns[3]
        metrics["llama_threads"] = columns[4]
        test_name = columns[5]
        tokens_per_second = re.sub(r"\s+.*$", "", columns[6])
        if test_name.startswith("pp"):
            metrics["llama_prefill_test"] = test_name
            metrics["llama_prefill_tps"] = tokens_per_second
        elif test_name.startswith("tg"):
            metrics["llama_decode_test"] = test_name
            metrics["llama_decode_tps"] = tokens_per_second
    return metrics


def run_llama_bench(cache_dir: Path, *, timeout_seconds: int) -> EngineRun | None:
    binary = llama_bench_binary(cache_dir)
    model = smollm2_model_path(cache_dir)
    if not binary.exists() or not os.access(binary, os.X_OK) or not model.exists():
        return None
    command = [
        str(binary),
        "-m",
        str(model),
        "-ngl",
        "0",
        "-p",
        "5",
        "-n",
        "4",
        "-r",
        "1",
    ]
    returncode, stdout, stderr = run_command(command, timeout_seconds=timeout_seconds)
    return EngineRun(
        name="llama_cpp_smollm2_q4_k_m",
```

```python
        command=command,
        returncode=returncode,
        stdout=stdout,
        stderr=stderr,
        metrics=parse_llama_bench_metrics(stdout),
    )


def write_report(path: Path, runs: list[EngineRun], *, prompt: str, max_new_tokens: int) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    lines = [
        "LCQI GPT-2 Benchmark Compare",
        "",
        f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
        f"repo_commit={current_commit()}",
        f"working_tree_dirty={working_tree_dirty()}",
        f"python={sys.version.split()[0]}",
        f"prompt={prompt}",
        f"max_new_tokens={max_new_tokens}",
        "",
        "scope=LCQI full_prefix and cached_kv run the same GPT-2 F32 safetensors model. "
        "llama.cpp uses SmolLM2-135M-Instruct Q4_K_M as an external mature-engine reference, "
        "so its numbers are engineering context, not same-model quality ranking.",
        "",
    ]
    for run in runs:
        lines.extend(
            [
                f"[{run.name}]",
                f"returncode={run.returncode}",
                f"command={command_line(run.command)}",
            ]
        )
        for key in sorted(run.metrics):
            lines.append(f"{key}={run.metrics[key]}")
        lines.extend(
            [
                "stdout_tail:",
                tail(run.stdout.strip(), 2000),
                "stderr_tail:",
                tail(run.stderr.strip(), 2000),
                "",
            ]
        )
    while lines and lines[-1] == "":
        lines.pop()
    path.write_text("\n".join(lines) + "\n", encoding="utf-8")
```

```python

def current_commit() -> str:
    result = subprocess.run(
        ["git", "-C", str(repo_dir()), "rev-parse", "--short", "HEAD"],
        text=True,
        capture_output=True,
        check=False,
    )
    return result.stdout.strip() if result.returncode == 0 else "unknown"


def working_tree_dirty() -> str:
    result = subprocess.run(
        ["git", "-C", str(repo_dir()), "status", "--porcelain"],
        text=True,
        capture_output=True,
        check=False,
    )
    if result.returncode != 0:
        return "unknown"
    return "true" if result.stdout.strip() else "false"


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Compare LCQI GPT-2 full-prefix and KV-cache paths, with optional llama.cpp context."
    )
    parser.add_argument("--cache-dir", type=Path, default=run_gpt2_smoke.default_cache_dir())
    parser.add_argument("--build-dir", type=Path, default=default_build_dir())
    parser.add_argument("--report", type=Path, default=default_report_path())
    parser.add_argument("--prompt", default=DEFAULT_PROMPT)
    parser.add_argument("--max-new-tokens", type=int, default=DEFAULT_MAX_NEW_TOKENS)
    parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
    parser.add_argument("--timeout-seconds", type=int, default=DEFAULT_TIMEOUT_SECONDS)
    parser.add_argument("--no-download", action="store_true")
    parser.add_argument("--skip-llama", action="store_true")
    parser.add_argument("--smollm2-cache-dir", type=Path, default=DEFAULT_SMOLLM2_CACHE_DIR)
    return parser.parse_args()


def main() -> int:
    args = parse_args()
    if args.max_new_tokens < 0:
        print("[lcqi-compare] --max-new-tokens cannot be negative", file=sys.stderr)
        return 1
    try:
```

```python
        run_gpt2_smoke.ensure_model_files(
            args.cache_dir,
            no_download=args.no_download,
            timeout_seconds=args.timeout_seconds,
        )
        binary = run_gpt2_smoke.build_lcqi_gpt2(args.build_dir, args.jobs)
        runs = [
            run_lcqi(
                binary,
                args.cache_dir,
                engine="full",
                prompt=args.prompt,
                max_new_tokens=args.max_new_tokens,
                timeout_seconds=args.timeout_seconds,
            ),
            run_lcqi(
                binary,
                args.cache_dir,
                engine="cached",
                prompt=args.prompt,
                max_new_tokens=args.max_new_tokens,
                timeout_seconds=args.timeout_seconds,
            ),
        ]
        if not args.skip_llama:
            llama_run = run_llama_bench(
                args.smollm2_cache_dir.expanduser().resolve(),
                timeout_seconds=args.timeout_seconds,
            )
            if llama_run is not None:
                runs.append(llama_run)
        write_report(args.report, runs, prompt=args.prompt, max_new_tokens=args.max_new_tokens)
    except Exception as error:
        print(f"[lcqi-compare] error: {error}", file=sys.stderr)
        return 1

    for run in runs:
        print(f"{run.name}: returncode={run.returncode}")
        for key in sorted(run.metrics):
            print(f"  {key}={run.metrics[key]}")
    print(f"report={args.report}")
    return 0 if all(run.returncode == 0 for run in runs) else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

`run_gpt2_optimization_ab.py` 专门比较 LCQI 自身改动前后。它从 baseline commit 建临时 worktree，baseline 和当前工作区都通过实践卷 CMake 入口构建 Release，再多轮运行同一个 GPT-2 F32 模型。报告中的 `median/min/max` 用来约束机器波动，`generated_text` 用来确认优化前后输出没有漂移，`benchmark_worker_count` 用来确认 cached decoder 到底是串行、自动

线程数还是固定线程数。

Listing 16.19 tools/run_gpt2_optimization_ab.py

```python
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import datetime as dt
import shutil
import statistics
import subprocess
import sys
import tempfile
from dataclasses import dataclass
from pathlib import Path


DEFAULT_BASELINE_REF = "4febf3f"
DEFAULT_PROMPT = "Hello, my name is"
DEFAULT_MAX_NEW_TOKENS = 4
DEFAULT_ROUNDS = 3
DEFAULT_BUILD_JOBS = 2
DEFAULT_TIMEOUT_SECONDS = 300

METRIC_KEYS = [
    "benchmark_prefill_ms",
    "benchmark_decode_ms",
    "benchmark_generate_ms",
    "benchmark_worker_count",
    "benchmark_generate_tokens_per_second",
    "benchmark_decode_tokens_per_second",
]


@dataclass(frozen=True)
class RunSample:
    variant: str
    engine: str
    round_id: int
    metrics: dict[str, float]
    generated_text: str


def script_path() -> Path:
    return Path(__file__).resolve()


def repo_dir() -> Path:
    return script_path().parents[3]


def volume_dir() -> Path:
```

```python
    return script_path().parents[1]


def practice_dir(root: Path) -> Path:
    return root / "books" / "cpu-volume-3-practice"


def default_report_path() -> Path:
    return volume_dir() / "results" / "lcqi-gpt2-optimization-ab.txt"


def default_model_dir() -> Path:
    return Path.home() / ".cache" / "lcqi-gpt2-smoke" / "openai-community--gpt2"


def run_command(
    args: list[str],
    *,
    cwd: Path,
    timeout_seconds: int,
) -> subprocess.CompletedProcess[str]:
    return subprocess.run(
        args,
        cwd=cwd,
        text=True,
        capture_output=True,
        timeout=timeout_seconds,
        check=False,
    )


def ensure_success(
    completed: subprocess.CompletedProcess[str],
    action: str,
) -> None:
    if completed.returncode == 0:
        return
    command = (
        " ".join(completed.args)
        if isinstance(completed.args, list)
        else str(completed.args)
    )
    raise RuntimeError(
        f"{action} failed with exit code {completed.returncode}\n"
        f"command: {command}\n"
        f"stdout tail:\n{completed.stdout[-2000:]}\n"
        f"stderr tail:\n{completed.stderr[-2000:]}"
    )


def git_text(args: list[str]) -> str:
```

```python
    completed = run_command(["git", *args], cwd=repo_dir(), timeout_seconds=60)
    ensure_success(completed, "git command")
    return completed.stdout.strip()


def working_tree_dirty() -> bool:
    completed = run_command(
        ["git", "status", "--porcelain"],
        cwd=repo_dir(),
        timeout_seconds=60,
    )
    ensure_success(completed, "git status")
    return bool(completed.stdout.strip())


def build_binary(root: Path, build_dir: Path, jobs: int, timeout_seconds: int) ↩
    ↪ -> Path:
    configure = run_command(
        [
            "cmake",
            "-S",
            str(practice_dir(root)),
            "-B",
            str(build_dir),
            "-DCMAKE_BUILD_TYPE=Release",
        ],
        cwd=root,
        timeout_seconds=timeout_seconds,
    )
    ensure_success(configure, f"configure {root}")
    build = run_command(
        [
            "cmake",
            "--build",
            str(build_dir),
            "--target",
            "lcqi_gpt2",
            "-j",
            str(jobs),
        ],
        cwd=root,
        timeout_seconds=timeout_seconds,
    )
    ensure_success(build, f"build {root}")
    binary = build_dir / "linux_cpu_inference" / "lcqi_gpt2"
    if not binary.exists():
        raise RuntimeError(f"lcqi_gpt2 binary not found: {binary}")
    return binary


def parse_metrics(stdout: str) -> tuple[dict[str, float], str]:
    metrics: dict[str, float] = {}
```

```python
    generated_text = ""
    for line in stdout.splitlines():
        key, _, value = line.partition(" ")
        if key in METRIC_KEYS:
            metrics[key] = float(value)
        elif key == "generated_text":
            generated_text = value
    metrics.setdefault("benchmark_worker_count", 1.0)
    missing = sorted(set(METRIC_KEYS) - metrics.keys())
    if missing:
        raise RuntimeError(f"missing benchmark metrics: {missing}")
    return metrics, generated_text


def run_sample(
    binary: Path,
    *,
    variant: str,
    engine: str,
    round_id: int,
    model_dir: Path,
    prompt: str,
    max_new_tokens: int,
    threads: int,
    timeout_seconds: int,
) -> RunSample:
    command = [
        str(binary),
        "--benchmark",
        "--engine",
        engine,
    ]
    if variant == "current":
        command.extend(["--threads", str(threads)])
    command.extend([
        str(model_dir),
        prompt,
        str(max_new_tokens),
    ])
    completed = run_command(
        command,
        cwd=repo_dir(),
        timeout_seconds=timeout_seconds,
    )
    ensure_success(completed, f"run {variant} {engine}")
    metrics, generated_text = parse_metrics(completed.stdout)
    return RunSample(
        variant=variant,
        engine=engine,
        round_id=round_id,
        metrics=metrics,
        generated_text=generated_text,
```

```python
    )


def summarize(samples: list[RunSample]) -> list[str]:
    lines: list[str] = []
    variants = sorted({sample.variant for sample in samples})
    engines = sorted({sample.engine for sample in samples})
    for variant in variants:
        for engine in engines:
            selected = [
                sample for sample in samples
                if sample.variant == variant and sample.engine == engine
            ]
            if not selected:
                continue
            lines.append(f"[{variant} {engine}]")
            for key in METRIC_KEYS:
                values = [sample.metrics[key] for sample in selected]
                lines.append(
                    f"{key}=median:{statistics.median(values):.6g},"
                    f"min:{min(values):.6g},max:{max(values):.6g}"
                )
            texts = sorted({sample.generated_text for sample in selected})
            lines.append(f"generated_text={texts[0] if len(texts) == 1 else ↩
    ↪ texts}")
            lines.append("")
    return lines


def write_report(
    path: Path,
    samples: list[RunSample],
    *,
    baseline_ref: str,
    baseline_commit: str,
    current_commit: str,
    prompt: str,
    max_new_tokens: int,
    rounds: int,
    current_threads: int,
) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    lines = [
        "LCQI GPT-2 Optimization A/B",
        "",
        f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
        f"baseline_ref={baseline_ref}",
        f"baseline_commit={baseline_commit}",
        f"current_commit={current_commit}",
        f"current_working_tree_dirty={str(working_tree_dirty()).lower()}",
        f"prompt={prompt}",
        f"max_new_tokens={max_new_tokens}",
```

```python
        f"rounds={rounds}",
        f"baseline_threads=1",
        f"current_threads={current_threads}",
        "",
        "scope=Both variants are built through books/cpu-volume-3-practice with "
        "CMAKE_BUILD_TYPE=Release and run on the same GPT-2 F32 safetensors model. "
        "The llama.cpp comparison remains in lcqi-gpt2-benchmark-compare.txt; this "
        "report isolates LCQI before/after code changes.",
        "",
        "[samples]",
    ]
    for sample in samples:
        metric_text = ",".join(
            f"{key}:{sample.metrics[key]:.6g}" for key in METRIC_KEYS
        )
        lines.append(
            f"round={sample.round_id},variant={sample.variant},engine={sample.engine},"
            f"{metric_text},generated_text={sample.generated_text}"
        )
    lines.append("")
    lines.extend(summarize(samples))
    while lines and lines[-1] == "":
        lines.pop()
    path.write_text("\n".join(lines) + "\n", encoding="utf-8")


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Run LCQI GPT-2 before/after A/B benchmark.")
    parser.add_argument("--baseline-ref", default=DEFAULT_BASELINE_REF)
    parser.add_argument("--model-dir", type=Path, default=default_model_dir())
    parser.add_argument("--report", type=Path, default=default_report_path())
    parser.add_argument("--prompt", default=DEFAULT_PROMPT)
    parser.add_argument("--max-new-tokens", type=int, default=DEFAULT_MAX_NEW_TOKENS)
    parser.add_argument("--rounds", type=int, default=DEFAULT_ROUNDS)
    parser.add_argument("--threads", type=int, default=0)
    parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
    parser.add_argument("--timeout-seconds", type=int, default=DEFAULT_TIMEOUT_SECONDS)
    return parser.parse_args()


def main() -> int:
    args = parse_args()
    if args.rounds <= 0:
        print("[lcqi-ab] --rounds must be positive", file=sys.stderr)
        return 1
```

```python
    if args.max_new_tokens < 0:
        print("[lcqi-ab] --max-new-tokens cannot be negative", file=sys.stderr)
        return 1
    if args.threads < 0:
        print("[lcqi-ab] --threads cannot be negative", file=sys.stderr)
        return 1
    if not args.model_dir.exists():
        print(f"[lcqi-ab] model directory not found: {args.model_dir}", file=↩
↪ sys.stderr)
        return 1

    baseline_commit = git_text(["rev-parse", "--short", args.baseline_ref])
    current_commit = git_text(["rev-parse", "--short", "HEAD"])
    with tempfile.TemporaryDirectory(prefix="lcqi-gpt2-ab-") as tmp:
        tmp_path = Path(tmp)
        baseline_root = tmp_path / "baseline"
        current_build = tmp_path / "current-build"
        baseline_build = tmp_path / "baseline-build"
        ensure_success(
            run_command(
                ["git", "worktree", "add", "--detach", str(baseline_root), args↩
↪ .baseline_ref],
                cwd=repo_dir(),
                timeout_seconds=args.timeout_seconds,
            ),
            "create baseline worktree",
        )
        try:
            baseline_binary = build_binary(
                baseline_root,
                baseline_build,
                args.jobs,
                args.timeout_seconds,
            )
            current_binary = build_binary(
                repo_dir(),
                current_build,
                args.jobs,
                args.timeout_seconds,
            )

            samples: list[RunSample] = []
            for round_id in range(1, args.rounds + 1):
                for variant, binary in [
                    ("baseline", baseline_binary),
                    ("current", current_binary),
                ]:
                    for engine in ["full", "cached"]:
                        sample = run_sample(
                            binary,
                            variant=variant,
                            engine=engine,
```

```
                            round_id=round_id,
                            model_dir=args.model_dir,
                            prompt=args.prompt,
                            max_new_tokens=args.max_new_tokens,
                            threads=1 if variant == "baseline" else args.threads,
                            timeout_seconds=args.timeout_seconds,
                        )
                        samples.append(sample)
                        print(
                            f"round={round_id} variant={variant} engine={engine} "
                            f"generate_ms={sample.metrics['benchmark_generate_ms']:.3f} "
                            f"workers={sample.metrics['benchmark_worker_count']:.0f} "
                            f"gen_tps={sample.metrics['benchmark_generate_tokens_per_second']:.5f}"
                        )
            write_report(
                args.report,
                samples,
                baseline_ref=args.baseline_ref,
                baseline_commit=baseline_commit,
                current_commit=current_commit,
                prompt=args.prompt,
                max_new_tokens=args.max_new_tokens,
                rounds=args.rounds,
                current_threads=args.threads,
            )
        finally:
            subprocess.run(
                ["git", "worktree", "remove", "--force", str(baseline_root)],
                cwd=repo_dir(),
                text=True,
                capture_output=True,
                check=False,
            )
    print(f"report={args.report}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

`run_gpt2_hotspot_profile.py` 专门收集 cached decode 内部热点。它构建 Release `lcqi_gpt2`，用 `--profile-hotspots` 输出 `hotspot_lm_head_pct`、`hotspot_mlp_fc_pct`、`hotspot_mlp_projection_pct`、`hotspot_qkv_projection_pct`、`hotspot_attention_projection_pct`、`hotspot_attention_pct` 和 `benchmark_worker_count`，再按 token 数和轮次计算中位数。这个脚本回答“优化应该打哪里”，不是回答“某次提交快了多少”。本轮线程优化就是靠它发现第一次 auto 阈值漏掉了 attention output projection，随后把行数和列数阈值一起纳入并行判定。

Listing 16.20 tools/run_gpt2_hotspot_profile.py

```python
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import datetime as dt
import statistics
import subprocess
from dataclasses import dataclass
from pathlib import Path


DEFAULT_PROMPT = "Hello, my name is"
DEFAULT_TOKEN_COUNTS = "4,16"
DEFAULT_ROUNDS = 3
DEFAULT_BUILD_JOBS = 2
DEFAULT_TIMEOUT_SECONDS = 900

METRIC_PREFIXES = ("benchmark_", "hotspot_")


@dataclass(frozen=True)
class HotspotSample:
    max_new_tokens: int
    round_id: int
    metrics: dict[str, float]
    generated_text: str


def script_path() -> Path:
    return Path(__file__).resolve()


def repo_dir() -> Path:
    return script_path().parents[3]


def volume_dir() -> Path:
    return script_path().parents[1]


def practice_dir() -> Path:
    return repo_dir() / "books" / "cpu-volume-3-practice"


def default_build_dir() -> Path:
    return volume_dir() / "build" / "lcqi-hotspot-profile"


def default_model_dir() -> Path:
    return Path.home() / ".cache" / "lcqi-gpt2-smoke" / "openai-community--gpt2"
```

```python


def default_report_path() -> Path:
    return volume_dir() / "results" / "lcqi-gpt2-hotspot-profile.txt"


def run_command(
    args: list[str],
    *,
    cwd: Path,
    timeout_seconds: int,
) -> subprocess.CompletedProcess[str]:
    return subprocess.run(
        args,
        cwd=cwd,
        text=True,
        capture_output=True,
        timeout=timeout_seconds,
        check=False,
    )


def ensure_success(completed: subprocess.CompletedProcess[str], action: str) -> None:
    if completed.returncode == 0:
        return
    command = (
        " ".join(completed.args)
        if isinstance(completed.args, list)
        else str(completed.args)
    )
    raise RuntimeError(
        f"{action} failed with exit code {completed.returncode}\n"
        f"command: {command}\n"
        f"stdout tail:\n{completed.stdout[-2000:]}\n"
        f"stderr tail:\n{completed.stderr[-2000:]}"
    )


def build_binary(build_dir: Path, jobs: int, timeout_seconds: int) -> Path:
    configure = run_command(
        [
            "cmake",
            "-S",
            str(practice_dir()),
            "-B",
            str(build_dir),
            "-DCMAKE_BUILD_TYPE=Release",
        ],
        cwd=repo_dir(),
        timeout_seconds=timeout_seconds,
    )
```

```python
    ensure_success(configure, "configure hotspot profile build")
    build = run_command(
        [
            "cmake",
            "--build",
            str(build_dir),
            "--target",
            "lcqi_gpt2",
            "-j",
            str(jobs),
        ],
        cwd=repo_dir(),
        timeout_seconds=timeout_seconds,
    )
    ensure_success(build, "build lcqi_gpt2")
    binary = build_dir / "linux_cpu_inference" / "lcqi_gpt2"
    if not binary.exists():
        raise RuntimeError(f"lcqi_gpt2 binary not found: {binary}")
    return binary


def parse_numeric_list(value: str) -> list[int]:
    result: list[int] = []
    for part in value.split(","):
        stripped = part.strip()
        if not stripped:
            continue
        parsed = int(stripped)
        if parsed <= 0:
            raise RuntimeError("--token-counts entries must be positive")
        result.append(parsed)
    if not result:
        raise RuntimeError("--token-counts must contain at least one positive integer")
    return result


def parse_stdout(stdout: str) -> tuple[dict[str, float], str]:
    metrics: dict[str, float] = {}
    generated_text = ""
    for line in stdout.splitlines():
        key, _, value = line.partition(" ")
        if key.startswith(METRIC_PREFIXES):
            metrics[key] = float(value)
        elif key == "generated_text":
            generated_text = value
    required = {
        "benchmark_generate_ms",
        "benchmark_worker_count",
        "hotspot_total_step_ms",
        "hotspot_lm_head_pct",
        "hotspot_mlp_fc_pct",
```

```
        "hotspot_mlp_projection_pct",
        "hotspot_qkv_projection_pct",
        "hotspot_attention_pct",
    }
    missing = sorted(required - metrics.keys())
    if missing:
        raise RuntimeError(f"missing hotspot metrics: {missing}")
    if not generated_text:
        raise RuntimeError("missing generated_text")
    return metrics, generated_text


def run_sample(
    binary: Path,
    *,
    model_dir: Path,
    prompt: str,
    max_new_tokens: int,
    round_id: int,
    threads: int,
    timeout_seconds: int,
) -> HotspotSample:
    completed = run_command(
        [
            str(binary),
            "--profile-hotspots",
            "--engine",
            "cached",
            "--threads",
            str(threads),
            str(model_dir),
            prompt,
            str(max_new_tokens),
        ],
        cwd=repo_dir(),
        timeout_seconds=timeout_seconds,
    )
    ensure_success(completed, f"run hotspot profile tokens={max_new_tokens}")
    metrics, generated_text = parse_stdout(completed.stdout)
    return HotspotSample(
        max_new_tokens=max_new_tokens,
        round_id=round_id,
        metrics=metrics,
        generated_text=generated_text,
    )


def summarize(samples: list[HotspotSample]) -> list[str]:
    lines: list[str] = []
    metric_keys = [
        "benchmark_load_ms",
        "benchmark_generate_ms",
```

```
        "benchmark_generate_tokens_per_second",
        "benchmark_decode_tokens_per_second",
        "hotspot_total_step_ms",
        "benchmark_worker_count",
        "hotspot_lm_head_pct",
        "hotspot_mlp_fc_pct",
        "hotspot_mlp_projection_pct",
        "hotspot_qkv_projection_pct",
        "hotspot_attention_projection_pct",
        "hotspot_attention_pct",
    ]
    for token_count in sorted({sample.max_new_tokens for sample in samples}):
        selected = [sample for sample in samples if sample.max_new_tokens == token_count]
        lines.append(f"[summary max_new_tokens={token_count}]")
        for key in metric_keys:
            values = [sample.metrics[key] for sample in selected if key in sample.metrics]
            if not values:
                continue
            lines.append(
                f"{key}=median:{statistics.median(values):.6g},"
                f"min:{min(values):.6g},max:{max(values):.6g}"
            )
        texts = sorted({sample.generated_text for sample in selected})
        lines.append(f"generated_text={texts[0] if len(texts) == 1 else texts}")
        lines.append("")
    return lines


def write_report(
    path: Path,
    samples: list[HotspotSample],
    *,
    model_dir: Path,
    prompt: str,
    token_counts: list[int],
    rounds: int,
    threads: int,
) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    lines = [
        "LCQI GPT-2 Hotspot Profile",
        "",
        f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
        f"model_dir={model_dir}",
        f"prompt={prompt}",
        f"token_counts={','.join(str(value) for value in token_counts)}",
        f"rounds={rounds}",
        f"requested_threads={threads}",
        "engine=cached",
```

```python
        "build=CMAKE_BUILD_TYPE=Release through books/cpu-volume-3-practice",
        "",
        "[samples]",
    ]
    for sample in samples:
        metric_text = ",".join(
            f"{key}:{sample.metrics[key]:.6g}"
            for key in sorted(sample.metrics)
            if key.startswith(METRIC_PREFIXES)
        )
        lines.append(
            f"round={sample.round_id},max_new_tokens={sample.max_new_tokens},"
            f"{metric_text},generated_text={sample.generated_text}"
        )
    lines.append("")
    lines.extend(summarize(samples))
    while lines and lines[-1] == "":
        lines.pop()
    path.write_text("\n".join(lines) + "\n", encoding="utf-8")


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Run LCQI GPT-2 cached hotspot profile.")
    parser.add_argument("--model-dir", type=Path, default=default_model_dir())
    parser.add_argument("--build-dir", type=Path, default=default_build_dir())
    parser.add_argument("--report", type=Path, default=default_report_path())
    parser.add_argument("--prompt", default=DEFAULT_PROMPT)
    parser.add_argument("--token-counts", default=DEFAULT_TOKEN_COUNTS)
    parser.add_argument("--rounds", type=int, default=DEFAULT_ROUNDS)
    parser.add_argument("--threads", type=int, default=0)
    parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
    parser.add_argument("--timeout-seconds", type=int, default=DEFAULT_TIMEOUT_SECONDS)
    return parser.parse_args()


def main() -> int:
    args = parse_args()
    if args.rounds <= 0:
        print("[lcqi-hotspot] --rounds must be positive")
        return 1
    if args.threads < 0:
        print("[lcqi-hotspot] --threads cannot be negative")
        return 1
    if not args.model_dir.exists():
        print(f"[lcqi-hotspot] model directory not found: {args.model_dir}")
        return 1
    token_counts = parse_numeric_list(args.token_counts)
    binary = build_binary(args.build_dir, args.jobs, args.timeout_seconds)
    samples: list[HotspotSample] = []
    for round_id in range(1, args.rounds + 1):
```

```python
        for token_count in token_counts:
            sample = run_sample(
                binary,
                model_dir=args.model_dir,
                prompt=args.prompt,
                max_new_tokens=token_count,
                round_id=round_id,
                threads=args.threads,
                timeout_seconds=args.timeout_seconds,
            )
            samples.append(sample)
            print(
                f"round={round_id} max_new_tokens={token_count} "
                f"generate_ms={sample.metrics['benchmark_generate_ms']:.3f} "
                f"workers={sample.metrics['benchmark_worker_count']:.0f} "
                f"lm_head_pct={sample.metrics['hotspot_lm_head_pct']:.3f} "
                f"mlp_pct={sample.metrics['hotspot_mlp_fc_pct'] + sample.metrics['hotspot_mlp_projection_pct']:.3f}"
            )
    write_report(
        args.report,
        samples,
        model_dir=args.model_dir,
        prompt=args.prompt,
        token_counts=token_counts,
        rounds=args.rounds,
        threads=args.threads,
    )
    print(f"report={args.report}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

`run_lcqi_benchmark.sh` 是本地 benchmark 收集脚本。它适合在固定机器上反复运行，把同一组 shape 和 backend 的结果落到报告文件。

Listing 16.21 tools/run_lcqi_benchmark.sh

```bash
#!/usr/bin/env bash
set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
VOLUME_DIR="$(cd "${SCRIPT_DIR}/.." && pwd)"
REPO_DIR="$(cd "${VOLUME_DIR}/../.." && pwd)"
BUILD_DIR="${VOLUME_DIR}/build/lcqi-release"
RESULT_DIR="${VOLUME_DIR}/results"
REPEAT="${1:-200}"
STAMP="$(date -u +%Y%m%dT%H%M%SZ)"
CSV_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.csv"
META_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.meta.txt"
PERF_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.perf.txt"
```

```

mkdir -p "${RESULT_DIR}"

cmake -S "${VOLUME_DIR}" -B "${BUILD_DIR}" -DCMAKE_BUILD_TYPE=Release
cmake --build "${BUILD_DIR}"
ctest --test-dir "${BUILD_DIR}" --output-on-failure

{
  echo "timestamp_utc=${STAMP}"
  echo "repeat=${REPEAT}"
  echo "commit=$(git -C "${REPO_DIR}" rev-parse --short HEAD)"
  echo "uname=$(uname -a)"
  echo "compiler=$(${CXX:-c++} --version | sed -n '1p')"
  echo "cmake=$(cmake --version | sed -n '1p')"
  echo "bench=${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench"
} > "${META_PATH}"

"${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench" "${REPEAT}" > "${CSV_PATH}"

if command -v perf >/dev/null 2>&1; then
  perf stat \
    -e cycles,instructions,cache-misses,branches,branch-misses \
    "${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench" "${REPEAT}" \
    > /dev/null 2> "${PERF_PATH}" || true
else
  echo "perf_unavailable=1" > "${PERF_PATH}"
fi

echo "csv=${CSV_PATH}"
echo "meta=${META_PATH}"
echo "perf=${PERF_PATH}"
```

完整测试

`lcqi_tests.cpp` 把 safetensors、tokenizer、tiny MLP、int8 kernel、reference decoder、GPT-2 tiny path、trace、ledger 和 serving probe 串成验收网。GPT-2 优化不是只看速度：测试会把 full-prefix logits、旧 cached logits、优化 decoder logits、强制两 worker 的并行 decoder logits 和 logits-free greedy token 放在同一个 tiny prompt 下对齐，确认 predicted token、logits、KV filled token 数和 generated ids 不漂移。新增优化或 shape 时，必须先扩展这里的 golden 和边界样例。

Listing 16.22 tests/lcqi_tests.cpp

```cpp
#include <lcqi/gpt2_reference.hpp>
#include <lcqi/f32_kernels.hpp>
#include <lcqi/ggml_matvec.hpp>
#include <lcqi/ggml_tensors.hpp>
#include <lcqi/gguf.hpp>
#include <lcqi/inference.hpp>
#include <lcqi/int8_kernels.hpp>
#include <lcqi/llama_gguf.hpp>
#include <lcqi/model.hpp>
#include <lcqi/q4_k.hpp>
```

```cpp
#include <lcqi/reference_decoder.hpp>
#include <lcqi/safetensors.hpp>
#include <lcqi/tokenizer.hpp>

#include <array>
#include <cmath>
#include <cstring>
#include <cstdlib>
#include <filesystem>
#include <fstream>
#include <iostream>
#include <span>
#include <sstream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

namespace {

constexpr float LCQI_TEST_EPSILON = 1.0e-5F;
constexpr std::int32_t LCQI_TEST_INPUT_SIZE = 4;
constexpr std::int32_t LCQI_TEST_HIDDEN_SIZE = 3;
constexpr std::int32_t LCQI_TEST_OUTPUT_SIZE = 2;
constexpr std::int32_t LCQI_TEST_PREDICTED_CLASS = 1;
constexpr float LCQI_TEST_LOGIT_0 = -0.48F;
constexpr float LCQI_TEST_LOGIT_1 = 1.06F;
constexpr float LCQI_TEST_HIDDEN_0 = 0.0F;
constexpr float LCQI_TEST_HIDDEN_1 = 0.4F;
constexpr float LCQI_TEST_HIDDEN_2 = 1.4F;
constexpr float LCQI_RMS_EXPECTED_0 = 0.848528F;
constexpr float LCQI_RMS_EXPECTED_1 = 0.565685F;
constexpr float LCQI_ATTENTION_EXPECTED_0 = 2.0F;
constexpr float LCQI_ATTENTION_EXPECTED_1 = 3.0F;
constexpr float LCQI_KERNEL_MAX_DIFF = 1.0e-5F;
constexpr float LCQI_F32_KERNEL_MAX_DIFF = 1.0e-4F;
constexpr std::int32_t LCQI_SIMD_TEST_BLOCK = 8;
constexpr std::size_t LCQI_F32_DOT_TEST_SIZE = 257;
constexpr std::size_t LCQI_F32_ROW_TEST_INPUT_SIZE = 65;
constexpr std::size_t LCQI_F32_ROW_TEST_OUTPUT_SIZE = 17;
constexpr std::int32_t LCQI_REFERENCE_DECODER_PREDICTED_TOKEN = 0;
constexpr std::size_t LCQI_REFERENCE_DECODER_VOCAB_SIZE = 5;
constexpr std::array<float, LCQI_REFERENCE_DECODER_VOCAB_SIZE> LCQI_REFERENCE_DECODER_LOGITS{
    0.352516979F,
    -0.126884326F,
    0.0797837451F,
    -0.29797405F,
    0.127030417F,
};
constexpr std::int32_t LCQI_REFERENCE_TRACE_TOKEN_COUNT = 3;
constexpr std::int32_t LCQI_REFERENCE_TRACE_HIDDEN_SIZE = 4;
```

```
constexpr std::int32_t LCQI_REFERENCE_TRACE_FIRST_TOKEN = 0;
constexpr std::uint64_t LCQI_TOKENIZER_HASH_EXPECTED = 1345290284538833734ULL;
constexpr std::int32_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
constexpr std::int32_t LCQI_TEST_BYTE_BITS = 8;
constexpr std::uint64_t LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES = 48;
constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_BEGIN = 0;
constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_END = 24;
constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_BEGIN = 24;
constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_END = 48;
constexpr unsigned int LCQI_BYTE_MASK = 0xFFU;
constexpr std::int32_t LCQI_GPT2_PREDICTED_TOKEN = 3;
constexpr std::size_t LCQI_GPT2_VOCAB_SIZE = 6;
constexpr std::array<float, LCQI_GPT2_VOCAB_SIZE> LCQI_GPT2_LOGITS{
    0.407358F,
    -0.504049F,
    -0.107834F,
    0.840337F,
    -0.450123F,
    -0.156763F,
};
constexpr std::int32_t LCQI_GPT2_GENERATED_LENGTH = 5;
constexpr std::int32_t LCQI_GPT2_TOKENIZER_HELLO_ID = 7;
constexpr std::uint16_t LCQI_TEST_F16_ONE = 0x3C00U;
constexpr std::uint16_t LCQI_TEST_F16_HALF = 0x3800U;
constexpr std::uint16_t LCQI_TEST_F16_NEGATIVE_TWO = 0xC000U;
constexpr std::uint16_t LCQI_TEST_BF16_ONE_POINT_FIVE = 0x3FC0U;
constexpr std::uint16_t LCQI_TEST_BF16_NEGATIVE_HALF = 0xBF00U;
constexpr std::uint32_t LCQI_GGUF_TEST_VERSION = 3;
constexpr std::uint32_t LCQI_GGUF_TEST_ALIGNMENT = 32;
constexpr std::int64_t LCQI_GGUF_TEST_TENSOR_COUNT = 1;
constexpr std::int64_t LCQI_GGUF_TEST_METADATA_COUNT = 3;
constexpr std::int32_t LCQI_GGUF_TYPE_UINT32 = 4;
constexpr std::int32_t LCQI_GGUF_TYPE_FLOAT32 = 6;
constexpr std::int32_t LCQI_GGUF_TYPE_STRING = 8;
constexpr std::int32_t LCQI_GGUF_TYPE_ARRAY = 9;
constexpr std::int32_t LCQI_GGUF_TENSOR_TYPE_Q4_K = 12;
constexpr std::uint64_t LCQI_GGUF_TEST_TENSOR_OFFSET = 0;
constexpr float LCQI_Q4K_TEST_BLOCK_SCALE = 0.5F;
constexpr float LCQI_Q4K_TEST_BLOCK_MIN = 0.25F;
constexpr float LCQI_Q4K_Q8_DIFF_LIMIT = 0.08F;
constexpr float LCQI_GGML_MATVEC_MAX_DIFF = 1.0e-4F;
constexpr float LCQI_GGML_Q8_MATVEC_MAX_DIFF = 0.2F;
constexpr std::int32_t LCQI_GGML_TEST_Q5_HALF_VALUES = 16;
constexpr std::int32_t LCQI_GGML_TEST_Q5_ZERO_POINT = 16;
constexpr std::int32_t LCQI_GGML_TEST_Q6_LANE = 5;
constexpr std::int32_t LCQI_LLAMA_TEST_HIDDEN = 4;
constexpr std::int32_t LCQI_LLAMA_TEST_HEADS = 2;
constexpr std::int32_t LCQI_LLAMA_TEST_KV_HEADS = 1;
constexpr std::int32_t LCQI_LLAMA_TEST_HEAD_DIM = 2;
constexpr std::int32_t LCQI_LLAMA_TEST_INTERMEDIATE = 4;
constexpr std::int32_t LCQI_LLAMA_TEST_VOCAB = 4;
constexpr std::int32_t LCQI_LLAMA_TEST_CONTEXT = 8;
```

```
constexpr std::int32_t LCQI_LLAMA_TEST_LAYER_COUNT = 1;
constexpr std::int32_t LCQI_LLAMA_TEST_EXPECTED_TOKEN = 2;
constexpr float LCQI_LLAMA_TEST_SMALL_ATTENTION_VALUE = 0.1F;
constexpr float LCQI_LLAMA_TEST_SMALL_OUTPUT_SCALE = 0.1F;
constexpr std::int32_t LCQI_LLAMA_Q4_TEST_HIDDEN = 256;
constexpr std::int32_t LCQI_LLAMA_Q4_TEST_HEADS = 1;
constexpr std::int32_t LCQI_LLAMA_Q4_TEST_KV_HEADS = 1;
constexpr std::int32_t LCQI_LLAMA_Q4_TEST_HEAD_DIM = 256;
constexpr std::int32_t LCQI_LLAMA_Q4_TEST_INTERMEDIATE = 256;
constexpr std::int32_t LCQI_LLAMA_Q4_TEST_VOCAB = 2;
constexpr std::int32_t LCQI_LLAMA_Q4_TEST_CONTEXT = 4;
constexpr std::int32_t LCQI_LLAMA_Q4_TEST_LAYER_COUNT = 1;
constexpr const char* LCQI_TEST_LLAMA_GGML_DIRECT_ENV = "LCQI_LLAMA_GGML_DIRECT";
constexpr const char* LCQI_TEST_TRUE_ENV = "1";

struct KernelShapeCase {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    std::int32_t output_block_size = 0;
};

struct RawTensorFixture {
    std::string name;
    std::string dtype;
    std::vector<std::int64_t> shape;
    std::vector<std::uint8_t> bytes;
};

struct FloatTensorFixture {
    std::string name;
    std::vector<std::int64_t> shape;
    std::vector<float> values;
};

struct GgufTensorFixture {
    std::string name;
    std::vector<std::int64_t> shape;
    lcqi::GgmlType type = lcqi::GgmlType::f32;
    std::vector<std::uint8_t> bytes;
    std::uint64_t relative_offset = 0;
};

struct ScopedEnvVar {
    std::string name;
    std::string old_value;
    bool had_old_value = false;

    ScopedEnvVar(const char* env_name, const char* value) : name(env_name) {
        const char* old = std::getenv(env_name);
        if (old != nullptr) {
            this->old_value = old;
```

```cpp
            this->had_old_value = true;
        }
        if (setenv(env_name, value, 1) != 0) {
            throw std::runtime_error("failed to set test environment variable");
        }
    }

    ScopedEnvVar(const ScopedEnvVar&) = delete;
    ScopedEnvVar& operator=(const ScopedEnvVar&) = delete;

    ~ScopedEnvVar() {
        if (this->had_old_value) {
            static_cast<void>(setenv(this->name.c_str(), this->old_value.c_str(), 1));
        } else {
            static_cast<void>(unsetenv(this->name.c_str()));
        }
    }
};

constexpr std::array<KernelShapeCase, 4> LCQI_KERNEL_SHAPE_CASES{{
    {17, 19, 8},
    {33, 7, 8},
    {64, 32, 16},
    {65, 31, 16},
}};

void require(bool condition, const char* message) {
    if (!condition) {
        throw std::runtime_error(message);
    }
}

void require_close(float actual, float expected, const char* message) {
    if (std::fabs(actual - expected) > LCQI_TEST_EPSILON) {
        throw std::runtime_error(message);
    }
}

std::filesystem::path test_model_path() {
    return std::filesystem::path(__FILE__).parent_path().parent_path() /
           "models" / "tiny_mlp_i8.txt";
}

std::filesystem::path test_tokenizer_path() {
    return std::filesystem::path(__FILE__).parent_path().parent_path() /
           "models" / "tiny_tokenizer.txt";
}

std::filesystem::path temp_file_path(const std::string& name) {
    return std::filesystem::temp_directory_path() / name;
```

```
}

void write_le_u64(std::ofstream& output, std::uint64_t value) {
    for (std::int32_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
        const char byte = static_cast<char>(
            (value >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK);
        output.write(&byte, 1);
    }
}

void append_le_u16(std::vector<std::uint8_t>& output, std::uint16_t value) {
    output.push_back(static_cast<std::uint8_t>(value & LCQI_BYTE_MASK));
    output.push_back(static_cast<std::uint8_t>((value >> LCQI_TEST_BYTE_BITS) &↩
    ↪ LCQI_BYTE_MASK));
}

void append_le_u32(std::vector<std::uint8_t>& output, std::uint32_t value) {
    for (std::int32_t i = 0; i < 4; ++i) {
        output.push_back(static_cast<std::uint8_t>(
            (value >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK));
    }
}

void append_le_i32(std::vector<std::uint8_t>& output, std::int32_t value) {
    std::uint32_t bits = 0;
    std::memcpy(&bits, &value, sizeof(bits));
    append_le_u32(output, bits);
}

void append_le_u64(std::vector<std::uint8_t>& output, std::uint64_t value) {
    for (std::int32_t i = 0; i < 8; ++i) {
        output.push_back(static_cast<std::uint8_t>(
            (value >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK));
    }
}

void append_le_i64(std::vector<std::uint8_t>& output, std::int64_t value) {
    std::uint64_t bits = 0;
    std::memcpy(&bits, &value, sizeof(bits));
    append_le_u64(output, bits);
}

void append_gguf_string(std::vector<std::uint8_t>& output, const std::string& ↩
↪ text) {
    append_le_u64(output, static_cast<std::uint64_t>(text.size()));
    output.insert(output.end(), text.begin(), text.end());
}

void append_le_f32(std::vector<std::uint8_t>& output, float value) {
    std::uint32_t bits = 0;
    std::memcpy(&bits, &value, sizeof(bits));
    for (std::int32_t i = 0; i < 4; ++i) {
```

```cpp
        output.push_back(static_cast<std::uint8_t>(
            (bits >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK));
    }
}

void write_safetensors_fixture(const std::filesystem::path& path,
                               const std::string& header,
                               std::uint64_t payload_bytes) {
    std::ofstream output(path, std::ios::binary);
    if (!output) {
        throw std::runtime_error("cannot create safetensors test fixture");
    }
    write_le_u64(output, static_cast<std::uint64_t>(header.size()));
    output.write(header.data(), static_cast<std::streamsize>(header.size()));
    for (std::uint64_t i = 0; i < payload_bytes; ++i) {
        const char byte = static_cast<char>(i & 0xFFU);
        output.write(&byte, 1);
    }
}

std::string json_shape(std::span<const std::int64_t> shape) {
    std::ostringstream stream;
    stream << '[';
    for (std::size_t index = 0; index < shape.size(); ++index) {
        if (index != 0) {
            stream << ',';
        }
        stream << shape[index];
    }
    stream << ']';
    return stream.str();
}

void write_safetensors_raw_fixture(const std::filesystem::path& path,
                                   std::span<const RawTensorFixture> tensors) {
    std::ostringstream header;
    header << "{\"__metadata__\":{\"format\":\"lcqi-test\"}";
    std::uint64_t offset = 0;
    for (const RawTensorFixture& tensor : tensors) {
        header << ",\"" << tensor.name << "\":{\"dtype\":\"" << tensor.dtype
               << "\",\"shape\":" << json_shape(tensor.shape)
               << ",\"data_offsets\":[" << offset << ','
               << offset + tensor.bytes.size() << "]}";
        offset += tensor.bytes.size();
    }
    header << '}';

    std::ofstream output(path, std::ios::binary);
    if (!output) {
        throw std::runtime_error("cannot create safetensors raw test fixture");
    }
    const std::string header_text = header.str();
```

```cpp
    write_le_u64(output, static_cast<std::uint64_t>(header_text.size()));
    output.write(header_text.data(), static_cast<std::streamsize>(header_text.size()));
    for (const RawTensorFixture& tensor : tensors) {
        output.write(reinterpret_cast<const char*>(tensor.bytes.data()),
                     static_cast<std::streamsize>(tensor.bytes.size()));
    }
}

std::vector<std::uint8_t> f32_payload(std::span<const float> values) {
    std::vector<std::uint8_t> bytes;
    bytes.reserve(values.size() * 4);
    for (const float value : values) {
        append_le_f32(bytes, value);
    }
    return bytes;
}

std::vector<float> transpose_linear_to_hf_conv1d(const lcqi::Gpt2LinearF32& linear) {
    std::vector<float> transposed(
        static_cast<std::size_t>(linear.input_size) *
            static_cast<std::size_t>(linear.output_size),
        0.0F);
    for (std::int32_t out = 0; out < linear.output_size; ++out) {
        for (std::int32_t in = 0; in < linear.input_size; ++in) {
            transposed[static_cast<std::size_t>(in) *
                           static_cast<std::size_t>(linear.output_size) +
                       static_cast<std::size_t>(out)] =
                linear.weights[static_cast<std::size_t>(out) *
                                   static_cast<std::size_t>(linear.input_size) +
                               static_cast<std::size_t>(in)];
        }
    }
    return transposed;
}

void add_f32_tensor(std::vector<RawTensorFixture>& tensors,
                    std::string name,
                    std::vector<std::int64_t> shape,
                    std::span<const float> values) {
    tensors.push_back(RawTensorFixture{
        std::move(name),
        "F32",
        std::move(shape),
        f32_payload(values),
    });
}

void write_text_file(const std::filesystem::path& path, const std::string& text) {
```

```cpp
    std::ofstream output(path);
    if (!output) {
        throw std::runtime_error("cannot create text fixture");
    }
    output << text;
}

std::vector<std::uint8_t> make_q4_k_test_block() {
    std::vector<std::uint8_t> block(
        static_cast<std::size_t>(lcqi::LCQI_Q4_K_BLOCK_BYTES),
        0);
    block[0] = 0x00U;
    block[1] = 0x38U;
    block[2] = 0x00U;
    block[3] = 0x34U;

    std::array<std::uint8_t, static_cast<std::size_t>(lcqi::LCQI_Q4_K_SUBBLOCKS)> scales{
        1, 2, 3, 4, 5, 6, 7, 8,
    };
    std::array<std::uint8_t, static_cast<std::size_t>(lcqi::LCQI_Q4_K_SUBBLOCKS)> mins{
        1, 1, 2, 2, 3, 3, 4, 4,
    };
    for (std::size_t index = 0; index < 4; ++index) {
        block[4 + index] = static_cast<std::uint8_t>(
            (scales[index] & 0x3FU) | ((scales[index + 4] >> 4U) << 6U));
        block[8 + index] = static_cast<std::uint8_t>(
            (mins[index] & 0x3FU) | ((mins[index + 4] >> 4U) << 6U));
        block[12 + index] = static_cast<std::uint8_t>(
            (scales[index + 4] & 0x0FU) | ((mins[index + 4] & 0x0FU) << 4U));
    }

    for (std::int32_t pair = 0; pair < 4; ++pair) {
        const std::int32_t low_subblock = pair * 2;
        const std::int32_t high_subblock = low_subblock + 1;
        for (std::int32_t index = 0; index < lcqi::LCQI_Q4_K_SUBBLOCK_VALUES; ++index) {
            const std::uint8_t low = static_cast<std::uint8_t>(
                (low_subblock + index) & 0x0F);
            const std::uint8_t high = static_cast<std::uint8_t>(
                (high_subblock + index + 1) & 0x0F);
            block[static_cast<std::size_t>(16 + pair * 32 + index)] =
                static_cast<std::uint8_t>(low | (high << 4U));
        }
    }
    return block;
}

std::vector<float> expected_q4_k_test_values() {
    const std::array<float, static_cast<std::size_t>(lcqi::LCQI_Q4_K_SUBBLOCKS)> scales{
```

```cpp
        1, 2, 3, 4, 5, 6, 7, 8,
    };
    const std::array<float, static_cast<std::size_t>(lcqi::LCQI_Q4_K_SUBBLOCKS)> mins{
        1, 1, 2, 2, 3, 3, 4, 4,
    };
    std::vector<float> values(
        static_cast<std::size_t>(lcqi::LCQI_QK_K_BLOCK_VALUES),
        0.0F);
    for (std::int32_t subblock = 0; subblock < lcqi::LCQI_Q4_K_SUBBLOCKS; ++subblock) {
        for (std::int32_t index = 0; index < lcqi::LCQI_Q4_K_SUBBLOCK_VALUES; ++index) {
            const std::int32_t quantized =
                (subblock % 2 == 0)
                    ? ((subblock + index) & 0x0F)
                    : ((subblock + index + 1) & 0x0F);
            values[static_cast<std::size_t>(
                subblock * lcqi::LCQI_Q4_K_SUBBLOCK_VALUES + index)] =
                LCQI_Q4K_TEST_BLOCK_SCALE * scales[static_cast<std::size_t>(subblock)] *
                    static_cast<float>(quantized) -
                LCQI_Q4K_TEST_BLOCK_MIN * mins[static_cast<std::size_t>(subblock)];
        }
    }
    return values;
}

std::vector<std::uint8_t> make_q8_0_test_block() {
    std::vector<std::uint8_t> block(
        static_cast<std::size_t>(lcqi::LCQI_Q8_0_BLOCK_BYTES),
        0);
    block[0] = static_cast<std::uint8_t>(LCQI_TEST_F16_HALF & LCQI_BYTE_MASK);
    block[1] = static_cast<std::uint8_t>((LCQI_TEST_F16_HALF >> LCQI_TEST_BYTE_BITS) &
                                         LCQI_BYTE_MASK);
    for (std::int32_t index = 0; index < lcqi::LCQI_Q8_0_BLOCK_VALUES; ++index) {
        block[static_cast<std::size_t>(2 + index)] =
            static_cast<std::uint8_t>(static_cast<std::int8_t>(index - 16));
    }
    return block;
}

std::vector<std::uint8_t> make_q5_0_test_block() {
    std::vector<std::uint8_t> block(
        static_cast<std::size_t>(lcqi::LCQI_Q5_0_BLOCK_BYTES),
        0);
    block[0] = static_cast<std::uint8_t>(LCQI_TEST_F16_HALF & LCQI_BYTE_MASK);
    block[1] = static_cast<std::uint8_t>((LCQI_TEST_F16_HALF >> LCQI_TEST_BYTE_BITS) &
```

```
                                                    LCQI_BYTE_MASK);

    std::uint32_t qh = 0;
    for (std::int32_t index = 0; index < lcqi::LCQI_Q5_0_BLOCK_VALUES / 2; ++index) {
        const std::int32_t first_quantized = index - LCQI_GGML_TEST_Q5_ZERO_POINT;
        const std::int32_t second_quantized = index;
        const std::uint8_t first_encoded =
            static_cast<std::uint8_t>(first_quantized + LCQI_GGML_TEST_Q5_ZERO_POINT);
        const std::uint8_t second_encoded =
            static_cast<std::uint8_t>(second_quantized + LCQI_GGML_TEST_Q5_ZERO_POINT);
        if ((first_encoded & 0x10U) != 0U) {
            qh |= (1U << index);
        }
        if ((second_encoded & 0x10U) != 0U) {
            qh |= (1U << (index + LCQI_GGML_TEST_Q5_HALF_VALUES));
        }
        const std::uint8_t low = static_cast<std::uint8_t>(first_encoded & 0x0FU);
        const std::uint8_t high = static_cast<std::uint8_t>(second_encoded & 0x0FU);
        block[static_cast<std::size_t>(6 + index)] =
            static_cast<std::uint8_t>(low | (high << 4U));
    }
    block[2] = static_cast<std::uint8_t>(qh & LCQI_BYTE_MASK);
    block[3] = static_cast<std::uint8_t>((qh >> LCQI_TEST_BYTE_BITS) & LCQI_BYTE_MASK);
    block[4] = static_cast<std::uint8_t>((qh >> (LCQI_TEST_BYTE_BITS * 2)) & LCQI_BYTE_MASK);
    block[5] = static_cast<std::uint8_t>((qh >> (LCQI_TEST_BYTE_BITS * 3)) & LCQI_BYTE_MASK);
    return block;
}

std::vector<std::uint8_t> make_q6_k_test_block() {
    std::vector<std::uint8_t> block(
        static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_BYTES),
        0);
    const std::size_t ql_offset = 0;
    const std::size_t qh_offset = static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_VALUES / 2);
    const std::size_t scales_offset =
        qh_offset + static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_VALUES / 4);
    for (std::int32_t index = 0; index < 16; ++index) {
        block[scales_offset + static_cast<std::size_t>(index)] = 1;
    }
    block[static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_BYTES - 2)] =
        static_cast<std::uint8_t>(LCQI_TEST_F16_ONE & LCQI_BYTE_MASK);
    block[static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_BYTES - 1)] =
```

```
        static_cast<std::uint8_t>((LCQI_TEST_F16_ONE >> LCQI_TEST_BYTE_BITS) &
                                  LCQI_BYTE_MASK);

    const auto store_q6 = [&block, ql_offset, qh_offset](std::int32_t position,
                                                         std::uint8_t encoded) {
        const std::int32_t half = position / 128;
        const std::int32_t local = position % 128;
        const std::size_t ql_base = ql_offset + static_cast<std::size_t>(half * 64);
        const std::size_t qh_base = qh_offset + static_cast<std::size_t>(half * 32);
        const std::uint8_t low = encoded & 0x0FU;
        const std::uint8_t high = static_cast<std::uint8_t>((encoded >> 4U) & 0x03U);
        if (local < 32) {
            block[ql_base + static_cast<std::size_t>(local)] =
                static_cast<std::uint8_t>(
                    (block[ql_base + static_cast<std::size_t>(local)] & 0xF0U) | low);
            block[qh_base + static_cast<std::size_t>(local)] =
                static_cast<std::uint8_t>(
                    (block[qh_base + static_cast<std::size_t>(local)] & 0xFCU) | high);
        } else if (local < 64) {
            const std::size_t lane = static_cast<std::size_t>(local - 32);
            block[ql_base + static_cast<std::size_t>(32) + lane] =
                static_cast<std::uint8_t>(
                    (block[ql_base + static_cast<std::size_t>(32) + lane] & 0xF0U) | low);
            block[qh_base + lane] =
                static_cast<std::uint8_t>((block[qh_base + lane] & 0xF3U) | (high << 2U));
        } else if (local < 96) {
            const std::size_t lane = static_cast<std::size_t>(local - 64);
            block[ql_base + lane] =
                static_cast<std::uint8_t>(
                    (block[ql_base + lane] & 0x0FU) | (low << 4U));
            block[qh_base + lane] =
                static_cast<std::uint8_t>((block[qh_base + lane] & 0xCFU) | (high << 4U));
        } else {
            const std::size_t lane = static_cast<std::size_t>(local - 96);
            block[ql_base + static_cast<std::size_t>(32) + lane] =
                static_cast<std::uint8_t>(
                    (block[ql_base + static_cast<std::size_t>(32) + lane] & 0x0FU) |
                    (low << 4U));
            block[qh_base + lane] =
                static_cast<std::uint8_t>((block[qh_base + lane] & 0x3FU) | (high << 6U));
        }
    };
```

```
    store_q6(LCQI_GGML_TEST_Q6_LANE, 35);
    store_q6(32 + LCQI_GGML_TEST_Q6_LANE, 31);
    store_q6(64 + LCQI_GGML_TEST_Q6_LANE, 63);
    store_q6(96 + LCQI_GGML_TEST_Q6_LANE, 0);
    store_q6(128 + LCQI_GGML_TEST_Q6_LANE, 36);
    return block;
}

void write_gguf_q4_k_fixture(const std::filesystem::path& path) {
    std::vector<std::uint8_t> bytes;
    bytes.push_back('G');
    bytes.push_back('G');
    bytes.push_back('U');
    bytes.push_back('F');
    append_le_u32(bytes, LCQI_GGUF_TEST_VERSION);
    append_le_i64(bytes, LCQI_GGUF_TEST_TENSOR_COUNT);
    append_le_i64(bytes, LCQI_GGUF_TEST_METADATA_COUNT);

    append_gguf_string(bytes, "general.alignment");
    append_le_i32(bytes, LCQI_GGUF_TYPE_UINT32);
    append_le_u32(bytes, LCQI_GGUF_TEST_ALIGNMENT);
    append_gguf_string(bytes, "general.architecture");
    append_le_i32(bytes, LCQI_GGUF_TYPE_STRING);
    append_gguf_string(bytes, "lcqi-test");
    append_gguf_string(bytes, "tokenizer.ggml.tokens");
    append_le_i32(bytes, LCQI_GGUF_TYPE_ARRAY);
    append_le_i32(bytes, LCQI_GGUF_TYPE_STRING);
    append_le_u64(bytes, 2);
    append_gguf_string(bytes, "<bos>");
    append_gguf_string(bytes, "hello");

    append_gguf_string(bytes, "blk.0.ffn_up.weight");
    append_le_u32(bytes, 2);
    append_le_i64(bytes, lcqi::LCQI_QK_K_BLOCK_VALUES);
    append_le_i64(bytes, 1);
    append_le_i32(bytes, LCQI_GGUF_TENSOR_TYPE_Q4_K);
    append_le_u64(bytes, LCQI_GGUF_TEST_TENSOR_OFFSET);
    while (bytes.size() % LCQI_GGUF_TEST_ALIGNMENT != 0) {
        bytes.push_back(0);
    }
    const std::vector<std::uint8_t> block = make_q4_k_test_block();
    bytes.insert(bytes.end(), block.begin(), block.end());

    std::ofstream output(path, std::ios::binary);
    if (!output) {
        throw std::runtime_error("cannot create GGUF fixture");
    }
    output.write(reinterpret_cast<const char*>(bytes.data()),
                 static_cast<std::streamsize>(bytes.size()));
}

```

```cpp
void append_gguf_uint32_metadata(std::vector<std::uint8_t>& bytes,
                                 const std::string& key,
                                 std::uint32_t value) {
    append_gguf_string(bytes, key);
    append_le_i32(bytes, LCQI_GGUF_TYPE_UINT32);
    append_le_u32(bytes, value);
}

void append_gguf_float32_metadata(std::vector<std::uint8_t>& bytes,
                                  const std::string& key,
                                  float value) {
    append_gguf_string(bytes, key);
    append_le_i32(bytes, LCQI_GGUF_TYPE_FLOAT32);
    append_le_f32(bytes, value);
}

void append_gguf_string_metadata(std::vector<std::uint8_t>& bytes,
                                 const std::string& key,
                                 const std::string& value) {
    append_gguf_string(bytes, key);
    append_le_i32(bytes, LCQI_GGUF_TYPE_STRING);
    append_gguf_string(bytes, value);
}

void write_gguf_fixture(const std::filesystem::path& path,
                        std::span<GgufTensorFixture> tensors,
                        const std::vector<std::uint8_t>& metadata_bytes) {
    std::uint64_t relative_offset = 0;
    for (GgufTensorFixture& tensor : tensors) {
        tensor.relative_offset = relative_offset;
        relative_offset += tensor.bytes.size();
        while (relative_offset % LCQI_GGUF_TEST_ALIGNMENT != 0) {
            ++relative_offset;
        }
    }

    std::vector<std::uint8_t> bytes;
    bytes.push_back('G');
    bytes.push_back('G');
    bytes.push_back('U');
    bytes.push_back('F');
    append_le_u32(bytes, LCQI_GGUF_TEST_VERSION);
    append_le_u64(bytes, static_cast<std::uint64_t>(tensors.size()));
    append_le_u64(bytes, 14);
    bytes.insert(bytes.end(), metadata_bytes.begin(), metadata_bytes.end());

    for (const GgufTensorFixture& tensor : tensors) {
        append_gguf_string(bytes, tensor.name);
        append_le_u32(bytes, static_cast<std::uint32_t>(tensor.shape.size()));
        for (const std::int64_t extent : tensor.shape) {
            append_le_i64(bytes, extent);
        }
```

```
        append_le_i32(bytes, static_cast<std::int32_t>(tensor.type));
        append_le_u64(bytes, tensor.relative_offset);
    }
    while (bytes.size() % LCQI_GGUF_TEST_ALIGNMENT != 0) {
        bytes.push_back(0);
    }
    std::uint64_t written_relative = 0;
    for (const GgufTensorFixture& tensor : tensors) {
        while (written_relative < tensor.relative_offset) {
            bytes.push_back(0);
            ++written_relative;
        }
        bytes.insert(bytes.end(), tensor.bytes.begin(), tensor.bytes.end());
        written_relative += tensor.bytes.size();
    }

    std::ofstream output(path, std::ios::binary);
    if (!output) {
        throw std::runtime_error("cannot create GGUF fixture");
    }
    output.write(reinterpret_cast<const char*>(bytes.data()),
                 static_cast<std::streamsize>(bytes.size()));
}

void add_gguf_f32_tensor(std::vector<GgufTensorFixture>& tensors,
                         std::string name,
                         std::vector<std::int64_t> shape,
                         std::span<const float> values) {
    tensors.push_back(GgufTensorFixture{
        std::move(name),
        std::move(shape),
        lcqi::GgmlType::f32,
        f32_payload(values),
        0,
    });
}

void add_gguf_q4_k_tensor(std::vector<GgufTensorFixture>& tensors,
                          std::string name,
                          std::vector<std::int64_t> shape,
                          const std::vector<std::uint8_t>& bytes) {
    tensors.push_back(GgufTensorFixture{
        std::move(name),
        std::move(shape),
        lcqi::GgmlType::q4_k,
        bytes,
        0,
    });
}

void add_gguf_quantized_tensor(std::vector<GgufTensorFixture>& tensors,
                               std::string name,
```

```
                              std::vector<std::int64_t> shape,
                              lcqi::GgmlType type,
                              const std::vector<std::uint8_t>& bytes) {
    tensors.push_back(GgufTensorFixture{
        std::move(name),
        std::move(shape),
        type,
        bytes,
        0,
    });
}

std::vector<std::uint8_t> repeated_ggml_block(const std::vector<std::uint8_t>& block,
                                             std::int32_t input_size,
                                             std::int32_t output_size,
                                             std::int64_t block_values) {
    const std::size_t block_count =
        static_cast<std::size_t>((input_size / block_values) * output_size);
    std::vector<std::uint8_t> bytes;
    bytes.reserve(block.size() * block_count);
    for (std::size_t index = 0; index < block_count; ++index) {
        bytes.insert(bytes.end(), block.begin(), block.end());
    }
    return bytes;
}

void write_tiny_llama_gguf_fixture(const std::filesystem::path& path) {
    std::vector<std::uint8_t> metadata;
    append_gguf_uint32_metadata(metadata, "general.alignment", LCQI_GGUF_TEST_ALIGNMENT);
    append_gguf_string_metadata(metadata, "general.architecture", "llama");
    append_gguf_string_metadata(metadata, "general.name", "lcqi tiny llama");
    append_gguf_uint32_metadata(metadata, "llama.embedding_length", LCQI_LLAMA_TEST_HIDDEN);
    append_gguf_uint32_metadata(metadata, "llama.block_count", LCQI_LLAMA_TEST_LAYER_COUNT);
    append_gguf_uint32_metadata(metadata, "llama.feed_forward_length", LCQI_LLAMA_TEST_INTERMEDIATE);
    append_gguf_uint32_metadata(metadata, "llama.attention.head_count", LCQI_LLAMA_TEST_HEADS);
    append_gguf_uint32_metadata(metadata, "llama.attention.head_count_kv", LCQI_LLAMA_TEST_KV_HEADS);
    append_gguf_uint32_metadata(metadata, "llama.rope.dimension_count", LCQI_LLAMA_TEST_HEAD_DIM);
    append_gguf_float32_metadata(metadata, "llama.rope.freq_base", 10000.0F);
    append_gguf_uint32_metadata(metadata, "llama.context_length", LCQI_LLAMA_TEST_CONTEXT);
    append_gguf_uint32_metadata(metadata, "llama.vocab_size", LCQI_LLAMA_TEST_VOCAB);
    append_gguf_float32_metadata(metadata, "llama.attention.layer_norm_rms_epsilon", 1.0e-5F);
```

```cpp
    append_gguf_uint32_metadata(metadata, "tokenizer.ggml.eos_token_id", 3);

    std::vector<GgufTensorFixture> tensors;
    const std::vector<float> embedding{
        1.0F, 0.0F, 0.0F, 0.0F,
        0.0F, 1.0F, 0.0F, 0.0F,
        0.0F, 2.0F, 0.0F, 0.0F,
        0.0F, 0.0F, 1.0F, 0.0F,
    };
    add_gguf_f32_tensor(tensors,
                        "token_embd.weight",
                        {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_VOCAB},
                        embedding);
    add_gguf_f32_tensor(tensors,
                        "output_norm.weight",
                        {LCQI_LLAMA_TEST_HIDDEN},
                        std::vector<float>(LCQI_LLAMA_TEST_HIDDEN, 1.0F));
    add_gguf_f32_tensor(tensors,
                        "blk.0.attn_norm.weight",
                        {LCQI_LLAMA_TEST_HIDDEN},
                        std::vector<float>(LCQI_LLAMA_TEST_HIDDEN, 1.0F));
    add_gguf_f32_tensor(tensors,
                        "blk.0.ffn_norm.weight",
                        {LCQI_LLAMA_TEST_HIDDEN},
                        std::vector<float>(LCQI_LLAMA_TEST_HIDDEN, 1.0F));
    const std::vector<float> hidden_to_hidden(
        static_cast<std::size_t>(LCQI_LLAMA_TEST_HIDDEN * LCQI_LLAMA_TEST_HIDDEN),
        0.0F);
    std::vector<float> hidden_to_kv(
        static_cast<std::size_t>(LCQI_LLAMA_TEST_HIDDEN * LCQI_LLAMA_TEST_HEAD_DIM),
        0.0F);
    hidden_to_kv[1] = LCQI_LLAMA_TEST_SMALL_ATTENTION_VALUE;
    std::vector<float> attention_output = hidden_to_hidden;
    attention_output[0] = LCQI_LLAMA_TEST_SMALL_OUTPUT_SCALE;
    attention_output[10] = LCQI_LLAMA_TEST_SMALL_OUTPUT_SCALE;
    const std::vector<float> hidden_to_intermediate(
        static_cast<std::size_t>(LCQI_LLAMA_TEST_HIDDEN * LCQI_LLAMA_TEST_INTERMEDIATE),
        0.0F);
    const std::vector<float> intermediate_to_hidden(
        static_cast<std::size_t>(LCQI_LLAMA_TEST_INTERMEDIATE * LCQI_LLAMA_TEST_HIDDEN),
        0.0F);
    add_gguf_f32_tensor(tensors,
                        "blk.0.attn_q.weight",
                        {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_HIDDEN},
                        hidden_to_hidden);
    add_gguf_f32_tensor(tensors,
                        "blk.0.attn_k.weight",
                        {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_HEAD_DIM},
```

```cpp
                        hidden_to_kv);
    add_gguf_f32_tensor(tensors,
                        "blk.0.attn_v.weight",
                        {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_HEAD_DIM},
                        hidden_to_kv);
    add_gguf_f32_tensor(tensors,
                        "blk.0.attn_output.weight",
                        {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_HIDDEN},
                        attention_output);
    add_gguf_f32_tensor(tensors,
                        "blk.0.ffn_gate.weight",
                        {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_INTERMEDIATE},
                        hidden_to_intermediate);
    add_gguf_f32_tensor(tensors,
                        "blk.0.ffn_up.weight",
                        {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_INTERMEDIATE},
                        hidden_to_intermediate);
    add_gguf_f32_tensor(tensors,
                        "blk.0.ffn_down.weight",
                        {LCQI_LLAMA_TEST_INTERMEDIATE, LCQI_LLAMA_TEST_HIDDEN},
                        intermediate_to_hidden);
    write_gguf_fixture(path, tensors, metadata);
}

void write_tiny_llama_ggml_direct_gguf_fixture(const std::filesystem::path& path) {
    std::vector<std::uint8_t> metadata;
    append_gguf_uint32_metadata(metadata, "general.alignment", LCQI_GGUF_TEST_ALIGNMENT);
    append_gguf_string_metadata(metadata, "general.architecture", "llama");
    append_gguf_string_metadata(metadata, "general.name", "lcqi ggml direct tiny llama");
    append_gguf_uint32_metadata(metadata, "llama.embedding_length", LCQI_LLAMA_Q4_TEST_HIDDEN);
    append_gguf_uint32_metadata(metadata, "llama.block_count", LCQI_LLAMA_Q4_TEST_LAYER_COUNT);
    append_gguf_uint32_metadata(metadata,
                                "llama.feed_forward_length",
                                LCQI_LLAMA_Q4_TEST_INTERMEDIATE);
    append_gguf_uint32_metadata(metadata, "llama.attention.head_count", LCQI_LLAMA_Q4_TEST_HEADS);
    append_gguf_uint32_metadata(metadata,
                                "llama.attention.head_count_kv",
                                LCQI_LLAMA_Q4_TEST_KV_HEADS);
    append_gguf_uint32_metadata(metadata,
                                "llama.rope.dimension_count",
                                LCQI_LLAMA_Q4_TEST_HEAD_DIM);
    append_gguf_float32_metadata(metadata, "llama.rope.freq_base", 10000.0F);
    append_gguf_uint32_metadata(metadata, "llama.context_length", LCQI_LLAMA_Q4_TEST_CONTEXT);
    append_gguf_uint32_metadata(metadata, "llama.vocab_size", LCQI_LLAMA_Q4_TEST_VOCAB);
```

```
    append_gguf_float32_metadata(metadata, "llama.attention.layer_norm_rms_epsilon", 1.0e-5F);
    append_gguf_uint32_metadata(metadata, "tokenizer.ggml.eos_token_id", 1);

    std::vector<GgufTensorFixture> tensors;
    std::vector<float> embedding(
        static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN * LCQI_LLAMA_Q4_TEST_VOCAB),
        0.0F);
    embedding[0] = 1.0F;
    embedding[static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN)] = 0.5F;
    add_gguf_f32_tensor(tensors,
                        "token_embd.weight",
                        {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_VOCAB},
                        embedding);
    add_gguf_f32_tensor(tensors,
                        "output_norm.weight",
                        {LCQI_LLAMA_Q4_TEST_HIDDEN},
                        std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));
    add_gguf_f32_tensor(tensors,
                        "blk.0.attn_norm.weight",
                        {LCQI_LLAMA_Q4_TEST_HIDDEN},
                        std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));
    add_gguf_f32_tensor(tensors,
                        "blk.0.ffn_norm.weight",
                        {LCQI_LLAMA_Q4_TEST_HIDDEN},
                        std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));

    const std::vector<std::uint8_t> q5_matrix = repeated_ggml_block(
        std::vector<std::uint8_t>(static_cast<std::size_t>(lcqi::LCQI_Q5_0_BLOCK_BYTES), 0),
        LCQI_LLAMA_Q4_TEST_HIDDEN,
        LCQI_LLAMA_Q4_TEST_HIDDEN,
        lcqi::LCQI_Q5_0_BLOCK_VALUES);
    const std::vector<std::uint8_t> q8_matrix = repeated_ggml_block(
        std::vector<std::uint8_t>(static_cast<std::size_t>(lcqi::LCQI_Q8_0_BLOCK_BYTES), 0),
        LCQI_LLAMA_Q4_TEST_HIDDEN,
        LCQI_LLAMA_Q4_TEST_HIDDEN,
        lcqi::LCQI_Q8_0_BLOCK_VALUES);
    const std::vector<std::uint8_t> q6_matrix = repeated_ggml_block(
        std::vector<std::uint8_t>(static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_BYTES), 0),
        LCQI_LLAMA_Q4_TEST_INTERMEDIATE,
        LCQI_LLAMA_Q4_TEST_HIDDEN,
        lcqi::LCQI_Q6_K_BLOCK_VALUES);

    add_gguf_quantized_tensor(tensors,
                              "blk.0.attn_q.weight",
                              {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_HIDDEN},
                              lcqi::GgmlType::q5_0,
```

```
                              q5_matrix);
    add_gguf_quantized_tensor(tensors,
                              "blk.0.attn_k.weight",
                              {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_HEAD_DIM},
                              lcqi::GgmlType::q5_0,
                              q5_matrix);
    add_gguf_quantized_tensor(tensors,
                              "blk.0.attn_v.weight",
                              {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_HEAD_DIM},
                              lcqi::GgmlType::q8_0,
                              q8_matrix);
    add_gguf_quantized_tensor(tensors,
                              "blk.0.attn_output.weight",
                              {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_HIDDEN},
                              lcqi::GgmlType::q5_0,
                              q5_matrix);
    add_gguf_quantized_tensor(tensors,
                              "blk.0.ffn_gate.weight",
                              {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_INTERMEDIATE},
                              lcqi::GgmlType::q5_0,
                              q5_matrix);
    add_gguf_quantized_tensor(tensors,
                              "blk.0.ffn_up.weight",
                              {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_INTERMEDIATE},
                              lcqi::GgmlType::q8_0,
                              q8_matrix);
    add_gguf_quantized_tensor(tensors,
                              "blk.0.ffn_down.weight",
                              {LCQI_LLAMA_Q4_TEST_INTERMEDIATE, LCQI_LLAMA_Q4_TEST_HIDDEN},
                              lcqi::GgmlType::q6_k,
                              q6_matrix);
    write_gguf_fixture(path, tensors, metadata);
}

void write_tiny_llama_q4_direct_gguf_fixture(const std::filesystem::path& path) {
    std::vector<std::uint8_t> metadata;
    append_gguf_uint32_metadata(metadata, "general.alignment", LCQI_GGUF_TEST_ALIGNMENT);
    append_gguf_string_metadata(metadata, "general.architecture", "llama");
    append_gguf_string_metadata(metadata, "general.name", "lcqi q4 direct tiny llama");
    append_gguf_uint32_metadata(metadata, "llama.embedding_length", LCQI_LLAMA_Q4_TEST_HIDDEN);
    append_gguf_uint32_metadata(metadata, "llama.block_count", LCQI_LLAMA_Q4_TEST_LAYER_COUNT);
```

```
    append_gguf_uint32_metadata(metadata, "llama.feed_forward_length", LCQI_LLAMA_Q4_TEST_INTERMEDIATE);
    append_gguf_uint32_metadata(metadata, "llama.attention.head_count", LCQI_LLAMA_Q4_TEST_HEADS);
    append_gguf_uint32_metadata(metadata, "llama.attention.head_count_kv", LCQI_LLAMA_Q4_TEST_KV_HEADS);
    append_gguf_uint32_metadata(metadata, "llama.rope.dimension_count", LCQI_LLAMA_Q4_TEST_HEAD_DIM);
    append_gguf_float32_metadata(metadata, "llama.rope.freq_base", 10000.0F);
    append_gguf_uint32_metadata(metadata, "llama.context_length", LCQI_LLAMA_Q4_TEST_CONTEXT);
    append_gguf_uint32_metadata(metadata, "llama.vocab_size", LCQI_LLAMA_Q4_TEST_VOCAB);
    append_gguf_float32_metadata(metadata, "llama.attention.layer_norm_rms_epsilon", 1.0e-5F);
    append_gguf_uint32_metadata(metadata, "tokenizer.ggml.eos_token_id", 1);

    std::vector<GgufTensorFixture> tensors;
    std::vector<float> embedding(
        static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN * LCQI_LLAMA_Q4_TEST_VOCAB),
        0.0F);
    embedding[0] = 1.0F;
    embedding[static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN)] = 0.5F;
    add_gguf_f32_tensor(tensors,
                        "token_embd.weight",
                        {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_VOCAB},
                        embedding);
    add_gguf_f32_tensor(tensors,
                        "output_norm.weight",
                        {LCQI_LLAMA_Q4_TEST_HIDDEN},
                        std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));
    add_gguf_f32_tensor(tensors,
                        "blk.0.attn_norm.weight",
                        {LCQI_LLAMA_Q4_TEST_HIDDEN},
                        std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));
    add_gguf_f32_tensor(tensors,
                        "blk.0.ffn_norm.weight",
                        {LCQI_LLAMA_Q4_TEST_HIDDEN},
                        std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));

    const std::vector<std::uint8_t> zero_q4_matrix(
        static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN) *
            static_cast<std::size_t>(lcqi::LCQI_Q4_K_BLOCK_BYTES),
        0);
    add_gguf_q4_k_tensor(tensors,
                         "blk.0.attn_q.weight",
                         {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_HIDDEN},
                         zero_q4_matrix);
    add_gguf_q4_k_tensor(tensors,
                         "blk.0.attn_k.weight",
```

```cpp
                         {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_HEAD_DIM},
                         zero_q4_matrix);
    add_gguf_q4_k_tensor(tensors,
                         "blk.0.attn_v.weight",
                         {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_HEAD_DIM},
                         zero_q4_matrix);
    add_gguf_q4_k_tensor(tensors,
                         "blk.0.attn_output.weight",
                         {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_HIDDEN},
                         zero_q4_matrix);
    add_gguf_q4_k_tensor(tensors,
                         "blk.0.ffn_gate.weight",
                         {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_INTERMEDIATE},
                         zero_q4_matrix);
    add_gguf_q4_k_tensor(tensors,
                         "blk.0.ffn_up.weight",
                         {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_INTERMEDIATE},
                         zero_q4_matrix);
    add_gguf_q4_k_tensor(tensors,
                         "blk.0.ffn_down.weight",
                         {LCQI_LLAMA_Q4_TEST_INTERMEDIATE, LCQI_LLAMA_Q4_TEST_HIDDEN},
                         zero_q4_matrix);
    write_gguf_fixture(path, tensors, metadata);
}

void write_tiny_gpt2_safetensors(const std::filesystem::path& path,
                                 const lcqi::Gpt2ReferenceModel& model) {
    std::vector<RawTensorFixture> tensors;
    add_f32_tensor(tensors,
                   "transformer.wte.weight",
                   {model.config.vocab_size, model.config.hidden_size},
                   model.token_embedding);
    add_f32_tensor(tensors,
                   "transformer.wpe.weight",
                   {model.config.max_positions, model.config.hidden_size},
                   model.position_embedding);
    for (std::int32_t layer_id = 0; layer_id < model.config.layer_count; ++layer_id) {
        const lcqi::Gpt2LayerWeightsF32& layer =
            model.layers[static_cast<std::size_t>(layer_id)];
        const std::string prefix = "transformer.h." + std::to_string(layer_id) + ".";
        add_f32_tensor(tensors, prefix + "ln_1.weight", {model.config.hidden_size}, layer.ln_1_weight);
        add_f32_tensor(tensors, prefix + "ln_1.bias", {model.config.hidden_size}, layer.ln_1_bias);
```

```cpp
        const std::vector<float> c_attn_hf = transpose_linear_to_hf_conv1d(layer.c_attn);
        add_f32_tensor(tensors,
                       prefix + "attn.c_attn.weight",
                       {model.config.hidden_size,
                        model.config.hidden_size * 3},
                       c_attn_hf);
        add_f32_tensor(tensors,
                       prefix + "attn.c_attn.bias",
                       {model.config.hidden_size * 3},
                       layer.c_attn.bias);
        const std::vector<float> c_proj_hf = transpose_linear_to_hf_conv1d(layer.c_proj);
        add_f32_tensor(tensors,
                       prefix + "attn.c_proj.weight",
                       {model.config.hidden_size, model.config.hidden_size},
                       c_proj_hf);
        add_f32_tensor(tensors,
                       prefix + "attn.c_proj.bias",
                       {model.config.hidden_size},
                       layer.c_proj.bias);
        add_f32_tensor(tensors, prefix + "ln_2.weight", {model.config.hidden_size}, layer.ln_2_weight);
        add_f32_tensor(tensors, prefix + "ln_2.bias", {model.config.hidden_size}, layer.ln_2_bias);
        const std::vector<float> c_fc_hf = transpose_linear_to_hf_conv1d(layer.c_fc);
        add_f32_tensor(tensors,
                       prefix + "mlp.c_fc.weight",
                       {model.config.hidden_size, model.config.intermediate_size},
                       c_fc_hf);
        add_f32_tensor(tensors,
                       prefix + "mlp.c_fc.bias",
                       {model.config.intermediate_size},
                       layer.c_fc.bias);
        const std::vector<float> mlp_proj_hf =
            transpose_linear_to_hf_conv1d(layer.mlp_c_proj);
        add_f32_tensor(tensors,
                       prefix + "mlp.c_proj.weight",
                       {model.config.intermediate_size, model.config.hidden_size},
                       mlp_proj_hf);
        add_f32_tensor(tensors,
                       prefix + "mlp.c_proj.bias",
                       {model.config.hidden_size},
                       layer.mlp_c_proj.bias);
    }
    add_f32_tensor(tensors,
                   "transformer.ln_f.weight",
                   {model.config.hidden_size},
                   model.final_ln_weight);
```

```cpp
    add_f32_tensor(tensors,
                   "transformer.ln_f.bias",
                   {model.config.hidden_size},
                   model.final_ln_bias);
    write_safetensors_raw_fixture(path, tensors);
}

void test_tiny_mlp() {
    const lcqi::TinyMlpModel model = lcqi::load_model(test_model_path());
    require(model.input_size == LCQI_TEST_INPUT_SIZE, "input size mismatch");
    require(model.hidden_size == LCQI_TEST_HIDDEN_SIZE, "hidden size mismatch");
    require(model.output_size == LCQI_TEST_OUTPUT_SIZE, "output size mismatch");

    const std::vector<float> input{1.0F, 2.0F, -1.0F, 0.5F};
    const lcqi::InferenceResult result = lcqi::run_inference(model, input);

    require(result.predicted_class == LCQI_TEST_PREDICTED_CLASS, "unexpected predicted class");
    require(result.logits.size() == static_cast<std::size_t>(LCQI_TEST_OUTPUT_SIZE),
            "unexpected logits size");
    require_close(result.logits[0], LCQI_TEST_LOGIT_0, "logit 0 mismatch");
    require_close(result.logits[1], LCQI_TEST_LOGIT_1, "logit 1 mismatch");

    std::vector<float> hidden(static_cast<std::size_t>(LCQI_TEST_HIDDEN_SIZE), 0.0F);
    lcqi::linear_i8(model.hidden, input, hidden);
    lcqi::relu_inplace(hidden);
    require_close(hidden[0], LCQI_TEST_HIDDEN_0, "hidden 0 mismatch");
    require_close(hidden[1], LCQI_TEST_HIDDEN_1, "hidden 1 mismatch");
    require_close(hidden[2], LCQI_TEST_HIDDEN_2, "hidden 2 mismatch");
}

void test_tokenizer_contract() {
    const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(test_tokenizer_path());
    require(tokenizer.bos_id == 0, "tokenizer BOS id mismatch");
    require(tokenizer.eos_id == 4, "tokenizer EOS id mismatch");
    require(tokenizer.unk_id == -1, "tokenizer UNK id mismatch");
    require(tokenizer.chat_template_hash == "tiny-chat-template-v1",
            "tokenizer template hash mismatch");

    const std::vector<std::int32_t> token_ids =
        lcqi::encode_prompt(tokenizer, "tok1 tok2 tok3");
    const std::vector<std::int32_t> expected{0, 1, 2, 3, 4};
    require(token_ids == expected, "tokenizer prompt ids mismatch");
    require(lcqi::tokenizer_contract_hash(tokenizer) == LCQI_TOKENIZER_HASH_EXPECTED,
            "tokenizer contract hash mismatch");
}
```

```cpp

void test_tokenizer_rejects_unknown_without_unk() {
    const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(test_tokenizer_path());
    bool threw = false;
    try {
        static_cast<void>(lcqi::encode_prompt(tokenizer, "tok1 missing_token"));
    } catch (const std::runtime_error&) {
        threw = true;
    }
    require(threw, "tokenizer should reject unknown token when unk id is absent");
}

void test_safetensors_manifest_contract() {
    const std::filesystem::path path =
        temp_file_path("lcqi_safetensors_manifest_contract.safetensors");
    const std::string header =
        R"({"__metadata__":{"format":"lcqi-fixture"},"model.embed_tokens.weight":{"dtype":"F32","shape":[2,3],"data_offsets":[0,24]},"model.layers.0.attn.q_proj.weight":{"dtype":"F16","shape":[4,3],"data_offsets":[24,48]}})";
    write_safetensors_fixture(path, header, LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES);

    const lcqi::SafeTensorManifest manifest = lcqi::load_safetensors_manifest(path);
    std::filesystem::remove(path);

    require(manifest.header_size == static_cast<std::uint64_t>(header.size()),
            "safetensors header size mismatch");
    require(manifest.data_start_offset ==
                static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) +
                    static_cast<std::uint64_t>(header.size()),
            "safetensors data start mismatch");
    require(manifest.metadata.size() == 1, "safetensors metadata count mismatch");
    require(manifest.metadata[0].first == "format" &&
                manifest.metadata[0].second == "lcqi-fixture",
            "safetensors metadata mismatch");
    require(manifest.tensors.size() == 2, "safetensors tensor count mismatch");

    const lcqi::SafeTensorEntry* embedding =
        manifest.find_tensor("model.embed_tokens.weight");
    require(embedding != nullptr, "safetensors embedding tensor missing");
    require(embedding->dtype == "F32", "safetensors embedding dtype mismatch");
    require(embedding->shape == std::vector<std::int64_t>({2, 3}),
            "safetensors embedding shape mismatch");
    require(embedding->data_begin == LCQI_SAFETENSORS_EMBED_BEGIN &&
                embedding->data_end == LCQI_SAFETENSORS_EMBED_END,
            "safetensors embedding offset mismatch");
```

```
    require(embedding->byte_size() ==
                LCQI_SAFETENSORS_EMBED_END - LCQI_SAFETENSORS_EMBED_BEGIN,
            "safetensors embedding byte size mismatch");

    const lcqi::SafeTensorEntry* q_proj =
        manifest.find_tensor("model.layers.0.attn.q_proj.weight");
    require(q_proj != nullptr, "safetensors q projection tensor missing");
    require(q_proj->dtype == "F16", "safetensors q projection dtype mismatch");
    require(q_proj->data_begin == LCQI_SAFETENSORS_QPROJ_BEGIN &&
                q_proj->data_end == LCQI_SAFETENSORS_QPROJ_END,
            "safetensors q projection offset mismatch");
}

void test_safetensors_rejects_bad_offsets() {
    const std::filesystem::path path =
        temp_file_path("lcqi_safetensors_bad_offsets.safetensors");
    const std::string header =
        R"({"tensor":{"dtype":"F32","shape":[4],"data_offsets":[0,32]}})";
    write_safetensors_fixture(path, header, 4);

    bool threw = false;
    try {
        static_cast<void>(lcqi::load_safetensors_manifest(path));
    } catch (const std::runtime_error&) {
        threw = true;
    }
    std::filesystem::remove(path);
    require(threw, "safetensors parser should reject offsets beyond payload");
}

void test_safetensors_rejects_bad_dtype_shape_size() {
    const std::filesystem::path path =
        temp_file_path("lcqi_safetensors_bad_dtype_shape_size.safetensors");
    const std::string header =
        R"({"tensor":{"dtype":"F32","shape":[4],"data_offsets":[0,8]}})";
    write_safetensors_fixture(path, header, 8);

    bool threw = false;
    try {
        static_cast<void>(lcqi::load_safetensors_manifest(path));
    } catch (const std::runtime_error&) {
        threw = true;
    }
    std::filesystem::remove(path);
    require(threw, "safetensors parser should reject dtype/shape byte mismatch"↩
    ↪ );
}

void test_safetensors_reads_float_tensors() {
    const std::filesystem::path path =
        temp_file_path("lcqi_safetensors_float_read.safetensors");
    std::vector<std::uint8_t> f16_bytes;
```

```
    append_le_u16(f16_bytes, LCQI_TEST_F16_ONE);
    append_le_u16(f16_bytes, LCQI_TEST_F16_NEGATIVE_TWO);
    std::vector<std::uint8_t> bf16_bytes;
    append_le_u16(bf16_bytes, LCQI_TEST_BF16_ONE_POINT_FIVE);
    append_le_u16(bf16_bytes, LCQI_TEST_BF16_NEGATIVE_HALF);

    const std::array<RawTensorFixture, 3> tensors{{
        {"f32", "F32", {2}, f32_payload(std::array<float, 2>{1.25F, -2.5F})},
        {"f16", "F16", {2}, f16_bytes},
        {"bf16", "BF16", {2}, bf16_bytes},
    }};
    write_safetensors_raw_fixture(path, tensors);

    const lcqi::SafeTensorFile file = lcqi::load_safetensors_file(path);
    std::filesystem::remove(path);
    const std::vector<float> f32 = lcqi::read_safetensor_f32_tensor(file, "f32");
    const std::vector<float> f16 = lcqi::read_safetensor_f32_tensor(file, "f16");
    const std::vector<float> bf16 = lcqi::read_safetensor_f32_tensor(file, "bf16");

    require_close(f32[0], 1.25F, "safetensors F32 value 0 mismatch");
    require_close(f32[1], -2.5F, "safetensors F32 value 1 mismatch");
    require_close(f16[0], 1.0F, "safetensors F16 value 0 mismatch");
    require_close(f16[1], -2.0F, "safetensors F16 value 1 mismatch");
    require_close(bf16[0], 1.5F, "safetensors BF16 value 0 mismatch");
    require_close(bf16[1], -0.5F, "safetensors BF16 value 1 mismatch");
}

void test_gguf_manifest_reads_q4_k_tensor() {
    const std::filesystem::path path = temp_file_path("lcqi_q4_k_fixture.gguf");
    write_gguf_q4_k_fixture(path);

    const lcqi::GgufManifest manifest = lcqi::load_gguf_manifest(path);
    const lcqi::GgufTensorInfo* tensor =
        manifest.find_tensor("blk.0.ffn_up.weight");
    require(tensor != nullptr, "GGUF Q4_K tensor missing");
    const std::vector<std::uint8_t> bytes =
        lcqi::read_gguf_tensor_bytes(path, *tensor);
    std::filesystem::remove(path);

    require(manifest.version == LCQI_GGUF_TEST_VERSION, "GGUF version mismatch");
    require(manifest.alignment == LCQI_GGUF_TEST_ALIGNMENT, "GGUF alignment mismatch");
    require(manifest.metadata.size() == LCQI_GGUF_TEST_METADATA_COUNT,
            "GGUF metadata count mismatch");
    const lcqi::GgufMetadataEntry* tokens = manifest.find_metadata("tokenizer.ggml.tokens");
    require(tokens != nullptr, "GGUF tokenizer tokens metadata missing");
```

```cpp
    require(tokens->string_values == std::vector<std::string>({"<bos>", "hello"}),
            "GGUF tokenizer string array mismatch");
    require(manifest.tensors.size() == LCQI_GGUF_TEST_TENSOR_COUNT,
            "GGUF tensor count mismatch");
    require(tensor->type == lcqi::GgmlType::q4_k, "GGUF tensor type mismatch");
    require(tensor->shape == std::vector<std::int64_t>({lcqi::LCQI_QK_K_BLOCK_VALUES, 1}),
            "GGUF tensor shape mismatch");
    require(tensor->byte_size == lcqi::LCQI_Q4_K_BLOCK_BYTES,
            "GGUF tensor byte size mismatch");
    require(bytes == make_q4_k_test_block(), "GGUF tensor payload mismatch");
}

void test_q4_k_dequant_and_dot_paths() {
    const std::vector<std::uint8_t> block = make_q4_k_test_block();
    const std::vector<float> expected = expected_q4_k_test_values();
    const std::vector<float> decoded =
        lcqi::dequantize_q4_k(block, lcqi::LCQI_QK_K_BLOCK_VALUES);

    require(decoded.size() == expected.size(), "Q4_K decoded size mismatch");
    for (std::size_t index = 0; index < expected.size(); ++index) {
        require_close(decoded[index], expected[index], "Q4_K decoded value mismatch");
    }

    std::vector<float> input(expected.size(), 0.0F);
    for (std::size_t index = 0; index < input.size(); ++index) {
        input[index] = static_cast<float>(static_cast<std::int32_t>(index % 11U) - 5) *
                       0.125F;
    }
    float expected_dot = 0.0F;
    for (std::size_t index = 0; index < expected.size(); ++index) {
        expected_dot += expected[index] * input[index];
    }
    const float q4_f32_dot = lcqi::dot_q4_k_f32(block, input);
    require_close(q4_f32_dot, expected_dot, "Q4_K F32 dot mismatch");

    const std::vector<lcqi::Q8KBlock> q8_input = lcqi::quantize_q8_k_input(input);
    const float q4_q8_dot = lcqi::dot_q4_k_q8(block, q8_input);
    require(std::fabs(q4_q8_dot - q4_f32_dot) <= LCQI_Q4K_Q8_DIFF_LIMIT,
            "Q4_K Q8 dot drift is too large");

    std::vector<float> matvec_output(1, 0.0F);
    lcqi::matvec_q4_k_q8(block, 1, lcqi::LCQI_QK_K_BLOCK_VALUES, q8_input, matvec_output);
    require_close(matvec_output[0], q4_q8_dot, "Q4_K Q8 matvec mismatch");
}

void test_ggml_mixed_dequantizers() {
```

```
    std::vector<std::uint8_t> f32_bytes;
    append_le_f32(f32_bytes, 1.25F);
    append_le_f32(f32_bytes, -2.5F);
    const std::vector<float> f32 =
        lcqi::dequantize_ggml_tensor(lcqi::GgmlType::f32, f32_bytes, 2);
    require_close(f32[0], 1.25F, "GGML F32 decoded value 0 mismatch");
    require_close(f32[1], -2.5F, "GGML F32 decoded value 1 mismatch");

    const std::vector<std::uint8_t> q8 = make_q8_0_test_block();
    const std::vector<float> q8_values =
        lcqi::dequantize_ggml_tensor(lcqi::GgmlType::q8_0,
                                     q8,
                                     lcqi::LCQI_Q8_0_BLOCK_VALUES);
    require_close(q8_values[0], -8.0F, "Q8_0 decoded value 0 mismatch");
    require_close(q8_values[16], 0.0F, "Q8_0 decoded value 16 mismatch");
    require_close(q8_values[31], 7.5F, "Q8_0 decoded value 31 mismatch");

    const std::vector<std::uint8_t> q5 = make_q5_0_test_block();
    const std::vector<float> q5_values =
        lcqi::dequantize_ggml_tensor(lcqi::GgmlType::q5_0,
                                     q5,
                                     lcqi::LCQI_Q5_0_BLOCK_VALUES);
    require_close(q5_values[0], -8.0F, "Q5_0 decoded value 0 mismatch");
    require_close(q5_values[1], -7.5F, "Q5_0 decoded value 1 mismatch");
    require_close(q5_values[15], -0.5F, "Q5_0 decoded value 15 mismatch");
    require_close(q5_values[16], 0.0F, "Q5_0 decoded value 16 mismatch");
    require_close(q5_values[31], 7.5F, "Q5_0 decoded value 31 mismatch");

    const std::vector<std::uint8_t> q6 = make_q6_k_test_block();
    const std::vector<float> q6_values =
        lcqi::dequantize_ggml_tensor(lcqi::GgmlType::q6_k,
                                     q6,
                                     lcqi::LCQI_Q6_K_BLOCK_VALUES);
    require_close(q6_values[LCQI_GGML_TEST_Q6_LANE], 3.0F, "Q6_K q1 mismatch");
    require_close(q6_values[32 + LCQI_GGML_TEST_Q6_LANE], -1.0F, "Q6_K q2 mismatch");
    require_close(q6_values[64 + LCQI_GGML_TEST_Q6_LANE], 31.0F, "Q6_K q3 mismatch");
    require_close(q6_values[96 + LCQI_GGML_TEST_Q6_LANE], -32.0F, "Q6_K q4 mismatch");
    require_close(q6_values[128 + LCQI_GGML_TEST_Q6_LANE], 4.0F, "Q6_K second half mismatch");

    bool rejected_bad_size = false;
    try {
        static_cast<void>(lcqi::dequantize_ggml_tensor(
            lcqi::GgmlType::q8_0,
            std::span<const std::uint8_t>(q8.data(), q8.size() - 1U),
            lcqi::LCQI_Q8_0_BLOCK_VALUES));
    } catch (const std::runtime_error&) {
        rejected_bad_size = true;
    }
```

```
    require(rejected_bad_size, "GGML dequantizer accepted a truncated Q8_0 block");
}

void test_ggml_quantized_f32_matvec_paths() {
    const std::vector<float> input32 = [] {
        std::vector<float> values(static_cast<std::size_t>(lcqi::LCQI_Q8_0_BLOCK_VALUES), 0.0F);
        for (std::size_t index = 0; index < values.size(); ++index) {
            values[index] =
                static_cast<float>(static_cast<std::int32_t>(index % 9U) - 4) * 0.125F;
        }
        return values;
    }();
    const std::vector<float> input256 = [] {
        std::vector<float> values(static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_VALUES), 0.0F);
        for (std::size_t index = 0; index < values.size(); ++index) {
            values[index] =
                static_cast<float>(static_cast<std::int32_t>(index % 13U) - 6) * 0.0625F;
        }
        return values;
    }();

    const auto assert_matvec_matches_dequantized =
        [](lcqi::GgmlType type,
           const std::vector<std::uint8_t>& block,
           std::span<const float> input,
           const char* message) {
            const std::int64_t columns = static_cast<std::int64_t>(input.size());
            const std::vector<std::uint8_t> matrix = [&] {
                std::vector<std::uint8_t> bytes;
                bytes.reserve(block.size() * 2U);
                bytes.insert(bytes.end(), block.begin(), block.end());
                bytes.insert(bytes.end(), block.begin(), block.end());
                return bytes;
            }();
            std::vector<float> direct(2, 0.0F);
            lcqi::matvec_ggml_quantized_f32(type, matrix, 2, columns, input, direct);
            const std::vector<lcqi::Q8_0InputBlock> q8_0_input =
                lcqi::quantize_q8_0_input(input);
            std::vector<float> q8_0_direct(2, 0.0F);
            lcqi::matvec_ggml_quantized_q8_0(type,
                                             matrix,
                                             2,
                                             columns,
                                             q8_0_input,
                                             q8_0_direct);
```

```
            const std::vector<float> weights =
                lcqi::dequantize_ggml_tensor(type, matrix, columns * 2);
            std::array<float, 2> reference{0.0F, 0.0F};
            for (std::size_t row = 0; row < reference.size(); ++row) {
                for (std::size_t column = 0; column < input.size(); ++column) {
                    reference[row] +=
                        weights[row * input.size() + column] * input[column];
                }
                if (std::fabs(reference[row] - direct[row]) > LCQI_GGML_MATVEC_MAX_DIFF) {
                    throw std::runtime_error(message);
                }
                if (std::fabs(reference[row] - q8_0_direct[row]) >
                    LCQI_GGML_Q8_MATVEC_MAX_DIFF) {
                    throw std::runtime_error(message);
                }
            }
        };

    require(lcqi::ggml_type_has_f32_direct_matvec(lcqi::GgmlType::q5_0),
            "Q5_0 direct matvec support flag mismatch");
    require(lcqi::ggml_type_has_f32_direct_matvec(lcqi::GgmlType::q6_k),
            "Q6_K direct matvec support flag mismatch");
    require(lcqi::ggml_type_has_f32_direct_matvec(lcqi::GgmlType::q8_0),
            "Q8_0 direct matvec support flag mismatch");
    require(!lcqi::ggml_type_has_f32_direct_matvec(lcqi::GgmlType::f32),
            "F32 should not be reported as quantized direct matvec");
    require(lcqi::ggml_type_has_q8_0_direct_matvec(lcqi::GgmlType::q5_0),
            "Q5_0 Q8_0 direct matvec support flag mismatch");
    require(lcqi::ggml_type_has_q8_0_direct_matvec(lcqi::GgmlType::q6_k),
            "Q6_K Q8_0 direct matvec support flag mismatch");
    require(lcqi::ggml_type_has_q8_0_direct_matvec(lcqi::GgmlType::q8_0),
            "Q8_0 Q8_0 direct matvec support flag mismatch");
    require(!lcqi::ggml_type_has_q8_0_direct_matvec(lcqi::GgmlType::f32),
            "F32 should not be reported as Q8_0 quantized direct matvec");

    assert_matvec_matches_dequantized(lcqi::GgmlType::q8_0,
                                      make_q8_0_test_block(),
                                      input32,
                                      "Q8_0 direct matvec mismatch");
    assert_matvec_matches_dequantized(lcqi::GgmlType::q5_0,
                                      make_q5_0_test_block(),
                                      input32,
                                      "Q5_0 direct matvec mismatch");
    assert_matvec_matches_dequantized(lcqi::GgmlType::q6_k,
                                      make_q6_k_test_block(),
                                      input256,
                                      "Q6_K direct matvec mismatch");
}

void test_llama_gguf_loader_and_generation() {
    const std::filesystem::path path = temp_file_path("lcqi_tiny_llama_fixture.
```

```
gguf");
    write_tiny_llama_gguf_fixture(path);

    const lcqi::LlamaGgufLoadedModel loaded =
        lcqi::load_llama_gguf_reference_model(path);
    const std::vector<std::int32_t> prompt{1};
    const lcqi::LlamaGgufGenerationResult result =
        lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
    std::filesystem::remove(path);

    require(loaded.model.architecture == "llama", "LLaMA GGUF architecture mismatch");
    require(loaded.model.decoder.config.hidden_size == LCQI_LLAMA_TEST_HIDDEN,
            "LLaMA GGUF hidden size mismatch");
    require(loaded.model.decoder.layers.size() == LCQI_LLAMA_TEST_LAYER_COUNT,
            "LLaMA GGUF layer count mismatch");
    require(loaded.model.lm_head_tied_to_embedding,
            "LLaMA GGUF tiny fixture should tie lm head to embeddings");
    require(result.generated_ids == std::vector<std::int32_t>({1, LCQI_LLAMA_TEST_EXPECTED_TOKEN}),
            "LLaMA GGUF generated ids mismatch");
    require(result.predicted_first_token == LCQI_LLAMA_TEST_EXPECTED_TOKEN,
            "LLaMA GGUF predicted token mismatch");
    require(loaded.report.tensors_loaded == 11, "LLaMA GGUF tensor load count mismatch");
    require(loaded.report.f32_weight_bytes == loaded.report.quantized_weight_bytes,
            "LLaMA GGUF F32 fixture byte accounting mismatch");
    require(loaded.report.q4_k_direct_tensors == 0,
            "LLaMA GGUF F32 fixture should not use direct Q4_K tensors");
    require(loaded.report.ggml_direct_tensors == 0,
            "LLaMA GGUF F32 fixture should not use direct GGML tensors");
    require(result.hotspots.f32_fallback_calls == 7,
            "LLaMA GGUF F32 fixture linear fallback call count mismatch");
    require(result.hotspots.q4_k_direct_calls == 0,
            "LLaMA GGUF F32 fixture should not call Q4_K direct matvec");
    require(result.hotspots.ggml_direct_calls == 0,
            "LLaMA GGUF F32 fixture should not call GGML direct matvec");
}

void test_llama_gguf_q4_direct_generation() {
    const std::filesystem::path path = temp_file_path("lcqi_tiny_llama_q4_direct.gguf");
    write_tiny_llama_q4_direct_gguf_fixture(path);

    const lcqi::LlamaGgufLoadedModel loaded =
        lcqi::load_llama_gguf_reference_model(path);
    const std::vector<std::int32_t> prompt{0};
    const lcqi::LlamaGgufGenerationResult result =
        lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
    std::filesystem::remove(path);

```

```
    require(loaded.model.decoder.config.hidden_size == LCQI_LLAMA_Q4_TEST_HIDDEN,
            "LLaMA Q4_K fixture hidden size mismatch");
    require(loaded.report.tensors_loaded == 11, "LLaMA Q4_K fixture tensor load count mismatch");
    require(loaded.report.q4_k_direct_tensors == 7,
            "LLaMA Q4_K fixture direct tensor count mismatch");
    require(loaded.report.ggml_direct_tensors == 0,
            "LLaMA Q4_K fixture should not use GGML direct tensors");
    require(loaded.report.f32_fallback_tensors == 4,
            "LLaMA Q4_K fixture fallback tensor count mismatch");
    require(loaded.report.direct_quantized_weight_bytes >
                loaded.report.fallback_dequantized_weight_bytes,
            "LLaMA Q4_K fixture should keep direct quantized matrix bytes");
    require(result.hotspots.q4_k_direct_calls == 7,
            "LLaMA Q4_K fixture direct matvec call count mismatch");
    require(result.hotspots.ggml_direct_calls == 0,
            "LLaMA Q4_K fixture should not call GGML direct matvec");
    require(result.hotspots.f32_fallback_calls == 0,
            "LLaMA Q4_K fixture should not call fallback linear matvec");
    require(result.generated_ids == std::vector<std::int32_t>({0, 0}),
            "LLaMA Q4_K fixture generated ids mismatch");
}

void test_llama_gguf_ggml_direct_generation() {
    const ScopedEnvVar enable_ggml_direct(LCQI_TEST_LLAMA_GGML_DIRECT_ENV,
                                          LCQI_TEST_TRUE_ENV);
    const std::filesystem::path path = temp_file_path("lcqi_tiny_llama_ggml_direct.gguf");
    write_tiny_llama_ggml_direct_gguf_fixture(path);

    const lcqi::LlamaGgufLoadedModel loaded =
        lcqi::load_llama_gguf_reference_model(path);
    const std::vector<std::int32_t> prompt{0};
    const lcqi::LlamaGgufGenerationResult result =
        lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
    std::filesystem::remove(path);

    require(loaded.model.decoder.config.hidden_size == LCQI_LLAMA_Q4_TEST_HIDDEN,
            "LLaMA GGML fixture hidden size mismatch");
    require(loaded.report.tensors_loaded == 11, "LLaMA GGML fixture tensor load count mismatch");
    require(loaded.report.q4_k_direct_tensors == 0,
            "LLaMA GGML fixture should not use Q4_K direct tensors");
    require(loaded.report.ggml_direct_tensors == 7,
            "LLaMA GGML fixture direct tensor count mismatch");
    require(loaded.report.f32_fallback_tensors == 4,
            "LLaMA GGML fixture fallback tensor count mismatch");
    require(result.hotspots.ggml_direct_calls == 7,
            "LLaMA GGML fixture direct matvec call count mismatch");
    require(result.hotspots.q4_k_direct_calls == 0,
```

```cpp
            "LLaMA GGML fixture should not call Q4_K direct matvec");
    require(result.hotspots.f32_fallback_calls == 0,
            "LLaMA GGML fixture should not call fallback linear matvec");
    require(result.generated_ids == std::vector<std::int32_t>({0, 0}),
            "LLaMA GGML fixture generated ids mismatch");
}

void test_gpt2_tiny_forward_and_generation() {
    const lcqi::Gpt2ReferenceModel model = lcqi::make_tiny_gpt2_reference_model();
    const std::vector<std::int32_t> tokens{1, 2, 3};
    const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, tokens);

    require(result.logits.size() == LCQI_GPT2_VOCAB_SIZE, "GPT-2 logits size mismatch");
    require(result.predicted_token == LCQI_GPT2_PREDICTED_TOKEN,
            "GPT-2 predicted token mismatch");
    for (std::size_t index = 0; index < LCQI_GPT2_LOGITS.size(); ++index) {
        require_close(result.logits[index], LCQI_GPT2_LOGITS[index], "GPT-2 logit mismatch");
    }

    const std::vector<std::int32_t> generated =
        lcqi::gpt2_generate_greedy(model, tokens, 2);
    const std::vector<std::int32_t> expected{1, 2, 3, 3, 3};
    require(generated == expected, "GPT-2 greedy generation mismatch");
    require(static_cast<std::int32_t>(generated.size()) == LCQI_GPT2_GENERATED_LENGTH,
            "GPT-2 generated length mismatch");

    lcqi::Gpt2KvCache cache(model.config);
    lcqi::Gpt2ForwardResult cached_result;
    for (const std::int32_t token : tokens) {
        cached_result = lcqi::run_gpt2_forward_cached(model, cache, token);
    }
    require(cache.filled_tokens() == static_cast<std::int32_t>(tokens.size()),
            "GPT-2 KV cache filled token count mismatch");
    require(cache.byte_size() > 0, "GPT-2 KV cache byte size should be non-zero");
    require(cached_result.predicted_token == result.predicted_token,
            "cached GPT-2 predicted token mismatch");
    require(cached_result.logits.size() == result.logits.size(),
            "cached GPT-2 logits size mismatch");
    for (std::size_t index = 0; index < result.logits.size(); ++index) {
        require_close(cached_result.logits[index],
                      result.logits[index],
                      "cached GPT-2 logit mismatch");
    }

    lcqi::Gpt2CachedGreedyDecoder decoder_with_logits(model);
    lcqi::Gpt2ForwardResult decoder_result;
```

```
    for (const std::int32_t token : tokens) {
        decoder_result = decoder_with_logits.step_with_logits(token);
    }
    require(decoder_with_logits.filled_tokens() == static_cast<std::int32_t>(tokens.size()),
            "GPT-2 optimized decoder filled token count mismatch");
    require(decoder_with_logits.kv_cache_bytes() == cache.byte_size(),
            "GPT-2 optimized decoder KV byte size mismatch");
    require(decoder_result.predicted_token == result.predicted_token,
            "optimized cached GPT-2 predicted token mismatch");
    require(decoder_result.logits.size() == result.logits.size(),
            "optimized cached GPT-2 logits size mismatch");
    for (std::size_t index = 0; index < result.logits.size(); ++index) {
        require_close(decoder_result.logits[index],
                      result.logits[index],
                      "optimized cached GPT-2 logit mismatch");
    }

    lcqi::Gpt2ExecutionOptions parallel_options;
    parallel_options.worker_count = 2;
    parallel_options.parallel_min_rows = 1;
    lcqi::Gpt2CachedGreedyDecoder parallel_decoder(model, nullptr, parallel_options);
    lcqi::Gpt2ForwardResult parallel_result;
    for (const std::int32_t token : tokens) {
        parallel_result = parallel_decoder.step_with_logits(token);
    }
    require(parallel_decoder.worker_count() == parallel_options.worker_count,
            "parallel GPT-2 decoder worker count mismatch");
    require(parallel_result.predicted_token == result.predicted_token,
            "parallel cached GPT-2 predicted token mismatch");
    require(parallel_result.logits.size() == result.logits.size(),
            "parallel cached GPT-2 logits size mismatch");
    for (std::size_t index = 0; index < result.logits.size(); ++index) {
        require_close(parallel_result.logits[index],
                      result.logits[index],
                      "parallel cached GPT-2 logit mismatch");
    }

    lcqi::Gpt2CachedGreedyDecoder greedy_decoder(model);
    std::int32_t greedy_predicted = 0;
    for (const std::int32_t token : tokens) {
        greedy_predicted = greedy_decoder.step(token);
    }
    require(greedy_predicted == result.predicted_token,
            "optimized logits-free GPT-2 predicted token mismatch");

    lcqi::Gpt2CachedGreedyDecoder prefill_decoder(model);
    for (std::size_t index = 0; index + 1 < tokens.size(); ++index) {
        prefill_decoder.advance_without_prediction(tokens[index]);
    }
    require(prefill_decoder.filled_tokens() ==
```

```
                static_cast<std::int32_t>(tokens.size() - 1U),
            "GPT-2 prefill-only decoder filled token count mismatch");
    const std::int32_t prefill_predicted = prefill_decoder.step(tokens.back());
    require(prefill_decoder.filled_tokens() == static_cast<std::int32_t>(tokens.size()),
            "GPT-2 prefill decoder final filled token count mismatch");
    require(prefill_predicted == result.predicted_token,
            "GPT-2 prefill-skipped predicted token mismatch");

    lcqi::Gpt2HotspotProfile hotspot_profile;
    lcqi::Gpt2CachedGreedyDecoder profiled_decoder(model, &hotspot_profile);
    std::int32_t profiled_predicted = 0;
    for (const std::int32_t token : tokens) {
        profiled_predicted = profiled_decoder.step(token);
    }
    require(profiled_predicted == result.predicted_token,
            "profiled GPT-2 predicted token mismatch");
    require(hotspot_profile.decoder_steps == static_cast<std::int64_t>(tokens.size()),
            "profiled GPT-2 decoder step count mismatch");
    require(hotspot_profile.layer_steps ==
                static_cast<std::int64_t>(tokens.size()) * model.config.layer_count,
            "profiled GPT-2 layer step count mismatch");
    require(hotspot_profile.total_step_ms >= hotspot_profile.lm_head_ms,
            "profiled GPT-2 total time should include lm head time");

    const std::vector<std::int32_t> cached_generated =
        lcqi::gpt2_generate_greedy_cached(model, tokens, 2);
    require(cached_generated == expected, "cached GPT-2 greedy generation mismatch");
}

void test_gpt2_byte_bpe_tokenizer() {
    const std::filesystem::path vocab_path = temp_file_path("lcqi_gpt2_vocab.json");
    const std::filesystem::path merges_path = temp_file_path("lcqi_gpt2_merges.txt");
    const std::string space_marker = "\xC4\xA0";
    const std::string vocab =
        "{\"Hello\":7,\",\":8,\"" + space_marker + "world\":9,\"!\":10}";
    const std::string merges =
        "#version: 0.2\n"
        "H e\n"
        "He l\n"
        "Hel l\n"
        "Hell o\n" +
        space_marker + " w\n" +
        space_marker + "w o\n" +
        space_marker + "wo r\n" +
        space_marker + "wor l\n" +
        space_marker + "worl d\n";
```

```
    write_text_file(vocab_path, vocab);
    write_text_file(merges_path, merges);

    const lcqi::Gpt2Tokenizer tokenizer =
        lcqi::load_gpt2_tokenizer(vocab_path, merges_path, -1, -1);
    std::filesystem::remove(vocab_path);
    std::filesystem::remove(merges_path);

    const std::vector<std::int32_t> ids =
        lcqi::gpt2_encode(tokenizer, "Hello, world!");
    const std::vector<std::int32_t> expected{
        LCQI_GPT2_TOKENIZER_HELLO_ID,
        8,
        9,
        10,
    };
    require(ids == expected, "GPT-2 BPE ids mismatch");
    require(lcqi::gpt2_decode(tokenizer, ids) == "Hello, world!",
            "GPT-2 BPE decode mismatch");
}

void test_gpt2_loads_hf_style_tiny_checkpoint() {
    const std::filesystem::path directory =
        temp_file_path("lcqi_gpt2_tiny_checkpoint_dir");
    std::filesystem::remove_all(directory);
    std::filesystem::create_directories(directory);

    const lcqi::Gpt2ReferenceModel original = lcqi::make_tiny_gpt2_reference_model();
    write_text_file(
        directory / "config.json",
        R"({"model_type":"gpt2","n_embd":4,"n_head":2,"n_layer":1,"n_positions":8,"n_ctx":8,"vocab_size":6,"n_inner":8,"layer_norm_epsilon":0.00001,"activation_function":"gelu_new","bos_token_id":0,"eos_token_id":5})");
    write_tiny_gpt2_safetensors(directory / "model.safetensors", original);

    const lcqi::Gpt2ReferenceModel loaded = lcqi::load_gpt2_from_directory(directory);
    const std::vector<std::int32_t> tokens{1, 2, 3};
    const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(loaded, tokens);
    std::filesystem::remove_all(directory);

    require(result.predicted_token == LCQI_GPT2_PREDICTED_TOKEN,
            "loaded GPT-2 predicted token mismatch");
    for (std::size_t index = 0; index < LCQI_GPT2_LOGITS.size(); ++index) {
        require_close(result.logits[index],
                      LCQI_GPT2_LOGITS[index],
                      "loaded GPT-2 logit mismatch");
    }
}
```

```cpp
void test_gpt2_loader_rejects_bad_shape() {
    const std::filesystem::path directory =
        temp_file_path("lcqi_gpt2_bad_shape_dir");
    std::filesystem::remove_all(directory);
    std::filesystem::create_directories(directory);

    const lcqi::Gpt2ReferenceModel original = lcqi::make_tiny_gpt2_reference_model();
    write_text_file(
        directory / "config.json",
        R"({"model_type":"gpt2","n_embd":5,"n_head":1,"n_layer":1,"n_positions":8,"n_ctx":8,"vocab_size":6,"n_inner":8,"layer_norm_epsilon":0.00001,"activation_function":"gelu_new","bos_token_id":0,"eos_token_id":5})");
    write_tiny_gpt2_safetensors(directory / "model.safetensors", original);

    bool threw = false;
    try {
        static_cast<void>(lcqi::load_gpt2_from_directory(directory));
    } catch (const std::runtime_error&) {
        threw = true;
    }
    std::filesystem::remove_all(directory);
    require(threw, "GPT-2 loader should reject bad tensor shape");
}

void test_reference_rms_norm() {
    const std::vector<float> input{3.0F, 4.0F};
    const std::vector<float> weight{1.0F, 0.5F};
    std::vector<float> output(2, 0.0F);
    lcqi::rms_norm(input, weight, 0.0F, output);
    require_close(output[0], LCQI_RMS_EXPECTED_0, "RMSNorm output 0 mismatch");
    require_close(output[1], LCQI_RMS_EXPECTED_1, "RMSNorm output 1 mismatch");
}

void test_reference_rope() {
    std::vector<float> heads{1.0F, 0.0F, 0.0F, 1.0F};
    lcqi::apply_rope(heads, 1, 4, 1, 10000.0F);
    require_close(heads[0], std::cos(1.0F), "RoPE pair 0 cosine mismatch");
    require_close(heads[1], std::sin(1.0F), "RoPE pair 0 sine mismatch");
    require_close(heads[2], -std::sin(0.01F), "RoPE pair 1 negative sine mismatch");
    require_close(heads[3], std::cos(0.01F), "RoPE pair 1 cosine mismatch");
}

void test_reference_kv_attention() {
    lcqi::DecoderConfig config;
    config.hidden_size = 4;
    config.query_heads = 2;
    config.kv_heads = 1;
    config.head_dim = 2;
    config.intermediate_size = 4;
    config.vocab_size = 4;
```

```cpp
    config.max_context = 4;

    lcqi::ReferenceKVCache cache(config, 1);
    const std::vector<float> key{1.0F, 0.0F};
    const std::vector<float> value{LCQI_ATTENTION_EXPECTED_0, LCQI_ATTENTION_EXPECTED_1};
    cache.append(0, 0, key, value);
    require(cache.filled_tokens() == 1, "KV filled token count mismatch");
    require_close(cache.key(0, 0, 0)[0], 1.0F, "KV key read mismatch");

    const std::vector<float> query{1.0F, 0.0F, 0.0F, 1.0F};
    std::vector<float> output(4, 0.0F);
    lcqi::attention_decode(config, cache, 0, 0, query, output);
    require_close(output[0], LCQI_ATTENTION_EXPECTED_0, "attention head 0 dim 0 mismatch");
    require_close(output[1], LCQI_ATTENTION_EXPECTED_1, "attention head 0 dim 1 mismatch");
    require_close(output[2], LCQI_ATTENTION_EXPECTED_0, "attention head 1 dim 0 mismatch");
    require_close(output[3], LCQI_ATTENTION_EXPECTED_1, "attention head 1 dim 1 mismatch");
}

void test_reference_decoder_end_to_end() {
    const lcqi::ReferenceDecoderModel model = lcqi::make_tiny_reference_decoder_model();
    const std::vector<std::int32_t> tokens{1, 2, 3};
    const lcqi::ReferenceDecodeResult result = lcqi::run_reference_decode(model, tokens);
    require(result.logits.size() == LCQI_REFERENCE_DECODER_VOCAB_SIZE,
            "reference decoder logits size mismatch");
    require(result.predicted_token == LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
            "reference decoder predicted token mismatch");
    for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.size(); ++index) {
        require_close(result.logits[index],
                      LCQI_REFERENCE_DECODER_LOGITS[index],
                      "reference decoder golden logit mismatch");
    }
}

void test_reference_decoder_trace_contract() {
    const lcqi::ReferenceDecoderModel model = lcqi::make_tiny_reference_decoder_model();
    const std::vector<std::int32_t> tokens{1, 2, 3};
    const lcqi::ReferenceDecodeTraceResult trace =
        lcqi::run_reference_decode_with_trace(model, tokens);

    require(trace.result.predicted_token == LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
            "reference trace predicted token mismatch");
    require(!trace.checkpoints.empty(), "reference trace checkpoints missing");
```

```cpp

    const lcqi::ReferenceTensorCheckpoint& first = trace.checkpoints.front();
    require(first.name == "embedding", "first checkpoint should be embedding");
    require(first.token_position == LCQI_REFERENCE_TRACE_FIRST_TOKEN,
            "first checkpoint token position mismatch");
    require(first.dtype == "f32", "first checkpoint dtype mismatch");
    require(first.layout == "row_major", "first checkpoint layout mismatch");
    require(first.shape.size() == 1 &&
                first.shape[0] == LCQI_REFERENCE_TRACE_HIDDEN_SIZE,
            "first checkpoint shape mismatch");
    require(first.stride.size() == 1 && first.stride[0] == 1,
            "first checkpoint stride mismatch");

    bool saw_logits = false;
    bool saw_kv_slot = false;
    for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints) {
        if (checkpoint.name == "kv_cache_key_slot") {
            saw_kv_slot = true;
            require(checkpoint.shape.size() == 4, "KV checkpoint rank mismatch");
            require(checkpoint.layout == "kv_contiguous", "KV checkpoint layout mismatch");
        }
        if (checkpoint.name == "logits") {
            saw_logits = true;
            require(checkpoint.token_position == LCQI_REFERENCE_TRACE_TOKEN_COUNT - 1,
                    "logits checkpoint token position mismatch");
            require(checkpoint.shape.size() == 1 &&
                        checkpoint.shape[0] ==
                            static_cast<std::int32_t>(LCQI_REFERENCE_DECODER_VOCAB_SIZE),
                    "logits checkpoint shape mismatch");
            require(checkpoint.values.size() == LCQI_REFERENCE_DECODER_LOGITS.size(),
                    "logits checkpoint value size mismatch");
            for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.size(); ++index) {
                require_close(checkpoint.values[index],
                              LCQI_REFERENCE_DECODER_LOGITS[index],
                              "trace logits checkpoint mismatch");
            }
        }
    }
    require(saw_kv_slot, "KV checkpoint missing");
    require(saw_logits, "logits checkpoint missing");
}

void test_packed_i8_kernel_matches_scalar() {
    for (const KernelShapeCase& shape_case : LCQI_KERNEL_SHAPE_CASES) {
        const lcqi::QuantizedLinearLayer layer =
```

```
            lcqi::make_deterministic_i8_layer(shape_case.input_size,
                                              shape_case.output_size,
                                              0.03125F);
        const lcqi::PackedLinearI8 packed =
            lcqi::pack_linear_i8(layer, shape_case.output_block_size);
        const std::vector<float> input = lcqi::make_deterministic_input(layer.input_size);
        std::vector<float> scalar_output(static_cast<std::size_t>(layer.output_size), 0.0F);
        std::vector<float> packed_output(static_cast<std::size_t>(layer.output_size), 0.0F);
        lcqi::linear_i8(layer, input, scalar_output);
        lcqi::linear_i8_packed(packed, input, packed_output);
        require(lcqi::max_abs_diff(scalar_output, packed_output) <= LCQI_KERNEL_MAX_DIFF,
                "packed int8 kernel output differs from scalar");

        const lcqi::PackedLinearI8 simd_packed =
            lcqi::pack_linear_i8(layer, LCQI_SIMD_TEST_BLOCK);
        if (lcqi::linear_i8_packed_avx2_available()) {
            std::vector<float> avx2_output(static_cast<std::size_t>(layer.output_size), 0.0F);
            lcqi::linear_i8_packed_avx2(simd_packed, input, avx2_output);
            require(lcqi::max_abs_diff(scalar_output, avx2_output) <= LCQI_KERNEL_MAX_DIFF,
                    "AVX2 int8 kernel output differs from scalar");
        }
    }
}

void test_f32_dot_kernel_matches_scalar() {
    std::vector<float> lhs(LCQI_F32_DOT_TEST_SIZE, 0.0F);
    std::vector<float> rhs(LCQI_F32_DOT_TEST_SIZE, 0.0F);
    for (std::size_t index = 0; index < LCQI_F32_DOT_TEST_SIZE; ++index) {
        lhs[index] = static_cast<float>(static_cast<std::int32_t>(index % 19U) + 1) *
                     0.03125F;
        rhs[index] = static_cast<float>(static_cast<std::int32_t>(index % 23U) - 11) *
                     0.0625F;
    }

    const float scalar =
        lcqi::dot_f32_scalar_unchecked(lhs.data(), rhs.data(), lhs.size());
    const float dispatched = lcqi::dot_f32(lhs, rhs);
    require(std::fabs(scalar - dispatched) <= LCQI_F32_KERNEL_MAX_DIFF,
            "F32 dot kernel output differs from scalar");

    if (lcqi::dot_f32_avx2_available()) {
        const float avx2 =
            lcqi::dot_f32_avx2_unchecked(lhs.data(), rhs.data(), lhs.size());
        require(std::fabs(scalar - avx2) <= LCQI_F32_KERNEL_MAX_DIFF,
```

```
                "AVX2 F32 dot kernel output differs from scalar");
    }

    bool rejected_mismatch = false;
    try {
        (void)lcqi::dot_f32(std::span<const float>(lhs.data(), lhs.size() - 1U), rhs);
    } catch (const std::runtime_error&) {
        rejected_mismatch = true;
    }
    require(rejected_mismatch, "F32 dot kernel accepted mismatched sizes");
}

void test_f32_row_kernels_match_scalar() {
    std::vector<float> weights(LCQI_F32_ROW_TEST_INPUT_SIZE *
                                   LCQI_F32_ROW_TEST_OUTPUT_SIZE,
                               0.0F);
    std::vector<float> input(LCQI_F32_ROW_TEST_INPUT_SIZE, 0.0F);
    std::vector<float> bias(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
    for (std::size_t index = 0; index < weights.size(); ++index) {
        weights[index] = static_cast<float>(static_cast<std::int32_t>(index % 29U) - 14) *
                         0.015625F;
    }
    for (std::size_t index = 0; index < input.size(); ++index) {
        input[index] = static_cast<float>(static_cast<std::int32_t>(index % 13U) - 6) *
                       0.03125F;
    }
    for (std::size_t index = 0; index < bias.size(); ++index) {
        bias[index] = static_cast<float>(static_cast<std::int32_t>(index % 7U) - 3) *
                      0.125F;
    }

    std::vector<float> scalar_output(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
    std::vector<float> dispatched_output(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
    lcqi::linear_f32_rows_scalar_unchecked(weights.data(),
                                           input.data(),
                                           bias.data(),
                                           input.size(),
                                           0,
                                           LCQI_F32_ROW_TEST_OUTPUT_SIZE,
                                           scalar_output.data());
    lcqi::linear_f32_rows_unchecked(weights.data(),
                                    input.data(),
                                    bias.data(),
                                    input.size(),
                                    0,
                                    LCQI_F32_ROW_TEST_OUTPUT_SIZE,
                                    dispatched_output.data());
    for (std::size_t index = 0; index < scalar_output.size(); ++index) {
```

```cpp
        require(std::fabs(scalar_output[index] - dispatched_output[index]) <=
                    LCQI_F32_KERNEL_MAX_DIFF,
                "F32 linear row kernel output differs from scalar");
    }

    const lcqi::F32RowMax scalar_max =
        lcqi::max_dot_f32_rows_scalar_unchecked(weights.data(),
                                                input.data(),
                                                input.size(),
                                                0,
                                                LCQI_F32_ROW_TEST_OUTPUT_SIZE);
    const lcqi::F32RowMax dispatched_max =
        lcqi::max_dot_f32_rows_unchecked(weights.data(),
                                         input.data(),
                                         input.size(),
                                         0,
                                         LCQI_F32_ROW_TEST_OUTPUT_SIZE);
    require(scalar_max.row == dispatched_max.row,
            "F32 row max kernel selected a different row");
    require(std::fabs(scalar_max.value - dispatched_max.value) <= LCQI_F32_KERNEL_MAX_DIFF,
            "F32 row max kernel value differs from scalar");

    if (lcqi::dot_f32_avx2_available()) {
        std::vector<float> avx2_output(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
        lcqi::linear_f32_rows_avx2_unchecked(weights.data(),
                                             input.data(),
                                             bias.data(),
                                             input.size(),
                                             0,
                                             LCQI_F32_ROW_TEST_OUTPUT_SIZE,
                                             avx2_output.data());
        for (std::size_t index = 0; index < scalar_output.size(); ++index) {
            require(std::fabs(scalar_output[index] - avx2_output[index]) <=
                        LCQI_F32_KERNEL_MAX_DIFF,
                    "AVX2 F32 linear row kernel output differs from scalar");
        }

        const lcqi::F32RowMax avx2_max =
            lcqi::max_dot_f32_rows_avx2_unchecked(weights.data(),
                                                  input.data(),
                                                  input.size(),
                                                  0,
                                                  LCQI_F32_ROW_TEST_OUTPUT_SIZE);
        require(scalar_max.row == avx2_max.row,
                "AVX2 F32 row max kernel selected a different row");
        require(std::fabs(scalar_max.value - avx2_max.value) <= LCQI_F32_KERNEL_MAX_DIFF,
                "AVX2 F32 row max kernel value differs from scalar");
    }
}
```

```cpp

}  // namespace

int main() {
    try {
        test_tiny_mlp();
        test_tokenizer_contract();
        test_tokenizer_rejects_unknown_without_unk();
        test_safetensors_manifest_contract();
        test_safetensors_rejects_bad_offsets();
        test_safetensors_rejects_bad_dtype_shape_size();
        test_safetensors_reads_float_tensors();
        test_gguf_manifest_reads_q4_k_tensor();
        test_q4_k_dequant_and_dot_paths();
        test_ggml_mixed_dequantizers();
        test_ggml_quantized_f32_matvec_paths();
        test_llama_gguf_loader_and_generation();
        test_llama_gguf_q4_direct_generation();
        test_llama_gguf_ggml_direct_generation();
        test_gpt2_tiny_forward_and_generation();
        test_gpt2_byte_bpe_tokenizer();
        test_gpt2_loads_hf_style_tiny_checkpoint();
        test_gpt2_loader_rejects_bad_shape();
        test_reference_rms_norm();
        test_reference_rope();
        test_reference_kv_attention();
        test_reference_decoder_end_to_end();
        test_reference_decoder_trace_contract();
        test_packed_i8_kernel_matches_scalar();
        test_f32_dot_kernel_matches_scalar();
        test_f32_row_kernels_match_scalar();

        std::cout << "lcqi tests passed\n";
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "lcqi test failure: " << error.what() << "\n";
        return EXIT_FAILURE;
    }
}
```

实践与代码总索引

本卷已经把实践任务、代码走读路线和完整代码清单合并到同一套内容目录。目录中不再存在一个独立的“源码卷部分”：构建入口、调用链、公共合同、tiny MLP、int8 kernel、reference decoder、GPT-2、工具脚本和测试文件都插入到相应主题之后，共同解释同一份 `books/cpu-volume-3/labs/linux_cpu_inference` 源码。

源码阅读任务

> 读者不需要再在实践卷和源码卷之间来回切换，也不需要先读完 16 个实践章再去末尾翻代码。实践章节提出任务，紧邻的代码走读解释完整实现，测试和 benchmark 章节给出回归证据；三者已经合并到这一本实践与代码卷里，共同构成一条可复查的工程证据链。

一、实践主题到代码走读的合并位置

第 1 章之后 插入“代码走读：构建入口、项目边界与调用链总图”，先把 CMake、README、模型资产、target、CLI、benchmark、trace、ledger、serving probe 和测试入口放到一张可复现地图里。

第 4 章之后 插入“代码走读：公共合同、Tiny MLP 与 Int8 Kernel”，把模型、推理、tokenizer、safetensors、reference decoder、GPT-2 头文件，以及 tiny MLP、基础 CLI、scalar/packed/AVX2 kernel 连起来。

第 10 章之后 插入“代码走读：Reference Decoder、Tokenizer、Safetensors 与 GPT-2”，把真实模型加载、byte-level BPE、safetensors、GPT-2 forward、cached KV 和 greedy generation 放在模型导入主题内部讲清楚。

第 16 章之后 插入“代码走读：工具、Smoke、Benchmark 与回归测试”，把 benchmark、trace、ledger、serving probe、SmolLM2 smoke、GPT-2 smoke、benchmark compare 和 CTest 回归证据集中收束。

二、最小复现命令

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
make cpu3p-pdf
make cpu3p-epub
```

完成这组命令后，读者应同时检查相邻实践章节的作业结果、对应代码走读、测试断言和报告字段，确认代码、测试、报告字段和章节讲解是一条证据链。

术语表

术语	实践含义
manifest	模型资产清单，记录 tensor name、dtype、shape、offset、checksum 和 tokenizer/配置版本。
shape verifier	在 kernel 运行前检查 shape、dtype、layout、量化 descriptor 和 state edge 是否匹配。
reference decoder	以清楚正确为目标的 decoder 实现，用来生成 trace 和 golden。
oracle	比较候选实现与 reference 的判定器，可使用精确相等或误差界。
trace checkpoint	推理链路中的可复查节点，至少包含 shape、stride、dtype、layout、checksum 和摘要值。
KV cache	attention 历史 K/V 状态，跨 token 存活，不能被普通 activation planner 复用。
packing	把权重改写成更适合热循环和 SIMD 的布局。
backend	执行计划落到机器的实现，例如 scalar、AVX2、AVX-512、GPU 或库调用。
fallback	后端能力不满足时显式降级到可验证路径，并记录原因。
SLO	服务目标，例如 TTFT、TPOT、queue wait、拒绝率和取消回收时间。